

C++ Zero to Godhood

The Definitive Guide from C++98 to C++26

Master the Beast.

Community Edition

February 4, 2026

Contents

1	Preface	1
2	The Complete C++ Programmer's Guide: From Zero to Godhood (C++98 to C++26)	2
2.1	Preface	2
2.1.1	About the Author	2
2.1.2	Book Purpose & Scope	2
2.1.3	Target Audience	2
2.1.4	Structure of the Book	3
2.1.5	Conventions & Style Guide	3
2.2	Table of Contents	4
2.2.1	Volume I: Foundations	4
3	ABSOLUTE BASICS (C++98)	6
3.1	Getting Started	6
3.1.1	What is C++?	6
3.1.2	Your First Program (C++98)	6
3.2	Basic Types & Variables	6
3.2.1	Fundamental Types (C++98)	6
3.2.2	Variable Declaration & Initialization	7
3.2.3	Scope & Lifetime (C++98)	7
3.2.4	Deep Dive: The Memory Model of Variables	8
3.3	Operators & Control Flow	8
3.3.1	Arithmetic Operators (C++98)	8
3.3.2	Deep Dive: Bitwise Mastery (Low-Level Optimization)	9
3.3.3	Comparison & Logical Operators (C++98)	10
3.3.4	If-Else Statements (C++98)	10
3.3.5	Loops (C++98)	10
3.3.6	Switch Statement (C++98)	11
3.4	Functions	11
3.4.1	Function Declaration & Definition (C++98)	11
3.4.2	Parameters & Return Values (C++98)	12
3.4.3	Default Parameters (C++98)	13
3.4.4	Function Overloading (C++98)	13
3.5	Arrays & Pointers	13
3.5.1	Arrays (C++98)	13
3.5.2	Pointers (C++98)	14
3.5.3	Dynamic Memory (C++98)	15

4	ADVANCED POINTERS & MEMORY	16
4.1	1.1 Pointer to Const vs Const Pointer	16
4.2	1.2 Void Pointers	16
4.3	1.3 Null Pointer Safety	17
4.4	1.4 Memory Layout & Alignment	17
5	ADVANCED FUNCTIONS	19
5.1	2.1 Variadic Functions	19
5.2	2.2 Function Recursion	19
5.3	2.3 Inline Functions	20
5.4	2.4 Static Functions	20
6	FUNCTION POINTERS & CALLBACKS	22
6.1	3.1 Function Pointers	22
6.2	3.2 Function Pointer Arrays	22
6.3	3.3 Callbacks	23
7	ADVANCED ARRAYS	24
7.1	4.1 Dynamic Arrays	24
7.2	4.2 Variable Length Arrays (Non-standard)	25
7.3	4.3 Array Bounds & Safety	25
8	ADVANCED STRINGS	26
8.1	5.1 String Manipulation	26
8.2	5.2 String Conversion	27
8.3	5.3 String Tokenization	27
9	BITWISE OPERATIONS	29
9.1	6.1 Bitwise Operators	29
9.2	6.2 Bit Manipulation Techniques	29
10	PREPROCESSOR DIRECTIVES	31
10.1	7.1 #define and #include	31
10.2	7.2 Conditional Compilation	31
10.3	7.3 Pragma Directives	32
11	TYPE CASTING	33
11.1	8.1 C-Style Casting	33
11.2	8.2 Implicit Conversions	33
12	ADVANCED CONTROL FLOW	35
12.1	9.1 Ternary Operator	35
12.2	9.2 goto Statement (Avoid)	35
12.3	9.3 Label & Goto for Error Handling	36
13	ENUMERATION & UNIONS	37
13.1	10.1 Enumerations	37
13.2	10.2 Unions	37
14	CONST & VOLATILE	39
14.1	11.1 Const Correctness	39
14.2	11.2 Volatile Keyword	39

15	INLINE FUNCTIONS & MACROS	41
15.1	12.1 Macro Functions vs Inline Functions	41
16	NAMESPACES	42
16.1	13.1 Namespace Basics	42
16.2	13.2 Namespace Aliases	42
17	FILE I/O ADVANCED	44
17.1	14.1 Binary File I/O	44
17.2	14.2 Stream Positioning	44
18	ERROR HANDLING & DEBUGGING	46
18.1	15.1 Assert Macro	46
18.2	15.2 Debug Output	46
19	THE C++ COMPILATION & EXECUTION MODEL	48
19.0.1	1.5.1 The Build Pipeline: From Source to Binary	48
19.0.2	1.5.2 Translation Units (TU) & Linkage	49
19.0.3	1.5.3 The One Definition Rule (ODR)	49
19.0.4	1.5.4 Process Memory Layout	50
19.0.5	1.5.5 Program Startup: Before main()	51
19.0.6	1.5.6 Deep Dive into Data Representation	51
20	OBJECT-ORIENTED PROGRAMMING FUNDAMENTALS	53
20.1	Classes & Objects	53
20.1.1	Basic Class (C++98)	53
20.1.2	Class Member Variables & Methods (C++98)	54
20.2	Classes & Objects - Complete Mastery	54
20.2.1	What is a Class?	54
20.2.2	What is an Object?	54
20.3	The Four Pillars of OOP	55
20.3.1	1. Encapsulation - Data Hiding	55
20.3.2	2. Inheritance - Code Reuse	55
20.3.3	3. Polymorphism - Flexible Behavior	55
20.3.4	4. Abstraction - Simplified Interface	55
20.4	Encapsulation	56
20.4.1	Access Levels (Access Modifiers)	56
20.5	Constructors & Destructors - Complete Guide	57
20.5.1	Types of Constructors	57
20.5.2	Initialization Lists (Member Initializer List)	59
20.6	Inheritance - All Types	60
20.6.1	Single Inheritance	60
20.6.2	Multiple Inheritance	62
20.6.3	Multilevel Inheritance	63
20.6.4	Hierarchical Inheritance	64
20.7	Polymorphism	65
20.7.1	Compile-Time Polymorphism (Static Polymorphism)	65
20.7.2	Run-Time Polymorphism (Dynamic Polymorphism)	67
20.8	Abstraction	71
20.9	Advanced Class Features	72
20.9.1	Nested Classes	72
20.9.2	Inner Classes with Access Control	73

20.10	Access Modifiers & Friend Classes	73
20.10.1	Public, Private, Protected	73
20.10.2	Friend Functions and Classes	74
20.11	Static Members	75
20.11.1	Static Variables & Methods	75
20.12	Const Correctness in OOP	76
20.13	Operator Overloading	77
20.13.1	Overloadable Operators	77
20.14	SOLID Principles	79
20.14.1	S - Single Responsibility Principle	79
20.14.2	O - Open/Closed Principle	80
20.14.3	L - Liskov Substitution Principle	81
20.14.4	I - Interface Segregation Principle	82
20.14.5	D - Dependency Inversion Principle	83
20.15	Constructors & Destructors	84
20.15.1	Constructors (C++98)	84
20.15.2	Destructors (C++98)	85
20.15.3	2.2 Advanced Constructor Features (C++11)	85
20.15.4	2.3 Construction & Destruction Order	86
20.15.5	2.4 The Rule of Three, Five, and Zero	86
20.16	Inheritance	87
20.16.1	Basic Inheritance (C++98)	87
20.16.2	Multiple Inheritance (C++98)	88
20.17	Virtual Functions & Polymorphism	88
20.17.1	Virtual Functions (C++98)	88
20.17.2	2.6 Advanced Polymorphism	89
21	DEEP OBJECT MODEL & VIRTUALIZATION	91
21.0.1	2.5.1 The Cost of Polymorphism (vptr & vtable)	91
21.0.2	2.5.2 Multiple Inheritance & Thunks	91
21.0.3	2.5.3 Virtual Inheritance (The Diamond Problem)	91
21.0.4	2.5.4 Alignment & Padding Rules	92
22	C++98/03 STANDARD LIBRARY	93
22.1	Standard Template Library	93
22.2	Introduction to STL	93
22.2.1	Key Advantages	93
22.3	STL Components Overview	93
22.4	CONTAINERS - COMPLETE REFERENCE	94
22.5	Container Characteristics	94
22.6	1.1 VECTOR - Dynamic Array	94
22.6.1	What is Vector?	94
22.6.2	Declaration & Initialization	94
22.6.3	Accessing Elements	95
22.6.4	Modifying Elements	95
22.6.5	Size & Capacity	95
22.6.6	Iterating Through Vector	96
22.6.7	Comparison & Assignment	96
22.6.8	Complete Vector Example	96
22.7	1.2 DEQUE - Double Ended Queue	97
22.7.1	What is Deque?	97
22.7.2	Deque vs Vector	98

22.8	1.3 LIST - Doubly Linked List	98
22.8.1	What is List?	98
22.8.2	List-Specific Operations	99
22.9	1.4 MAP - Sorted Key-Value Pairs	99
22.9.1	What is Map?	99
22.9.2	Map Operations	100
22.9.3	Map with Custom Comparator	101
22.10	1.5 SET - Sorted Unique Elements	101
22.10.1	What is Set?	101
22.10.2	Set with Strings	102
22.10.3	Multiset - Allows Duplicates	102
22.11	1.6 MULTIMAP - Key-Value Pairs with Duplicate Keys	102
22.12	1.7 UNORDERED_MAP - Hash-Based Key-Value Pairs	103
22.12.1	What is Unordered Map?	103
22.12.2	Unordered Set	104
22.13	1.8 QUEUE - FIFO (First In, First Out)	104
22.13.1	What is Queue?	104
22.13.2	Queue Example - Task Processing	104
22.14	1.9 STACK - LIFO (Last In, First Out)	105
22.14.1	What is Stack?	105
22.14.2	Stack Example - Balanced Parentheses	105
22.15	1.10 PRIORITY_QUEUE - Heap-Based Container	106
22.15.1	What is Priority Queue?	106
22.15.2	Priority Queue with Custom Comparator	106
22.16	ITERATORS - DEEP DIVE	107
22.17	Iterator Categories	107
22.17.1	Iterator Types	107
22.17.2	Iterator Operations	108
22.17.3	Reverse Iterators	108
22.17.4	Const Iterators	108
22.18	ALGORITHMS - COMPLETE REFERENCE	110
23	C++ STL Advanced - Extended Reference & Algorithms Library	111
23.1	COMPREHENSIVE STL ALGORITHMS REFERENCE	111
23.1.1	All Algorithms by Category (60+ Algorithms)	111
23.2	NON-MODIFYING SEQUENCE ALGORITHMS	111
23.2.1	1. find Family	111
23.2.2	2. count Family	111
23.2.3	3. mismatch	111
23.2.4	4. equal	111
23.2.5	5. search & adjacent_find	112
23.2.6	6. Logical Operations	112
23.2.7	7. Min/Max	112
23.3	MODIFYING SEQUENCE ALGORITHMS	112
23.3.1	1. copy Family	112
23.3.2	2. move (C++11)	112
23.3.3	3. transform	112
23.3.4	4. fill & generate	112
23.3.5	5. replace	113
23.3.6	6. swap & reverse	113
23.3.7	7. rotate	113

23.3.8	8. unique	113
23.3.9	9. shuffle & random	113
23.3.10	10. remove	113
23.4	SORTING & PARTITIONING ALGORITHMS	113
23.4.1	Sorting	113
23.4.2	Partitioning	114
23.4.3	Binary Search (requires sorted range)	114
23.5	NUMERIC ALGORITHMS	114
23.6	SET OPERATIONS (require sorted ranges)	115
23.7	HEAP OPERATIONS	115
23.8	PERMUTATION ALGORITHMS	115
23.9	COMPLETE STL ALGORITHMS QUICK REFERENCE	116
23.10	CONTAINERS DETAILED OPERATIONS	116
23.10.1	Vector Operations	116
23.10.2	Map Operations	117
23.10.3	Set Operations	117
23.11	ALGORITHM COMPLEXITY CHEAT SHEET	117
23.12	CONTAINER COMPLEXITY COMPARISON	118
23.13	ITERATOR COMPARISON TABLE	118
23.14	PRACTICAL STL PATTERNS	119
23.14.1	Pattern 1: Find & Remove	119
23.14.2	Pattern 2: Filter & Copy	119
23.14.3	Pattern 3: Transform & Collect	119
23.14.4	Pattern 4: Partition & Process	119
23.14.5	Pattern 5: Sort & Deduplicate	120
23.14.6	Pattern 6: Group & Count	120
23.14.7	Pattern 7: Custom Sorting	120
23.14.8	Pattern 8: Merge Ranges	120
23.15	STL WITH LAMBDA (C++11 and later)	121
23.16	STRINGS - MASTER GUIDE	121
23.17	String Basics	121
23.17.1	Size and Capacity	122
23.18	Accessing Characters	122
23.19	String Concatenation	122
23.20	String Searching	123
23.21	String Manipulation	124
23.22	String Iteration	124
23.23	String Conversion	125
23.24	String Comparison	125
23.25	String from Stringstream	126
23.26	FILE I/O - COMPLETE COVERAGE	126
23.27	File I/O Basics	126
23.28	File Operations	127
23.28.1	Open Modes	127
23.29	Writing to Files	127
23.30	Reading from Files	128
23.30.1	Line by Line	128
23.30.2	Word by Word	128
23.30.3	Character by Character	128
23.30.4	Specific Format	128
23.31	Binary File I/O	129

23.325.6 File Position	129
23.335.7 Error Handling	130
23.345.8 Complete File I/O Example	130
23.35FUNCTION OBJECTS & COMPARATORS	131
23.366.1 Function Objects (Functors)	131
23.376.2 Standard Comparators	132
23.38STL BEST PRACTICES	132
23.397.1 Container Selection	132
23.407.2 Algorithm Selection	133
23.417.3 Memory Management	133
23.427.4 Performance Tips	134
23.42.1 Containers Intro (C++98)	134
24 STL INTERNALS DEEP DIVE	135
24.0.1 3.5.1 The Truth About std::vector	135
24.0.2 3.5.2 The std::deque Implementation	135
24.0.3 3.5.3 Why std::list is (Almost) Always Wrong	136
24.0.4 3.5.4 Associative Containers (Map/Set)	136
24.0.5 3.5.5 Unordered Containers (Hash Maps)	136
24.0.6 3.5.6 Iterator Invalidation Cheat Sheet	137
25 C++11 REVOLUTION	138
25.1 C++11 Overview & History	138
25.1.1 Timeline	138
25.1.2 Why C++11 Matters	138
25.1.3 Key Themes	138
25.2 AUTO & TYPE DEDUCTION	139
25.3 1.1 Auto Keyword	139
25.3.1 Basic Auto	139
25.3.2 Auto With Pointers & References	139
25.3.3 Auto in Templates	139
25.3.4 Auto Type Deduction Rules	140
25.3.5 Auto Benefits & Limitations	140
25.4 1.2 decltype	140
25.4.1 decltype vs auto	141
25.5 SMART POINTERS - MEMORY REVOLUTION	142
25.6 2.1 Unique Pointer (unique_ptr)	142
25.6.1 Basic Usage	142
25.6.2 Unique Pointer Operations	142
25.6.3 Unique Pointer with Custom Deleters	143
25.6.4 Unique Pointer in Collections	143
25.7 2.2 Shared Pointer (shared_ptr)	144
25.7.1 Basic Usage	144
25.7.2 Shared Pointer Operations	144
25.7.3 Shared Pointer with Arrays & Custom Deleters	145
25.7.4 Shared Pointers in Collections	145
25.8 2.3 Weak Pointer (weak_ptr)	145
25.8.1 Circular Reference Problem	145
25.8.2 Solution with Weak Pointer	146
25.8.3 Weak Pointer Usage	146
25.8.4 Practical Circular Reference Solutions	146
25.9 Smart Pointer Comparison	147

25.10	MOVE SEMANTICS & RVALUE REFERENCES	147
25.113.1	Lvalue vs Rvalue	147
25.11.1	Lvalue & Rvalue Examples	148
25.123.2	Move Constructor & Move Assignment	148
25.12.1	Before C++11 - Expensive Copy	148
25.12.2	With C++11 - Efficient Move	149
25.12.3	Move Constructor Details	150
25.12.4	std::move	150
25.133.3	Perfect Forwarding	151
25.13.1	The Problem	151
25.13.2	Implementation	151
25.13.3	Perfect Forwarding with Multiple Parameters	152
25.13.4	Universal Reference	152
25.14	LAMBDA FUNCTIONS	153
25.154.1	Lambda Basics	153
25.15.1	Lambda with Capture	153
25.15.2	Lambda Capture Examples	154
25.164.2	Lambda with Algorithms	154
25.174.3	Advanced Lambda Features	155
25.17.1	Init Capture (Generalized Capture - C++14)	155
25.17.2	Recursive Lambdas	155
25.17.3	Const and Mutable Lambdas	156
25.184.4	Lambda Under the Hood	156
25.19	VARIADIC TEMPLATES	156
25.205.1	Parameter Packs	156
25.20.1	Basic Variadic Template	157
25.20.2	Understanding Parameter Packs	157
25.20.3	Pack Expansion	157
25.20.4	Variadic Function Print Example	157
25.20.5	Fold Expressions (C++17)	158
25.21	RANGE-BASED FOR LOOPS	158
25.226.1	Iterating Over Containers	158
25.22.1	Range-Based For with Different Containers	159
25.22.2	Custom Range-Based For	159
25.23	UNIFORM INITIALIZATION	160
25.247.1	Brace Initialization	160
25.24.1	Initializer Lists	161
25.25	NULLPTR & STRONGLY TYPED ENUMS	161
25.268.1	nullptr	161
25.278.2	Strongly Typed Enums	161
25.28	TUPLE, ARRAY, & UNORDERED CONTAINERS	162
25.299.1	Tuple	162
25.309.2	Array	163
25.319.3	Unordered Containers	163
25.32	DECLTYPE & TYPE TRAITS	164
25.3310.1	decltype	164
25.3410.2	Type Traits	164
25.35	CONCURRENCY & THREADING	165
25.3611.1	Threads	165
25.36.1	Threading with Parameters	166
25.36.2	Thread with Lambda	166

25.36.3 Multiple Threads	166
25.3711.2 Mutex & Locks	167
25.3811.3 Atomic Operations	167
25.39REGULAR EXPRESSIONS	168
25.4012.1 Basic Regex	168
25.41NEW LIBRARY FEATURES	168
25.4213.1 Standard Library Additions	168
25.42.1 Chrono Library	168
25.42.2 Random Number Generation	169
25.42.3 Function	169
26 ADVANCED MOVE SEMANTICS & VALUE CATEGORIES	170
26.0.1 4.5.1 The C++17 Value Category Taxonomy	170
26.0.2 4.5.2 std::move and std::forward Internals	170
26.0.3 4.5.3 Reference Collapsing Rules	171
27 C++14 ENHANCEMENTS	172
27.1 C++14 Overview & Philosophy	172
27.1.1 Timeline & Context	172
27.1.2 C++14 Philosophy	172
27.1.3 Key Features	172
27.1.4 Why C++14 Matters	173
27.2 GENERIC LAMBDA	173
27.3 1.1 Auto Parameters in Lambdas	173
27.3.1 Basic Generic Lambda	173
27.3.2 Generic Lambda Deduction	173
27.3.3 Generic Lambdas with std::vector	173
27.3.4 Generic Lambdas with Algorithms	174
27.3.5 Generic Lambda Compile-Time Behavior	174
27.3.6 When to Use Generic Lambdas	174
27.4 RETURN TYPE DEDUCTION FOR ALL FUNCTIONS	175
27.5 2.1 Return Type Deduction (C++14 Enhancement)	175
27.5.1 Basic Return Type Deduction	175
27.5.2 Multiple Return Statements	175
27.5.3 Return Type Deduction with Complex Types	175
27.5.4 Return Type Deduction in Templates	176
27.5.5 Recursion with Return Type Deduction	176
27.5.6 Benefits of Return Type Deduction	176
27.6 AUTO FOR VARIABLES IN LAMBDA	177
27.7 3.1 Init Capture with Auto (C++14)	177
27.7.1 Basic Init Capture	177
27.7.2 Init Capture with Values	177
27.7.3 Init Capture with Move	177
27.7.4 Init Capture with Complex Types	178
27.7.5 Init Capture Patterns	178
27.8 BINARY LITERALS & DIGIT SEPARATORS	179
27.9 4.1 Binary Literals	179
27.9.1 Binary Literal Syntax	179
27.9.2 Binary Literals Use Cases	179
27.104.2 Digit Separators	179
27.10.1 Digit Separator Examples	179
27.10.2 Readability Improvement	180

27.10.3 Digit Separator Rules	180
27.11 <code>STD::MAKE_UNIQUE</code>	181
27.125.1 <code>std::make_unique</code> (C++14)	181
27.12.1 Before C++14	181
27.12.2 With <code>std::make_unique</code>	181
27.12.3 <code>make_unique</code> with Classes	181
27.12.4 <code>make_unique</code> with Arrays (C++20)	181
27.12.5 <code>make_unique</code> vs <code>new</code>	182
27.12.6 <code>make_unique</code> Best Practices	182
27.13 <code>RELAXED CONSTEXPR RESTRICTIONS</code>	183
27.146.1 Enhanced <code>constexpr</code> Functions	183
27.14.1 C++11 <code>constexpr</code> Limitations	183
27.14.2 C++14 <code>constexpr</code> Enhancements	183
27.14.3 C++14 <code>constexpr</code> Features	183
27.14.4 <code>constexpr</code> Still Has Limitations	184
27.14.5 Practical <code>constexpr</code> Uses	184
27.15 <code>VARIABLE TEMPLATES</code>	185
27.167.1 Template Variables (C++14)	185
27.16.1 Basic Variable Template	185
27.16.2 Variable Template with Type Traits	186
27.16.3 Useful Variable Templates	186
27.16.4 C++17 Standard Variable Templates	186
27.17 <code>AGGREGATE MEMBER INITIALIZATION</code>	187
27.188.1 Extended Aggregate Initialization	187
27.18.1 Basic Aggregate Initialization (C++98)	187
27.18.2 With Base Classes (C++14)	187
27.18.3 Nested Aggregates	187
27.18.4 C++14 vs C++11 Differences	188
27.19 <code>MEMBER FUNCTION REF/CONST-REF QUALIFIERS</code>	188
27.209.1 Lvalue vs Rvalue Member Functions	188
27.20.1 Member Function Overloading	188
27.20.2 Practical Use Case	189
27.20.3 Const/Volatile Combinations	189
27.21 <code>STD::INTEGER_SEQUENCE</code>	190
27.2210.1 Compile-Time Integer Sequences	190
27.22.1 Basic Usage	190
27.22.2 Unpacking Tuple	190
27.22.3 Array Initialization	191
27.23 <code>LIBRARY IMPROVEMENTS</code>	191
27.2411.1 STL Enhancements in C++14	191
27.24.1 <code>std::quoted</code> for String I/O	191
27.24.2 <code>std::less</code> and Comparators	191
27.24.3 Algorithms Returning Pair	192
27.24.4 <code>std::exchange</code>	192
27.24.5 <code>std::get</code> with Type	192
27.25 <code>DEPRECATED FEATURES & REMOVALS</code>	192
27.2612.1 Features Deprecated in C++14	192
27.26.1 12.5 Shared Locks (Reader-Writer Mutex)	193
27.27 C++14 BEST PRACTICES	193
27.28 What's Better with C++14	193

28 C++17 MODERN FEATURES	195
28.1 C++17 Overview & Significance	195
28.1.1 Timeline & Context	195
28.1.2 C++17 Philosophy	195
28.1.3 Key Themes	195
28.1.4 Why C++17 Matters	196
28.2 STRUCTURED BINDINGS	196
28.3 1.1 Introduction to Structured Bindings	196
28.3.1 Basic Structured Bindings	196
28.3.2 Structured Bindings with Pairs	196
28.3.3 Structured Bindings with Arrays	196
28.3.4 Structured Bindings with Structs	197
28.3.5 Structured Bindings with Return Values	197
28.3.6 Structured Bindings with References	197
28.3.7 Practical Use Cases	198
28.3.8 Structured Bindings Rules	198
28.4 1.2 Structured Bindings Under the Hood	199
28.5 OPTIONAL & VARIANT	199
28.6 2.1 std::optional - Safe Nullable Values	199
28.6.1 Basic optional Usage	199
28.6.2 optional with Complex Types	199
28.6.3 optional Operations	200
28.6.4 optional with Chaining	200
28.7 2.2 std::variant - Type-Safe Union	201
28.7.1 Basic variant Usage	201
28.7.2 variant with Type Checking	201
28.7.3 variant with Visitor Pattern	201
28.7.4 variant with Lambdas (C++20 or using overload trick)	202
28.7.5 Practical variant Example	202
28.8 STD::ANY	202
28.9 3.1 Type-Erased Storage with std::any	202
28.9.1 Basic any Usage	202
28.9.2 any with Type Checking	203
28.9.3 any in Collections	203
28.9.4 any vs variant	204
28.10 STD::STRING_VIEW	204
28.11 4.1 Non-Owning String References	204
28.11.1 Basic string_view	204
28.11.2 string_view from Different Sources	204
28.11.3 string_view Operations	204
28.11.4 Performance Benefits of string_view	205
28.11.5 string_view Limitations	205
28.11.6 string_view Use Cases	206
28.12 IF CONSTEXPR	206
28.13 5.1 Compile-Time Conditional Code	206
28.13.1 Basic if constexpr	206
28.13.2 if constexpr Advantages	207
28.13.3 if constexpr with Complex Logic	207
28.13.4 if constexpr with Concepts (C++20-like)	207
28.14 5.2 The Death of SFINAE?	208
28.15 FOLD EXPRESSIONS	208

28.166.1 Operating on Parameter Packs	208
28.16.1 Basic Fold Expressions	208
28.16.2 Fold Directions	209
28.16.3 Fold with Default Value	209
28.16.4 Practical Fold Examples	209
28.176.2 Advanced Fold Expressions	210
28.17.1 Calling Function on Pack	210
28.17.2 Validating All Arguments	210
28.18 CLASS TEMPLATE ARGUMENT DEDUCTION (CTAD)	210
28.197.1 Automatic Template Type Deduction	210
28.19.1 Before CTAD (C++14)	210
28.19.2 With CTAD (C++17)	211
28.19.3 CTAD with Custom Deduction Guides	211
28.19.4 Practical CTAD Examples	211
28.19.5 CTAD Benefits	211
28.20 STD::FILESYSTEM	212
28.218.1 Portable File System Operations	212
28.21.1 Basic Path Operations	212
28.21.2 File Existence & Type Checking	212
28.21.3 Directory Operations	213
28.21.4 File Operations	213
28.21.5 Practical filesystem Example	213
28.228.5 Filesystem Deep Dive	214
28.22.1 Exception-Free API	214
28.22.2 Space & Permissions	214
28.23 STD::INVOKE	215
28.249.1 Uniform Callable Invocation	215
28.24.1 Basic invoke	215
28.24.2 invoke with Methods	215
28.24.3 invoke with Data Members	216
28.24.4 Practical invoke Pattern	216
28.25 PARALLEL ALGORITHMS	216
28.2610.1 Parallel Algorithm Execution	216
28.26.1 Execution Policies	216
28.26.2 Parallel Algorithm Examples	217
28.26.3 Performance Considerations	217
28.27 CORE LANGUAGE FEATURES	217
28.2811.1 Additional C++17 Features	217
28.28.1 Nested namespaces	217
28.28.2 Inline Variables	218
28.28.3 Structured Exception Handling (still mostly the same, but with improve- ments)	218
28.28.4 Deduced Return Types in Lambdas	218
28.28.5 std::byte	218
28.29 LIBRARY IMPROVEMENTS	218
28.3012.1 STL Enhancements in C++17	218
28.30.1 std::optional Algorithms	218
28.30.2 Improved Algorithms	219
28.30.3 std::charconv	219
28.31 POLYMORPHIC MEMORY RESOURCES (PMR)	219
28.3213.1 Introduction to std::pmr	219

28.32.1 The Problem with Traditional Allocators	219
28.3313.2 Memory Resources	220
28.33.1 Standard Memory Resources	220
28.33.2 Chaining Resources	220
28.33.3 Performance Benefits	221
28.34C++17 BEST PRACTICES	221
28.35What's Better with C++17	221
29 C++20 REVOLUTIONARY FEATURES	222
29.1 C++20 Overview & Revolutionary Scope	222
29.1.1 Timeline & Context	222
29.1.2 C++20 Philosophy	222
29.1.3 Key Themes	222
29.1.4 Why C++20 Matters	223
29.2 CONCEPTS & CONSTRAINTS	223
29.3 1.1 Introduction to Concepts	223
29.3.1 Basic Concept Definition	223
29.3.2 Standard Library Concepts	223
29.3.3 Complex Concept Definition	224
29.3.4 Concept Benefits	225
29.4 1.2 Requires Expressions	225
29.4.1 Basic Requires Expression	225
29.4.2 Requires with Return Type Checking	225
29.4.3 Practical Requires Examples	226
29.5 1.3 Concepts & Overload Resolution	226
29.6 RANGES LIBRARY	227
29.7 2.1 Introduction to Ranges	227
29.7.1 Basic Range Operations	227
29.7.2 Range Views	227
29.7.3 Range Algorithms	228
29.7.4 Composing Multiple Views	228
29.8 2.2 Ranges Deep Dive	229
29.8.1 Projections	229
29.8.2 Dangling Iterators	229
29.9 COROUTINES	229
29.103.1 Introduction to Coroutines	229
29.10.1 Basic Generator Coroutine	229
29.10.2 Async Coroutine	231
29.10.3 Practical Coroutine: Fibonacci Generator	232
29.113.2 Coroutines Deep Dive	232
29.11.1 The Awaitable Interface	232
29.11.2 Symmetric Transfer	232
29.12SPACESHIP OPERATOR (THREE-WAY COMPARISON)	233
29.134.1 The Spaceship Operator $\lt=\gt$	233
29.13.1 Basic Spaceship Usage	233
29.13.2 Spaceship with Custom Types	233
29.13.3 Defaulted Comparison	233
29.13.4 Comparison Categories	234
29.13.5 Spaceship Benefits	234
29.14MODULES	235
29.155.1 Introduction to Modules	235

29.15.1 Module Definition	235
29.15.2 Using Modules	235
29.15.3 Module Partitions	235
29.15.4 Module Benefits	236
29.165.2 Modules Deep Dive	236
29.16.1 Global Module Fragment	236
29.16.2 Private Module Partition	236
29.17DESIGNATED INITIALIZERS	237
29.186.1 Named Member Initialization	237
29.18.1 Basic Designated Initializers	237
29.18.2 With Classes and Inheritance	237
29.18.3 Practical Designated Initializers	237
29.19CALENDAR & TIME ZONES	238
29.207.1 Advanced Chrono Features	238
29.20.1 Calendar Types	238
29.20.2 Time Zones	238
29.20.3 Formatted Time Output	239
29.21STD::FORMAT	239
29.228.1 Type-Safe String Formatting	239
29.22.1 Basic format Usage	239
29.22.2 Formatting Specifications	239
29.22.3 Format with Custom Types	240
29.23CONSTEVAL & CONSTINIT	240
29.249.1 Immediate Functions and Constants	240
29.24.1 consteval - Immediate Functions	240
29.24.2 constinit - Compile-Time Initialization	241
29.24.3 Difference: constexpr vs consteval	241
29.25LAMBDA ENHANCEMENTS	241
29.2610.1 C++20 Lambda Improvements	241
29.26.1 Default Constructible Lambdas	241
29.26.2 Stateless Lambda as Template Parameter	242
29.27ADVANCED FEATURES	242
29.2811.1 Additional C++20 Features	242
29.28.1 Spaceship Operator with Library Support	242
29.28.2 Bit Operations	242
29.28.3 std::atomic_ref	243
29.28.4 Concepts in Standard Library	243
29.29LIBRARY IMPROVEMENTS	243
29.3012.1 STL Enhancements in C++20	243
29.30.1 std::span (Non-owning Array View)	243
29.30.2 std::semaphore	244
29.30.3 std::latch & std::barrier	244
29.30.4 std::source_location (Reflection for Logging)	245
29.30.5 std::osyncstream (Synchronized Output)	245
29.30.6 Ranges with Algorithms	245
30 C++23 LATEST FEATURES	247
30.1 C++23 Overview & Direction	247
30.1.1 Timeline & Context	247
30.1.2 C++23 Philosophy	247
30.1.3 Key Themes	247

30.1.4	Why C++23 Matters	248
30.2	STD::PRINT & FORMATTED OUTPUT	248
30.3	1.1 std::print - Simple Output	248
30.3.1	Basic print Usage	248
30.3.2	print vs format vs iostream	248
30.3.3	print with File Streams	249
30.4	1.2 std::format Enhancements	249
30.4.1	Format Improvements in C++23	249
30.5	DEDUCING THIS	249
30.6	2.1 Explicit Member Function Parameters	249
30.6.1	Basic Deducing This	249
30.6.2	Deducing This for Const/Non-Const	250
30.6.3	Practical Deducing This	250
30.7	2.2 Deducing This - Beyond the Basics	251
30.7.1	Recursive Lambdas	251
30.7.2	Replacing CRTP	251
30.8	RANGE-BASED FOR LOOP ENHANCEMENTS	251
30.9	3.1 For Loop Initializers	251
30.9.1	For Loop with Init	251
30.9.2	For Loop with Init and Structured Binding	252
30.10	STD::EXPECTED	252
30.11.1	Result Type for Error Handling	252
30.11.1	Basic expected Usage	252
30.11.2	expected with Transform	253
30.11.3	expected vs optional	253
30.12	STD::OPTIONAL IMPROVEMENTS	253
30.13.1	Enhanced optional Operations	253
30.13.1	optional with Deref Operator	253
30.13.2	optional::value_or_else	254
30.14	MULTIDIMENSIONAL SUBSCRIPT OPERATOR	254
30.15.1	Multiple Index Support	254
30.15.1	Multi-Index Subscript	254
30.15.2	Dynamic 2D Array Wrapper	254
30.16.2	std::mdspan (Multidimensional View)	255
30.16.1	Basic mdspan Usage	255
30.17.3	mdspan Layouts	255
30.18	STD::STACKTRACE	256
30.19.1	Runtime Stack Trace Capture	256
30.19.1	Basic Stacktrace	256
30.19.2	Stacktrace in Error Handling	256
30.20	CONSTEXPR ENHANCEMENTS	257
30.21.1	More Compile-Time Capabilities	257
30.21.1	constexpr std::string	257
30.21.2	constexpr vector Operations	257
30.21.3	constexpr Algorithms	258
30.22	ADAPTOR IMPROVEMENTS	258
30.23.1	Ranges::to Conversion	258
30.23.1	Basic ranges::to	258
30.23.2	ranges::to with Construction Args	258
30.24	LIBRARY IMPROVEMENTS	259
30.25.1	Utility Improvements	259

30.25.1	<code>std::out_ptr</code> & <code>std::inout_ptr</code>	259
30.25.2	<code>std::move_iterator</code> Improvements	259
30.25.3	Bit Manipulation Improvements	259
30.26	ATTRIBUTES & FEATURES	260
30.2711.1	<code>[[assume]]</code> Attribute	260
30.27.1	Using <code>assume</code>	260
30.2811.2	<code>[[stdcall]]</code> and ABI Attributes	260
30.29	STANDARD LIBRARY ADDITIONS	260
30.3012.1	Container & Utility Additions	260
30.30.1	<code>std::debug_assert</code> (Conditional Assertion)	260
30.30.2	<code>std::repeat_view</code>	261
30.30.3	<code>std::stride_view</code>	261
30.30.4	<code>std::chunk_by</code>	261
30.30.5	<code>std::flat_map</code> and <code>std::flat_set</code>	261
30.30.6	<code>std::generator</code> (Synchronous Coroutine)	262
30.30.7	12.3 Generator Internals	262
30.31	C++23 BEST PRACTICES	262
30.32	What's Better with C++23	262
31	THE FUTURE - C++26 PREVIEW	264
31.0.1	13.1 Static Reflection (<code>std::meta</code>)	264
31.0.2	13.2 Contracts	264
31.0.3	13.3 Senders & Receivers (<code>std::execution</code>)	265
31.0.4	13.4 Linear Algebra (<code>std::linalg</code>)	265
32	ADVANCED TOPICS	267
32.1	TEMPLATE METAPROGRAMMING	267
32.2	1.1 Compile-Time Computation	267
32.2.1	Factorial at Compile-Time	267
32.2.2	Fibonacci at Compile-Time	267
32.2.3	Compile-Time Power Check	268
32.3	1.2 Template Specialization	268
32.3.1	Full Specialization	268
32.3.2	Partial Specialization	268
32.4	1.3 Advanced Metaprogramming Patterns	269
32.4.1	Typelists	269
32.5	SFINAE & TYPE TRAITS	270
32.6	2.1 SFINAE - Substitution Failure Is Not An Error	270
32.6.1	Basic SFINAE	270
32.6.2	Detector Pattern	270
32.6.3	Advanced SFINAE with Multiple Conditions	270
32.7	2.2 Type Traits Mastery	271
32.7.1	Custom Type Traits	271
32.8	EXPRESSION TEMPLATES	271
32.9	3.1 Lazy Evaluation Pattern	271
32.9.1	Vector Operations	271
32.103.2	Triggering Computation (Assignment)	273
32.11	POLICY-BASED DESIGN	273
32.124.1	Template Policies	273
32.12.1	Thread Safety Policy	273
32.12.2	Storage Policy	274
32.134.2	Policy-Based Smart Pointer (Advanced Example)	274

32.14	MEMORY MANAGEMENT & OPTIMIZATION	275
32.155.1	Custom Allocators	275
32.165.2	Small Object Optimization (SOO)	275
32.17	CONCURRENCY & PARALLELISM	276
32.186.1	Advanced Threading Patterns	276
32.18.1	Thread Pool	276
32.18.2	Lock-Free Queue	277
32.19	TYPE ERASURE PATTERNS	278
32.207.1	Virtual Function-Based Type Erasure	278
32.217.2	std::function Implementation	279
32.227.3	Concept-Based Type Erasure (C++20)	279
32.23	CRTP (CURIOUSLY RECURRING TEMPLATE PATTERN)	280
32.248.1	Static Polymorphism	280
32.258.2	CRTP for Comparisons	281
32.26	PERFECT FORWARDING & MOVE SEMANTICS	282
32.279.1	Forwarding Problems	282
32.289.2	Move Semantics Implementation	282
32.29	COMPILE-TIME PROGRAMMING	283
32.3010.1	Tuple Operations at Compile-Time	283
32.3110.2	Compile-Time String Processing	284
32.32	META-OBJECT PROTOCOL	284
32.3311.1	Reflection-Like Patterns	284
32.3411.2	Magic Enum (Reflection Hack)	285
32.35	ADVANCED CONTAINER TECHNIQUES	285
32.3612.1	COW (Copy-On-Write) String	285
32.3712.2	Intrusive Containers	286
32.38	ABI & BINARY COMPATIBILITY	286
32.3913.1	Stable ABI Design	286
32.40	PERFORMANCE PROFILING & OPTIMIZATION	287
32.4114.1	Benchmarking Framework	287
32.4214.2	Cache-Friendly Algorithms	288
32.43	DOMAIN-SPECIFIC LANGUAGE DESIGN	288
32.4415.1	Expression DSL	288
32.45	MODERN DESIGN PATTERNS	289
32.4616.1	Strategy Pattern (Functional Approach)	289
32.4716.2	Visitor Pattern (std::variant)	290
32.48	HARDWARE SYMPATHY	291
32.4917.1	Cache Locality & False Sharing	291
32.49.1	False Sharing	291
32.5017.2	Branch Prediction	291
32.5117.3	SIMD (Single Instruction, Multiple Data)	291
33	PRODUCTION & PROFESSIONAL	292
33.1	LARGE-SCALE PROJECT ARCHITECTURE	292
33.2	1.1 Layered Architecture	292
33.3	1.2 Microservices Architecture	294
33.4	CODE ORGANIZATION & PROJECT STRUCTURE	294
33.5	2.1 Modern CMake Project Structure	294
33.6	2.2 Header Organization	295
33.7	2.3 Modern CMake with Modules (C++20)	296
33.8	BUILD SYSTEMS & COMPILATION	297

33.9 3.1 Conan Package Manager	297
33.103.2 Incremental Build Optimization	297
33.11 TESTING STRATEGIES	298
33.124.1 Unit Testing with Catch2	298
33.134.2 Integration Testing	298
33.144.3 Test Fixtures & Mocks	299
33.154.4 Advanced Mocking with Google Mock (GMock)	299
33.16 DEBUGGING & PROFILING	300
33.175.1 Debug Utilities	300
33.185.2 Performance Profiling	301
33.195.3 Memory Profiling with AddressSanitizer	302
33.20 VERSION CONTROL & COLLABORATION	302
33.216.1 Git Workflow	302
33.226.2 Feature Branch Workflow	303
33.23 DOCUMENTATION & KNOWLEDGE TRANSFER	303
33.247.1 Code Documentation with Doxygen	303
33.257.2 Architecture Decision Records (ADR)	304
33.26 SECURITY & SAFETY	305
33.278.1 Input Validation	305
33.288.2 SQL Injection Prevention	305
33.29 PERFORMANCE ENGINEERING	306
33.309.1 Benchmarking Framework	306
33.319.2 Load Testing	307
33.32 ERROR HANDLING & RECOVERY	308
33.3310.1 Exception Hierarchy	308
33.3410.2 Error Recovery Patterns	309
33.35 DEPLOYMENT & DEVOPS	309
33.3611.1 Docker Containerization	309
33.3711.2 Kubernetes Deployment	310
33.3811.3 CI/CD Pipeline (GitHub Actions)	311
33.39 CODE REVIEW & QUALITY	311
33.4012.1 Code Review Checklist	311
33.4112.2 Static Code Analysis	312
33.42 TECHNICAL DEBT MANAGEMENT	312
33.4313.1 Tracking Technical Debt	312
33.4413.2 Refactoring Strategy	313
33.45 LEGACY CODE MODERNIZATION	314
33.4614.1 Incremental Modernization	314
33.47 LEADERSHIP & TEAM MANAGEMENT	315
33.4815.1 Mentoring Framework	315
33.4915.2 Technical Decision Making	315
34 SYSTEM DESIGN CASE STUDIES (C++ EDITION)	317
34.0.1 10.5.1 LRU Cache	317
34.0.2 10.5.2 Token Bucket Rate Limiter	318
35 CONCURRENCY DESIGN PATTERNS	319
35.0.1 10.6.1 Active Object Pattern	319
35.0.2 10.6.2 Monitor Object (Thread-Safe Interface)	320

36 THE C++ BUILD ECOSYSTEM MASTERY	321
36.0.1 30.1 Package Managers Deep Dive	321
36.0.2 30.2 Sanitizers: The Developer’s Best Friend	321
36.0.3 30.3 Profiling Tools	322
37 LOW-LATENCY C++ OPTIMIZATION	323
37.0.1 17.1 CPU Pipelines & Branch Prediction	323
37.0.2 17.2 Data-Oriented Design (DoD)	323
37.0.3 17.3 Prefetching	323
37.0.4 17.4 Micro-Benchmarking (Google Benchmark)	324
37.0.5 17.5 System Warm-up	324
37.0.6 17.6 False Sharing Prevention	324
38 LOW-LATENCY SYSTEM ARCHITECTURE	325
38.0.1 24.1 The Disruptor Pattern (C++ Implementation)	325
38.0.2 24.2 Kernel Bypass Networking (Concept)	325
38.0.3 24.3 OS Tuning for C++	325
38.0.4 24.4 Zero-Copy Serialization (Cap’n Proto / FlatBuffers)	326
38.0.5 24.5 LMAX Disruptor Internals	326
39 EXTREME LOW LATENCY & HARDWARE MASTERY	327
39.0.1 31.1 CPU Architecture & Cache Topology	327
39.0.2 31.2 NUMA (Non-Uniform Memory Access)	327
39.0.3 31.3 Compiler Optimizations (The “Free Lunch”)	327
39.0.4 31.4 Lock-Free Stack Implementation (Wait-Free Push)	328
39.0.5 31.5 Measurable Performance Targets	328
40 ADVANCED SIMD (AVX2 & AVX-512)	329
40.0.1 32.1 SIMD Basics & Registers	329
40.0.2 32.2 Intrinsics Example (Vector Addition)	329
40.0.3 32.3 Measurable Outcome	329
41 CUSTOM MEMORY ALLOCATORS	330
41.0.1 33.1 Linear Allocator (Arena)	330
41.0.2 33.2 Pool Allocator	330
42 APPENDICES	332
43 Appendix A: C++ Keywords & Operators Reference	333
43.0.1 Essential Keywords (Non-Exhaustive)	333
43.0.2 Special Operators	334
44 Appendix B: Common Acronyms	335
45 Appendix C: Recommended Tooling	336
45.0.1 Compilers	336
45.0.2 Build Systems	336
45.0.3 Package Managers	336
45.0.4 Static Analysis & Sanitizers	336

46 Appendix D: Common C++ Traps & Pitfalls	337
46.0.1 I. General & Syntax Traps	337
46.0.2 II. Pointers, References & Memory	337
46.0.3 III. Classes & OOP	338
46.0.4 IV. Concurrency	339
46.0.5 V. Modern C++ & Macros	339
47 Appendix H: Professional C++ Idioms	340
47.0.1 1. RAII (Resource Acquisition Is Initialization)	340
47.0.2 2. Pimpl (Pointer to Implementation)	340
47.0.3 3. Copy-and-Swap	340
47.0.4 4. NVI (Non-Virtual Interface)	340
47.0.5 5. Erase-Remove Idiom	341
47.0.6 6. SFINAE (Substitution Failure Is Not An Error)	341
47.0.7 7. CRTP (Curiously Recurring Template Pattern)	341
48 Appendix E: C++ Interview Cheat Sheet	342
48.0.1 Core Concepts	342
48.0.2 Modern C++ (C++11/14/17/20)	342
48.0.3 System Design Questions	343
48.0.4 Quick Coding	343
49 Appendix F: The C++ Standard Evolution Matrix	344
49.0.1 1. Versioned Changelog	344
49.0.2 2. Feature Matrix	345
49.0.3 3. Timeline & Release Accuracy	345
50 Appendix G: C++ Standard Library Headers Reference	346
50.0.1 Concepts & Utilities	346
50.0.2 Containers	346
50.0.3 Algorithms & Iterators	347
50.0.4 Concurrency	347
50.0.5 Input/Output	347
50.0.6 Numerics & Math	347
51 C++ UNDER THE HOOD	348
51.0.1 14.1 Object Layout & ABI (Itanium C++ ABI)	348
51.0.2 14.2 Small String Optimization (SSO)	348
51.0.3 14.3 Return Value Optimization (RVO)	349
52 MASTERING THE MEMORY MODEL	350
52.0.1 15.1 Atomicity vs Ordering	350
52.0.2 15.2 Memory Orders Deep Dive	350
52.0.3 15.3 The Happens-Before Relationship	351
53 WRITING A C++ COMPILER (BASICS)	352
53.0.1 18.1 Lexical Analysis (Tokenizer)	352
53.0.2 18.2 Parsing (Recursive Descent)	352
53.0.3 18.3 Semantic Analysis (Types & Scopes)	352
54 WRITING A GARBAGE COLLECTOR	354
54.0.1 29.1 Mark-and-Sweep Basics	354

55 THE STANDARD LIBRARY FROM SCRATCH	356
55.0.1 19.1 Implementing my::vector	356
55.0.2 19.2 Implementing my::shared_ptr	356
56 DISTRIBUTED C++	358
56.0.1 16.1 Serialization (Binary Protocols)	358
56.0.2 16.2 RPC (Remote Procedure Call) Concept	358
56.0.3 16.3 Consensus (Raft Basics)	359
57 NETWORKING FROM SCRATCH	360
57.0.1 28.1 Berkeley Sockets API	360
57.0.2 28.2 Non-Blocking I/O & Epoll (Linux)	360
58 C++ IN THE CLOUD	362
58.0.1 20.1 Microservices with C++	362
58.0.2 20.2 Serverless C++ (AWS Lambda)	362
59 CROSS-PLATFORM DEVELOPMENT	363
59.0.1 21.1 WebAssembly (Wasm) with Emscripten	363
59.0.2 21.2 Mobile C++ (Android NDK & JNI)	363
60 GUI DEVELOPMENT WITH C++	364
60.0.1 22.1 Qt Framework (Retained Mode)	364
60.0.2 22.2 Dear ImGui (Immediate Mode)	364
61 SCIENTIFIC COMPUTING & GPU	365
61.0.1 23.1 Eigen (Linear Algebra)	365
61.0.2 23.2 CUDA (GPU Programming)	365
62 INTEROPERABILITY	366
62.0.1 35.1 Python Bindings (pybind11)	366
62.0.2 35.2 Java Native Interface (JNI)	366
62.0.3 35.3 Stable C ABI	366
63 SECURITY ENGINEERING	367
63.0.1 36.1 Fuzzing (libFuzzer / AFL++)	367
63.0.2 36.2 Cryptography & Timing Attacks	367
63.0.3 36.3 Side-Channel Mitigations (Spectre)	367
64 SPECIALIZED DOMAINS	368
64.0.1 37.1 Game Development (Data-Oriented Design)	368
64.0.2 37.2 Embedded Systems	368
64.0.3 37.3 High-Frequency Trading (HFT)	368
64.0.4 37.4 Automotive (MISRA C++ / AUTOSAR)	368
64.1 FINAL COMPREHENSIVE CHECKLIST	369
64.1.1 C++98/03 Foundation	369
64.1.2 C++11 Major Features	369
64.1.3 C++14 Improvements	369
64.1.4 C++17 Modern Features	370
64.1.5 C++20 Revolutionary	370
64.1.6 C++23 Latest	370
64.1.7 Advanced Concepts	370
64.1.8 Concurrency	371

64.1.9 Performance & Optimization	371
64.1.10 STL Mastery	371
64.1.11 Design Patterns	371
64.1.12 Professional Development	371
64.2 Key Insights for C++ Mastery	372
64.3 Learning Path	372
64.4 Resources	372
64.4.1 Official Documentation	372
64.4.2 Books	373
64.4.3 Practice	373
65 ABA PROBLEM & MEMORY RECLAMATION	374
65.0.1 1. The ABA Problem Explained	374
65.0.2 2. Solution I: Tagged Pointers (Version Counters)	374
65.0.3 3. Solution II: Hazard Pointers (The Gold Standard)	374
65.0.4 4. Solution III: Epoch-Based Reclamation (EBR)	375
66 TEMPLATE METAPROGRAMMING PATTERNS	376
66.0.1 1. Expression Templates (Lazy Evaluation)	376
66.0.2 2. Type Erasure (The <code>std::any</code> Pattern)	376
66.0.3 3. The Detection Idiom (<code>void_t</code>)	376
66.0.4 4. Policy-Based Design	377
67 HIGH-PERFORMANCE DATA STRUCTURES	378
67.0.1 1. The Disruptor (Ring Buffer on Steroids)	378
67.0.2 2. Swiss Table (Open Addressing + Metadata)	378
67.0.3 3. Burst Tries / Judy Arrays	378
67.0.4 4. Slot Map	378
68 REAL-TIME AUDIO & SIGNAL PROCESSING	379
68.0.1 1. Lock-Free IPC (SPSC Queue)	379
68.0.2 2. SIMD for DSP	379
68.0.3 3. Double Buffering	379
69 ROBOTICS & ROS2 DEVELOPMENT	380
69.0.1 1. ROS2 Architecture & DDS	380
69.0.2 2. Zero-Copy Transport (Iceoryx)	380
69.0.3 3. Real-Time Executors	380
69.0.4 4. Custom Allocators (<code>std::pmr</code>)	380
70 MACHINE LEARNING INFRASTRUCTURE	381
70.0.1 1. Tensor Memory Layout	381
70.0.2 2. Broadcasting Implementation	381
70.0.3 3. Automatic Differentiation (Autograd)	381
70.0.4 4. Operator Fusion	381
71 DATABASE INTERNALS (LSM TREES)	382
71.0.1 1. The Write Problem	382
71.0.2 2. LSM Tree (Log-Structured Merge Tree)	382
71.0.3 3. Bloom Filters	382
71.0.4 4. Memory Mapped I/O (<code>mmap</code>)	382

72 THE ULTIMATE ALGORITHM REFERENCE	383
72.0.1 27.1 Non-Modifying Sequence Operations	383
72.0.2 27.2 Modifying Sequence Operations	383
72.0.3 27.3 Partitioning	383
72.0.4 27.4 Sorting	384
72.0.5 27.5 Binary Search (On Sorted Ranges)	384
72.0.6 27.6 Numeric Operations ()	384
73 CAPSTONE PROJECT - HIGH-PERFORMANCE ORDER BOOK	385
73.0.1 Project Structure	385
73.0.2 1. Types Module (types.cppm)	385
73.0.3 2. Order Module (order.cppm)	385
73.0.4 3. Order Book Module (book.cppm)	386
73.0.5 4. Main Application (main.cpp)	387
74 APPENDICES	388
75 Appendix A: C++ Keywords & Operators Reference	389
75.0.1 Essential Keywords (Non-Exhaustive)	389
75.0.2 Special Operators	390
76 Appendix B: Common Acronyms	391
77 Appendix C: Recommended Tooling	392
77.0.1 Compilers	392
77.0.2 Build Systems	392
77.0.3 Package Managers	392
77.0.4 Static Analysis & Sanitizers	392
78 Appendix D: Common C++ Traps & Pitfalls	393
78.0.1 I. General & Syntax Traps	393
78.0.2 II. Pointers, References & Memory	393
78.0.3 III. Classes & OOP	394
78.0.4 IV. Concurrency	395
78.0.5 V. Modern C++ & Macros	395
79 Appendix H: Professional C++ Idioms	396
79.0.1 1. RAII (Resource Acquisition Is Initialization)	396
79.0.2 2. Pimpl (Pointer to Implementation)	396
79.0.3 3. Copy-and-Swap	396
79.0.4 4. NVI (Non-Virtual Interface)	396
79.0.5 5. Erase-Remove Idiom	397
79.0.6 6. SFINAE (Substitution Failure Is Not An Error)	397
79.0.7 7. CRTP (Curiously Recurring Template Pattern)	397
80 Appendix E: C++ Interview Cheat Sheet	398
80.0.1 Core Concepts	398
80.0.2 Modern C++ (C++11/14/17/20)	398
80.0.3 System Design Questions	399
80.0.4 Quick Coding	399

81 Appendix F: The C++ Standard Evolution Matrix	400
81.0.1 1. Versioned Changelog	400
81.0.2 2. Feature Matrix	401
81.0.3 3. Timeline & Release Accuracy	401
 82 Appendix G: C++ Standard Library Headers Reference	 402
82.0.1 Concepts & Utilities	402
82.0.2 Containers	402
82.0.3 Algorithms & Iterators	403
82.0.4 Concurrency	403
82.0.5 Input/Output	403
82.0.6 Numerics & Math	403

Chapter 1

Preface

Chapter 2

The Complete C++ Programmer's Guide: From Zero to Godhood (C++98 to C++26)

Author: Shreejit Verma

2.1 Preface

2.1.1 About the Author

Shreejit Verma is a dedicated software engineer and quantitative developer with a passion for high-performance systems. With deep expertise in C++, distributed systems, and algorithmic trading, this book distills years of “in the trenches” experience into a single, comprehensive volume. The goal is simple: to provide the resource I wish I had when I started a path that doesn’t just teach syntax, but teaches **architecture, performance, and hardware sympathy**.

2.1.2 Book Purpose & Scope

This book is not just a language reference; it is a **career accelerator**. It aims to bridge the gap between academic C++ (often stuck in 1998) and the high-frequency trading/low-latency engineering standards of 2026.

We cover:

- * **Legacy to Modern:** From `void*` and `malloc` to `std::unique_ptr` and `std::pmr`.
- * **Standards Evolution:** A strict chronological progression from C++98 through C++23, with a preview of C++26.
- * **Systems Programming:** Memory models, atomics, networking, and compiler internals.
- * **Optimization:** SIMD, cache locality, branch prediction, and lock-free data structures.

2.1.3 Target Audience

1. **The Student:** Start at **Chapter 1**. Do not skip the “Deep Dive” sections; they lay the groundwork for understanding *why* things crash.
2. **The Professional:** Use **Chapters 4-8** to update your stack to Modern C++.
3. **The Specialist:** Jump to **Part 17 (Low Latency)** or **Part 24 (Architecture)** for domain-specific mastery.

2.1.4 Structure of the Book

The book is divided into **33 Chapters** (formerly “Parts”) organized into **5 Phases**:

- **Phase I: The Foundation (Chapters 1-3.5)**
 - Core syntax, OOP mechanics, and the “Classic” STL.
 - *Goal*: Write correct, compiling C++98 code.
- **Phase II: The Modern Renaissance (Chapters 4-6)**
 - C++11, C++14, and C++17. Move semantics, Lambdas, Smart Pointers.
 - *Goal*: Write safe, expressive, and efficient code.
- **Phase III: The Conceptual Revolution (Chapters 7-8)**
 - C++20/23. Concepts, Ranges, Coroutines, Modules.
 - *Goal*: Write generic, composable, and modular libraries.
- **Phase IV: Systems & Architecture (Chapters 9-16)**
 - Metaprogramming, Memory Models, Distributed Systems.
 - *Goal*: Design scalable systems that span threads and machines.
- **Phase V: Godhood (Chapters 17-33)**
 - Hardware Sympathy, SIMD, Custom Allocators, Compiler Construction.
 - *Goal*: Squeeze every nanosecond out of the CPU.

2.1.5 Conventions & Style Guide

To ensure clarity, this book follows strict conventions:

- **Code Standards**: All code examples use `Allman` or `K&R` brace styles and `snake\_case` for variables, consistent with the C++ Standard Library.
 - **Terminology**:
 - **UB**: Undefined Behavior (The compiler can do anything).
 - **Ill-formed**: The code will not compile.
 - **ODR**: One Definition Rule.
 - **Measurable Outcomes**: Each major section concludes with a “Skill Check” or “Implementation Task” to verify mastery.
-

2.2 Table of Contents

2.2.1 Volume I: Foundations

- Chapter 1: [ABSOLUTE BASICS \(C++98\)](#)
- Chapter 2: [THE C++ COMPILATION & EXECUTION MODEL](#)
- Chapter 3: [OBJECT-ORIENTED PROGRAMMING FUNDAMENTALS](#)
- Chapter 4: [DEEP OBJECT MODEL & VIRTUALIZATION](#)
- Chapter 5: [C++98/03 STANDARD LIBRARY](#)
- Chapter 6: [STL INTERNALS DEEP DIVE](#) ### Volume II: The Modern Renaissance
- Chapter 7: [C++11 REVOLUTION](#)
- Chapter 8: [ADVANCED MOVE SEMANTICS & VALUE CATEGORIES](#)
- Chapter 9: [C++14 ENHANCEMENTS](#)
- Chapter 10: [C++17 MODERN FEATURES](#) ### Volume III: Modern Mastery
- Chapter 11: [C++20 REVOLUTIONARY FEATURES](#)
- Chapter 12: [C++23 LATEST FEATURES](#)
- Chapter 13: [THE FUTURE - C++26 PREVIEW](#) ### Volume IV: Systems & Architecture
- Chapter 14: [ADVANCED TOPICS](#)
- Chapter 15: [PRODUCTION & PROFESSIONAL](#)
- Chapter 16: [SYSTEM DESIGN CASE STUDIES \(C++ EDITION\)](#)
- Chapter 17: [CONCURRENCY DESIGN PATTERNS](#)
- Chapter 18: [THE C++ BUILD ECOSYSTEM MASTERY](#) ### Volume V: High Performance & Low Latency
- Chapter 19: [LOW-LATENCY C++ OPTIMIZATION](#)
- Chapter 20: [LOW-LATENCY SYSTEM ARCHITECTURE](#)
- Chapter 21: [EXTREME LOW LATENCY & HARDWARE MASTERY](#)
- Chapter 22: [ADVANCED SIMD \(AVX2 & AVX-512\)](#)
- Chapter 23: [CUSTOM MEMORY ALLOCATORS](#) ### Volume VI: Deep Internals
- Chapter 24: [C++ UNDER THE HOOD](#)
- Chapter 25: [MASTERING THE MEMORY MODEL](#)
- Chapter 26: [WRITING A C++ COMPILER \(BASICS\)](#)
- Chapter 27: [WRITING A GARBAGE COLLECTOR](#)
- Chapter 28: [THE STANDARD LIBRARY FROM SCRATCH](#) ### Volume VII: Specialized Domains

- Chapter 29: [DISTRIBUTED C++](#)
 - Chapter 30: [NETWORKING FROM SCRATCH](#)
 - Chapter 31: [C++ IN THE CLOUD](#)
 - Chapter 32: [CROSS-PLATFORM DEVELOPMENT](#)
 - Chapter 33: [GUI DEVELOPMENT WITH C++](#)
 - Chapter 34: [SCIENTIFIC COMPUTING & GPU](#)
 - Chapter 35: [INTEROPERABILITY](#)
 - Chapter 36: [SECURITY ENGINEERING](#)
 - Chapter 37: [SPECIALIZED DOMAINS ### Volume VIII: Expert Mastery](#)
 - Chapter 38: [ABA PROBLEM & MEMORY RECLAMATION](#)
 - Chapter 39: [TEMPLATE METAPROGRAMMING PATTERNS](#)
 - Chapter 40: [HIGH-PERFORMANCE DATA STRUCTURES](#)
 - Chapter 41: [REAL-TIME AUDIO & SIGNAL PROCESSING](#)
 - Chapter 42: [ROBOTICS & ROS2 DEVELOPMENT](#)
 - Chapter 43: [MACHINE LEARNING INFRASTRUCTURE](#)
 - Chapter 44: [DATABASE INTERNALS \(LSM TREES\) ### Volume IX: Final Reference](#)
 - Chapter 45: [THE ULTIMATE ALGORITHM REFERENCE](#)
 - Chapter 46: [CAPSTONE PROJECT - HIGH-PERFORMANCE ORDER BOOK](#)
-

Part I

Volume I: C++98/03 - The Foundation

Chapter 3

ABSOLUTE BASICS (C++98)

3.1 Getting Started

3.1.1 What is C++?

C++ is a statically-typed, compiled programming language that combines low-level memory manipulation with high-level abstractions. It's the language of choice for performance-critical applications.

3.1.2 Your First Program (C++98)

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello, World!" << std::endl;
5     return 0;
6 }
```

Breakdown: - `\#include <iostream>` - Include input/output library - `std::cout` - Standard output stream (print to console) - `std::endl` - End line and flush buffer - `main()` - Entry point of program - `return 0` - Exit code (0 = success)

Compile and run:

```
1 g++ -o hello hello.cpp
2 ./hello
```

3.2 Basic Types & Variables

3.2.1 Fundamental Types (C++98)

```
1 #include <iostream>
2 #include <limits>
3
4 int main() {
5     // Integer types
6     int x = 42;                // 32-bit integer
7     short y = 10;             // 16-bit integer
8     long z = 1000000;          // 32 or 64-bit integer
9     long long w = 9999999999;  // 64-bit integer
10
11     // Floating-point types
```

```

12 float f = 3.14f;           // 32-bit (4 bytes)
13 double d = 3.14159265;    // 64-bit (8 bytes)
14 long double ld = 3.14159265359L; // 80+ bits
15
16 // Character and boolean types
17 char c = 'A';             // Single byte
18 bool b = true;            // true or false
19
20 // Print sizes
21 std::cout << "int: " << sizeof(int) << " bytes\n";
22 std::cout << "double: " << sizeof(double) << " bytes\n";
23
24 // Min/max values
25 std::cout << "int max: " << std::numeric_limits<int>::max() << "\n";
26 std::cout << "int min: " << std::numeric_limits<int>::min() << "\n";
27
28 return 0;
29 }

```

3.2.2 Variable Declaration & Initialization

```

1 #include <iostream>
2
3 int main() {
4     // C-style initialization
5     int x = 5;
6     float f = 3.14f;
7
8     // Multiple variables
9     int a, b, c;
10
11     // Uninitialized (dangerous - contains garbage)
12     int uninitialized;
13     std::cout << uninitialized << "\n"; // Undefined behavior!
14
15     // Constants
16     const int MAX_SIZE = 100;
17     // MAX_SIZE = 200; // Error: can't modify const
18
19     return 0;
20 }

```

3.2.3 Scope & Lifetime (C++98)

```

1 #include <iostream>
2
3 int global = 100; // Global scope - lives entire program
4
5 void function() {
6     int local = 5; // Local scope - lives only in function
7     {
8         int nested = 10; // Block scope - lives only in block
9         std::cout << nested << "\n";
10    }
11    // std::cout << nested << "\n"; // Error: nested out of scope
12 }
13
14 int main() {
15     {
16         int x = 5;

```

```

17     }
18     // std::cout << x << "\n"; // Error: x out of scope
19
20     return 0;
21 }

```

3.2.4 Deep Dive: The Memory Model of Variables

Understanding *where* your variables live is the first step to Godhood.

1. The Stack (Automatic Storage):

- **What:** Local variables (`int x`).
- **Speed:** Extremely fast (just moving a pointer).
- **Lifetime:** Scope-based (die at `}`).
- **Limit:** Small (typically 1MB-8MB). Recursion depth is limited by this.

2. The Heap (Dynamic Storage):

- **What:** `new int`, `malloc`.
- **Speed:** Slower (allocation requires finding free block).
- **Lifetime:** Manual (until `delete` / `free`).
- **Limit:** RAM size (Gigabytes).

3. Static/Global (Static Storage):

- **What:** Global variables, `static` locals.
- **Speed:** Fast access, but initialization order is tricky.
- **Lifetime:** Program start to program end.

4. Registers:

- **What:** CPU internal storage.
- **Speed:** Instant (0 cycles).
- **Note:** Variables are often optimized into registers, never touching RAM!

3.3 Operators & Control Flow

3.3.1 Arithmetic Operators (C++98)

```

1 #include <iostream>
2
3 int main() {
4     int a = 10, b = 3;
5
6     std::cout << a + b << "\n"; // 13 (addition)
7     std::cout << a - b << "\n"; // 7 (subtraction)
8     std::cout << a * b << "\n"; // 30 (multiplication)
9     std::cout << a / b << "\n"; // 3 (integer division)
10    std::cout << a % b << "\n"; // 1 (modulo)

```

```

11
12 // Compound assignment
13 int x = 5;
14 x += 3; // x = 8
15 x -= 2; // x = 6
16 x *= 2; // x = 12
17 x /= 3; // x = 4
18
19 // Increment/Decrement
20 int y = 5;
21 y++; // Post-increment: 6
22 ++y; // Pre-increment: 7
23 y--; // Post-decrement: 6
24 --y; // Pre-decrement: 5
25
26 return 0;
27 }

```

3.3.2 Deep Dive: Bitwise Mastery (Low-Level Optimization)

Bitwise operators manipulate individual bits. Essential for embedded systems, graphics, and cryptography.

The Operators

- `&` (AND): Both bits must be 1.
- `|` (OR): At least one bit must be 1.
- `^` (XOR): Bits must be different.
- `~` (NOT): Flip all bits.
- `<<` (Left Shift): Multiply by 2^N .
- `>>` (Right Shift): Divide by 2^N .

God-Tier Tricks

1. **Check Odd/Even:** `(x & 1) == 0` (Even). Faster than `% 2`.
2. **Multiply by 2:** `x << 1`.
3. **Divide by 2:** `x >> 1`.
4. **Clear Last Set Bit:** `x & (x - 1)`. Used to count set bits (Kernighan's Algorithm).
5. **Check Power of 2:** `(x > 0) && ((x & (x - 1)) == 0)`.
6. **Toggle Bit N:** `x ^= (1 << N)`.
7. **Set Bit N:** `x |= (1 << N)`.
8. **Clear Bit N:** `x &= ~(1 << N)`.

```

1 // Fast Power of 2 check
2 bool isPowerOf2(int x) {
3     return x && !(x & (x - 1));
4 }

```

3.3.3 Comparison & Logical Operators (C++98)

```
1 #include <iostream>
2
3 int main() {
4     int a = 10, b = 5;
5
6     // Comparison (return true/false)
7     std::cout << (a > b) << "\n";    // 1 (true)
8     std::cout << (a < b) << "\n";    // 0 (false)
9     std::cout << (a == b) << "\n";   // 0 (false)
10    std::cout << (a != b) << "\n";   // 1 (true)
11    std::cout << (a >= b) << "\n";   // 1 (true)
12    std::cout << (a <= b) << "\n";   // 0 (false)
13
14    // Logical operators
15    bool x = true, y = false;
16    std::cout << (x && y) << "\n";    // 0 (AND)
17    std::cout << (x || y) << "\n";    // 1 (OR)
18    std::cout << (!x) << "\n";       // 0 (NOT)
19
20    return 0;
21 }
```

3.3.4 If-Else Statements (C++98)

```
1 #include <iostream>
2
3 int main() {
4     int score = 85;
5
6     // Basic if-else
7     if (score >= 90) {
8         std::cout << "Grade: A\n";
9     } else if (score >= 80) {
10        std::cout << "Grade: B\n";
11    } else if (score >= 70) {
12        std::cout << "Grade: C\n";
13    } else {
14        std::cout << "Grade: F\n";
15    }
16
17    // Ternary operator
18    std::string grade = (score >= 80) ? "Pass" : "Fail";
19    std::cout << grade << "\n";
20
21    return 0;
22 }
```

3.3.5 Loops (C++98)

```
1 #include <iostream>
2
3 int main() {
4     // While loop
5     int i = 0;
6     while (i < 5) {
7         std::cout << i << " ";
8         i++;
9     }
10 }
```

```
10 std::cout << "\n";
11
12 // Do-while loop (executes at least once)
13 int j = 0;
14 do {
15     std::cout << j << " ";
16     j++;
17 } while (j < 5);
18 std::cout << "\n";
19
20 // For loop
21 for (int k = 0; k < 5; k++) {
22     std::cout << k << " ";
23 }
24 std::cout << "\n";
25
26 // Break and continue
27 for (int m = 0; m < 10; m++) {
28     if (m == 3) continue; // Skip 3
29     if (m == 7) break;    // Exit at 7
30     std::cout << m << " ";
31 }
32 std::cout << "\n";
33
34 return 0;
35 }
```

3.3.6 Switch Statement (C++98)

```
1 #include <iostream>
2
3 int main() {
4     int day = 3;
5
6     switch (day) {
7         case 1:
8             std::cout << "Monday\n";
9             break;
10        case 2:
11            std::cout << "Tuesday\n";
12            break;
13        case 3:
14            std::cout << "Wednesday\n";
15            break;
16        default:
17            std::cout << "Unknown day\n";
18    }
19
20    return 0;
21 }
```

3.4 Functions

3.4.1 Function Declaration & Definition (C++98)

```
1 #include <iostream>
2
```

```
3 // Function declaration (prototype)
4 int add(int a, int b);
5 void print_hello();
6
7 // Function definition
8 int add(int a, int b) {
9     return a + b;
10 }
11
12 void print_hello() {
13     std::cout << "Hello!\n";
14 }
15
16 int main() {
17     print_hello();
18     std::cout << add(5, 3) << "\n"; // 8
19     return 0;
20 }
```

3.4.2 Parameters & Return Values (C++98)

```
1 #include <iostream>
2
3 // Pass by value (copy)
4 void increment_value(int x) {
5     x++;
6     std::cout << "Inside: " << x << "\n";
7 }
8
9 // Pass by reference (same variable)
10 void increment_ref(int& x) {
11     x++;
12     std::cout << "Inside: " << x << "\n";
13 }
14
15 // Pass by const reference (can't modify)
16 void print_value(const int& x) {
17     std::cout << x << "\n";
18 }
19
20 // Returning by value
21 int get_value() {
22     return 42;
23 }
24
25 // Returning by reference (dangerous!)
26 int& get_global() {
27     static int x = 100;
28     return x;
29 }
30
31 int main() {
32     int a = 5;
33
34     increment_value(a); // Copy passed
35     std::cout << a << "\n"; // Still 5
36
37     increment_ref(a); // Reference passed
38     std::cout << a << "\n"; // Now 6
39
40     print_value(a); // Can't modify a
41 }
```



```
42     return 0;
43 }
```

3.4.3 Default Parameters (C++98)

```
1 #include <iostream>
2
3 void greet(const std::string& name = "World") {
4     std::cout << "Hello, " << name << "!\n";
5 }
6
7 int main() {
8     greet();                // Uses default: "World"
9     greet("Alice");         // Uses provided: "Alice"
10    return 0;
11 }
```

3.4.4 Function Overloading (C++98)

```
1 #include <iostream>
2
3 // Same function name, different parameters
4 int add(int a, int b) {
5     return a + b;
6 }
7
8 double add(double a, double b) {
9     return a + b;
10 }
11
12 void print(int x) {
13     std::cout << "Integer: " << x << "\n";
14 }
15
16 void print(double x) {
17     std::cout << "Double: " << x << "\n";
18 }
19
20 void print(const std::string& s) {
21     std::cout << "String: " << s << "\n";
22 }
23
24 int main() {
25     std::cout << add(5, 3) << "\n";    // 8 (int version)
26     std::cout << add(2.5, 3.7) << "\n"; // 6.2 (double version)
27
28     print(42);           // Integer version
29     print(3.14);         // Double version
30     print("Hello");      // String version
31
32     return 0;
33 }
```

3.5 Arrays & Pointers

3.5.1 Arrays (C++98)

```
1 #include <iostream>
2
3 int main() {
4     // Array declaration and initialization
5     int arr[5] = {1, 2, 3, 4, 5};
6
7     // Access elements (0-indexed)
8     std::cout << arr[0] << "\n"; // 1
9     std::cout << arr[4] << "\n"; // 5
10
11    // Array size (local arrays only)
12    int size = 5;
13    for (int i = 0; i < size; i++) {
14        std::cout << arr[i] << " ";
15    }
16    std::cout << "\n";
17
18    // 2D array
19    int matrix[3][3] = {
20        {1, 2, 3},
21        {4, 5, 6},
22        {7, 8, 9}
23    };
24
25    std::cout << matrix[1][2] << "\n"; // 6
26
27    return 0;
28 }
```

3.5.2 Pointers (C++98)

```
1 #include <iostream>
2
3 int main() {
4     int x = 42;
5
6     // Create a pointer
7     int* ptr = &x; // & = address-of operator
8
9     // Dereference pointer
10    std::cout << *ptr << "\n"; // 42 (dereference with *)
11
12    // Modify through pointer
13    *ptr = 100;
14    std::cout << x << "\n"; // 100
15
16    // Pointer arithmetic
17    int arr[5] = {10, 20, 30, 40, 50};
18    int* p = arr; // Array decays to pointer
19
20    std::cout << p[0] << "\n"; // 10
21    std::cout << *(p+1) << "\n"; // 20
22    std::cout << p[2] << "\n"; // 30
23
24    // Null pointer
25    int* null_ptr = nullptr; // or NULL in C++98
26
27    // Pointer to pointer
28    int** pp = &ptr;
29    std::cout << **pp << "\n"; // 100
30 }
```

```
31     return 0;
32 }
```

3.5.3 Dynamic Memory (C++98)

```
1  #include <iostream>
2
3  int main() {
4      // Allocate single value on heap
5      int* ptr = new int;
6      *ptr = 42;
7      std::cout << *ptr << "\n";
8      delete ptr;          // Must deallocate
9      ptr = nullptr;       // Good practice
10
11     // Allocate array on heap
12     int* arr = new int[10];
13     for (int i = 0; i < 10; i++) {
14         arr[i] = i * i;
15     }
16     delete[] arr;         // Note the [] for arrays
17
18     // Forgetting to delete = memory leak
19     int* leaked = new int(100);
20     // delete leaked;    // Forgot this!
21
22     return 0;
23 }
```

Chapter 4

ADVANCED POINTERS & MEMORY

4.1 1.1 Pointer to Const vs Const Pointer

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 5, y = 10;
6
7     // Pointer to const - can't modify data
8     const int* ptr1 = &x;
9     // *ptr1 = 10; // ERROR
10    ptr1 = &y; // OK - can change pointer
11
12    // Const pointer - can't modify pointer
13    int* const ptr2 = &x;
14    *ptr2 = 10; // OK - can change data
15    // ptr2 = &y; // ERROR
16
17    // Const pointer to const - can't modify either
18    const int* const ptr3 = &x;
19    // *ptr3 = 10; // ERROR
20    // ptr3 = &y; // ERROR
21
22    return 0;
23 }
```

4.2 1.2 Void Pointers

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 42;
6     double y = 3.14;
7
8     // Void pointer can point to any type
9     void* ptr = &x;
10    cout << *(int*)ptr << endl; // 42
11
12    ptr = &y;
13    cout << *(double*)ptr << endl; // 3.14
14 }
```

```

14
15 // Generic function using void*
16 void print_value(void* ptr, char type) {
17     if (type == 'i') {
18         cout << *(int*)ptr << endl;
19     } else if (type == 'd') {
20         cout << *(double*)ptr << endl;
21     }
22 }
23
24 print_value(&x, 'i'); // 42
25 print_value(&y, 'd'); // 3.14
26
27 return 0;
28 }

```

4.3 1.3 Null Pointer Safety

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int* ptr = NULL; // Set to null
6
7     // Always check before dereferencing
8     if (ptr != NULL) {
9         cout << *ptr << endl;
10    } else {
11        cout << "Pointer is NULL" << endl;
12    }
13
14    // Safer approach
15    ptr = new int(42);
16    if (ptr) {
17        cout << *ptr << endl;
18        delete ptr;
19        ptr = NULL;
20    }
21
22    return 0;
23 }

```

4.4 1.4 Memory Layout & Alignment

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     struct Data {
6         char a; // 1 byte
7         int b; // 4 bytes
8         double c; // 8 bytes
9     };
10
11    cout << "Size of Data: " << sizeof(Data) << endl;
12    // Likely 16 or 24 (due to alignment padding)
13
14    cout << "Size of char: " << sizeof(char) << endl; // 1

```

```
15     cout << "Size of int: " << sizeof(int) << endl;           // 4
16     cout << "Size of double: " << sizeof(double) << endl;     // 8
17
18     Data data;
19     cout << "Address of a: " << (void*)&data.a << endl;
20     cout << "Address of b: " << (void*)&data.b << endl;
21     cout << "Address of c: " << (void*)&data.c << endl;
22
23     return 0;
24 }
```

Chapter 5

ADVANCED FUNCTIONS

5.1 2.1 Variadic Functions

```
1 #include <iostream>
2 #include <cstdarg>
3 using namespace std;
4
5 // Function with variable number of arguments
6 int sum(int count, ...) {
7     va_list args;
8     va_start(args, count);
9
10    int total = 0;
11    for (int i = 0; i < count; i++) {
12        total += va_arg(args, int);
13    }
14
15    va_end(args);
16    return total;
17 }
18
19 int main() {
20     cout << sum(3, 10, 20, 30) << endl;    // 60
21     cout << sum(5, 1, 2, 3, 4, 5) << endl; // 15
22
23     return 0;
24 }
```

5.2 2.2 Function Recursion

```
1 #include <iostream>
2 using namespace std;
3
4 // Factorial using recursion
5 int factorial(int n) {
6     if (n <= 1) {
7         return 1; // Base case
8     }
9     return n * factorial(n - 1); // Recursive case
10 }
11
12 // Fibonacci using recursion (inefficient)
13 int fibonacci(int n) {
14     if (n <= 1) return n;
15     return fibonacci(n - 1) + fibonacci(n - 2);
16 }
```

```
16 }
17
18 // Fibonacci with memoization (efficient)
19 int fib_memo(int n, int memo[]) {
20     if (n <= 1) return n;
21     if (memo[n] != -1) return memo[n];
22
23     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo);
24     return memo[n];
25 }
26
27 int main() {
28     cout << factorial(5) << endl; // 120
29     cout << fibonacci(10) << endl; // 55
30
31     int memo[11];
32     for (int i = 0; i < 11; i++) memo[i] = -1;
33     cout << fib_memo(10, memo) << endl; // 55
34
35     return 0;
36 }
```

5.3 2.3 Inline Functions

```
1 #include <iostream>
2 using namespace std;
3
4 // Inline function - compiler may expand code
5 inline int square(int x) {
6     return x * x;
7 }
8
9 // Inline with condition
10 inline double max_value(double a, double b) {
11     return (a > b) ? a : b;
12 }
13
14 int main() {
15     cout << square(5) << endl; // 25
16     cout << max_value(3.5, 2.1) << endl; // 3.5
17
18     return 0;
19 }
```

5.4 2.4 Static Functions

```
1 #include <iostream>
2 using namespace std;
3
4 // File scope - only visible in this file
5 static void internal_function() {
6     cout << "Internal function" << endl;
7 }
8
9 // Static with counter
10 int get_call_count() {
11     static int count = 0; // Persists between calls
12     return ++count;
13 }
```



```
13 }  
14  
15 int main() {  
16     cout << get_call_count() << endl; // 1  
17     cout << get_call_count() << endl; // 2  
18     cout << get_call_count() << endl; // 3  
19  
20     internal_function(); // OK in same file  
21  
22     return 0;  
23 }
```

Chapter 6

FUNCTION POINTERS & CALLBACKS

6.1 3.1 Function Pointers

```
1 #include <iostream>
2 using namespace std;
3
4 // Function pointer declaration: return_type (*name)(parameters)
5 int add(int a, int b) {
6     return a + b;
7 }
8
9 int subtract(int a, int b) {
10    return a - b;
11 }
12
13 int multiply(int a, int b) {
14    return a * b;
15 }
16
17 int main() {
18     // Declare function pointer
19     int (*operation)(int, int);
20
21     // Assign function to pointer
22     operation = add;
23     cout << operation(5, 3) << endl;    // 8
24
25     operation = subtract;
26     cout << operation(5, 3) << endl;    // 2
27
28     operation = multiply;
29     cout << operation(5, 3) << endl;    // 15
30
31     return 0;
32 }
```

6.2 3.2 Function Pointer Arrays

```
1 #include <iostream>
2 using namespace std;
3
4 int add(int a, int b) { return a + b; }
```

```
5 int subtract(int a, int b) { return a - b; }
6 int multiply(int a, int b) { return a * b; }
7 int divide(int a, int b) { return a / b; }
8
9 int main() {
10     // Array of function pointers
11     int (*operations[4])(int, int) = {
12         add, subtract, multiply, divide
13     };
14
15     int a = 20, b = 4;
16
17     for (int i = 0; i < 4; i++) {
18         cout << "Result: " << operations[i](a, b) << endl;
19     }
20     // Output: 24, 16, 80, 5
21
22     return 0;
23 }
```

6.3 3.3 Callbacks

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Callback function type
6 typedef void (*Callback)(const string&);
7
8 class Button {
9 private:
10     Callback on_click;
11
12 public:
13     void set_click_handler(Callback callback) {
14         on_click = callback;
15     }
16
17     void click() {
18         if (on_click) {
19             on_click("Button clicked!");
20         }
21     }
22 };
23
24 void handle_click(const string& message) {
25     cout << message << endl;
26 }
27
28 int main() {
29     Button button;
30     button.set_click_handler(handle_click);
31     button.click(); // Output: Button clicked!
32
33     return 0;
34 }
```

Chapter 7

ADVANCED ARRAYS

7.1 4.1 Dynamic Arrays

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // 1D dynamic array
6     int size = 5;
7     int* arr = new int[size];
8
9     for (int i = 0; i < size; i++) {
10         arr[i] = i * 10;
11     }
12
13     for (int i = 0; i < size; i++) {
14         cout << arr[i] << " ";
15     }
16     cout << endl;
17
18     delete[] arr;
19     arr = NULL;
20
21     // 2D dynamic array
22     int rows = 3, cols = 4;
23     int** matrix = new int*[rows];
24     for (int i = 0; i < rows; i++) {
25         matrix[i] = new int[cols];
26     }
27
28     // Fill matrix
29     for (int i = 0; i < rows; i++) {
30         for (int j = 0; j < cols; j++) {
31             matrix[i][j] = i * cols + j;
32         }
33     }
34
35     // Delete matrix
36     for (int i = 0; i < rows; i++) {
37         delete[] matrix[i];
38     }
39     delete[] matrix;
40
41     return 0;
42 }
```

7.2 4.2 Variable Length Arrays (Non-standard)

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int size;
6     cout << "Enter size: ";
7     cin >> size;
8
9     // VLA - not standard but supported by many compilers
10    int arr[size]; // GCC extension
11
12    for (int i = 0; i < size; i++) {
13        arr[i] = i;
14    }
15
16    for (int i = 0; i < size; i++) {
17        cout << arr[i] << " ";
18    }
19    cout << endl;
20
21    return 0;
22 }
```

7.3 4.3 Array Bounds & Safety

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int arr[5] = {10, 20, 30, 40, 50};
6
7     // No bounds checking in C++
8     cout << arr[0] << endl; // 10 (OK)
9     cout << arr[10] << endl; // Undefined behavior!
10
11    // Manual bounds checking
12    int index = 5;
13    if (index >= 0 && index < 5) {
14        cout << arr[index] << endl;
15    } else {
16        cout << "Index out of bounds" << endl;
17    }
18
19    return 0;
20 }
```

Chapter 8

ADVANCED STRINGS

8.1 5.1 String Manipulation

```
1 #include <iostream>
2 #include <string>
3 #include <cstring>
4 using namespace std;
5
6 int main() {
7     string s = "Hello World";
8
9     // Length and capacity
10    cout << "Length: " << s.length() << endl;
11    cout << "Capacity: " << s.capacity() << endl;
12
13    // Access characters
14    cout << "First char: " << s[0] << endl;
15    cout << "Last char: " << s[s.length() - 1] << endl;
16
17    // Finding substrings
18    size_t pos = s.find("World");
19    if (pos != string::npos) {
20        cout << "Found at position: " << pos << endl;
21    }
22
23    // Replace
24    s.replace(6, 5, "C++");
25    cout << s << endl; // Hello C++
26
27    // Insert
28    s.insert(5, " there");
29    cout << s << endl; // Hello there C++
30
31    // Erase
32    s.erase(5, 6);
33    cout << s << endl; // Hello C++
34
35    // Substring
36    cout << s.substr(0, 5) << endl; // Hello
37
38    // Reverse
39    reverse(s.begin(), s.end());
40    cout << s << endl; // ++C olleH
41
42    return 0;
43 }
```

8.2 5.2 String Conversion

```
1 #include <iostream>
2 #include <string>
3 #include <sstream>
4 #include <cstdlib>
5 using namespace std;
6
7 int main() {
8     // String to number (C style)
9     string s1 = "42";
10    int num = atoi(s1.c_str());
11    cout << num << endl;
12
13    string s2 = "3.14";
14    double dbl = atof(s2.c_str());
15    cout << dbl << endl;
16
17    // Number to string (using stringstream)
18    stringstream ss;
19    ss << 42 << " " << 3.14 << " " << true;
20    string result = ss.str();
21    cout << result << endl; // 42 3.14 1
22
23    // Reverse conversion
24    stringstream ss2("100 200 300");
25    int a, b, c;
26    ss2 >> a >> b >> c;
27    cout << a << " " << b << " " << c << endl; // 100 200 300
28
29    return 0;
30 }
```

8.3 5.3 String Tokenization

```
1 #include <iostream>
2 #include <string>
3 #include <cstring>
4 using namespace std;
5
6 int main() {
7     string line = "apple,banana,orange,grape";
8
9     // Using stringstream and getline
10    stringstream ss(line);
11    string token;
12
13    while (getline(ss, token, ',')) {
14        cout << token << endl;
15    }
16    // Output: apple, banana, orange, grape
17
18    // Using strtok (C style)
19    char str[] = "hello world how are you";
20    char* ptr = strtok(str, " ");
21
22    while (ptr != NULL) {
23        cout << ptr << endl;
24        ptr = strtok(NULL, " ");
25    }
```

```
26 // Output: hello, world, how, are, you
27
28 return 0;
29 }
```

Chapter 9

BITWISE OPERATIONS

9.1 6.1 Bitwise Operators

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     unsigned char a = 5;    // 0101
6     unsigned char b = 3;    // 0011
7
8     // AND
9     cout << (a & b) << endl; // 0001 = 1
10
11    // OR
12    cout << (a | b) << endl; // 0111 = 7
13
14    // XOR
15    cout << (a ^ b) << endl; // 0110 = 6
16
17    // NOT (bitwise complement)
18    cout << (~a) << endl;    // 1010 = 250 (for unsigned char)
19
20    // Left shift
21    cout << (a << 1) << endl; // 1010 = 10
22
23    // Right shift
24    cout << (b >> 1) << endl; // 0001 = 1
25
26    return 0;
27 }
```

9.2 6.2 Bit Manipulation Techniques

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     unsigned int num = 5;    // 0101
6
7     // Check if bit is set
8     int bit_pos = 2;
9     bool is_set = (num >> bit_pos) & 1;
10    cout << "Bit " << bit_pos << " is: " << is_set << endl;
11
12    // Set a bit
```

```
13 num |= (1 << 1); // Set bit 1
14 cout << "After setting bit 1: " << num << endl; // 7 (0111)
15
16 // Clear a bit
17 num &= ~(1 << 1); // Clear bit 1
18 cout << "After clearing bit 1: " << num << endl; // 5 (0101)
19
20 // Toggle a bit
21 num ^= (1 << 0); // Toggle bit 0
22 cout << "After toggling bit 0: " << num << endl; // 4 (0100)
23
24 // Count set bits
25 unsigned int count = 0;
26 unsigned int temp = num;
27 while (temp) {
28     count += temp & 1;
29     temp >>= 1;
30 }
31 cout << "Number of set bits: " << count << endl;
32
33 return 0;
34 }
```

Chapter 10

PREPROCESSOR DIRECTIVES

10.1 7.1 #define and #include

```
1 // Define constants
2 #define PI 3.14159
3 #define MAX_SIZE 100
4 #define SQUARE(x) ((x) * (x))
5
6 // Conditional compilation
7 #define DEBUG
8
9 #ifdef DEBUG
10     #define LOG(msg) cout << msg << endl
11 #else
12     #define LOG(msg) // Do nothing in release
13 #endif
14
15 #include <iostream>
16 using namespace std;
17
18 int main() {
19     cout << "PI = " << PI << endl;
20
21     int arr[MAX_SIZE];
22     cout << "Array size: " << sizeof(arr) << endl;
23
24     cout << "Square of 5: " << SQUARE(5) << endl;
25
26     LOG("Debug message");
27
28     return 0;
29 }
```

10.2 7.2 Conditional Compilation

```
1 #include <iostream>
2 using namespace std;
3
4 // Platform-specific code
5 #ifdef _WIN32
6     #define OS "Windows"
7 #elif __APPLE__
8     #define OS "macOS"
9 #elif __linux__
10    #define OS "Linux"
```

```
11 #else
12     #define OS "Unknown"
13 #endif
14
15 int main() {
16     cout << "Running on: " << OS << endl;
17
18     #if defined(DEBUG)
19         cout << "Debug mode" << endl;
20     #else
21         cout << "Release mode" << endl;
22     #endif
23
24     return 0;
25 }
```

10.3 7.3 Pragma Directives

```
1 #include <iostream>
2 using namespace std;
3
4 // Disable specific warnings
5 #pragma warning(disable: 4996) // MSVC
6
7 // Pack structure
8 #pragma pack(1)
9 struct PackedData {
10     char a;
11     int b;
12     double c;
13 };
14 #pragma pack()
15
16 int main() {
17     cout << "Size of PackedData: " << sizeof(PackedData) << endl;
18     // Without pragma pack: 24 (aligned)
19     // With pragma pack: 13 (packed)
20
21     return 0;
22 }
```

Chapter 11

TYPE CASTING

11.1 8.1 C-Style Casting

```
1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4
5 int main() {
6     double d = 3.14;
7
8     // C-style cast (avoid in modern C++)
9     int i = (int)d; // 3
10    cout << i << endl;
11
12    int x = 65;
13    char c = (char)x; // 'A'
14    cout << c << endl;
15
16    float f = (float)d;
17    cout << f << endl;
18
19    return 0;
20 }
```

11.2 8.2 Implicit Conversions

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // Implicit conversions
6     int x = 5;
7     double d = x; // int to double (automatic)
8     cout << d << endl; // 5.0
9
10    double d2 = 3.9;
11    int y = d2; // double to int (loses precision)
12    cout << y << endl; // 3
13
14    // Char arithmetic
15    char c = 'A';
16    int code = c; // char to int (gets ASCII)
17    cout << code << endl; // 65
18
19    return 0;
20 }
```

20 }

Chapter 12

ADVANCED CONTROL FLOW

12.1 9.1 Ternary Operator

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 10, y = 5;
6
7     // condition ? true_value : false_value
8     int max = (x > y) ? x : y;
9     cout << "Max: " << max << endl;    // 10
10
11    // Nested ternary (use with caution)
12    int age = 20;
13    string status = (age < 18) ? "Minor" : (age < 65) ? "Adult" : "Senior";
14    cout << status << endl;
15
16    // String ternary
17    cout << (x % 2 == 0 ? "Even" : "Odd") << endl;
18
19    return 0;
20 }
```

12.2 9.2 goto Statement (Avoid)

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // goto is generally discouraged
6     int x = 0;
7
8 loop:
9     cout << x << " ";
10    x++;
11
12    if (x < 5) {
13        goto loop;
14    }
15    cout << endl;
16
17    // Better alternative: use loops
18    for (int i = 0; i < 5; i++) {
19        cout << i << " ";
20    }
```

```
20     }
21     cout << endl;
22
23     return 0;
24 }
```

12.3 9.3 Label & Goto for Error Handling

```
1 #include <iostream>
2 #include <cstdlib>
3 using namespace std;
4
5 int main() {
6     FILE* file = NULL;
7     char* buffer = NULL;
8
9     // Using goto for cleanup (rare acceptable use)
10    file = fopen("test.txt", "r");
11    if (!file) {
12        cout << "Failed to open file" << endl;
13        goto cleanup;
14    }
15
16    buffer = new char[100];
17    if (!buffer) {
18        cout << "Memory allocation failed" << endl;
19        goto cleanup;
20    }
21
22    // Do work...
23
24 cleanup:
25     if (buffer) delete[] buffer;
26     if (file) fclose(file);
27
28     return 0;
29 }
```


Chapter 13

ENUMERATION & UNIONS

13.1 10.1 Enumerations

```
1 #include <iostream>
2 using namespace std;
3
4 // Enum definition
5 enum Color { RED, GREEN, BLUE };
6
7 enum Direction {
8     NORTH = 0,
9     EAST = 1,
10    SOUTH = 2,
11    WEST = 3
12 };
13
14 int main() {
15     Color c = RED;
16     cout << c << endl;    // 0
17
18     Color colors[3] = {RED, GREEN, BLUE};
19
20     // Switching on enum
21     switch (c) {
22         case RED:
23             cout << "Red" << endl;
24             break;
25         case GREEN:
26             cout << "Green" << endl;
27             break;
28         case BLUE:
29             cout << "Blue" << endl;
30             break;
31     }
32
33     // Iterate through enum values
34     for (int dir = NORTH; dir <= WEST; dir++) {
35         cout << "Direction: " << dir << endl;
36     }
37
38     return 0;
39 }
```

13.2 10.2 Unions

```
1 #include <iostream>
2 using namespace std;
3
4 // Union - all members share same memory
5 union Data {
6     int i;
7     float f;
8     char c;
9 };
10
11 int main() {
12     Data data;
13
14     cout << "Size of Data: " << sizeof(data) << endl; // 4 (size of largest
15 // member)
16
17     data.i = 10;
18     cout << "data.i: " << data.i << endl; // 10
19     cout << "data.f: " << data.f << endl; // Garbage (overwrites data.i)
20
21     data.f = 3.14;
22     cout << "data.i: " << data.i << endl; // Garbage (overwrites by data.f)
23     cout << "data.f: " << data.f << endl; // 3.14
24
25 // Union useful for memory-constrained systems
26 union Variant {
27     int int_val;
28     double double_val;
29     char char_val;
30 };
31
32 cout << "Size of Variant: " << sizeof(Variant) << endl;
33
34 return 0;
35 }
```

Chapter 14

CONST & VOLATILE

14.1 11.1 Const Correctness

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int x = 10;
6
7     // const variable
8     const int constant = 5;
9     // constant = 10; // ERROR
10
11    // pointer to const
12    const int* ptr1 = &x;
13    // *ptr1 = 20; // ERROR
14    ptr1 = &constant; // OK
15
16    // const pointer
17    int* const ptr2 = &x;
18    *ptr2 = 20; // OK
19    // ptr2 = &constant; // ERROR
20
21    // const reference
22    const int& ref = x;
23    // ref = 20; // ERROR
24
25    cout << x << endl;
26    cout << *ptr1 << endl;
27    cout << *ptr2 << endl;
28    cout << ref << endl;
29
30    return 0;
31 }
```

14.2 11.2 Volatile Keyword

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // volatile - tells compiler value may change unexpectedly
6     volatile int sensor_reading = 0; // From hardware
7
8     // Compiler won't optimize away reads
```

```
9  while (sensor_reading < 100) {  
10     // Check actual value each time, not cached  
11 }  
12  
13 // Common use: hardware registers  
14 volatile int* hardware_register = (volatile int*)0x1000;  
15  
16 // Each access reads from actual location  
17 int val1 = *hardware_register;  
18 int val2 = *hardware_register;  
19  
20 // Without volatile, compiler might optimize one read  
21  
22 return 0;  
23 }
```

Chapter 15

INLINE FUNCTIONS & MACROS

15.1 12.1 Macro Functions vs Inline Functions

```
1 #include <iostream>
2 using namespace std;
3
4 // Macro function - preprocessor substitution
5 #define ADD_MACRO(a, b) ((a) + (b))
6
7 // Inline function - type-safe
8 inline int add_inline(int a, int b) {
9     return a + b;
10 }
11
12 int main() {
13     cout << ADD_MACRO(5, 3) << endl;           // 8
14     cout << add_inline(5, 3) << endl;           // 8
15
16     // Macro danger: side effects
17     int x = 5, y = 3;
18     cout << ADD_MACRO(x++, y++) << endl;        // Evaluates as: ((x++) + (y++))
19     cout << "x = " << x << ", y = " << y << endl; // x = 6, y = 4
20
21     // Inline function is safer
22     x = 5, y = 3;
23     cout << add_inline(x++, y++) << endl;        // 8
24     cout << "x = " << x << ", y = " << y << endl; // x = 6, y = 4 (correct)
25
26     return 0;
27 }
```

Chapter 16

NAMESPACES

16.1 13.1 Namespace Basics

```
1 #include <iostream>
2 using namespace std;
3
4 // Define namespace
5 namespace Math {
6     double PI = 3.14159;
7
8     double circle_area(double radius) {
9         return PI * radius * radius;
10    }
11 }
12
13 namespace Graphics {
14     double PI = 3.14;    // Different PI
15
16     void draw_circle(double radius) {
17         cout << "Drawing circle with radius: " << radius << endl;
18     }
19 }
20
21 int main() {
22     // Access with namespace::name
23     cout << Math::PI << endl;
24     cout << Graphics::PI << endl;
25
26     cout << Math::circle_area(5) << endl;
27     Graphics::draw_circle(5);
28
29     return 0;
30 }
```

16.2 13.2 Namespace Aliases

```
1 #include <iostream>
2 using namespace std;
3
4 namespace Very {
5     namespace Long {
6         namespace Namespace {
7             void function() {
8                 cout << "Long namespace function" << endl;
9             }
10        }
11    }
12 }
```

```
10     }
11 }
12 }
13
14 int main() {
15     // Use alias to shorten
16     namespace VLN = Very::Long::Namespace;
17
18     VLN::function();
19
20     // or use using
21     using namespace Very::Long::Namespace;
22     function();
23
24     return 0;
25 }
```

Chapter 17

FILE I/O ADVANCED

17.1 14.1 Binary File I/O

```
1 #include <iostream>
2 #include <fstream>
3 using namespace std;
4
5 int main() {
6     // Write binary data
7     ofstream outfile("data.bin", ios::binary);
8
9     int numbers[] = {10, 20, 30, 40, 50};
10    outfile.write((char*)numbers, sizeof(numbers));
11    outfile.close();
12
13    // Read binary data
14    ifstream infile("data.bin", ios::binary);
15
16    int buffer[5];
17    infile.read((char*)buffer, sizeof(buffer));
18
19    for (int i = 0; i < 5; i++) {
20        cout << buffer[i] << " ";
21    }
22    cout << endl;
23
24    infile.close();
25
26    return 0;
27 }
```

17.2 14.2 Stream Positioning

```
1 #include <iostream>
2 #include <fstream>
3 using namespace std;
4
5 int main() {
6     // Write to file
7     ofstream outfile("test.txt");
8     outfile << "0123456789";
9     outfile.close();
10
11    // Read with positioning
12    ifstream infile("test.txt");
```



```
13
14 // Tell position
15 cout << "Current position: " << infile.tellg() << endl;
16
17 // Seek to position
18 infile.seekg(5);
19 char c;
20 infile.get(c);
21 cout << "Character at position 5: " << c << endl; // '5'
22
23 // Seek from end
24 infile.seekg(-3, ios::end);
25 infile.get(c);
26 cout << "Third from end: " << c << endl; // '7'
27
28 infile.close();
29
30 return 0;
31 }
```

Chapter 18

ERROR HANDLING & DEBUGGING

18.1 15.1 Assert Macro

```
1 #include <iostream>
2 #include <cassert>
3 using namespace std;
4
5 int divide(int a, int b) {
6     assert(b != 0); // b must not be zero
7     return a / b;
8 }
9
10 int main() {
11     cout << divide(10, 2) << endl; // 5
12
13     // cout << divide(10, 0) << endl; // Assertion fails!
14
15     return 0;
16 }
```

18.2 15.2 Debug Output

```
1 #include <iostream>
2 #include <cstdio>
3 using namespace std;
4
5 #ifdef DEBUG
6     #define DPRINTF(fmt, ...) printf(fmt, __VA_ARGS__)
7 #else
8     #define DPRINTF(fmt, ...) (void)0
9 #endif
10
11 int main() {
12     int x = 42;
13
14     DPRINTF("Debug: x = %d\n", x);
15
16     cout << "Regular output" << endl;
17
18
19     return 0;
20 }
```


Chapter 19

THE C++ COMPILATION & EXECUTION MODEL

To truly understand C++, you must understand how your code transforms from text to a running process. This section demystifies the “black box” of the compiler.

19.0.1 1.5.1 The Build Pipeline: From Source to Binary

The “compilation” process actually consists of four distinct stages:

1. **Preprocessing:** Text substitution.

- Removes comments.
- Expands macros (`\#define`).
- Includes headers (`\#include`) recursively.
- Handles conditionals (`\#ifdef`).
- *Output:* A single “Translation Unit” (pure C++ source code).

2. **Compilation:** Syntax to Assembly.

- **Lexical Analysis:** Tokenizes source (e.g., `int`, `main`, `{`).
- **Parsing:** Builds Abstract Syntax Tree (AST).
- **Semantic Analysis:** Type checking, overload resolution.
- **Optimization:** Dead code elimination, loop unrolling, inlining (O1, O2, O3).
- **Code Generation:** Generates assembly code for the target architecture (x86_64, ARM64).
- *Output:* Assembly file (`.s` or `.asm`).

3. **Assembly:** Assembly to Machine Code.

- Translates mnemonics (`MOV`, `ADD`) to opcodes (`0x89`, `0x01`).
- *Output:* Object file (`.o` or `.obj`). Contains machine code but with *unresolved symbols*.

4. **Linking:** Combining Object Files.

- Combines multiple `.o` files and static libraries (`.a/.lib`).
- Resolves symbols: Matches function calls in one TU to definitions in another.
- Relocates addresses: Adjusts internal pointers.
- *Output:* Executable (`.exe` or ELF/Mach-O) or Shared Library (`.so/.dll`).

19.0.2 1.5.2 Translation Units (TU) & Linkage

A **Translation Unit** is the input to the compiler after preprocessing. It is the fundamental unit of compilation.

Linkage Types

Linkage determines if a name (variable/function) is visible to other TUs.

1. No Linkage:

- Visible only within the current scope (block).
- Example: Local variables.

2. Internal Linkage:

- Visible only within the current Translation Unit.
- Hidden from the linker.
- Examples: `static` global variables, `static` free functions, `const` globals (by default).
- *Use Case*: Private helper functions.

3. External Linkage:

- Visible to the linker and other TUs.
- Examples: Global variables, non-static functions, `extern` variables.
- *Danger*: Requires strict ODR compliance.

```
1 // TU1.cpp
2 static int internal_var = 10; // Internal Linkage
3 int global_var = 20;         // External Linkage
4
5 // TU2.cpp
6 extern int global_var;        // Declares existence of external symbol
7 // extern int internal_var;    // ERROR: Linker error (symbol not found)
```

19.0.3 1.5.3 The One Definition Rule (ODR)

The ODR is the most important rule in C++ linking.

1. **ODR for Translation Units**: A function/variable can have only **one definition** in a single TU. (Declarations can be repeated).
2. **ODR for Programs**: A non-inline function/variable can have only **one definition** in the entire program.
3. **ODR for Classes/Templates**: Can be defined in multiple TUs (via headers), but definitions must be **token-for-token identical**.

Violation Example:

```
1 // header.h
2 int x = 10; // DEFINITION (allocates memory)
3
4 // a.cpp
5 #include "header.h"
6
7 // b.cpp
8 #include "header.h"
9
10 // Linker Error: multiple definition of 'x'
11 // Fix: Use 'extern int x;' in header, definition in one .cpp
12 // Fix (C++17): Use 'inline int x = 10;'
```

19.0.4 1.5.4 Process Memory Layout

When your C++ program runs, the OS allocates virtual memory segments:

1. Text Segment (.text):

- Contains executable machine instructions.
- Read-Only (prevents self-modifying code exploits).
- Shared between multiple instances of the app.

2. Data Segment (.data):

- Initialized global and static variables.
- `int g = 10;` lives here.

3. BSS Segment (.bss):

- Uninitialized global/static variables.
- `int g;` lives here.
- Automatically zero-initialized by OS before `main()`.

4. Heap:

- Dynamic memory (`new`, `malloc`).
- Grows upwards (typically).
- Managed manually or by smart pointers.

5. Stack:

- Local variables, function parameters, return addresses.
- Grows downwards (typically).
- LIFO (Last-In, First-Out).
- *Stack Overflow*: Exceeding stack size (e.g., deep recursion).

19.0.5 1.5.5 Program Startup: Before main()

main() is NOT the first thing to run.

1. **OS Loader:** Loads EXE into memory.
2. **Entry Point** (`_start`): Defined by C Runtime (CRT).
3. **Global Initialization:**
 - Constructors of global/static objects run **before** main.
 - *Static Initialization Order Fiasco*: Order between TUs is undefined.
4. **main():** Your code runs.
5. **Global Destruction:**
 - Destructors of global/static objects run **after** main returns.
 - `atexit` handlers run.

19.0.6 1.5.6 Deep Dive into Data Representation

To understand bugs like integer overflow and floating-point inaccuracy, you must know how data is stored bits-by-bits.

Integer Representation (Two's Complement)

Most modern systems use **Two's Complement** for signed integers. * **Positive Numbers:** Standard binary. 0000 0101 (5). * **Negative Numbers:** Invert all bits and add 1. * 5 = 0000 0101 * Invert: 1111 1010 * Add 1: 1111 1011 (-5) * **Advantage:** Addition/Subtraction works identically for signed/unsigned. * **Range:** $-2^{(N-1)}$ to $2^{(N-1)} - 1$. (One extra negative number).

Floating Point (IEEE 754)

`float` (32-bit) and `double` (64-bit) follow IEEE 754. * **Components:** 1. **Sign Bit:** 0 (+) or 1 (-). 2. **Exponent:** Biased integer (allows very large/small numbers). 3. **Mantissa (Significand):** The precision bits (normalized to 1.xxxxx). * **Implication:** * $0.1 + 0.2 \neq 0.3$ because 0.1 cannot be represented exactly in binary (infinite repeating fraction). * **NaN (Not a Number):** Result of 0/0 or `sqrt(-1)`. `NaN != NaN` is always true. * **Infinity:** Result of $1.0/0.0$.

```

1 #include <iostream>
2 #include <cmath>
3 #include <limits>
4
5 int main() {
6     float a = 0.1f;
7     float b = 0.2f;
8     if (a + b == 0.3f) {
9         std::cout << "Math works!\n";
10    } else {
11        std::cout << "Math is broken: " << (a+b) << "\n"; // Prints 0.30000001
12    }
13
14    // Correct comparison
15    if (std::abs((a + b) - 0.3f) < 1e-5) {
16        std::cout << "Close enough!\n";
17    }
18 }

```


Chapter 20

OBJECT-ORIENTED PROGRAMMING FUNDAMENTALS

20.1 Classes & Objects

20.1.1 Basic Class (C++98)

```
1 #include <iostream>
2 #include <string>
3
4 class Person {
5 public:          // Publicly accessible
6     std::string name;
7     int age;
8
9     void introduce() {
10         std::cout << "I am " << name << ", age " << age << "\n";
11     }
12
13 private:       // Private to class
14     std::string secret;
15
16     void hidden_method() {
17         // Can't be called from outside
18     }
19
20 protected:    // Protected (for inheritance)
21     std::string protected_data;
22 };
23
24 int main() {
25     Person p;
26     p.name = "Alice";
27     p.age = 30;
28     p.introduce();
29
30     // p.secret = "xyz"; // Error: private
31     // p.hidden_method(); // Error: private
32
33     return 0;
34 }
```

20.1.2 Class Member Variables & Methods (C++98)

```

1 #include <iostream>
2
3 class BankAccount {
4 private:
5     double balance;
6     std::string owner;
7 public:
8     // Constructor
9     BankAccount(const std::string& name, double initial)
10         : owner(name), balance(initial) {}
11
12     // Methods
13     void deposit(double amount) {
14         if (amount > 0) {
15             balance += amount;
16         }
17     }
18
19     void withdraw(double amount) {
20         if (amount > 0 && amount <= balance) {
21             balance -= amount;
22         }
23     }
24
25     double get_balance() const { // const = doesn't modify
26         return balance;
27     }
28
29     std::string get_owner() const {
30         return owner;
31     }
32 };
33
34 int main() {
35     BankAccount account("Alice", 1000);
36     account.deposit(500);
37     account.withdraw(200);
38
39     std::cout << account.get_owner() << ": $"
40               << account.get_balance() << "\n";
41
42     return 0;
43 }

```

20.2 Classes & Objects - Complete Mastery

20.2.1 What is a Class?

A **class** is a blueprint for creating objects. It defines: - **Attributes** (member variables) - what the object has - **Methods** (member functions) - what the object does

20.2.2 What is an Object?

An **object** is an instance of a class - a concrete entity with specific values.

20.3 The Four Pillars of OOP

Object-Oriented Programming is built on four fundamental concepts that distinguish it from procedural programming:

20.3.1 1. Encapsulation - Data Hiding

20.3.2 2. Inheritance - Code Reuse

20.3.3 3. Polymorphism - Flexible Behavior

20.3.4 4. Abstraction - Simplified Interface

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Class definition
6 class Car {
7 public: // Public members accessible from outside
8     // Member variables
9     string brand;
10    string color;
11    int year;
12    int speed;
13
14    // Member functions
15    void accelerate() {
16        speed += 10;
17        cout << brand << " accelerates to " << speed << " mph\n";
18    }
19
20    void brake() {
21        if (speed >= 10) {
22            speed -= 10;
23        } else {
24            speed = 0;
25        }
26        cout << brand << " brakes to " << speed << " mph\n";
27    }
28
29    void displayInfo() {
30        cout << year << " " << color << " " << brand
31            << " at " << speed << " mph\n";
32    }
33 };
34
35 int main() {
36     // Creating objects (instances)
37     Car car1;
38     car1.brand = "Toyota";
39     car1.color = "Red";
40     car1.year = 2020;
41     car1.speed = 0;
42
43     Car car2;
44     car2.brand = "BMW";
45     car2.color = "Blue";
46     car2.year = 2022;
47     car2.speed = 0;
48
49     // Using objects
```

```

50     car1.displayInfo();
51     car1.accelerate();
52     car1.accelerate();
53     car1.brake();
54
55     car2.displayInfo();
56     car2.accelerate();
57     car2.accelerate();
58     car2.accelerate();
59
60     return 0;
61 }

```

Output:

```

1 2020 Red Toyota at 0 mph
2 Toyota accelerates to 10 mph
3 Toyota accelerates to 20 mph
4 Toyota brakes to 10 mph
5 2022 Blue BMW at 0 mph
6 BMW accelerates to 10 mph
7 BMW accelerates to 20 mph
8 BMW accelerates to 30 mph

```

20.4 Encapsulation

Encapsulation is hiding internal details and exposing only what's necessary.

20.4.1 Access Levels (Access Modifiers)

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class BankAccount {
6 private: // Only accessible within this class
7     double balance;
8     string accountNumber;
9
10    // Private helper function
11    bool validateAmount(double amount) {
12        return amount > 0;
13    }
14
15 public: // Accessible from anywhere
16    // Constructor
17    BankAccount(string accNum, double initialBalance)
18        : accountNumber(accNum), balance(initialBalance) {
19        cout << "Account created: " << accountNumber << "\n";
20    }
21
22    // Public methods to access private data
23    void deposit(double amount) {
24        if (validateAmount(amount)) {
25            balance += amount;
26            cout << "Deposited: $" << amount
27                << ". New balance: $" << balance << "\n";
28        } else {

```

```

29         cout << "Invalid deposit amount\n";
30     }
31 }
32
33 void withdraw(double amount) {
34     if (validateAmount(amount) && amount <= balance) {
35         balance -= amount;
36         cout << "Withdrew: $" << amount
37             << ". New balance: $" << balance << "\n";
38     } else {
39         cout << "Cannot withdraw that amount\n";
40     }
41 }
42
43 double getBalance() const { // const = doesn't modify
44     return balance;
45 }
46
47 string getAccountNumber() const {
48     return accountNumber;
49 }
50
51 protected: // Accessible in this class and derived classes
52 void logTransaction(string type) {
53     cout << "Transaction (" << type << ") logged\n";
54 }
55 };
56
57 int main() {
58     BankAccount account("ACC123456", 1000);
59
60     account.deposit(500);
61     account.withdraw(200);
62     account.withdraw(2000); // Won't work - insufficient funds
63
64     cout << "Final balance: $" << account.getBalance() << "\n";
65
66     // These would cause compile errors:
67     // account.balance = 10000; // Error: private
68     // account.validateAmount(100); // Error: private
69     // account.accountNumber = "123"; // Error: private
70
71     return 0;
72 }

```

Output:

```

1 Account created: ACC123456
2 Deposited: $500. New balance: $1500
3 Withdrew: $200. New balance: $1300
4 Cannot withdraw that amount
5 Final balance: $1300

```

Why Encapsulation? - Data Protection: Prevents invalid states - **Control:** You control how data is modified - **Flexibility:** Can change internal implementation without affecting users - **Security:** Sensitive data is hidden - **Maintainability:** Easy to modify implementation later

20.5 Constructors & Destructors - Complete Guide

20.5.1 Types of Constructors

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Student {
6 private:
7     string name;
8     int rollNumber;
9     double gpa;
10
11 public:
12     // 1. Default Constructor (no parameters)
13     Student() : name("Unknown"), rollNumber(0), gpa(0.0) {
14         cout << "Default constructor called\n";
15     }
16
17     // 2. Parameterized Constructor
18     Student(string n, int roll, double g)
19         : name(n), rollNumber(roll), gpa(g) {
20         cout << "Parameterized constructor called\n";
21     }
22
23     // 3. Copy Constructor (copies another object)
24     Student(const Student& other)
25         : name(other.name), rollNumber(other.rollNumber), gpa(other.gpa) {
26         cout << "Copy constructor called\n";
27     }
28
29     // 4. Move Constructor (C++11) - transfers ownership
30     Student(Student&& other) noexcept
31         : name(move(other.name)), rollNumber(other.rollNumber), gpa(other.gpa)
32     {
33         other.rollNumber = 0;
34         other.gpa = 0.0;
35         cout << "Move constructor called\n";
36     }
37
38     // Destructor - called when object is destroyed
39     ~Student() {
40         cout << "Destructor called for " << name << "\n";
41     }
42
43     // Getter methods
44     void display() const {
45         cout << "Name: " << name << ", Roll: " << rollNumber
46             << ", GPA: " << gpa << "\n";
47     }
48 };
49
50 int main() {
51     cout << "--- Creating objects ---\n";
52
53     // Default constructor
54     Student s1;
55     s1.display();
56     cout << "\n";
57
58     // Parameterized constructor
59     Student s2("Alice", 101, 3.8);
60     s2.display();
61     cout << "\n";
62 }
```

```

62 // Copy constructor (explicit)
63 Student s3 = s2; // Calls copy constructor
64 s3.display();
65 cout << "\n";
66
67 // Move constructor
68 Student s4 = move(s2); // Calls move constructor
69 s4.display();
70 cout << "\n";
71
72 cout << "--- Objects going out of scope ---\n";
73
74 return 0; // Destructors called here for all objects
75 }

```

Output:

```

1 --- Creating objects ---
2 Default constructor called
3 Name: Unknown, Roll: 0, GPA: 0
4
5 Parameterized constructor called
6 Name: Alice, Roll: 101, GPA: 3.8
7
8 Copy constructor called
9 Name: Alice, Roll: 101, GPA: 3.8
10
11 Move constructor called
12 Name: Alice, Roll: 101, GPA: 3.8
13
14 --- Objects going out of scope ---
15 Destructor called for Alice
16 Destructor called for Alice
17 Destructor called for Unknown
18 Destructor called for Unknown

```

20.5.2 Initialization Lists (Member Initializer List)

```

1 #include <iostream>
2 using namespace std;
3
4 class Rectangle {
5 private:
6     int width;
7     int height;
8
9 public:
10    // Using initialization list
11    // This is ALWAYS better than assignment in constructor body
12    Rectangle(int w, int h) : width(w), height(h) {
13        cout << "Rectangle created\n";
14    }
15
16    // Alternative (NOT recommended) - less efficient
17    // Rectangle(int w, int h) {
18    //     width = w; // Assignment, not initialization
19    //     height = h;
20    // }
21
22    int getArea() const {
23        return width * height;
24    }

```

```

25 };
26
27 int main() {
28     Rectangle rect(5, 10);
29     cout << "Area: " << rect.getArea() << "\n";
30
31     return 0;
32 }

```

Why initialization lists? - Efficiency: Initializes variables once (not create then assign)
- Const members: Can only be initialized with initializer list - **Reference members:** Must use initializer list - **Base class initialization:** Only way to initialize base class

20.6 Inheritance - All Types

Inheritance allows a class to inherit properties and methods from another class.

20.6.1 Single Inheritance

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // Base class (Parent class)
6  class Animal {
7  protected:  // Accessible in derived classes
8      string name;
9      int age;
10
11 public:
12     Animal(string n, int a) : name(n), age(a) {
13         cout << "Animal constructor called\n";
14     }
15
16     virtual void eat() {
17         cout << name << " is eating\n";
18     }
19
20     virtual void sleep() {
21         cout << name << " is sleeping\n";
22     }
23
24     virtual void makeSound() {
25         cout << name << " makes a sound\n";
26     }
27
28     virtual ~Animal() {
29         cout << "Animal destructor called\n";
30     }
31 };
32
33 // Derived class (Child class)
34 class Dog : public Animal {
35 private:
36     string breed;
37
38 public:
39     Dog(string n, int a, string b)

```



```

40     : Animal(n, a), breed(b) {
41     cout << "Dog constructor called\n";
42     }
43
44     // Override methods from base class
45     void makeSound() override {
46         cout << name << " barks: Woof! Woof!\n";
47     }
48
49     void fetch() {
50         cout << name << " is fetching the ball\n";
51     }
52
53     ~Dog() {
54         cout << "Dog destructor called\n";
55     }
56 };
57
58 class Cat : public Animal {
59 private:
60     bool isIndoor;
61
62 public:
63     Cat(string n, int a, bool indoor)
64         : Animal(n, a), isIndoor(indoor) {
65         cout << "Cat constructor called\n";
66     }
67
68     void makeSound() override {
69         cout << name << " meows: Meow! Meow!\n";
70     }
71
72     void scratch() {
73         cout << name << " is scratching the furniture\n";
74     }
75
76     ~Cat() {
77         cout << "Cat destructor called\n";
78     }
79 };
80
81 int main() {
82     cout << "=== Creating Dog ===\n";
83     Dog dog("Rex", 3, "Golden Retriever");
84     dog.eat();
85     dog.sleep();
86     dog.makeSound();
87     dog.fetch();
88
89     cout << "\n=== Creating Cat ===\n";
90     Cat cat("Whiskers", 2, true);
91     cat.eat();
92     cat.makeSound();
93     cat.scratch();
94
95     cout << "\n=== Using Polymorphism ===\n";
96     Animal* animals[2] = {&dog, &cat};
97     for (int i = 0; i < 2; i++) {
98         animals[i]->makeSound();
99     }
100
101     cout << "\n=== Destructors ===\n";
102     return 0;

```

103 }

Output:

```

1  === Creating Dog ===
2  Animal constructor called
3  Dog constructor called
4  Rex is eating
5  Rex is sleeping
6  Rex barks: Woof! Woof!
7  Rex is fetching the ball
8
9  === Creating Cat ===
10 Animal constructor called
11 Cat constructor called
12 Whiskers is eating
13 Whiskers meows: Meow! Meow!
14 Whiskers is scratching the furniture
15
16 === Using Polymorphism ===
17 Rex barks: Woof! Woof!
18 Whiskers meows: Meow! Meow!
19
20 === Destructors ===
21 Cat destructor called
22 Animal destructor called
23 Dog destructor called
24 Animal destructor called

```

20.6.2 Multiple Inheritance

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Flyer {
6  public:
7      virtual void fly() {
8          cout << "Flying...\n";
9      }
10     virtual ~Flyer() {}
11 };
12
13 class Swimmer {
14 public:
15     virtual void swim() {
16         cout << "Swimming...\n";
17     }
18     virtual ~Swimmer() {}
19 };
20
21 // Duck inherits from both Flyer and Swimmer
22 class Duck : public Flyer, public Swimmer {
23 private:
24     string name;
25
26 public:
27     Duck(string n) : name(n) {}
28
29     void fly() override {
30         cout << name << " is flying\n";
31     }

```

```
32
33     void swim() override {
34         cout << name << " is swimming\n";
35     }
36
37     void quack() {
38         cout << name << " quacks: Quack! Quack!\n";
39     }
40 };
41
42 int main() {
43     Duck duck("Donald");
44     duck.fly();
45     duck.swim();
46     duck.quack();
47
48     return 0;
49 }
```

20.6.3 Multilevel Inheritance

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Level 1: Base class
6 class Vehicle {
7 protected:
8     string brand;
9
10 public:
11     Vehicle(string b) : brand(b) {}
12     virtual void start() {
13         cout << brand << " is starting\n";
14     }
15     virtual ~Vehicle() {}
16 };
17
18 // Level 2: Derived from Vehicle
19 class Car : public Vehicle {
20 protected:
21     int numDoors;
22
23 public:
24     Car(string b, int doors) : Vehicle(b), numDoors(doors) {}
25     void start() override {
26         cout << brand << " car with " << numDoors
27             << " doors is starting\n";
28     }
29     virtual ~Car() {}
30 };
31
32 // Level 3: Derived from Car
33 class ElectricCar : public Car {
34 private:
35     int batteryPercentage;
36
37 public:
38     ElectricCar(string b, int doors, int battery)
39         : Car(b, doors), batteryPercentage(battery) {}
40
41     void start() override {
```

```
42         cout << "Electric " << brand << " (Battery: "
43             << batteryPercentage << "%) starting\n";
44     }
45 };
46
47 int main() {
48     ElectricCar tesla("Tesla", 4, 95);
49     tesla.start();
50
51     return 0;
52 }
```

20.6.4 Hierarchical Inheritance

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Employee {
6  protected:
7      string name;
8      double salary;
9
10 public:
11     Employee(string n, double s) : name(n), salary(s) {}
12     virtual void work() = 0;
13     virtual ~Employee() {}
14 };
15
16 class Manager : public Employee {
17 public:
18     Manager(string n, double s) : Employee(n, s) {}
19     void work() override {
20         cout << name << " is managing the team\n";
21     }
22 };
23
24 class Developer : public Employee {
25 private:
26     string language;
27
28 public:
29     Developer(string n, double s, string lang)
30         : Employee(n, s), language(lang) {}
31     void work() override {
32         cout << name << " is coding in " << language << "\n";
33     }
34 };
35
36 class Designer : public Employee {
37 public:
38     Designer(string n, double s) : Employee(n, s) {}
39     void work() override {
40         cout << name << " is designing UI/UX\n";
41     }
42 };
43
44 int main() {
45     Manager manager("Alice", 80000);
46     Developer dev("Bob", 70000, "C++");
47     Designer designer("Carol", 65000);
48 }
```

```
49     manager.work();
50     dev.work();
51     designer.work();
52
53     return 0;
54 }
```

20.7 Polymorphism

Polymorphism means “many forms” - the ability of objects to take multiple forms.

20.7.1 Compile-Time Polymorphism (Static Polymorphism)

1. Function Overloading

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Calculator {
6  public:
7      // Overload for integers
8      int add(int a, int b) {
9          cout << "Adding integers\n";
10         return a + b;
11     }
12
13     // Overload for doubles
14     double add(double a, double b) {
15         cout << "Adding doubles\n";
16         return a + b;
17     }
18
19     // Overload for strings (concatenation)
20     string add(string a, string b) {
21         cout << "Concatenating strings\n";
22         return a + b;
23     }
24
25     // Overload with different number of parameters
26     int add(int a, int b, int c) {
27         cout << "Adding three integers\n";
28         return a + b + c;
29     }
30 };
31
32 int main() {
33     Calculator calc;
34
35     cout << "Result: " << calc.add(5, 3) << "\n";
36     cout << "Result: " << calc.add(2.5, 3.7) << "\n";
37     cout << "Result: " << calc.add("Hello", " World") << "\n";
38     cout << "Result: " << calc.add(5, 3, 2) << "\n";
39
40     return 0;
41 }
```

2. Operator Overloading

```

1 #include <iostream>
2 using namespace std;
3
4 class Complex {
5 private:
6     double real, imag;
7
8 public:
9     Complex(double r = 0, double i = 0) : real(r), imag(i) {}
10
11     // Overload + operator
12     Complex operator+(const Complex& other) const {
13         return Complex(real + other.real, imag + other.imag);
14     }
15
16     // Overload - operator
17     Complex operator-(const Complex& other) const {
18         return Complex(real - other.real, imag - other.imag);
19     }
20
21     // Overload * operator
22     Complex operator*(const Complex& other) const {
23         double r = real * other.real - imag * other.imag;
24         double i = real * other.imag + imag * other.real;
25         return Complex(r, i);
26     }
27
28     // Overload == operator
29     bool operator==(const Complex& other) const {
30         return real == other.real && imag == other.imag;
31     }
32
33     // Overload << operator for output
34     friend ostream& operator<<(ostream& os, const Complex& c) {
35         os << c.real << " + " << c.imag << "i";
36         return os;
37     }
38
39     // Overload = operator (assignment)
40     Complex& operator=(const Complex& other) {
41         if (this != &other) {
42             real = other.real;
43             imag = other.imag;
44         }
45         return *this;
46     }
47 };
48
49 int main() {
50     Complex c1(3, 4);
51     Complex c2(2, 5);
52
53     Complex c3 = c1 + c2;
54     cout << c1 << " + " << c2 << " = " << c3 << "\n";
55
56     Complex c4 = c1 * c2;
57     cout << c1 << " * " << c2 << " = " << c4 << "\n";
58
59     if (c1 == c2) {
60         cout << "Complex numbers are equal\n";
61     } else {

```

```

62     cout << "Complex numbers are not equal\n";
63 }
64
65 return 0;
66 }

```

3. Template Specialization

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // Generic template
6  template <typename T>
7  class Printer {
8  public:
9      void print(T value) {
10         cout << "Generic: " << value << "\n";
11     }
12 };
13
14 // Template specialization for bool
15 template <>
16 class Printer<bool> {
17 public:
18     void print(bool value) {
19         cout << "Boolean: " << (value ? "true" : "false") << "\n";
20     }
21 };
22
23 // Template specialization for string
24 template <>
25 class Printer<string> {
26 public:
27     void print(string value) {
28         cout << "String: \"" << value << "\"\n";
29     }
30 };
31
32 int main() {
33     Printer<int> intPrinter;
34     intPrinter.print(42);
35
36     Printer<double> doublePrinter;
37     doublePrinter.print(3.14);
38
39     Printer<bool> boolPrinter;
40     boolPrinter.print(true);
41
42     Printer<string> stringPrinter;
43     stringPrinter.print("Hello, World!");
44
45     return 0;
46 }

```

20.7.2 Run-Time Polymorphism (Dynamic Polymorphism)

Virtual Functions & Override

```

1  #include <iostream>

```

```
2 #include <memory>
3 #include <vector>
4 using namespace std;
5
6 class Shape {
7 public:
8     virtual void draw() = 0;           // Pure virtual
9     virtual double area() = 0;
10    virtual string getName() = 0;
11    virtual ~Shape() {}                // Virtual destructor (important!)
12};
13
14 class Circle : public Shape {
15 private:
16     double radius;
17
18 public:
19     Circle(double r) : radius(r) {}
20
21     void draw() override {
22         cout << "Drawing Circle\n";
23     }
24
25     double area() override {
26         return 3.14159 * radius * radius;
27     }
28
29     string getName() override {
30         return "Circle";
31     }
32};
33
34 class Rectangle : public Shape {
35 private:
36     double width, height;
37
38 public:
39     Rectangle(double w, double h) : width(w), height(h) {}
40
41     void draw() override {
42         cout << "Drawing Rectangle\n";
43     }
44
45     double area() override {
46         return width * height;
47     }
48
49     string getName() override {
50         return "Rectangle";
51     }
52};
53
54 class Triangle : public Shape {
55 private:
56     double base, height;
57
58 public:
59     Triangle(double b, double h) : base(b), height(h) {}
60
61     void draw() override {
62         cout << "Drawing Triangle\n";
63     }
64}
```



```

65     double area() override {
66         return 0.5 * base * height;
67     }
68
69     string getName() override {
70         return "Triangle";
71     }
72 };
73
74 int main() {
75     // Using smart pointers (C++11)
76     vector<unique_ptr<Shape>> shapes;
77     shapes.push_back(make_unique<Circle>(5));
78     shapes.push_back(make_unique<Rectangle>(4, 6));
79     shapes.push_back(make_unique<Triangle>(3, 4));
80
81     cout << "=== Drawing all shapes ===\n";
82     for (auto& shape : shapes) {
83         shape->draw();
84         cout << "Area of " << shape->getName() << ": "
85              << shape->area() << "\n\n";
86     }
87
88     return 0; // Smart pointers automatically deleted
89 }

```

Output:

```

1 === Drawing all shapes ===
2 Drawing Circle
3 Area of Circle: 78.5385
4
5 Drawing Rectangle
6 Area of Rectangle: 24
7
8 Drawing Triangle
9 Area of Triangle: 6

```

Abstract Classes & Interfaces

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Abstract class (interface-like)
6 class DatabaseConnection {
7 public:
8     virtual void connect() = 0;
9     virtual void disconnect() = 0;
10    virtual bool isConnected() = 0;
11    virtual ~DatabaseConnection() {}
12 };
13
14 class MySQLConnection : public DatabaseConnection {
15 private:
16     bool connected;
17
18 public:
19     MySQLConnection() : connected(false) {}
20
21     void connect() override {
22         cout << "Connecting to MySQL...\n";

```

```

23     connected = true;
24     cout << "Connected to MySQL\n";
25 }
26
27 void disconnect() override {
28     cout << "Disconnecting from MySQL...\n";
29     connected = false;
30 }
31
32 bool isConnected() override {
33     return connected;
34 }
35 };
36
37 class PostgreSQLConnection : public DatabaseConnection {
38 private:
39     bool connected;
40
41 public:
42     PostgreSQLConnection() : connected(false) {}
43
44     void connect() override {
45         cout << "Connecting to PostgreSQL...\n";
46         connected = true;
47         cout << "Connected to PostgreSQL\n";
48     }
49
50     void disconnect() override {
51         cout << "Disconnecting from PostgreSQL...\n";
52         connected = false;
53     }
54
55     bool isConnected() override {
56         return connected;
57     }
58 };
59
60 int main() {
61     // Cannot create DatabaseConnection directly
62     // DatabaseConnection db; // Error: abstract class
63
64     // Create through derived classes
65     MySQLConnection mysql;
66     PostgreSQLConnection postgres;
67
68     // Use polymorphically
69     DatabaseConnection* db1 = &mysql;
70     DatabaseConnection* db2 = &postgres;
71
72     db1->connect();
73     cout << "MySQL connected: " << (db1->isConnected() ? "Yes" : "No") << "\n\n";
74
75     db2->connect();
76     cout << "PostgreSQL connected: " << (db2->isConnected() ? "Yes" : "No") <<
77     "\n\n";
78
79     db1->disconnect();
80     db2->disconnect();
81
82     return 0;
83 }

```

20.8 Abstraction

Abstraction is showing only essential features and hiding unnecessary details.

```
1 #include <iostream>
2 #include <string>
3 #include <cmath>
4 using namespace std;
5
6 // Abstract class - hides implementation details
7 class Shape {
8 public:
9     virtual void draw() = 0;
10    virtual double area() = 0;
11    virtual double perimeter() = 0;
12    virtual ~Shape() {}
13 };
14
15 // Concrete implementation
16 class Circle : public Shape {
17 private:
18     double radius;
19     const double PI = 3.14159;
20
21 public:
22     Circle(double r) : radius(r) {}
23
24     void draw() override {
25         cout << "Displaying Circle\n";
26     }
27
28     double area() override {
29         return PI * radius * radius;
30     }
31
32     double perimeter() override {
33         return 2 * PI * radius;
34     }
35 };
36
37 class Rectangle : public Shape {
38 private:
39     double width, height;
40
41 public:
42     Rectangle(double w, double h) : width(w), height(h) {}
43
44     void draw() override {
45         cout << "Displaying Rectangle\n";
46     }
47
48     double area() override {
49         return width * height;
50     }
51
52     double perimeter() override {
53         return 2 * (width + height);
54     }
55 };
56
```

```

57 // Client code doesn't know HOW things are calculated
58 void printShapeInfo(Shape* shape) {
59     shape->draw();
60     cout << "Area: " << shape->area() << "\n";
61     cout << "Perimeter: " << shape->perimeter() << "\n";
62 }
63
64 int main() {
65     Circle circle(5);
66     Rectangle rectangle(4, 6);
67
68     cout << "=== Circle ===\n";
69     printShapeInfo(&circle);
70
71     cout << "\n=== Rectangle ===\n";
72     printShapeInfo(&rectangle);
73
74     return 0;
75 }

```

20.9 Advanced Class Features

20.9.1 Nested Classes

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Car {
6 public:
7     // Nested class
8     class Engine {
9     private:
10         double horsepower;
11
12     public:
13         Engine(double hp) : horsepower(hp) {}
14
15         void startEngine() {
16             cout << "Engine with " << horsepower
17                 << " HP is starting\n";
18         }
19     };
20
21 private:
22     string brand;
23     Engine engine;
24
25 public:
26     Car(string b, double hp) : brand(b), engine(hp) {}
27
28     void display() {
29         cout << brand << " car\n";
30         engine.startEngine();
31     }
32 };
33
34 int main() {
35     Car car("Ferrari", 800);

```

```
36     car.display();
37
38     // Can also create nested class independently
39     Car::Engine standaloneEngine(500);
40     standaloneEngine.startEngine();
41
42     return 0;
43 }
```

20.9.2 Inner Classes with Access Control

```
1 #include <iostream>
2 using namespace std;
3
4 class Outer {
5 private:
6     int outerPrivate = 10;
7
8 public:
9     class Inner {
10    public:
11        void accessOuterPrivate(Outer& outer) {
12            // Inner class can access Outer's private members
13            cout << "Accessing outer private: "
14                << outer.outerPrivate << "\n";
15        }
16    };
17
18    Inner createInner() {
19        return Inner();
20    }
21 };
22
23 int main() {
24     Outer outer;
25     Outer::Inner inner = outer.createInner();
26     inner.accessOuterPrivate(outer);
27
28     return 0;
29 }
```

20.10 Access Modifiers & Friend Classes

20.10.1 Public, Private, Protected

```
1 #include <iostream>
2 using namespace std;
3
4 class Base {
5 public:
6     int publicData = 1;
7     void publicMethod() {
8         cout << "Public method\n";
9     }
10
11 protected:
12     int protectedData = 2;
```

```

13     void protectedMethod() {
14         cout << "Protected method\n";
15     }
16
17 private:
18     int privateData = 3;
19     void privateMethod() {
20         cout << "Private method\n";
21     }
22 };
23
24 class Derived : public Base {
25 public:
26     void testAccess() {
27         cout << "Public data: " << publicData << "\n";
28         cout << "Protected data: " << protectedData << "\n";
29         // cout << "Private data: " << privateData << "\n"; // Error!
30     }
31 };
32
33 int main() {
34     Base b;
35     cout << "Public: " << b.publicData << "\n";
36     // cout << "Protected: " << b.protectedData << "\n"; // Error!
37     // cout << "Private: " << b.privateData << "\n"; // Error!
38
39     Derived d;
40     d.testAccess();
41
42     return 0;
43 }

```

20.10.2 Friend Functions and Classes

```

1 #include <iostream>
2 using namespace std;
3
4 class MyClass {
5 private:
6     int secretValue;
7
8 public:
9     MyClass(int value) : secretValue(value) {}
10
11     // Friend function - can access private members
12     friend void revealSecret(MyClass& obj);
13
14     // Friend class - can access private members
15     friend class FriendClass;
16 };
17
18 // Friend function
19 void revealSecret(MyClass& obj) {
20     cout << "Secret value: " << obj.secretValue << "\n";
21 }
22
23 // Friend class
24 class FriendClass {
25 public:
26     void accessPrivate(MyClass& obj) {
27         cout << "Friend class accessing: " << obj.secretValue << "\n";
28     }

```

```
29 };
30
31 int main() {
32     MyClass obj(42);
33     revealSecret(obj);
34
35     FriendClass friend;
36     friend.accessPrivate(obj);
37
38     return 0;
39 }
```

20.11 Static Members

20.11.1 Static Variables & Methods

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Employee {
6 private:
7     string name;
8     int id;
9     static int nextId;      // Shared by all instances
10    static int totalCount;  // Count of employees
11
12 public:
13     Employee(string n) : name(n), id(nextId++) {
14         totalCount++;
15     }
16
17     ~Employee() {
18         totalCount--;
19     }
20
21     // Static method - can only access static members
22     static void displayStats() {
23         cout << "Total employees: " << totalCount << "\n";
24         cout << "Next ID will be: " << nextId << "\n";
25     }
26
27     void display() {
28         cout << "ID: " << id << ", Name: " << name << "\n";
29     }
30 };
31
32 // Initialize static members
33 int Employee::nextId = 1;
34 int Employee::totalCount = 0;
35
36 int main() {
37     cout << "Creating employees...\n";
38     Employee emp1("Alice");
39     emp1.display();
40
41     Employee emp2("Bob");
42     emp2.display();
43 }
```

```

44 Employee emp3("Carol");
45 emp3.display();
46
47 // Call static method
48 Employee::displayStats();
49
50 {
51     Employee emp4("David");
52     emp4.display();
53     Employee::displayStats();
54 }
55
56 cout << "\nAfter scope ends:\n";
57 Employee::displayStats();
58
59 return 0;
60 }

```

Output:

```

1 Creating employees...
2 ID: 1, Name: Alice
3 ID: 2, Name: Bob
4 ID: 3, Name: Carol
5 Total employees: 3
6 Next ID will be: 4
7 ID: 4, Name: David
8 Total employees: 4
9 Next ID will be: 5
10
11 After scope ends:
12 Total employees: 3
13 Next ID will be: 5

```

20.12 Const Correctness in OOP

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Person {
6 private:
7     mutable int accessCount; // Can be modified even in const functions
8     string name;
9     int age;
10
11 public:
12     Person(string n, int a) : name(n), age(a), accessCount(0) {}
13
14     // Const method - cannot modify member variables
15     string getName() const {
16         accessCount++; // OK because accessCount is mutable
17         return name;
18     }
19
20     // Const method returning const reference
21     const string& getNameRef() const {
22         accessCount++;
23         return name;

```



```

24     }
25
26     // Non-const method - can modify members
27     void setAge(int newAge) {
28         age = newAge; // OK in non-const method
29     }
30
31     // Const method
32     int getAge() const {
33         return age;
34     }
35
36     int getAccessCount() const {
37         return accessCount;
38     }
39
40     void display() const {
41         cout << "Name: " << name << ", Age: " << age << "\n";
42     }
43 };
44
45 int main() {
46     Person p("Alice", 30);
47
48     // Can call both const and non-const methods
49     cout << p.getName() << "\n";
50     cout << p.getAge() << "\n";
51     p.setAge(31);
52
53     // Const object
54     const Person cp("Bob", 25);
55
56     // Can only call const methods
57     cout << cp.getName() << "\n";
58     cout << cp.getAge() << "\n";
59     // cp.setAge(26); // Error: cannot call non-const method on const object
60
61     cp.display();
62     cout << "Access count: " << cp.getAccessCount() << "\n";
63
64     return 0;
65 }

```

20.13 Operator Overloading

20.13.1 Overloadable Operators

```

1 #include <iostream>
2 using namespace std;
3
4 class Vector {
5 private:
6     int x, y;
7
8 public:
9     Vector(int x = 0, int y = 0) : x(x), y(y) {}
10
11     // Arithmetic operators
12     Vector operator+(const Vector& v) const {

```

```

13     return Vector(x + v.x, y + v.y);
14 }
15
16 Vector operator-(const Vector& v) const {
17     return Vector(x - v.x, y - v.y);
18 }
19
20 Vector operator*(int scalar) const {
21     return Vector(x * scalar, y * scalar);
22 }
23
24 // Comparison operators
25 bool operator==(const Vector& v) const {
26     return x == v.x && y == v.y;
27 }
28
29 bool operator!=(const Vector& v) const {
30     return !(*this == v);
31 }
32
33 // Assignment operator
34 Vector& operator=(const Vector& v) {
35     if (this != &v) {
36         x = v.x;
37         y = v.y;
38     }
39     return *this;
40 }
41
42 // Unary operators
43 Vector operator-() const {
44     return Vector(-x, -y);
45 }
46
47 // Increment/Decrement
48 Vector& operator++() { // Pre-increment
49     x++; y++;
50     return *this;
51 }
52
53 Vector operator++(int) { // Post-increment
54     Vector temp = *this;
55     x++; y++;
56     return temp;
57 }
58
59 // Subscript operator
60 int operator[](int index) const {
61     if (index == 0) return x;
62     if (index == 1) return y;
63     throw out_of_range("Invalid index");
64 }
65
66 // Stream operators (must be friend or free functions)
67 friend ostream& operator<<(ostream& os, const Vector& v) {
68     os << "(" << v.x << ", " << v.y << ")";
69     return os;
70 }
71
72 friend istream& operator>>(istream& is, Vector& v) {
73     is >> v.x >> v.y;
74     return is;
75 }

```

```

76 };
77
78 int main() {
79     Vector v1(3, 4);
80     Vector v2(1, 2);
81
82     cout << "v1 = " << v1 << "\n";
83     cout << "v2 = " << v2 << "\n";
84
85     Vector v3 = v1 + v2;
86     cout << "v1 + v2 = " << v3 << "\n";
87
88     Vector v4 = v1 - v2;
89     cout << "v1 - v2 = " << v4 << "\n";
90
91     Vector v5 = v1 * 2;
92     cout << "v1 * 2 = " << v5 << "\n";
93
94     cout << "v1 == v2: " << (v1 == v2 ? "true" : "false") << "\n";
95
96     cout << "-v1 = " << (-v1) << "\n";
97
98     cout << "v1[0] = " << v1[0] << ", v1[1] = " << v1[1] << "\n";
99
100    Vector v6 = v1;
101    cout << "After v6 = v1: v6 = " << v6 << "\n";
102
103    return 0;
104 }

```

20.14 SOLID Principles

20.14.1 S - Single Responsibility Principle

```

1  #include <iostream>
2  #include <string>
3  #include <fstream>
4  using namespace std;
5
6  // BAD: Class doing multiple things
7  // class User {
8  //     void save() { } // Saving logic
9  //     void sendEmail() { } // Sending email
10 //     void validate() { } // Validation
11 // };
12
13 // GOOD: Each class has one responsibility
14 class User {
15 private:
16     string name, email;
17
18 public:
19     User(string n, string e) : name(n), email(e) {}
20     string getName() { return name; }
21     string getEmail() { return email; }
22 };
23
24 class UserValidator {
25 public:

```

```

26     bool isValid(const User& user) {
27         return !user.getName().empty() &&
28                !user.getEmail().empty();
29     }
30 };
31
32 class UserRepository {
33 public:
34     void save(const User& user) {
35         // Save to database or file
36         cout << "Saving user: " << user.getName() << "\n";
37     }
38 };
39
40 class EmailService {
41 public:
42     void sendWelcomeEmail(const User& user) {
43         cout << "Sending welcome email to: " << user.getEmail() << "\n";
44     }
45 };
46
47 int main() {
48     User user("Alice", "alice@example.com");
49     UserValidator validator;
50     UserRepository repo;
51     EmailService emailService;
52
53     if (validator.isValid(user)) {
54         repo.save(user);
55         emailService.sendWelcomeEmail(user);
56     }
57
58     return 0;
59 }

```

20.14.2 O - Open/Closed Principle

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  // GOOD: Open for extension, closed for modification
6  class Shape {
7  public:
8      virtual double area() = 0;
9      virtual ~Shape() {}
10 };
11
12 class Circle : public Shape {
13 private:
14     double radius;
15 public:
16     Circle(double r) : radius(r) {}
17     double area() override {
18         return 3.14 * radius * radius;
19     }
20 };
21
22 class Rectangle : public Shape {
23 private:
24     double width, height;
25 public:

```

```

26     Rectangle(double w, double h) : width(w), height(h) {}
27     double area() override {
28         return width * height;
29     }
30 };
31
32 // New shape can be added without modifying existing code
33 class Triangle : public Shape {
34 private:
35     double base, height;
36 public:
37     Triangle(double b, double h) : base(b), height(h) {}
38     double area() override {
39         return 0.5 * base * height;
40     }
41 };
42
43 class AreaCalculator {
44 public:
45     double totalArea(vector<Shape*> shapes) {
46         double total = 0;
47         for (Shape* shape : shapes) {
48             total += shape->area();
49         }
50         return total;
51     }
52 };
53
54 int main() {
55     Circle c(5);
56     Rectangle r(4, 6);
57     Triangle t(3, 4);
58
59     vector<Shape*> shapes = {&c, &r, &t};
60     AreaCalculator calc;
61
62     cout << "Total area: " << calc.totalArea(shapes) << "\n";
63
64     return 0;
65 }

```

20.14.3 L - Liskov Substitution Principle

```

1  #include <iostream>
2  using namespace std;
3
4  class Bird {
5  public:
6      virtual void fly() = 0;
7      virtual ~Bird() {}
8  };
9
10 // GOOD: Penguin shouldn't be Bird if it can't fly
11 // Instead, create separate hierarchy
12 class FlyingBird : public Bird {
13 public:
14     void fly() override {
15         cout << "Flying...\n";
16     }
17 };
18
19 class Sparrow : public FlyingBird {

```

```

20 public:
21     void fly() override {
22         cout << "Sparrow flying\n";
23     }
24 };
25
26 // Penguin is a Bird, but not a FlyingBird
27 class NonFlyingBird {
28 public:
29     virtual void move() = 0;
30     virtual ~NonFlyingBird() {}
31 };
32
33 class Penguin : public NonFlyingBird {
34 public:
35     void move() override {
36         cout << "Penguin swimming/waddling\n";
37     }
38 };
39
40 int main() {
41     Sparrow sparrow;
42     sparrow.fly();
43
44     Penguin penguin;
45     penguin.move();
46
47     return 0;
48 }

```

20.14.4 I - Interface Segregation Principle

```

1  #include <iostream>
2  using namespace std;
3
4  // BAD: Worker interface too bloated
5  // class Worker {
6  //     virtual void work() = 0;
7  //     virtual void eat() = 0;
8  //     virtual void sleep() = 0;
9  // };
10
11 // GOOD: Segregated interfaces
12 class Workable {
13 public:
14     virtual void work() = 0;
15     virtual ~Workable() {}
16 };
17
18 class Eatable {
19 public:
20     virtual void eat() = 0;
21     virtual ~Eatable() {}
22 };
23
24 class Sleepable {
25 public:
26     virtual void sleep() = 0;
27     virtual ~Sleepable() {}
28 };
29
30 class Human : public Workable, public Eatable, public Sleepable {

```

```

31 public:
32     void work() override {
33         cout << "Human working\n";
34     }
35     void eat() override {
36         cout << "Human eating\n";
37     }
38     void sleep() override {
39         cout << "Human sleeping\n";
40     }
41 };
42
43 class Robot : public Workable {
44 public:
45     void work() override {
46         cout << "Robot working\n";
47     }
48 };
49
50 int main() {
51     Human human;
52     human.work();
53     human.eat();
54     human.sleep();
55
56     Robot robot;
57     robot.work();
58     // robot.eat(); // Robot doesn't have eat, which is correct!
59
60     return 0;
61 }

```

20.14.5 D - Dependency Inversion Principle

```

1  #include <iostream>
2  #include <memory>
3  using namespace std;
4
5  // Abstraction
6  class DataStore {
7  public:
8      virtual void save(const string& data) = 0;
9      virtual ~DataStore() {}
10 };
11
12 // Concrete implementations
13 class DatabaseStore : public DataStore {
14 public:
15     void save(const string& data) override {
16         cout << "Saving to database: " << data << "\n";
17     }
18 };
19
20 class FileStore : public DataStore {
21 public:
22     void save(const string& data) override {
23         cout << "Saving to file: " << data << "\n";
24     }
25 };
26
27 // High-level module depends on abstraction, not concrete class
28 class UserService {

```

```

29 private:
30     shared_ptr<DataStore> dataStore;
31
32 public:
33     UserService(shared_ptr<DataStore> store) : dataStore(store) {}
34
35     void registerUser(const string& username) {
36         cout << "Registering user: " << username << "\n";
37         dataStore->save(username);
38     }
39 };
40
41 int main() {
42     // Can easily switch implementations
43     auto dbStore = make_shared<DatabaseStore>();
44     UserService service1(dbStore);
45     service1.registerUser("Alice");
46
47     cout << "\n";
48
49     auto fileStore = make_shared<FileStore>();
50     UserService service2(fileStore);
51     service2.registerUser("Bob");
52
53     return 0;
54 }

```

20.15 Constructors & Destructors

20.15.1 Constructors (C++98)

```

1  #include <iostream>
2  #include <string>
3
4  class Car {
5  private:
6      std::string brand;
7      int year;
8  public:
9      // Default constructor (no parameters)
10     Car() : brand("Unknown"), year(2000) {
11         std::cout << "Default constructor called\n";
12     }
13
14     // Parameterized constructor
15     Car(const std::string& b, int y) : brand(b), year(y) {
16         std::cout << "Constructor called\n";
17     }
18
19     // Copy constructor
20     Car(const Car& other) : brand(other.brand), year(other.year) {
21         std::cout << "Copy constructor called\n";
22     }
23
24     void show() const {
25         std::cout << brand << " (" << year << ")\n";
26     }
27 };
28

```



```
29 int main() {
30     Car c1;                      // Calls default
31     Car c2("Toyota", 2020);      // Calls parameterized
32     Car c3 = c2;                 // Calls copy
33
34     c1.show();
35     c2.show();
36     c3.show();
37
38     return 0;
39 }
```

20.15.2 Destructors (C++98)

```
1 #include <iostream>
2 #include <fstream>
3
4 class File {
5 private:
6     std::ofstream file;
7 public:
8     File(const std::string& filename) {
9         file.open(filename);
10        if (file) {
11            std::cout << "File opened\n";
12        }
13    }
14
15    // Destructor (called when object destroyed)
16    ~File() {
17        if (file.is_open()) {
18            file.close();
19            std::cout << "File closed\n";
20        }
21    }
22
23    void write(const std::string& data) {
24        file << data << "\n";
25    }
26 };
27
28 int main() {
29     {
30         File f("output.txt");
31         f.write("Hello, World!");
32     } // Destructor called here, file closed
33
34     return 0;
35 }
```

20.15.3 2.2 Advanced Constructor Features (C++11)

Delegating Constructors

One constructor can call another constructor of the same class to reduce code duplication.

```
1 class Data {
2     int x, y;
3     std::string s;
4 public:
5     // Target constructor
```

```

6   Data(int x, int y, std::string s) : x(x), y(y), s(s) {}
7
8   // Delegating constructor
9   Data() : Data(0, 0, "default") {}
10
11  // Another delegating constructor
12  Data(int x) : Data(x, 0, "default") {}
13 };

```

Inheriting Constructors

Using using to expose base class constructors.

```

1  class Base {
2  public:
3      Base(int x) { std::cout << "Base(int)\n"; }
4  };
5
6  class Derived : public Base {
7  public:
8      using Base::Base; // Inherits Base(int)
9      // Implicitly generates Derived(int x) : Base(x) {}
10 };

```

20.15.4 2.3 Construction & Destruction Order

The order is strict and deterministic (Stack logic: LIFO).

Construction Order: 1. Base Classes (in order of inheritance) 2. Member Objects (in order of declaration in class) 3. Constructor Body

Destruction Order: 1. Destructor Body 2. Member Objects (reverse order of declaration) 3. Base Classes (reverse order of inheritance)

```

1  class Base { public: Base() { cout << "Base "; } ~Base() { cout << "~Base "; }
2  };
3
4  class Member { public: Member() { cout << "Member "; } ~Member() { cout << "~
5  Member "; } };
6
7  class Derived : public Base {
8  public:
9      Derived() { cout << "Derived "; }
10     ~Derived() { cout << "~Derived "; }
11 };
12
13 int main() {
14     Derived d; // Output: Base Member Derived
15     // End of scope: ~Derived ~Member ~Base
16 }

```

20.15.5 2.4 The Rule of Three, Five, and Zero

This is the cornerstone of resource management.

1. **Rule of Three (C++98):** If you implement one of: Destructor, Copy Constructor, Copy Assignment Operator; you likely need to implement all three.
2. **Rule of Five (C++11):** For Move semantics, add Move Constructor and Move Assignment Operator.

3. **Rule of Zero:** If your class uses RAII types (`std::string`, `std::vector`, `std::unique_ptr`), do **NOT** declare any of the special member functions. Let the compiler generate them.

```
1 // Rule of Zero Example (Best Practice)
2 class User {
3     std::string name; // Manages its own memory
4     std::vector<int> scores; // Manages its own memory
5     // No destructor needed!
6 };
```

20.16 Inheritance

20.16.1 Basic Inheritance (C++98)

```
1 #include <iostream>
2 #include <string>
3
4 // Base class
5 class Animal {
6 protected:
7     std::string name;
8 public:
9     Animal(const std::string& n) : name(n) {}
10
11     virtual void speak() { // virtual allows override
12         std::cout << name << " makes a sound\n";
13     }
14
15     virtual ~Animal() {}
16 };
17
18 // Derived class
19 class Dog : public Animal {
20 public:
21     Dog(const std::string& n) : Animal(n) {}
22
23     void speak() override { // override keyword (C++11)
24         std::cout << name << " barks\n";
25     }
26 };
27
28 class Cat : public Animal {
29 public:
30     Cat(const std::string& n) : Animal(n) {}
31
32     void speak() override {
33         std::cout << name << " meows\n";
34     }
35 };
36
37 int main() {
38     Dog dog("Rex");
39     Cat cat("Whiskers");
40
41     dog.speak();
42     cat.speak();
43
44     // Polymorphism
```

```
45     Animal* animals[2] = {&dog, &cat};
46     for (int i = 0; i < 2; i++) {
47         animals[i]->speack();
48     }
49
50     return 0;
51 }
```

20.16.2 Multiple Inheritance (C++98)

```
1  #include <iostream>
2
3  class A {
4  public:
5      void func_a() { std::cout << "A::func\n"; }
6  };
7
8  class B {
9  public:
10     void func_b() { std::cout << "B::func\n"; }
11 };
12
13 class C: public A, public B {
14 public:
15     void func_c() { std::cout << "C::func\n"; }
16 };
17
18 int main() {
19     C c;
20     c.func_a();
21     c.func_b();
22     c.func_c();
23
24     return 0;
25 }
```

20.17 Virtual Functions & Polymorphism

20.17.1 Virtual Functions (C++98)

```
1  #include <iostream>
2  #include <vector>
3  #include <memory>
4
5  class Shape {
6  public:
7      virtual void draw() = 0; // Pure virtual (abstract)
8      virtual double area() = 0;
9      virtual ~Shape() {}
10 };
11
12 class Circle : public Shape {
13 private:
14     double radius;
15 public:
16     Circle(double r) : radius(r) {}
17 }
```

```

18     void draw() override {
19         std::cout << "Drawing circle\n";
20     }
21
22     double area() override {
23         return 3.14159 * radius * radius;
24     }
25 };
26
27 class Rectangle : public Shape {
28 private:
29     double width, height;
30 public:
31     Rectangle(double w, double h) : width(w), height(h) {}
32
33     void draw() override {
34         std::cout << "Drawing rectangle\n";
35     }
36
37     double area() override {
38         return width * height;
39     }
40 };
41
42 int main() {
43     std::vector<Shape*> shapes;
44     shapes.push_back(new Circle(5));
45     shapes.push_back(new Rectangle(4, 6));
46
47     for (Shape* s : shapes) {
48         s->draw();
49         std::cout << "Area: " << s->area() << "\n";
50     }
51
52     // Cleanup
53     for (Shape* s : shapes) {
54         delete s;
55     }
56
57     return 0;
58 }

```

20.17.2 2.6 Advanced Polymorphism

1. Virtual Destructors (CRITICAL)

If you delete a derived object through a base pointer, the base destructor must be virtual. Otherwise, the derived destructor is **never called**, causing memory leaks.

```

1 class Base {
2 public:
3     // virtual ~Base() {} // Correct
4     ~Base() {} // Dangerous!
5 };
6
7 class Derived : public Base {
8     int* ptr;
9 public:
10     Derived() { ptr = new int[100]; }
11     ~Derived() { delete[] ptr; }
12 };
13

```

```
14 Base* b = new Derived();  
15 delete b; // If ~Base is not virtual, ~Derived is NOT called! Leak!
```

2. Covariant Return Types

An override can return a pointer/reference to a *derived* class, not just the base.

```
1 class Shape {  
2 public:  
3     virtual Shape* clone() = 0;  
4 };  
5  
6 class Circle : public Shape {  
7 public:  
8     // Returns Circle* instead of Shape* - Valid!  
9     Circle* clone() override { return new Circle(*this); }  
10};
```

3. RTTI & dynamic_cast

Run-Time Type Information allows safe downcasting. It uses the `vptr` to check the actual type.

```
1 Shape* s = new Circle(5);  
2  
3 // Safe cast: returns nullptr if s is not a Circle  
4 if (Circle* c = dynamic_cast<Circle*>(s)) {  
5     c->special_circle_method();  
6 } else {  
7     std::cout << "Not a circle\n";  
8 }
```

4. Static Polymorphism (CRTP)

Curiously Recurring Template Pattern. Faster than virtual functions (compile-time resolution).

```
1 template<typename Derived>  
2 class Shape {  
3 public:  
4     void draw() {  
5         // Compile-time dispatch  
6         static_cast<Derived*>(this)->draw_impl();  
7     }  
8 };  
9  
10 class Circle : public Shape<Circle> {  
11 public:  
12     void draw_impl() { cout << "Circle\n"; }  
13};
```

Chapter 21

DEEP OBJECT MODEL & VIRTUALIZATION

Understanding the “C++ Object Model” distinguishes a user from a master. This section explains what the compiler generates for your classes.

21.0.1 2.5.1 The Cost of Polymorphism (vptr & vtable)

Every class with *at least one* virtual function has a hidden overhead.

1. **vptr (Virtual Pointer)**: A hidden member added to the *layout* of the object.
2. **vtable (Virtual Table)**: A static table of function pointers for that class.

```
1 class Base {  
2     int data;  
3     virtual void func() {}  
4 };  
5 // sizeof(Base) = sizeof(int) + sizeof(void*) + padding  
6 // On 64-bit: 4 bytes (int) + 4 bytes (padding) + 8 bytes (ptr) = 16 bytes
```

21.0.2 2.5.2 Multiple Inheritance & Thunks

When inheriting from multiple classes, pointer arithmetic gets tricky.

```
1 class A { int a; virtual void f() {} };  
2 class B { int b; virtual void g() {} };  
3 class C : public A, public B { int c; };  
4  
5 C obj;  
6 A* pa = &obj; // Points to start of obj  
7 B* pb = &obj; // Points to obj + sizeof(A) !!
```

- **Thunk**: A small piece of assembly code generated by the compiler to adjust the `this` pointer when calling a virtual function from a base class pointer that isn't at offset 0.

21.0.3 2.5.3 Virtual Inheritance (The Diamond Problem)

```
1 class Top { int t; };  
2 class Left : virtual public Top { int l; };  
3 class Right : virtual public Top { int r; };  
4 class Bottom : public Left, public Right { int b; };
```

To solve the Diamond Problem (Top appearing twice), `virtual` inheritance ensures `Top` is shared. * **Cost:** `Left` and `Right` now contain a **vbptr** (Virtual Base Pointer) pointing to the shared `Top` instance. Accessing members of `Top` becomes slower (indirection).

21.0.4 2.5.4 Alignment & Padding Rules

CPU reads are efficient at specific addresses (multiples of 4 or 8). Compilers insert “padding bytes”.

Rule: A member of size N must sit at an offset divisible by N .

```
1 struct Mixed {  
2     char a;        // 1 byte  
3                     // 3 bytes PADDING  
4     int b;         // 4 bytes  
5     short c;       // 2 bytes  
6                     // 6 bytes PADDING (to align structure size to 8)  
7 };  
8 // sizeof(Mixed) = 16 (on 64-bit)
```

Optimization: Sort members by size (Largest to Smallest) to minimize padding.

Chapter 22

C++98/03 STANDARD LIBRARY

22.1 Standard Template Library

22.2 Introduction to STL

The Standard Template Library (STL) is a collection of template classes and functions that provide: - **Containers** - Data structures to hold objects - **Iterators** - Objects to traverse containers - **Algorithms** - Functions to manipulate data - **Function Objects** - Objects that act like functions

22.2.1 Key Advantages

- Generic programming (templates)
 - High performance (optimized)
 - Code reuse
 - Type-safe
 - Well-tested and standardized
-

22.3 STL Components Overview

```
1      STL (Standard Template Library)
2
3
4
5      CONTAINERS          ITERATORS
6
7      Sequence            Input
8      Associative         Output
9      Adapters            Forward
10     Bidirectional|      |
11                   Random Access
12
13     --
14     ALGORITHMS
15     FUNCTION OBJ.
16     Searching
```

16	Sorting	Predicates
17	Modifying	Comparators
18	Numeric	Functors

22.4 CONTAINERS - COMPLETE REFERENCE

22.5 Container Characteristics

Container	Type	Insert	Delete	Search	Random Access	Memory
vector	Sequence	O(n)	O(n)	O(n)	O(1)	Contiguous
list	Sequence	O(1)	O(1)	O(n)	O(n)	Scattered
deque	Sequence	O(n)	O(n)	O(n)	O(1)	Blocks
map	Associative	O(log n)	O(log n)	O(log n)	-	Tree
set	Associative	O(log n)	O(log n)	O(log n)	-	Tree
multimap	Associative	O(log n)	O(log n)	O(log n)	-	Tree
multiset	Associative	O(log n)	O(log n)	O(log n)	-	Tree
unordered_map	Hash	O(1) avg	O(1) avg	O(1) avg	-	Hash
unordered_set	Hash	O(1) avg	O(1) avg	O(1) avg	-	Hash
priority_queue	Adapter	O(log n)	O(log n)	O(n)	-	Heap
queue	Adapter	O(1)	O(1)	O(n)	-	-
stack	Adapter	O(1)	O(1)	O(n)	-	-

22.6 1.1 VECTOR - Dynamic Array

22.6.1 What is Vector?

A dynamic array that grows automatically. Use this most of the time.

22.6.2 Declaration & Initialization

```

1 #include <vector>
2 using namespace std;
3
4 // Empty vector
5 vector<int> v1;
6
7 // Vector with initial size
8 vector<int> v2(10);           // 10 elements, initialized to 0
9
10 // Vector with initial values
11 vector<int> v3(5, 10);        // 5 elements, all set to 10
12
13 // Copy constructor
14 vector<int> v4(v3);           // Copy of v3
15
16 // From array (C++11)
17 int arr[] = {1, 2, 3, 4, 5};
18 vector<int> v5(arr, arr + 5); // From array
19

```

```
20 // Initializer list (C++11)
21 vector<int> v6 = {1, 2, 3, 4, 5};
22
23 // Deduction (C++17)
24 vector v7 = {1, 2, 3};           // Type deduced as vector<int>
```

22.6.3 Accessing Elements

```
1 vector<int> v = {10, 20, 30, 40, 50};
2
3 // 1. Using operator[]
4 cout << v[0] << "\n";           // 10 - No bounds checking
5
6 // 2. Using at()
7 cout << v.at(0) << "\n";        // 10 - With bounds checking, throws exception
8
9 // 3. Front and back
10 cout << v.front() << "\n";      // 10 (first element)
11 cout << v.back() << "\n";       // 50 (last element)
12
13 // 4. Direct pointer access (C++11)
14 int* ptr = v.data();             // Pointer to underlying array
15 cout << ptr[0] << "\n";         // 10
```

22.6.4 Modifying Elements

```
1 vector<int> v = {10, 20, 30};
2
3 // Assignment
4 v[0] = 100;
5 v.at(1) = 200;
6
7 // Adding elements
8 v.push_back(40);                 // Add to end: {10, 200, 30, 40}
9 v.insert(v.begin() + 1, 15);     // Insert 15 at index 1: {10, 15, 200, 30, 40}
10
11 // Removing elements
12 v.pop_back();                   // Remove last: {10, 15, 200, 30}
13 v.erase(v.begin() + 1);         // Remove at index 1: {10, 200, 30}
14 v.erase(v.begin(), v.begin() + 2); // Remove first 2: {30}
15
16 // Clear all
17 v.clear();                      // Empty vector
```

22.6.5 Size & Capacity

```
1 vector<int> v = {1, 2, 3};
2
3 cout << v.size() << "\n";       // 3 - Number of elements
4 cout << v.capacity() << "\n";   // 3+ - Allocated space
5 cout << v.empty() << "\n";     // false
6
7 // Reserve space (optimization)
8 v.reserve(100);                 // Allocate for 100 elements
9 cout << v.capacity() << "\n";   // 100
10
11 // Resize
12 v.resize(5, 0);                 // Resize to 5, new elements = 0
```

```

13 v.resize(2); // Shrink to 2 elements
14
15 // Shrink to fit (C++11)
16 v.shrink_to_fit(); // Release unused capacity

```

22.6.6 Iterating Through Vector

```

1 vector<int> v = {10, 20, 30, 40, 50};
2
3 // 1. Traditional for loop
4 for (int i = 0; i < v.size(); i++) {
5     cout << v[i] << " ";
6 }
7
8 // 2. Iterator loop
9 for (vector<int>::iterator it = v.begin(); it != v.end(); ++it) {
10     cout << *it << " ";
11 }
12
13 // 3. Auto with iterator (C++11)
14 for (auto it = v.begin(); it != v.end(); ++it) {
15     cout << *it << " ";
16 }
17
18 // 4. Range-based for (C++11)
19 for (int val : v) {
20     cout << val << " ";
21 }
22
23 // 5. Reverse iteration
24 for (auto it = v.rbegin(); it != v.rend(); ++it) {
25     cout << *it << " ";
26 }

```

22.6.7 Comparison & Assignment

```

1 vector<int> v1 = {1, 2, 3};
2 vector<int> v2 = {1, 2, 3};
3 vector<int> v3 = {1, 2, 4};
4
5 cout << (v1 == v2) << "\n"; // 1 (true)
6 cout << (v1 != v3) << "\n"; // 1 (true)
7 cout << (v1 < v3) << "\n"; // 1 (true) - lexicographic
8
9 // Assignment
10 v1 = v3; // Copy v3 to v1
11 v1.swap(v2); // Swap v1 and v2
12
13 // Swap single elements
14 swap(v1[0], v1[1]);

```

22.6.8 Complete Vector Example

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int main() {

```

```

6     vector<int> nums;
7
8     // Adding elements
9     for (int i = 1; i <= 5; i++) {
10         nums.push_back(i * 10);
11     }
12
13     // Display
14     cout << "Vector contents: ";
15     for (int num : nums) {
16         cout << num << " ";
17     }
18     cout << "\n";
19
20     // Modify
21     nums[2] = 999;
22
23     // Insert
24     nums.insert(nums.begin() + 2, 777);
25
26     // Remove
27     nums.erase(nums.begin() + 1);
28
29     // Display after modifications
30     cout << "After modifications: ";
31     for (int num : nums) {
32         cout << num << " ";
33     }
34     cout << "\nSize: " << nums.size() << "\n";
35
36     return 0;
37 }

```

22.7 1.2 DEQUE - Double Ended Queue

22.7.1 What is Deque?

Fast insertion/deletion at both ends. Like vector but with efficient operations on both sides.

```

1 #include <deque>
2 using namespace std;
3
4 // Declaration
5 deque<int> dq;
6
7 // Adding elements
8 dq.push_back(10);           // Add to end
9 dq.push_front(5);          // Add to front: {5, 10}
10
11 // Removing elements
12 dq.pop_back();              // Remove from end: {5}
13 dq.pop_front();             // Remove from front: {}
14
15 // Accessing
16 dq.push_back(20);
17 dq.push_back(30);
18 cout << dq.front() << "\n"; // 20
19 cout << dq.back() << "\n";  // 30
20 cout << dq[0] << "\n";      // 20 - supports random access

```

```

21
22 // Iteration
23 for (int val : dq) {
24     cout << val << " ";
25 }
26
27 // Size operations
28 cout << dq.size() << "\n";
29 cout << dq.empty() << "\n";
30
31 // Clearing
32 dq.clear();

```

22.7.2 Deque vs Vector

- **Deque:** Better for push_front/pop_front operations
 - **Vector:** Better for push_back/pop_back and random access
-

22.8 1.3 LIST - Doubly Linked List

22.8.1 What is List?

Efficient insertion/deletion anywhere. No random access.

```

1 #include <list>
2 using namespace std;
3
4 // Declaration
5 list<int> lst;
6
7 // Adding elements
8 lst.push_back(10);           // Add to end
9 lst.push_front(5);           // Add to front: {5, 10}
10
11 // Insert at position
12 auto it = lst.begin();
13 ++it;
14 lst.insert(it, 7);           // Insert 7 at position 1: {5, 7, 10}
15
16 // Removing elements
17 lst.pop_back();              // Remove from end
18 lst.pop_front();             // Remove from front
19 lst.erase(it);               // Remove at iterator
20 lst.remove(5);                // Remove all elements with value 5
21 lst.clear();                  // Clear all
22
23 // Accessing
24 cout << lst.front() << "\n"; // First element
25 cout << lst.back() << "\n";  // Last element
26 // NO random access: lst[0] - NOT AVAILABLE
27
28 // Iteration
29 for (int val : lst) {
30     cout << val << " ";
31 }
32
33 // Reverse iteration
34 for (auto it = lst.rbegin(); it != lst.rend(); ++it) {

```

```

35     cout << *it << " ";
36 }
37
38 // Size
39 cout << lst.size() << "\n";
40
41 // Useful operations
42 lst.reverse();           // Reverse the list
43 lst.sort();             // Sort the list
44 lst.unique();           // Remove consecutive duplicates
45 lst.sort(greater<int>()); // Sort in descending order

```

22.8.2 List-Specific Operations

```

1 list<int> lst1 = {1, 2, 3};
2 list<int> lst2 = {4, 5, 6};
3
4 // Splice - move elements from one list to another
5 lst1.splice(lst1.end(), lst2); // Append lst2 to lst1
6 // lst1: {1, 2, 3, 4, 5, 6}
7 // lst2: {} (empty)
8
9 // Merge - combine two sorted lists
10 list<int> a = {1, 3, 5};
11 list<int> b = {2, 4, 6};
12 a.merge(b); // a: {1, 2, 3, 4, 5, 6}, b: {}
13
14 // Remove if
15 list<int> nums = {1, 2, 3, 4, 5};
16 nums.remove_if([](int x) { return x % 2 == 0; }); // Remove even numbers
17 // nums: {1, 3, 5}

```

22.9 1.4 MAP - Sorted Key-Value Pairs

22.9.1 What is Map?

Stores key-value pairs, sorted by key. Logarithmic operations.

```

1 #include <map>
2 using namespace std;
3
4 // Declaration
5 map<string, int> ages;
6
7 // Insertion
8 ages["Alice"] = 30;
9 ages["Bob"] = 25;
10 ages["Carol"] = 28;
11 ages.insert({"David", 32});
12 ages.insert({"Eve", 27});
13
14 // Accessing
15 cout << ages["Alice"] << "\n"; // 30
16 cout << ages.at("Bob") << "\n"; // 25 - with bounds checking
17
18 // Check existence
19 if (ages.find("Alice") != ages.end()) {
20     cout << "Alice found\n";

```

```

21 }
22
23 // Safe access with count
24 if (ages.count("Bob")) {
25     cout << "Bob exists\n";
26 }
27
28 // Size
29 cout << ages.size() << "\n";
30
31 // Iteration
32 for (auto& pair : ages) {
33     cout << pair.first << ": " << pair.second << "\n";
34 }
35
36 // With auto (C++11)
37 for (const auto& [name, age] : ages) { // Structured binding (C++17)
38     cout << name << ": " << age << "\n";
39 }
40
41 // Reverse iteration
42 for (auto it = ages.rbegin(); it != ages.rend(); ++it) {
43     cout << it->first << ": " << it->second << "\n";
44 }

```

22.9.2 Map Operations

```

1 map<int, string> dict;
2
3 // Insert
4 dict[1] = "one";
5 dict[2] = "two";
6 dict[3] = "three";
7
8 // Erase
9 dict.erase(2); // Erase by key
10 dict.erase(dict.begin()); // Erase by iterator
11
12 // Find
13 auto it = dict.find(1);
14 if (it != dict.end()) {
15     cout << it->second << "\n";
16 }
17
18 // Lower and upper bound
19 map<int, string> scores = {{1, "A"}, {2, "B"}, {3, "C"}};
20
21 // find first >= key
22 auto it1 = scores.lower_bound(2); // Points to {2, "B"}
23
24 // Find first > key
25 auto it2 = scores.upper_bound(2); // Points to {3, "C"}
26
27 // Range [lower_bound, upper_bound)
28 auto range = scores.equal_range(2); // Pair of iterators
29 for (auto it = range.first; it != range.second; ++it) {
30     cout << it->second << "\n";
31 }
32
33 // Clear
34 dict.clear();

```


22.9.3 Map with Custom Comparator

```

1 // Descending order
2 map<int, string, greater<int>> descending;
3 descending[3] = "three";
4 descending[1] = "one";
5 descending[2] = "two";
6 // Iteration order: 3, 2, 1
7
8 // Custom comparator
9 struct Compare {
10     bool operator()(const string& a, const string& b) const {
11         return a.length() < b.length(); // Sort by string length
12     }
13 };
14
15 map<string, int, Compare> byLength;
16 byLength["a"] = 1;
17 byLength["abc"] = 3;
18 byLength["ab"] = 2;

```

22.10 1.5 SET - Sorted Unique Elements

22.10.1 What is Set?

Stores unique, sorted elements. Like map but key only (no value).

```

1 #include <set>
2 using namespace std;
3
4 // Declaration
5 set<int> nums;
6
7 // Insertion
8 nums.insert(30);
9 nums.insert(10);
10 nums.insert(20);
11 nums.insert(10); // Duplicate - ignored
12 // {10, 20, 30}
13
14 // Accessing
15 auto it = nums.find(20);
16 if (it != nums.end()) {
17     cout << "Found: " << *it << "\n";
18 }
19
20 // Count
21 cout << nums.count(20) << "\n"; // 1 or 0
22
23 // Iteration
24 for (int val : nums) {
25     cout << val << " ";
26 }
27
28 // Erase
29 nums.erase(20); // Erase by value
30 nums.erase(nums.begin()); // Erase by iterator
31 nums.erase(nums.lower_bound(15), nums.upper_bound(25)); // Range erase
32

```

```
33 // Size and empty
34 cout << nums.size() << "\n";
35 cout << nums.empty() << "\n";
36
37 // Clear
38 nums.clear();
```

22.10.2 Set with Strings

```
1 set<string> words;
2
3 words.insert("zebra");
4 words.insert("apple");
5 words.insert("mango");
6 words.insert("apple"); // Duplicate ignored
7
8 // Prints in alphabetical order
9 for (const string& word : words) {
10     cout << word << "\n";
11 }
12 // Output: apple, mango, zebra
```

22.10.3 Multiset - Allows Duplicates

```
1 #include <set>
2
3 multiset<int> nums;
4
5 nums.insert(10);
6 nums.insert(20);
7 nums.insert(10);
8 nums.insert(20);
9 // {10, 10, 20, 20}
10
11 cout << nums.count(10) << "\n"; // 2
12
13 // All operations similar to set
14 for (int val : nums) {
15     cout << val << " ";
16 }
```

22.11 1.6 MULTIMAP - Key-Value Pairs with Duplicate Keys

```
1 #include <map>
2
3 multimap<string, int> students;
4
5 // Multiple values for same key
6 students.insert({"Math", 95});
7 students.insert({"Math", 87});
8 students.insert({"English", 92});
9 students.insert({"English", 88});
10
11 // Find all with key "Math"
12 auto range = students.equal_range("Math");
```

```

13 for (auto it = range.first; it != range.second; ++it) {
14     cout << it->first << ": " << it->second << "\n";
15 }
16 // Output: Math: 95, Math: 87
17
18 // Count how many with key "Math"
19 cout << students.count("Math") << "\n"; // 2
20
21 // Iterate all
22 for (const auto& [subject, score] : students) {
23     cout << subject << ": " << score << "\n";
24 }

```

22.12 1.7 UNORDERED_MAP - Hash-Based Key-Value Pairs

22.12.1 What is Unordered Map?

Like map but no sorting, $O(1)$ average operations.

```

1 #include <unordered_map>
2
3 // Declaration
4 unordered_map<string, int> ages;
5
6 // Insertion - same as map
7 ages["Alice"] = 30;
8 ages["Bob"] = 25;
9
10 // Accessing - same as map
11 cout << ages["Alice"] << "\n";
12
13 // Find
14 if (ages.find("Bob") != ages.end()) {
15     cout << "Found Bob\n";
16 }
17
18 // Erase
19 ages.erase("Alice");
20
21 // Iteration - ORDER IS ARBITRARY
22 for (const auto& [name, age] : ages) {
23     cout << name << ": " << age << "\n";
24 }
25
26 // Size
27 cout << ages.size() << "\n";
28
29 // Bucket information
30 cout << ages.bucket_count() << "\n"; // Number of buckets
31 cout << ages.load_factor() << "\n"; // Load factor
32 cout << ages.max_load_factor() << "\n"; // Max load factor
33
34 // Rehash
35 ages.rehash(100); // Rehash with hint 100
36 ages.reserve(50); // Reserve space for 50 elements
37
38 // Clear
39 ages.clear();

```

22.12.2 Unordered Set

```

1 #include <unordered_set>
2
3 unordered_set<int> nums = {30, 10, 20};
4
5 // Similar to set but unordered
6 if (nums.count(10)) {
7     cout << "Found 10\n";
8 }
9
10 for (int val : nums) {
11     cout << val << " "; // Order is arbitrary
12 }

```

22.13 1.8 QUEUE - FIFO (First In, First Out)

22.13.1 What is Queue?

Adapter container. Elements added at back, removed from front.

```

1 #include <queue>
2
3 queue<int> q;
4
5 // Adding (enqueue)
6 q.push(10);
7 q.push(20);
8 q.push(30);
9
10 // Size and check empty
11 cout << q.size() << "\n";
12 cout << q.empty() << "\n";
13
14 // Access front and back
15 cout << q.front() << "\n"; // 10 (first element)
16 cout << q.back() << "\n"; // 30 (last element)
17
18 // Removing (dequeue)
19 q.pop(); // Removes 10
20
21 // Typical queue pattern
22 while (!q.empty()) {
23     cout << q.front() << " ";
24     q.pop();
25 }

```

22.13.2 Queue Example - Task Processing

```

1 queue<string> taskQueue;
2
3 taskQueue.push("Task 1");
4 taskQueue.push("Task 2");
5 taskQueue.push("Task 3");
6
7 while (!taskQueue.empty()) {
8     cout << "Processing: " << taskQueue.front() << "\n";

```

```

9     taskQueue.pop();
10 }
11 // Output: Task 1, Task 2, Task 3

```

22.14 1.9 STACK - LIFO (Last In, First Out)

22.14.1 What is Stack?

Adapter container. Elements added and removed from top.

```

1 #include <stack>
2
3 stack<int> st;
4
5 // Adding (push)
6 st.push(10);
7 st.push(20);
8 st.push(30);
9
10 // Size and check empty
11 cout << st.size() << "\n";
12 cout << st.empty() << "\n";
13
14 // Access top
15 cout << st.top() << "\n"; // 30 (last added)
16
17 // Removing (pop)
18 st.pop(); // Removes 30
19
20 // Typical stack pattern
21 while (!st.empty()) {
22     cout << st.top() << " ";
23     st.pop();
24 }
25 // Output: 20 10

```

22.14.2 Stack Example - Balanced Parentheses

```

1 #include <stack>
2 #include <string>
3
4 bool isBalanced(string expr) {
5     stack<char> st;
6
7     for (char c : expr) {
8         if (c == '(' || c == '[' || c == '{') {
9             st.push(c);
10        } else if (c == ')' || c == ']' || c == '}') {
11            if (st.empty()) return false;
12
13            char top = st.top();
14            if ((c == ')' && top == '(') ||
15                (c == ']' && top == '[') ||
16                (c == '}' && top == '{')) {
17                st.pop();
18            } else {
19                return false;
20            }
21        }
22    }
23    return st.empty();
24 }

```

```

21     }
22 }
23
24 return st.empty();
25 }

```

22.15 1.10 PRIORITY_QUEUE - Heap-Based Container

22.15.1 What is Priority Queue?

Elements removed in order of priority (max/min).

```

1 #include <queue>
2
3 // Max heap (largest element has highest priority)
4 priority_queue<int> pq;
5
6 pq.push(10);
7 pq.push(30);
8 pq.push(20);
9
10 while (!pq.empty()) {
11     cout << pq.top() << " "; // Largest first
12     pq.pop();
13 }
14 // Output: 30 20 10
15
16 // Min heap (smallest element has highest priority)
17 priority_queue<int, vector<int>, greater<int>> minPQ;
18
19 minPQ.push(10);
20 minPQ.push(30);
21 minPQ.push(20);
22
23 while (!minPQ.empty()) {
24     cout << minPQ.top() << " "; // Smallest first
25     minPQ.pop();
26 }
27 // Output: 10 20 30

```

22.15.2 Priority Queue with Custom Comparator

```

1 struct Task {
2     string name;
3     int priority;
4
5     // For priority_queue to work
6     bool operator<(const Task& other) const {
7         return priority < other.priority; // Max heap on priority
8     }
9 };
10
11 priority_queue<Task> tasks;
12
13 tasks.push({"Task A", 5});
14 tasks.push({"Task B", 10});
15 tasks.push({"Task C", 3});
16

```

```

17 while (!tasks.empty()) {
18     cout << tasks.top().name << " (P: " << tasks.top().priority << ")\n";
19     tasks.pop();
20 }
21 // Output: Task B (P: 10), Task A (P: 5), Task C (P: 3)

```

22.16 ITERATORS - DEEP DIVE

22.17 Iterator Categories

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28

```

	Iterator		
	Single pass		
	Basic operations		
	Input	Output	Forward
	Read	Write	Read+Write
	++, *	++, =	All ops
		Bidirectional	
		++, --, *, =	
		Random Access	
		All ops + []	

22.17.1 Iterator Types

```

1 #include <vector>
2 #include <list>
3 #include <map>
4
5 vector<int> vec;           // Random access
6 list<int> lst;             // Bidirectional
7 map<int, int> mp;          // Bidirectional
8 deque<int> dq;            // Random access
9 set<int> st;              // Bidirectional
10
11 // Declaring iterators
12 vector<int>::iterator it1; // Random access
13 list<int>::iterator it2;   // Bidirectional
14 map<int, int>::iterator it3; // Bidirectional
15 const vector<int>::iterator it4; // Const iterator

```

```

16
17 // Auto (C++11)
18 auto it = vec.begin();           // Type deduced

```

22.17.2 Iterator Operations

```

1 vector<int> v = {10, 20, 30, 40, 50};
2
3 auto it = v.begin();
4
5 // Navigation
6 ++it;           // Move forward
7 --it;           // Move backward (if bidirectional)
8 it++;           // Post-increment
9 it--;           // Post-decrement
10
11 // Dereference
12 cout << *it << "\n";    // Get value
13
14 // Arithmetic (random access only)
15 it = it + 3;       // Move 3 positions forward
16 it = it - 2;       // Move 2 positions backward
17 it += 5;
18 it -= 3;
19
20 // Comparison
21 if (it == v.end()) {}    // Check equality
22 if (it != v.end()) {}    // Check inequality
23 if (it < v.end()) {}     // Less than (random access)
24
25 // Distance
26 int dist = distance(v.begin(), it);
27
28 // Advance
29 advance(it, 5);          // Move 5 positions forward

```

22.17.3 Reverse Iterators

```

1 vector<int> v = {10, 20, 30, 40, 50};
2
3 // Reverse iteration
4 for (auto it = v.rbegin(); it != v.rend(); ++it) {
5     cout << *it << " ";    // 50 40 30 20 10
6 }
7
8 // Reverse iteration with auto
9 for (auto val : v) {
10     cout << val << " ";    // 10 20 30 40 50
11 }
12
13 // Reverse iteration (explicit)
14 for (int i = v.size() - 1; i >= 0; --i) {
15     cout << v[i] << " ";    // 50 40 30 20 10
16 }

```

22.17.4 Const Iterators


```

1  const vector<int> v = {10, 20, 30};
2
3  // Const iterator
4  auto it = v.begin();           // const_iterator
5  cout << *it << "\n";         // OK - read
6  // *it = 100;                 // Error - can't modify
7
8  // Explicit const iterator
9  const_iterator cit = v.cbegin();
10
11 // Reverse const iterator
12 auto rit = v.crbegin();
13
14 // Using const_iterator with non-const vector
15 vector<int> v2 = {1, 2, 3};
16 const_iterator it2 = v2.cbegin();

```

```
const_iterator it2 = v2.cbegin();
```

```

1  ### Advanced Iterator Concepts
2
3  #### 1. Iterator Traits (std::iterator_traits)
4  Algorithms use 'iterator_traits' to know what an iterator can do.
5
6  '''cpp
7  #include <iterator>
8
9  template<typename Iter>
10 void my_advance(Iter& it, int n) {
11     using category = typename std::iterator_traits<Iter>::iterator_category;
12
13     if constexpr (std::is_base_of_v<std::random_access_iterator_tag, category>)
14     {
15         it += n; // O(1)
16     } else {
17         while (n-- > 0) ++it; // O(N)
18     }
19 }

```

2. Writing a Custom Iterator

To make a class compatible with STL algorithms (like `std::find`), you need a conformant iterator.

```

1  class Integers {
2      struct Iterator {
3          using iterator_category = std::forward_iterator_tag;
4          using difference_type   = std::ptrdiff_t;
5          using value_type        = int;
6          using pointer            = int*;
7          using reference          = int&;
8
9          int value;
10         Iterator(int v) : value(v) {}
11
12         reference operator*() { return value; }
13         pointer operator->() { return &value; }
14
15         Iterator& operator++() { value++; return *this; }
16         Iterator operator++(int) { Iterator tmp = *this; ++(*this); return tmp; }
17     }
18 }

```

```

18     friend bool operator== (const Iterator& a, const Iterator& b) { return
    a.value == b.value; };
19     friend bool operator!= (const Iterator& a, const Iterator& b) { return
    a.value != b.value; };
20 };
21
22 public:
23     Iterator begin() { return Iterator(0); }
24     Iterator end()   { return Iterator(10); } // Range [0, 10)
25 };

```

3. Stream Iterators

Treat IO streams as containers.

```

1 #include <iterator>
2 #include <algorithm>
3
4 // Read ints from cin until EOF or invalid input
5 std::istream_iterator<int> input_it(std::cin);
6 std::istream_iterator<int> eos;
7
8 // Write ints to cout with ", " delimiter
9 std::ostream_iterator<int> output_it(std::cout, ", ");
10
11 std::copy(input_it, eos, output_it);

```

4. Insert Iterators

Special output iterators that grow the container.

- `std::back_inserter(c)`: Calls `c.push_back(val)`. (Vector, List, Deque)
- `std::front_inserter(c)`: Calls `c.push_front(val)`. (List, Deque)
- `std::inserter(c, it)`: Calls `c.insert(it, val)`. (Map, Set, List, Vector)

```

1 std::vector<int> v;
2 std::fill_n(std::back_inserter(v), 5, 42); // v becomes {42, 42, 42, 42, 42}

```

Chapter 23

C++ STL Advanced - Extended Reference & Algorithms Library

23.1 COMPREHENSIVE STL ALGORITHMS REFERENCE

23.1.1 All Algorithms by Category (60+ Algorithms)

23.2 NON-MODIFYING SEQUENCE ALGORITHMS

23.2.1 1. find Family

```
1 #include <algorithm>
2
3 auto it = find(first, last, value);           // Find element
4 auto it = find_if(first, last, predicate);    // Find matching condition
5 auto it = find_if_not(first, last, predicate); // Find non-matching (C++11)
```

23.2.2 2. count Family

```
1 int n = count(first, last, value);           // Count occurrences
2 int n = count_if(first, last, predicate);    // Count matching
```

23.2.3 3. mismatch

```
1 auto [it1, it2] = mismatch(first1, last1, first2); // Find first difference
2 auto [it1, it2] = mismatch(first1, last1, first2, comp); // With comparator
```

23.2.4 4. equal

```
1 bool eq = equal(first1, last1, first2);       // Compare ranges
2 bool eq = equal(first1, last1, first2, comp); // With comparator
```

23.2.5 5. search & adjacent_find

```

1 auto it = search(first, last, s_first, s_last); // Find subsequence
2 auto it = search_n(first, last, count, value); // Find N equal elements
3 auto it = adjacent_find(first, last);           // Find adjacent equal
  elements
4 auto it = adjacent_find(first, last, comp);      // With comparator

```

23.2.6 6. Logical Operations

```

1 bool b = all_of(first, last, predicate); // All match
2 bool b = any_of(first, last, predicate); // Any matches
3 bool b = none_of(first, last, predicate); // None match

```

23.2.7 7. Min/Max

```

1 auto it = min_element(first, last); // Find minimum
2 auto it = max_element(first, last); // Find maximum
3 auto [minIt, maxIt] = minmax_element(first, last); // Both (C++11)
4
5 auto it = min_element(first, last, comp); // With comparator
6 auto it = max_element(first, last, comp);
7 auto [minIt, maxIt] = minmax_element(first, last, comp);

```

23.3 MODIFYING SEQUENCE ALGORITHMS

23.3.1 1. copy Family

```

1 copy(first, last, d_first); // Copy range
2 copy_n(first, count, d_first); // Copy N elements
3 copy_if(first, last, d_first, predicate); // Conditional copy
4 copy_backward(first, last, d_last); // Copy backwards

```

23.3.2 2. move (C++11)

```

1 move(first, last, d_first); // Move range
2 move_backward(first, last, d_last); // Move backwards

```

23.3.3 3. transform

```

1 transform(first, last, d_first, op); // Apply function
2 transform(first1, last1, first2, d_first, op); // Apply to two ranges

```

23.3.4 4. fill & generate

```

1 fill(first, last, value); // Fill with value
2 fill_n(first, count, value); // Fill N elements
3 generate(first, last, gen); // Generate values
4 generate_n(first, count, gen); // Generate N values

```

23.3.5 5. replace

```

1 replace(first, last, old_value, new_value);           // Replace values
2 replace_if(first, last, predicate, new_value);       // Conditional replace
3 replace_copy(first, last, d_first, old, new);        // Copy with replace
4 replace_copy_if(first, last, d_first, pred, new);    // Conditional copy-replace

```

23.3.6 6. swap & reverse

```

1 swap(a, b);                                           // Swap two values
2 iter_swap(it1, it2);                                 // Swap via iterators
3 reverse(first, last);                                // Reverse range
4 reverse_copy(first, last, d_first);                  // Copy reversed

```

23.3.7 7. rotate

```

1 rotate(first, middle, last);                         // Rotate range
2 rotate_copy(first, middle, last, d_first);           // Copy rotated

```

23.3.8 8. unique

```

1 auto it = unique(first, last);                       // Remove consecutive
   duplicates
2 auto it = unique(first, last, comp);                 // With comparator
3 auto it = unique_copy(first, last, d_first);        // Copy unique
4 auto it = unique_copy(first, last, d_first, comp);

```

23.3.9 9. shuffle & random

```

1 shuffle(first, last, rng);                          // Random shuffle
2 random_shuffle(first, last);                        // Legacy shuffle
3 random_shuffle(first, last, randFunc);              // With random function

```

23.3.10 10. remove

```

1 auto it = remove(first, last, value);               // Remove all matching
2 auto it = remove_if(first, last, predicate);        // Conditional remove
3 auto it = remove_copy(first, last, d_first, val);   // Copy without matching
4 auto it = remove_copy_if(first, last, d_first, pred);

```

23.4 SORTING & PARTITIONING ALGORITHMS

23.4.1 Sorting

```

1 sort(first, last);                                  // Sort (introsort)
2 sort(first, last, comp);                           // With comparator
3 stable_sort(first, last);                          // Stable sort
4 stable_sort(first, last, comp);
5
6 partial_sort(first, middle, last);                 // Sort first part

```

```

7 partial_sort(first, middle, last, comp);
8 partial_sort_copy(first, last, d_first, d_last); // Copy partially sorted
9
10 nth_element(first, nth, last); // Sort around nth
11 nth_element(first, nth, last, comp);

```

23.4.2 Partitioning

```

1 auto it = partition(first, last, predicate); // Partition
2 auto it = stable_partition(first, last, pred); // Stable partition
3 auto it = partition_copy(first, last, d_true, d_false, pred); // Copy
  partitions
4
5 bool b = is_partitioned(first, last, predicate); // Check if partitioned
6 auto it = partition_point(first, last, predicate); // Find partition point

```

23.4.3 Binary Search (requires sorted range)

```

1 auto it = lower_bound(first, last, value); // First >= value
2 auto it = upper_bound(first, last, value); // First > value
3 auto [lo, hi] = equal_range(first, last, value); // Range of value
4 bool b = binary_search(first, last, value); // Check existence
5
6 auto it = lower_bound(first, last, value, comp);
7 auto it = upper_bound(first, last, value, comp);
8 auto [lo, hi] = equal_range(first, last, value, comp);
9 bool b = binary_search(first, last, value, comp);

```

23.5 NUMERIC ALGORITHMS

```

1 #include <numeric>
2
3 // Accumulation
4 int sum = accumulate(first, last, init); // Sum
5 auto prod = accumulate(first, last, init, op); // Custom operation
6
7 // Inner product (dot product)
8 int dot = inner_product(first1, last1, first2, init);
9 auto result = inner_product(first1, last1, first2, init, op1, op2);
10
11 // Partial sums
12 partial_sum(first, last, d_first); // Cumulative sum
13 partial_sum(first, last, d_first, op); // Custom operation
14
15 // Adjacent differences
16 adjacent_difference(first, last, d_first); // Differences
17 adjacent_difference(first, last, d_first, op); // Custom operation

```

23.6 SET OPERATIONS (require sorted ranges)

```

1 // Union - all unique elements
2 auto it = set_union(first1, last1, first2, last2, d_first);
3 auto it = set_union(first1, last1, first2, last2, d_first, comp);
4
5 // Intersection - common elements
6 auto it = set_intersection(first1, last1, first2, last2, d_first);
7 auto it = set_intersection(first1, last1, first2, last2, d_first, comp);
8
9 // Difference - in first but not in second
10 auto it = set_difference(first1, last1, first2, last2, d_first);
11 auto it = set_difference(first1, last1, first2, last2, d_first, comp);
12
13 // Symmetric difference - in one but not both
14 auto it = set_symmetric_difference(first1, last1, first2, last2, d_first);
15 auto it = set_symmetric_difference(first1, last1, first2, last2, d_first, comp)
16     ;
17
18 // Check relationship
19 bool b = includes(first1, last1, first2, last2); // first1 includes first2
20 bool b = includes(first1, last1, first2, last2, comp);

```

23.7 HEAP OPERATIONS

```

1 #include <algorithm>
2
3 make_heap(first, last); // Create heap
4 make_heap(first, last, comp); // With comparator
5
6 push_heap(first, last); // Add element to heap
7 pop_heap(first, last); // Extract max from heap
8 sort_heap(first, last); // Sort heap
9
10 bool b = is_heap(first, last); // Check if valid heap
11 bool b = is_heap(first, last, comp);
12 auto it = is_heap_until(first, last); // Find where heap property
    breaks

```

23.8 PERMUTATION ALGORITHMS

```

1 bool b = next_permutation(first, last); // Next lexicographic
    permutation
2 bool b = next_permutation(first, last, comp);
3 bool b = prev_permutation(first, last); // Previous permutation
4 bool b = prev_permutation(first, last, comp);
5
6 bool b = is_permutation(first1, last1, first2); // Check if permutation
7 bool b = is_permutation(first1, last1, first2, comp);

```

23.9 COMPLETE STL ALGORITHMS QUICK REFERENCE

Algorithm	Purpose	Returns
<code>find</code>	Find element	Iterator
<code>find_if</code>	Find matching	Iterator
<code>count</code>	Count occurrences	Count
<code>equal</code>	Compare ranges	Bool
<code>search</code>	Find subsequence	Iterator
<code>sort</code>	Sort range	Void
<code>binary_search</code>	Check existence (sorted)	Bool
<code>lower_bound</code>	First $\geq$ value (sorted)	Iterator
<code>partition</code>	Divide by condition	Iterator
<code>copy</code>	Copy range	Iterator
<code>transform</code>	Apply function	Iterator
<code>remove</code>	Remove matching	Iterator
<code>unique</code>	Remove duplicates	Iterator
<code>reverse</code>	Reverse range	Void
<code>rotate</code>	Rotate range	Void
<code>min_element</code>	Find minimum	Iterator
<code>max_element</code>	Find maximum	Iterator
<code>accumulate</code>	Sum/aggregate	Value
<code>inner_product</code>	Dot product	Value
<code>set_union</code>	Union of sets	Iterator
<code>set_intersection</code>	Intersection	Iterator

23.10 CONTAINERS DETAILED OPERATIONS

23.10.1 Vector Operations

```

1  v.push_back(val);           // Add to end
2  v.pop_back();              // Remove from end
3  v.insert(pos, val);        // Insert at position
4  v.erase(pos);             // Erase at position
5  v.clear();                 // Remove all
6  v.resize(n);               // Change size
7  v.reserve(n);              // Pre-allocate
8  v.shrink_to_fit();         // Release excess memory
9  v.swap(other);             // Swap two vectors
10
11 // Access
12 v[i];                      // O(1) random access
13 v.at(i);                   // O(1) with bounds check
14 v.front();                 // First element
15 v.back();                  // Last element
16 v.data();                  // Raw pointer (C++11)
17
18 // Iteration
19 begin(v), end(v);           // Iterators
20 rbegin(v), rend(v);         // Reverse iterators
21 cbegin(v), cend(v);         // Const iterators (C++11)
22
23 // Properties

```



```

24 v.size();           // Number of elements
25 v.capacity();      // Allocated space
26 v.empty();         // Check if empty
27 v.max_size();      // Maximum possible size

```

23.10.2 Map Operations

```

1 m[key] = value;      // Insert/update
2 m.insert({key, value}); // Insert
3 m.erase(key);       // Erase by key
4 m.clear();          // Remove all
5 m.swap(other);      // Swap two maps
6
7 // Access
8 m[key];             // Access (creates if missing)
9 m.at(key);          // Access (throws if missing)
10 m.find(key);        // Find key
11 m.count(key);       // Check existence
12
13 // Range operations
14 m.lower_bound(key); // First >= key
15 m.upper_bound(key); // First > key
16 m.equal_range(key); // All equal keys
17
18 // Iteration
19 m.begin(), m.end(); // Forward
20 m.rbegin(), m.rend(); // Reverse
21
22 // Properties
23 m.size();           // Number of elements
24 m.empty();          // Check if empty

```

23.10.3 Set Operations

```

1 s.insert(val);      // Insert element
2 s.erase(val);      // Erase by value
3 s.clear();          // Remove all
4 s.swap(other);     // Swap
5
6 // Access
7 s.find(val);        // Find element
8 s.count(val);       // Check existence (1 or 0)
9 s.lower_bound(val); // First >= value
10 s.upper_bound(val); // First > value
11 s.equal_range(val); // All equal values
12
13 // Iteration
14 s.begin(), s.end(); // Forward
15 s.rbegin(), s.rend(); // Reverse
16
17 // Properties
18 s.size();
19 s.empty();

```

23.11 ALGORITHM COMPLEXITY CHEAT SHEET

1	find, find_if, find_if_not:	$O(n)$
2	count, count_if:	$O(n)$
3	search, search_n:	$O(n*m)$
4	all_of, any_of, none_of:	$O(n)$
5		
6	sort:	$O(n \log n)$ avg
7	stable_sort:	$O(n \log n)$
8	partial_sort:	$O(n \log k)$ $k=\text{distance}(\text{first}, \text{last})$
9	nth_element:	$O(n)$ avg
10	make_heap:	$O(n)$
11	push_heap:	$O(\log n)$
12	pop_heap:	$O(\log n)$
13		
14	copy:	$O(n)$
15	transform:	$O(n)$
16	fill:	$O(n)$
17	reverse:	$O(n)$
18	rotate:	$O(n)$
19	unique:	$O(n)$
20	remove:	$O(n)$
21	partition:	$O(n)$
22	binary_search:	$O(\log n)$
23	lower_bound:	$O(\log n)$
24	upper_bound:	$O(\log n)$
25		
26	accumulate:	$O(n)$
27	inner_product:	$O(n)$
28	partial_sum:	$O(n)$
29		
30	set_union:	$O(n+m)$
31	set_intersection:	$O(n+m)$
32	set_difference:	$O(n+m)$

23.12 CONTAINER COMPLEXITY COMPARISON

	Insert	Delete	Search	Random Access	Memory
1					
2	vector	$O(n)$	$O(n)$	$O(1)$	Contiguous
3	deque	$O(n)$	$O(n)$	$O(1)$	Chunks
4	list	$O(1)$	$O(n)$	$O(n)$	Scattered
5	map	$O(\log n)$	$O(\log n)$	$O(\log n)$	Tree
6	set	$O(\log n)$	$O(\log n)$	$O(\log n)$	Tree
7	multimap	$O(\log n)$	$O(\log n)$	$O(\log n)$	Tree
8	multiset	$O(\log n)$	$O(\log n)$	$O(\log n)$	Tree
9	unordered_map	$O(1)$	$O(1)$	-	Hash
10	unordered_set	$O(1)$	$O(1)$	-	Hash
11	queue	$O(1)$	$O(1)$	$O(n)$	- (adapter)
12	stack	$O(1)$	$O(1)$	$O(n)$	- (adapter)
13	priority_queue	$O(\log n)$	$O(\log n)$	$O(n)$	Heap

23.13 ITERATOR COMPARISON TABLE

1	Container	Iterator Type	Bidirectional	Random Access
2	vector	random access	Yes	Yes

3	deque	random access	Yes	Yes
4	list	bidirectional	Yes	No
5	map	bidirectional	Yes	No
6	set	bidirectional	Yes	No
7	multimap	bidirectional	Yes	No
8	multiset	bidirectional	Yes	No
9	unordered_map	forward	No	No
10	unordered_set	forward	No	No

23.14 PRACTICAL STL PATTERNS

23.14.1 Pattern 1: Find & Remove

```

1 vector<int> v = {1, 2, 3, 2, 4, 2};
2 auto it = find(v.begin(), v.end(), 2);
3 if (it != v.end()) {
4     v.erase(it); // Remove first occurrence
5 }
6 // v: {1, 3, 2, 4, 2}
7
8 // Remove all
9 v.erase(remove(v.begin(), v.end(), 2), v.end());
10 // v: {1, 3, 4}

```

23.14.2 Pattern 2: Filter & Copy

```

1 vector<int> v = {1, 2, 3, 4, 5};
2 vector<int> even;
3
4 copy_if(v.begin(), v.end(), back_inserter(even),
5         [](int x) { return x % 2 == 0; });
6 // even: {2, 4}

```

23.14.3 Pattern 3: Transform & Collect

```

1 vector<int> v = {1, 2, 3};
2 vector<int> squared;
3
4 transform(v.begin(), v.end(), back_inserter(squared),
5           [](int x) { return x * x; });
6 // squared: {1, 4, 9}

```

23.14.4 Pattern 4: Partition & Process

```

1 vector<int> v = {1, 2, 3, 4, 5, 6};
2
3 auto it = partition(v.begin(), v.end(),
4                    [](int x) { return x % 2 == 0; });
5
6 // Process even numbers
7 for (auto i = v.begin(); i != it; ++i) {
8     cout << *i << " "; // 2 4 6
9 }

```

23.14.5 Pattern 5: Sort & Deduplicate

```
1 vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};
2
3 sort(v.begin(), v.end());
4 auto it = unique(v.begin(), v.end());
5 v.erase(it, v.end());
6 // v: {1, 2, 3, 4, 5, 6, 9}
```

23.14.6 Pattern 6: Group & Count

```
1 map<string, int> frequency;
2
3 vector<string> words = {"apple", "banana", "apple", "cherry", "banana"};
4 for (const string& word : words) {
5     frequency[word]++;
6 }
7
8 for (const auto& [word, count] : frequency) {
9     cout << word << ": " << count << "\n";
10 }
11 // Output: apple: 2, banana: 2, cherry: 1
```

23.14.7 Pattern 7: Custom Sorting

```
1 struct Person {
2     string name;
3     int age;
4 };
5
6 vector<Person> people = {
7     {"Alice", 30}, {"Bob", 25}, {"Carol", 30}
8 };
9
10 // Sort by age, then by name
11 sort(people.begin(), people.end(),
12     [](const Person& a, const Person& b) {
13         if (a.age != b.age) return a.age < b.age;
14         return a.name < b.name;
15     });
```

23.14.8 Pattern 8: Merge Ranges

```
1 vector<int> v1 = {1, 3, 5};
2 vector<int> v2 = {2, 4, 6};
3 vector<int> merged;
4
5 merge(v1.begin(), v1.end(), v2.begin(), v2.end(),
6     back_inserter(merged));
7 // merged: {1, 2, 3, 4, 5, 6}
```

23.15 STL WITH LAMBIDAS (C++11 and later)

```

1 #include <algorithm>
2 #include <vector>
3
4 vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
5
6 // Simple lambda
7 auto isEven = [](int x) { return x % 2 == 0; };
8
9 // With capture by value
10 int threshold = 5;
11 auto gt = [threshold](int x) { return x > threshold; };
12
13 // With capture by reference
14 int sum = 0;
15 for_each(v.begin(), v.end(),
16         [&sum](int x) { sum += x; });
17
18 // Mutable lambda
19 auto counter = [count = 0]() mutable { return ++count; };
20
21 // Generic lambda (C++14)
22 auto print = [](auto x) { cout << x << " "; };
23 for_each(v.begin(), v.end(), print);
24
25 // Find even numbers
26 auto it = find_if(v.begin(), v.end(),
27                 [](int x) { return x % 2 == 0; });
28
29 // Count odd numbers
30 int oddCount = count_if(v.begin(), v.end(),
31                        [](int x) { return x % 2 != 0; });
32
33 // Transform to squares
34 vector<int> squared;
35 transform(v.begin(), v.end(), back_inserter(squared),
36          [](int x) { return x * x; });
37
38 // Filter and collect
39 vector<int> filtered;
40 copy_if(v.begin(), v.end(), back_inserter(filtered),
41        [](int x) { return x % 2 == 0; });

```

23.16 STRINGS - MASTER GUIDE

23.17 4.1 String Basics

```

1 #include <string>
2 using namespace std;
3
4 // Declaration
5 string s1; // Empty string
6 string s2 = "Hello"; // Initialize with C-string
7 string s3("World"); // Constructor
8 string s4(5, 'a'); // 5 'a' characters: "aaaaa"
9 string s5 = s2; // Copy

```

```

10 string s6(s2, 1, 3);           // Substring of s2 from pos 1, length 3: "ell"
11 string s7(s2.begin(), s2.end()); // From iterators
12
13 // C-string
14 const char* cstr = s2.c_str(); // Convert to C-string
15 const char* data = s2.data();  // Get data pointer

```

23.17.1 Size and Capacity

```

1 string s = "Hello";
2
3 cout << s.length() << "\n";    // 5
4 cout << s.size() << "\n";      // 5 (same as length)
5 cout << s.capacity() << "\n";  // >= 5
6
7 cout << s.empty() << "\n";     // 0 (false)
8 cout << s.max_size() << "\n";  // Maximum possible size
9
10 // Resize
11 s.resize(10, '*');              // Resize to 10, fill with '*'
12 // s: "Hello*****"
13
14 // Reserve
15 s.reserve(100);                 // Reserve space for 100 characters
16
17 // Clear
18 s.clear();                      // Empty the string

```

23.18 4.2 Accessing Characters

```

1 string s = "Hello";
2
3 // Using operator[]
4 cout << s[0] << "\n";          // 'H' - No bounds checking
5 cout << s.at(0) << "\n";       // 'H' - With bounds checking
6
7 // Front and back
8 cout << s.front() << "\n";     // 'H'
9 cout << s.back() << "\n";      // 'o'
10
11 // Modifying characters
12 s[0] = 'J';                     // "Jello"
13 s.at(1) = 'A';                  // "JAello"
14 s.front() = 'B';                // "BAello"
15 s.back() = 'z';                 // "BAellz"

```

23.19 4.3 String Concatenation

```

1 string s1 = "Hello";
2 string s2 = "World";
3
4 // Using + operator

```

```

5 string s3 = s1 + " " + s2;    // "Hello World"
6
7 // Using += operator
8 s1 += " ";
9 s1 += s2;                    // s1: "Hello World"
10
11 // Using append()
12 s1.append(" C++");          // "Hello World C++"
13 s1.append(3, '!');          // "Hello World C+++!!"
14
15 // Using push_back()
16 s1.push_back('*');          // "Hello World C+++!!*"
17
18 // Using insert()
19 s1.insert(5, "_");           // Insert "_" at position 5
20 // s1: "Hello_ World C+++!!*"

```

23.20 4.4 String Searching

```

1 string s = "Hello World Hello";
2
3 // find() - find substring
4 size_t pos = s.find("World");
5 if (pos != string::npos) {
6     cout << "Found at position: " << pos << "\n";    // 6
7 }
8
9 // find() with starting position
10 pos = s.find("Hello", 2);    // Find "Hello" starting from position 2
11 // Returns 12 (second "Hello")
12
13 // rfind() - find from right
14 pos = s.rfind("Hello");      // Position of last "Hello"
15 // Returns 12
16
17 // find_first_of() - find first occurrence of any character in string
18 pos = s.find_first_of("aeiou");
19 // Returns position of first vowel: 1 ('e')
20
21 // find_last_of() - find last occurrence of any character
22 pos = s.find_last_of("aeiou");
23 // Returns position of last vowel: 14 ('o')
24
25 // find_first_not_of() - find first character NOT in string
26 pos = s.find_first_not_of("He");
27 // Returns 2 ('l')
28
29 // find_last_not_of() - find last character NOT in string
30 pos = s.find_last_not_of("o");
31 // Returns 15
32
33 // Check if string starts with (C++20)
34 // bool starts = s.starts_with("Hello");
35 // bool ends = s.ends_with("ello");

```

23.21 4.5 String Manipulation

```
1 string s = "Hello World";
2
3 // Substring
4 string sub = s.substr(0, 5); // "Hello"
5 string sub2 = s.substr(6);  // "World"
6
7 // Replace
8 s.replace(0, 5, "Hi");      // Replace first 5 chars with "Hi"
9 // s: "Hi World"
10
11 // Erase
12 s.erase(2, 1);             // Erase 1 char starting at position 2
13 // s: "HiWorld"
14
15 // remove() + erase() idiom (for removing specific characters)
16 string s2 = "H-e-l-l-o";
17 s2.erase(remove(s2.begin(), s2.end(), '-'), s2.end());
18 // s2: "Hello"
19
20 // Compare
21 string a = "apple";
22 string b = "apple";
23 cout << (a == b) << "\n"; // 1 (true)
24 cout << a.compare(b) << "\n"; // 0 (equal)
25
26 // Case conversion (manual)
27 for (char& c : s) {
28     c = toupper(c); // Convert to uppercase
29 }
30 // s: "HI WORLD"
31
32 // Transform with algorithm
33 transform(s.begin(), s.end(), s.begin(), ::toupper);
```

23.22 4.6 String Iteration

```
1 string s = "Hello";
2
3 // Iterator
4 for (auto it = s.begin(); it != s.end(); ++it) {
5     cout << *it << " ";
6 }
7
8 // Range-based for (C++11)
9 for (char c : s) {
10     cout << c << " ";
11 }
12
13 // Index-based
14 for (int i = 0; i < s.length(); i++) {
15     cout << s[i] << " ";
16 }
17
18 // Reverse iteration
19 for (auto it = s.rbegin(); it != s.rend(); ++it) {
20     cout << *it << " ";
```


21 }

23.23 4.7 String Conversion

```

1 #include <string>
2
3 // String to numbers
4 string num = "123";
5 int intVal = stoi(num);           // 123
6 long longVal = stol(num);        // 123L
7 float floatVal = stof("3.14");   // 3.14
8 double doubleVal = stod("3.14159"); // 3.14159
9
10 // Number to string
11 int x = 42;
12 string s1 = to_string(x);        // "42"
13 string s2 = to_string(3.14);     // "3.140000"
14 string s3 = to_string(true);     // "1"
15
16 // With radix (base)
17 string hex = to_string(255);     // "255" (decimal)
18 // For hex, use stringstream or manual conversion
    
```

23.24 4.8 String Comparison

```

1 string s1 = "apple";
2 string s2 = "apple";
3 string s3 = "banana";
4
5 // Equality
6 cout << (s1 == s2) << "\n";     // 1 (true)
7 cout << (s1 != s3) << "\n";     // 1 (true)
8
9 // Ordering (lexicographic)
10 cout << (s1 < s3) << "\n";      // 1 (true) - "apple" < "banana"
11 cout << (s1 > s3) << "\n";      // 0 (false)
12
13 // compare() method
14 cout << s1.compare(s2) << "\n"; // 0 (equal)
15 cout << s1.compare(s3) << "\n"; // -1 (s1 < s3)
16 cout << s3.compare(s1) << "\n"; // 1 (s3 > s1)
17
18 // Compare substring
19 cout << s1.compare(0, 3, "app") << "\n"; // 0 (equal)
20
21 // Case-insensitive comparison (manual)
22 bool caseInsensitive = true;
23 for (int i = 0; i < s1.length() && i < s3.length(); i++) {
24     if (tolower(s1[i]) != tolower(s3[i])) {
25         caseInsensitive = false;
26         break;
27     }
28 }
    
```

23.25 4.9 String from Stringstream

```
1 #include <sstream>
2
3 // Building string
4 ostream oss;
5 oss << "Value: " << 42 << ", Name: " << "Alice";
6 string result = oss.str(); // "Value: 42, Name: Alice"
7
8 // Parsing string
9 istream iss("10 20 30");
10 int a, b, c;
11 iss >> a >> b >> c; // a=10, b=20, c=30
12
13 // Convert various types
14 int num = 42;
15 double pi = 3.14159;
16 string name = "Alice";
17
18 ostream convert;
19 convert << num << " " << pi << " " << name;
20 string combined = convert.str();
21
22 // Parse line with specific delimiter
23 istream lineStream("apple,banana,mango");
24 string fruit;
25 while (getline(lineStream, fruit, ',')) {
26     cout << fruit << "\n";
27 }
```

23.26 FILE I/O - COMPLETE COVERAGE

23.27 5.1 File I/O Basics

```
1 #include <fstream>
2 #include <iostream>
3 using namespace std;
4
5 // Output file stream (write)
6 ofstream outFile;
7 outFile.open("output.txt");
8
9 if (outFile.is_open()) {
10     outFile << "Hello, File!\n";
11     outFile << "This is a test.\n";
12     outFile.close();
13 } else {
14     cout << "Error opening file\n";
15 }
16
17 // Input file stream (read)
18 ifstream inFile;
19 inFile.open("output.txt");
20
21 if (inFile.is_open()) {
22     string line;
23     while (getline(inFile, line)) {
```

```

24         cout << line << "\n";
25     }
26     inFile.close();
27 } else {
28     cout << "Error opening file\n";
29 }
    
```

23.28 5.2 File Operations

23.28.1 Open Modes

```

1 #include <fstream>
2
3 // Write (truncate if exists)
4 ofstream file1("data.txt"); // Default
5 ofstream file2("data.txt", ios::out); // Explicit
6
7 // Append
8 ofstream file3("data.txt", ios::app);
9
10 // Read
11 ifstream file4("data.txt");
12 ifstream file5("data.txt", ios::in);
13
14 // Read and Write
15 fstream file6("data.txt", ios::in | ios::out);
16
17 // Binary mode
18 ofstream binFile("data.bin", ios::binary);
19 ifstream binRead("data.bin", ios::binary);
20
21 // Truncate (discards existing content)
22 ofstream file7("data.txt", ios::trunc);
23
24 // Seek position
25 fstream file8("data.txt", ios::in | ios::out);
26 file8.seekg(10); // Seek to position 10 for reading
27 file8.seekp(10); // Seek to position 10 for writing
    
```

23.29 5.3 Writing to Files

```

1 ofstream outFile("output.txt");
2
3 if (outFile) {
4     // Write strings
5     outFile << "Hello, World!\n";
6     outFile << "Line 2\n";
7
8     // Write numbers
9     outFile << 42 << " " << 3.14 << "\n";
10
11    // Write characters
12    outFile << 'A' << 'B' << 'C' << "\n";
13
14    // Write using put() for single character
15    outFile.put('X');
    
```

```
16
17 // Write raw data
18 string data = "Raw data";
19 outFile.write(data.c_str(), data.length());
20
21 outFile.close();
22 }
```

23.30 5.4 Reading from Files

23.30.1 Line by Line

```
1 #include <fstream>
2 #include <string>
3
4 ifstream inFile("input.txt");
5
6 if (inFile) {
7     string line;
8     while (getline(inFile, line)) {
9         cout << line << "\n";
10    }
11    inFile.close();
12 }
```

23.30.2 Word by Word

```
1 ifstream inFile("input.txt");
2
3 if (inFile) {
4     string word;
5     while (inFile >> word) {
6         cout << word << "\n";
7     }
8     inFile.close();
9 }
```

23.30.3 Character by Character

```
1 ifstream inFile("input.txt");
2
3 if (inFile) {
4     char ch;
5     while (inFile.get(ch)) {
6         cout << ch;
7     }
8     inFile.close();
9 }
```

23.30.4 Specific Format

```

1 ifstream inFile("data.txt");
2
3 if (inFile) {
4     int id;
5     string name;
6     double salary;
7
8     while (inFile >> id >> name >> salary) {
9         cout << "ID: " << id << ", Name: " << name
10            << ", Salary: " << salary << "\n";
11     }
12     inFile.close();
13 }

```

23.31 5.5 Binary File I/O

```

1 #include <fstream>
2
3 // Writing binary
4 struct Person {
5     int age;
6     double height;
7     char initial;
8 };
9
10 ofstream binOut("people.bin", ios::binary);
11 Person p = {30, 5.9, 'A'};
12 binOut.write(reinterpret_cast<char*>(&p), sizeof(Person));
13 binOut.close();
14
15 // Reading binary
16 ifstream binIn("people.bin", ios::binary);
17 Person p2;
18 binIn.read(reinterpret_cast<char*>(&p2), sizeof(Person));
19 cout << "Age: " << p2.age << ", Height: " << p2.height << "\n";
20 binIn.close();
21
22 // Reading multiple binary objects
23 vector<Person> people;
24 Person p3;
25 while (binIn.read(reinterpret_cast<char*>(&p3), sizeof(Person))) {
26     people.push_back(p3);
27 }

```

23.32 5.6 File Position

```

1 fstream file("data.txt", ios::in | ios::out);
2
3 // Get position
4 streampos pos = file.tellg(); // Get read position
5 pos = file.tellp();          // Get write position
6
7 // Set position

```

```
8 file.seekg(0, ios::beg);      // Seek to beginning
9 file.seekg(0, ios::end);      // Seek to end
10 file.seekg(-10, ios::end);    // Seek 10 bytes from end
11 file.seekp(5);                // Seek write position to 5
12
13 // File size
14 file.seekg(0, ios::end);
15 int fileSize = file.tellg();
16 cout << "File size: " << fileSize << " bytes\n";
17
18 file.close();
```

23.33 5.7 Error Handling

```
1 ifstream file("input.txt");
2
3 // Check if file opened
4 if (!file) {
5     cout << "Failed to open file\n";
6     return;
7 }
8
9 // Check read state
10 if (file.fail()) {
11     cout << "Read operation failed\n";
12 }
13
14 if (file.bad()) {
15     cout << "Severe error\n";
16 }
17
18 if (file.eof()) {
19     cout << "End of file reached\n";
20 }
21
22 // Clear error flags
23 file.clear();
24
25 // Check if good
26 if (file.good()) {
27     cout << "File is in good state\n";
28 }
29
30 file.close();
```

23.34 5.8 Complete File I/O Example

```
1 #include <fstream>
2 #include <sstream>
3 #include <vector>
4 using namespace std;
5
6 struct Student {
7     int id;
```

```

8     string name;
9     double gpa;
10 };
11
12 // Write students to file
13 void writeStudents(const string& filename, const vector<Student>& students) {
14     ofstream file(filename);
15
16     for (const auto& student : students) {
17         file << student.id << " " << student.name << " " << student.gpa << "\n"
18     ;
19     }
20
21     file.close();
22 }
23
24 // Read students from file
25 vector<Student> readStudents(const string& filename) {
26     vector<Student> students;
27     ifstream file(filename);
28
29     int id;
30     string name;
31     double gpa;
32
33     while (file >> id >> name >> gpa) {
34         students.push_back({id, name, gpa});
35     }
36
37     file.close();
38     return students;
39 }
40
41 // Main
42 int main() {
43     vector<Student> students = {
44         {101, "Alice", 3.8},
45         {102, "Bob", 3.5},
46         {103, "Carol", 3.9}
47     };
48
49     // Write
50     writeStudents("students.txt", students);
51
52     // Read
53     auto readData = readStudents("students.txt");
54
55     for (const auto& s : readData) {
56         cout << s.id << " " << s.name << " " << s.gpa << "\n";
57     }
58
59     return 0;
60 }

```

23.35 FUNCTION OBJECTS & COMPARATORS

23.36 6.1 Function Objects (Functors)

```

1 // Function object for greater than comparison
2 struct GreaterThan {
3     int value;
4
5     GreaterThan(int v) : value(v) {}
6
7     bool operator()(int x) const {
8         return x > value;
9     }
10 };
11
12 vector<int> v = {10, 20, 30, 40, 50};
13
14 // Use with algorithm
15 auto it = find_if(v.begin(), v.end(), GreaterThan(25));
16 if (it != v.end()) {
17     cout << "Found: " << *it << "\n"; // 30
18 }
19
20 // Count elements greater than 25
21 int count = count_if(v.begin(), v.end(), GreaterThan(25));
22 cout << "Count: " << count << "\n"; // 3

```

23.37 6.2 Standard Comparators

```

1 #include <functional>
2
3 // less - ascending order
4 sort(v.begin(), v.end(), less<int>());
5
6 // greater - descending order
7 sort(v.begin(), v.end(), greater<int>());
8
9 // equal_to, not_equal_to
10 count_if(v.begin(), v.end(), bind(equal_to<int>(), placeholders::_1, 20));
11
12 // Map with custom comparator
13 map<string, int, less<string>> ascending; // A-Z
14 map<string, int, greater<string>> descending; // Z-A

```

23.38 STL BEST PRACTICES

23.39 7.1 Container Selection

```

1 Use VECTOR when:
2   - Need random access
3   - Need cache locality
4   - Mostly append operations
5
6 Use LIST when:
7   - Need frequent insertion/deletion in middle
8   - Don't need random access
9
10 Use DEQUE when:
11   - Need efficient push_front/pop_front

```



```

12 - Need random access
13
14 Use MAP/SET when:
15 - Need sorted, unique elements
16 - Need  $O(\log n)$  lookup
17
18 Use UNORDERED_MAP/SET when:
19 - Need  $O(1)$  average lookup
20 - Don't care about order
21
22 Use QUEUE when:
23 - Need FIFO behavior
24
25 Use STACK when:
26 - Need LIFO behavior
27
28 Use PRIORITY_QUEUE when:
29 - Need elements processed by priority

```

23.40 7.2 Algorithm Selection

```

1 // For small data: simple loop
2 for (int i = 0; i < v.size(); i++) {
3     // Process v[i]
4 }
5
6 // For searching: find_if
7 auto it = find_if(v.begin(), v.end(), predicate);
8
9 // For filtering: copy_if
10 copy_if(v.begin(), v.end(), back_inserter(result), predicate);
11
12 // For transforming: transform
13 transform(v.begin(), v.end(), result.begin(), function);
14
15 // For aggregating: accumulate
16 int sum = accumulate(v.begin(), v.end(), 0);
17
18 // For sorting: sort
19 sort(v.begin(), v.end());

```

23.41 7.3 Memory Management

```

1 // Reserve space when size is known
2 vector<int> v;
3 v.reserve(1000); // Avoid reallocations
4
5 // Clear and shrink
6 v.clear();
7 v.shrink_to_fit(); // Release memory
8
9 // Use move semantics
10 vector<int> getVector() {
11     vector<int> v;
12     // ... fill v
13     return v; // Move, not copy (C++11)
14 }

```

23.42 7.4 Performance Tips

```
1 // Prefer iterators over indices in generic code
2 for (auto it = v.begin(); it != v.end(); ++it) {
3     // Optimized for all container types
4 }
5
6 // Use const references to avoid copying
7 void process(const vector<int>& v);
8
9 // Pre-allocate space
10 map<string, int> m;
11 m.reserve(1000);
12
13 // Use stable algorithms when order matters
14 stable_sort(v.begin(), v.end());
15
16 // Avoid repeated function calls in loops
17 int size = v.size();
18 for (int i = 0; i < size; i++) {
19     // Use cached size
20 }
```

23.42.1 Containers Intro (C++98)

```
1 #include <iostream>
2 #include <vector>
3 #include <list>
4 #include <map>
5 #include <set>
6
7 int main() {
8     // Vector (dynamic array)
9     std::vector<int> v;
10    v.push_back(1);
11    v.push_back(2);
12    v.push_back(3);
13
14    for (int i = 0; i < v.size(); i++) {
15        std::cout << v[i] << " ";
16    }
17    std::cout << "\n";
18
19    // List (linked list)
20    std::list<int> l;
21    l.push_back(10);
22    l.push_front(5);
23
24    for (std::list<int>::iterator it = l.begin(); it != l.end(); ++it) {
25        std::cout << *it << " ";
26    }
27    std::cout << "\n";
28
29    return 0;
30 }
```

Chapter 24

STL INTERNALS DEEP DIVE

To master the STL, you must understand what happens under the hood.

24.0.1 3.5.1 The Truth About `std::vector`

`std::vector` is a dynamic array. It guarantees contiguous memory.

- **Layout:** Three pointers: `start`, `finish`, `end_of_storage`.
 - `start`: Points to first element.
 - `finish`: Points to one-past-the-last active element (size).
 - `end_of_storage`: Points to end of allocated buffer (capacity).
- **Growth Strategy:** Geometric growth.
 - When `size() == capacity()`, a new buffer is allocated (usually 2x or 1.5x larger).
 - **Elements are MOVED** (or copied) to the new buffer.
 - Old buffer is deleted.
 - *Cost*: Amortized $O(1)$ `push_back`, but worst-case $O(N)$.
- **Iterator Invalidation:**
 - **Reallocation:** Invalidates ALL iterators, pointers, and references.
 - **Insertion/Erasure:** Invalidates iterators at and after the point of operation.

24.0.2 3.5.2 The `std::deque` Implementation

`std::deque` (Double-Ended Queue) is NOT a contiguous array.

- **Layout:** A “Map” (dynamic array) of pointers to fixed-size “Chunks” (blocks).
 - Iterators are smart pointers that know how to jump between chunks.
- **Performance:**
 - $O(1)$ random access (double dereference).
 - $O(1)$ push/pop at BOTH ends (no full reallocation needed, just add a new chunk).
- **Cache Locality:** Worse than vector, better than list.

24.0.3 3.5.3 Why `std::list` is (Almost) Always Wrong

`std::list` is a Doubly Linked List.

- **Layout:** Nodes allocated individually on the heap.

```
– struct Node \{ T val; Node* prev; Node* next; \}
```

- **The Cache Problem:** Nodes are scattered in memory. Traversing a list causes constant Cache Misses.
- **Benchmark:** Iterating a `vector` is orders of magnitude faster than a `list`, even for large types, due to prefetching.
- **Use Case:** Only when you need **Reference Stability** (insertions never invalidate references to other elements).

24.0.4 3.5.4 Associative Containers (Map/Set)

`std::map`, `std::set`, `std::multimap`, `std::multiset`.

- **Implementation:** Balanced Binary Search Tree (usually **Red-Black Tree**).
- **Node Layout:**

```
struct Node \{ T val; Node* left; Node* right; Node* parent; Color color; \}
```
- **Complexity:** $O(\log N)$ for insert, lookup, delete.
- **Overhead:** 3 pointers + enum per element (heavy memory overhead).

24.0.5 3.5.5 Unordered Containers (Hash Maps)

`std::unordered_map`, `std::unordered_set`.

- **Implementation:** Array of “Buckets” (Linked Lists).
 - Hash function maps Key -> Bucket Index.
 - Collisions handled by Chaining (linked list in bucket).
- **Complexity:**
 - Average: $O(1)$.
 - Worst Case: $O(N)$ (if all keys hash to same bucket).
- **Rehashing:** When `load_factor > max_load_factor`, bucket array grows, all elements re-hashed.

Container	Operation	Invalidates
-----------	-----------	-------------

24.0.6 3.5.6 Iterator Invalidation Cheat Sheet

Container	Operation	Invalidates
Vector	Capacity Change	ALL
Vector	Insert/Erase	Current & After
Deque	Insert/Erase (ends)	Iterators only (Refs valid!)
Deque	Insert/Erase (middle)	ALL
List	Insert/Erase	Only deleted element
Map/Set	Insert/Erase	Only deleted element
Unordered	Rehash	ALL
Unordered	Insert (no rehash)	None

Part II

Volume II: C++11 - The Modern Revolution

Chapter 25

C++11 REVOLUTION

The C++11 standard was a massive upgrade. This is where modern C++ begins!

25.1 C++11 Overview & History

C++11 (also called C++0x) is the most significant C++ update since the language's creation:

25.1.1 Timeline

- **1998:** C++98 released (first standard)
- **2003:** C++03 (minor fixes)
- **2011:** C++11 (revolutionary update - 13 years later!)
- **2014:** C++14 (maintenance release)
- **2017:** C++17 (major update)
- **2020:** C++20 (revolutionary)
- **2023:** C++23 (latest)

25.1.2 Why C++11 Matters

C++11 introduced **70+ new features**, transforming C++ from error-prone to modern and safe.

25.1.3 Key Themes

1. **Memory Safety** - Smart pointers
 2. **Performance** - Move semantics
 3. **Readability** - Auto, lambdas, range-based for
 4. **Correctness** - nullptr, strongly-typed enums
 5. **Flexibility** - Variadic templates
 6. **Concurrency** - Threads, atomic operations
 7. **Convenience** - Uniform initialization, tuple
-

25.2 AUTO & TYPE DEDUCTION

25.3 1.1 Auto Keyword

The `auto` keyword instructs the compiler to deduce the type from context.

25.3.1 Basic Auto

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 using namespace std;
5
6 // Type deduction - compiler figures out the type
7 auto x = 5; // int
8 auto y = 3.14; // double
9 auto z = "hello"; // const char*
10 auto s = string("hello"); // string
11 auto v = vector<int>{1, 2, 3}; // vector<int>
12
13 // Without auto (verbose)
14 vector<int>::iterator it1 = v.begin();
15
16 // With auto (concise)
17 auto it2 = v.begin(); // Same type, much cleaner!
18
19 // Auto in loops
20 for (auto val : v) { // Type deduced as int
21     cout << val << " ";
22 }
23
24 // Auto with complex types
25 map<string, vector<int>>> m;
26 auto it = m.begin(); // Type: map<string, vector<int>>::iterator

```

25.3.2 Auto With Pointers & References

```

1 int value = 42;
2
3 auto ptr = &value; // int* (pointer)
4 auto ref = value; // int (by value)
5 auto& ref2 = value; // int& (reference)
6 auto* ptr2 = &value; // int* (explicit pointer)
7
8 const int cv = 10;
9 auto a = cv; // int (const lost!)
10 auto b = &cv; // const int*
11 const auto c = cv; // const int (preserve const)
12
13 // Reference to const
14 const auto& d = cv; // const int&

```

25.3.3 Auto in Templates

```

1 template<typename T>
2 void process(T value) {
3     auto copy = value; // Type is T
4     // ...

```



```

5 }
6
7 // Without auto - would need explicit template parameter
8 template<typename T>
9 void oldWay(T value) {
10     T copy = value;           // Must use T explicitly
11 }

```

25.3.4 Auto Type Deduction Rules

```

1 // Rule 1: Const is stripped unless explicitly written
2 const int ci = 5;
3 auto a = ci;           // int (const stripped)
4 const auto b = ci;     // const int (const kept)
5
6 // Rule 2: Reference is stripped for value
7 int i = 10;
8 auto& ref = i;
9 auto c = ref;          // int (& stripped)
10 auto d = &i;           // int*
11
12 // Rule 3: Initializer list
13 auto e = {1, 2, 3};    // initializer_list<int>
14 vector<int> v = {1, 2, 3}; // vector<int> (different!)
15
16 // Rule 4: Function return type
17 auto add = [](int a, int b) { return a + b; }; // Return type deduced

```

25.3.5 Auto Benefits & Limitations

Benefits: - Reduces verbosity - Refactoring-friendly (type changes automatically) - Prevents narrowing conversions - Works with complex types

Limitations: - Readability can suffer (what's the type?) - Can hide bugs - Not available in C++98/03

Best Practices:

```

1 // Good - obvious type
2 auto count = 0;           // int - clear
3
4 // Questionable - unclear type
5 auto result = calculateSomething(); // What type?
6
7 // Solution - use explicit type or meaningful name
8 int result = calculateCount(); // Clear!
9 auto result_count = calculateCount(); // Name clarifies

```

25.4 1.2 decltype

The `decltype` keyword queries the type of an expression.

```

1 #include <iostream>
2 using namespace std;
3
4 int x = 5;
5
6 // Get type of x

```

```

7  decltype(x) y = 10;           // int y = 10
8
9  // Get type of expression
10 auto result = 5 + 3;          // int
11 decltype(5 + 3) z = 20;      // int z = 20
12
13 // With references
14 int& ref = x;
15 decltype(ref) ref2 = x;      // int& ref2 = x
16
17 // Complex types
18 vector<int> v;
19 decltype(v.begin()) it = v.begin(); // vector<int>::iterator
20
21 // Function return type deduction (C++11)
22 template<typename T, typename U>
23 auto add(T a, U b) -> decltype(a + b) {
24     return a + b;
25 }
26
27 // Type matching
28 int i = 5;
29 double d = 3.14;
30 decltype(i + d) result = i + d; // double (int + double = double)
31
32 // With function calls
33 int getNumber() { return 42; }
34 decltype(getNumber()) num = getNumber(); // int
35
36 // Advanced - trailing return type
37 template<typename T, typename U>
38 auto multiply(T a, U b) -> decltype(a * b) {
39     return a * b;
40 }
41
42 // Checking types match
43 static_assert(is_same<decltype(x), int>::value); // Compile-time check

```

25.4.1 decltype vs auto

```

1  // auto - deduces from initializer
2  auto a = 5;           // int
3
4  // decltype - deduces from type
5  int i = 5;
6  decltype(i) b = 10;   // int
7
8  // Difference with references
9  int& ref = i;
10 auto c = ref;          // int (reference stripped)
11 decltype(ref) d = i;   // int& (reference preserved)
12
13 // Practical use - return type deduction
14 template<typename T, typename U>
15 auto divide(T a, U b) -> decltype(a / b) {
16     return a / b;
17 }

```

25.5 SMART POINTERS - MEMORY REVOLUTION

25.6 2.1 Unique Pointer (unique_ptr)

unique_ptr - exclusive ownership of dynamic object. One owner only.

25.6.1 Basic Usage

```

1 #include <memory>
2 using namespace std;
3
4 // Old way (C++98) - manual management, error-prone
5 int* old_ptr = new int(42);
6 cout << *old_ptr << "\n";
7 delete old_ptr; // Easy to forget!
8 old_ptr = nullptr;
9
10 // New way (C++11) - automatic management
11 unique_ptr<int> smart_ptr(new int(42));
12 cout << *smart_ptr << "\n";
13 // Automatically deleted when smart_ptr goes out of scope
14
15 // Better - use make_unique (C++14)
16 auto ptr = make_unique<int>(42);
17 cout << *ptr << "\n";
18 // Automatically deleted

```

25.6.2 Unique Pointer Operations

```

1 class Resource {
2 public:
3     Resource() { cout << "Resource created\n"; }
4     ~Resource() { cout << "Resource destroyed\n"; }
5     void use() { cout << "Using resource\n"; }
6 };
7
8 // Create unique_ptr
9 unique_ptr<Resource> ptr1(new Resource());
10 unique_ptr<Resource> ptr2 = make_unique<Resource>();
11
12 // Access
13 ptr1->use(); // Arrow operator
14 (*ptr1).use(); // Dereference
15 if (ptr1) { } // Check null
16 ptr1.get(); // Get raw pointer
17
18 // Move semantics (ownership transfer)
19 unique_ptr<Resource> ptr3 = move(ptr1);
20 // Now ptr3 owns the resource
21 // ptr1 is now null
22 cout << (ptr1 ? "owns" : "empty") << "\n"; // "empty"
23
24 // Array version
25 unique_ptr<int[]> arr(new int[10]);
26 arr[0] = 5;
27 arr[1] = 10;
28 // Automatically deleted with delete[]
29
30 // Reset
31 ptr3.reset(); // Delete and become null

```

```

32 ptr3.reset(new Resource());           // Delete old, manage new
33
34 // Release (give up ownership)
35 Resource* raw = ptr3.release();       // Ownership transferred to you!
36 delete raw;                          // You must delete it
37
38 // Custom deleter
39 unique_ptr<FILE, decltype(&fclose)> file(fopen("test.txt", "r"), &fclose);
40 // File automatically closed

```

25.6.3 Unique Pointer with Custom Deleters

```

1  class Database {
2  public:
3      Database() { cout << "DB connected\n"; }
4      ~Database() { cout << "DB disconnected\n"; }
5  };
6
7  // Custom deleter function
8  void closeDB(Database* db) {
9      cout << "Closing database...\n";
10     delete db;
11 }
12
13 // Using custom deleter
14 unique_ptr<Database, decltype(&closeDB)> db(
15     new Database(),
16     &closeDB
17 );
18 // Output: DB connected
19 // When db goes out of scope:
20 // Output: Closing database...
21 //         DB disconnected
22
23 // With lambda deleter
24 auto deleter = [](Database* db) {
25     cout << "Lambda deleting database\n";
26     delete db;
27 };
28 unique_ptr<Database, decltype(deleter)> db2(new Database(), deleter);

```

25.6.4 Unique Pointer in Collections

```

1  vector<unique_ptr<Resource>> resources;
2
3  resources.push_back(make_unique<Resource>());
4  resources.push_back(make_unique<Resource>());
5  resources.push_back(make_unique<Resource>());
6
7  // Iterate and use
8  for (auto& res : resources) {
9      res->use();
10 }
11 // All automatically cleaned up when vector destroyed

```

25.7 2.2 Shared Pointer (shared_ptr)

`shared_ptr` - shared ownership. Reference counting tracks multiple owners.

25.7.1 Basic Usage

```

1 #include <memory>
2
3 class Resource {
4 public:
5     Resource() { cout << "Created\n"; }
6     ~Resource() { cout << "Destroyed\n"; }
7 };
8
9 // Create shared_ptr
10 shared_ptr<Resource> ptr1 = make_shared<Resource>();
11 cout << ptr1.use_count() << "\n";           // 1
12
13 // Copy creates new reference
14 shared_ptr<Resource> ptr2 = ptr1;
15 cout << ptr1.use_count() << "\n";           // 2
16 cout << ptr2.use_count() << "\n";           // 2
17
18 // Create third reference
19 shared_ptr<Resource> ptr3 = ptr1;
20 cout << ptr1.use_count() << "\n";           // 3
21
22 // When ptr3 goes out of scope
23 {
24     shared_ptr<Resource> ptr4 = ptr1;
25     cout << ptr1.use_count() << "\n";           // 4
26 }
27 cout << ptr1.use_count() << "\n";           // 3 - ptr4 destroyed
28
29 // When all references gone, resource deleted
30 ptr1 = nullptr;
31 cout << ptr2.use_count() << "\n";           // 2
32 ptr2 = nullptr;
33 cout << ptr3.use_count() << "\n";           // 1
34 ptr3 = nullptr;
35 // Output: Destroyed (all references gone)

```

25.7.2 Shared Pointer Operations

```

1 shared_ptr<Resource> ptr = make_shared<Resource>();
2
3 // Access
4 ptr->use();
5 (*ptr).use();
6
7 // Check null
8 if (ptr) { }
9 if (ptr.get()) { }
10
11 // Reference counting
12 ptr.use_count();           // Number of owners
13
14 // Reset
15 ptr.reset();               // Decrement count, may delete
16

```

```

17 // Get raw pointer
18 Resource* raw = ptr.get();           // Don't delete raw!
19
20 // Convert to raw and create new shared_ptr from same resource
21 // WARNING: This is dangerous!
22 Resource* raw2 = ptr.get();
23 // shared_ptr<Resource> ptr2(raw2); // DANGER! Two separate reference counts!
24
25 // Safe way - use enable_shared_from_this
26 class SafeResource : public enable_shared_from_this<SafeResource> {
27 public:
28     shared_ptr<SafeResource> getSelf() {
29         return shared_from_this();
30     }
31 };

```

25.7.3 Shared Pointer with Arrays & Custom Deleters

```

1 // Array version
2 shared_ptr<int[]> arr(new int[10]); // C++20
3 arr[0] = 5;
4
5 // Custom deleter
6 auto deleter = [](FILE* f) {
7     cout << "Closing file\n";
8     if (f) fclose(f);
9 };
10
11 shared_ptr<FILE> file(fopen("test.txt", "r"), deleter);
12 // File automatically closed when last reference gone

```

25.7.4 Shared Pointers in Collections

```

1 vector<shared_ptr<Resource>> resources;
2
3 resources.push_back(make_shared<Resource>());
4 resources.push_back(make_shared<Resource>());
5
6 shared_ptr<Resource> copy = resources[0];
7 cout << resources[0].use_count() << "\n";           // 2 (vector + copy)
8
9 // When we erase
10 resources.erase(resources.begin());
11 cout << copy.use_count() << "\n";                   // Still 1 (copy still owns it)

```

25.8 2.3 Weak Pointer (weak_ptr)

weak_ptr - non-owning reference to object owned by **shared_ptr**. Prevents circular references.

25.8.1 Circular Reference Problem

```

1 // PROBLEM: Circular references cause memory leak
2 class Node {
3 public:

```

```

4     shared_ptr<Node> next;
5 };
6
7 auto node1 = make_shared<Node>();
8 auto node2 = make_shared<Node>();
9
10 node1->next = node2;
11 node2->next = node1; // CIRCULAR REFERENCE!
12
13 // When we set node1 and node2 to null:
14 node1 = nullptr;
15 node2 = nullptr;
16 // Leak! Both nodes are never deleted because they keep owning each other

```

25.8.2 Solution with Weak Pointer

```

1 class Node {
2 public:
3     shared_ptr<Node> next;
4     weak_ptr<Node> prev; // Non-owning reference
5 };
6
7 auto node1 = make_shared<Node>();
8 auto node2 = make_shared<Node>();
9
10 node1->next = node2; // node2 owned by node1
11 node2->prev = node1; // node1 not owned by node2
12
13 // When node1 out of scope, node2 is deleted
14 // When node2 out of scope, node1 is deleted (if no other owners)
15 // No leak!

```

25.8.3 Weak Pointer Usage

```

1 weak_ptr<Resource> weak = ptr; // Create weak_ptr from shared_ptr
2
3 // weak_ptr doesn't increment reference count
4 cout << ptr.use_count() << "\n"; // 1 (not incremented by weak)
5
6 // To use weak_ptr, must lock it to get shared_ptr
7 if (auto shared = weak.lock()) {
8     // shared_ptr valid, resource still exists
9     shared->use();
10 } else {
11     // Resource was deleted
12     cout << "Resource deleted\n";
13 }
14
15 // Check if expired
16 if (weak.expired()) {
17     cout << "Resource deleted\n";
18 }

```

25.8.4 Practical Circular Reference Solutions

```

1 // Parent-Child relationship
2 class Parent;
3

```

```

4 class Child {
5     weak_ptr<Parent> parent; // Non-owning back-reference
6 public:
7     void setParent(shared_ptr<Parent> p) {
8         parent = p;
9     }
10 };
11
12 class Parent {
13     shared_ptr<Child> child; // Owns child
14 public:
15     void setChild(shared_ptr<Child> c) {
16         child = c;
17     }
18 };
19
20 auto parent = make_shared<Parent>();
21 auto child = make_shared<Child>();
22
23 parent->setChild(child);
24 child->setParent(parent);
25 // No circular reference! Safe to use

```

25.9 Smart Pointer Comparison

	unique_ptr	shared_ptr	weak_ptr
Ownership	Exclusive	Shared	None
Copy	No (move)	Yes	No
Reference Count	No	Yes	No
Use Count	N/A	Yes	No
Overhead	Minimal	Medium	Medium
Thread Safe	No	Yes (atomic)	Yes (atomic)
Circular Ref	No issue	Problem	Solution
When to use:			
unique_ptr	- Single owner, exclusive access		
shared_ptr	- Multiple owners needed		
weak_ptr	- Break circular references		

25.10 MOVE SEMANTICS & RVALUE REFERENCES

25.11 3.1 Lvalue vs Rvalue

Understanding the difference is crucial for C++11.

```

1 // LVALUE - has memory address, survives beyond expression
2 int x = 5; // x is lvalue (persists)
3 int& ref = x; // Can bind lvalue reference
4
5 // RVALUE - temporary, doesn't have persistent storage
6 int y = x + 3; // (x+3) is rvalue (temporary)
7 // int& ref2 = (x+3); // ERROR! Can't bind lvalue ref to rvalue
8
9 // Rvalue reference binding (C++11)

```



```

10 int&& rref = x + 3;    // OK! Rvalue reference to temporary
11 cout << rref << "\n"; // 8
12
13 // But can't bind to lvalue
14 int a = 10;
15 // int&& rref2 = a;    // ERROR! Can't bind to lvalue

```

25.11.1 Lvalue & Rvalue Examples

```

1 int a = 5;
2
3 // Lvalue expressions (have addresses)
4 a;                // lvalue
5 a + 0;            // rvalue (result is temporary)
6 a = 10;           // lvalue (a itself is lvalue)
7 a++;             // rvalue (returns temporary copy)
8 ++a;             // lvalue (returns reference to a)
9
10 // Function returning by reference - lvalue
11 int& getRef() { static int x; return x; }
12 getRef() = 20;    // lvalue - can assign to
13
14 // Function returning by value - rvalue
15 int getValue() { return 42; }
16 // getValue() = 20; // ERROR - rvalue can't be assigned to
17
18 // String examples
19 string s = "hello"; // s is lvalue
20 string t = s;       // s is lvalue
21 string u = "world"; // "world" is rvalue
22 // u = getValue();  // rvalue expression

```

25.12 3.2 Move Constructor & Move Assignment

Move semantics avoid expensive copying by transferring ownership.

25.12.1 Before C++11 - Expensive Copy

```

1 class String {
2 private:
3     char* data;
4     size_t size;
5
6 public:
7     // Copy constructor - EXPENSIVE
8     String(const String& other) {
9         cout << "Copy constructor\n";
10        size = other.size;
11        data = new char[size + 1];
12        strcpy(data, other.data); // Copy memory
13    }
14
15    // Assignment - EXPENSIVE
16    String& operator=(const String& other) {
17        if (this != &other) {
18            delete[] data;

```

```

19         size = other.size;
20         data = new char[size + 1];
21         strcpy(data, other.data);
22     }
23     return *this;
24 }
25
26 ~String() {
27     delete[] data;
28 }
29 };
30
31 // This calls copy constructor - expensive!
32 String s1 = "hello";
33 String s2 = s1;           // Copy all data

```

25.12.2 With C++11 - Efficient Move

```

1 class String {
2 private:
3     char* data;
4     size_t size;
5
6 public:
7     // Copy constructor
8     String(const String& other) {
9         cout << "Copy constructor\n";
10        size = other.size;
11        data = new char[size + 1];
12        strcpy(data, other.data);
13    }
14
15    // MOVE constructor (C++11) - EFFICIENT
16    String(String&& other) noexcept {
17        cout << "Move constructor\n";
18        data = other.data;           // Transfer ownership
19        size = other.size;
20        other.data = nullptr;       // Leave other empty
21        other.size = 0;
22    }
23
24    // Copy assignment
25    String& operator=(const String& other) {
26        if (this != &other) {
27            delete[] data;
28            size = other.size;
29            data = new char[size + 1];
30            strcpy(data, other.data);
31        }
32        return *this;
33    }
34
35    // MOVE assignment (C++11) - EFFICIENT
36    String& operator=(String&& other) noexcept {
37        cout << "Move assignment\n";
38        if (this != &other) {
39            delete[] data;
40            data = other.data;       // Transfer ownership
41            size = other.size;
42            other.data = nullptr;   // Leave other empty
43            other.size = 0;
44        }

```

```

45     return *this;
46 }
47
48 ~String() {
49     delete[] data;
50 }
51 };
52
53 // Compiler chooses move constructor (rvalue)
54 String s1 = String("hello"); // Move constructor called, not copy!
55
56 // Explicit move
57 String s2 = "world";
58 String s3 = move(s2);        // Move assignment called

```

25.12.3 Move Constructor Details

```

1  class Vector {
2  private:
3      int* data;
4      int size;
5
6  public:
7      // Regular constructor
8      Vector(int n) : size(n), data(new int[n]) {}
9
10     // Move constructor signature: T(T&& other) noexcept
11     Vector(Vector&& other) noexcept
12         : size(other.size), data(other.data) {
13         other.data = nullptr; // Critical: leave source empty!
14         other.size = 0;
15     }
16
17     // Move assignment
18     Vector& operator=(Vector&& other) noexcept {
19         if (this != &other) {
20             delete[] data; // Clean up old data
21             data = other.data;
22             size = other.size;
23             other.data = nullptr;
24             other.size = 0;
25         }
26         return *this;
27     }
28
29     ~Vector() {
30         delete[] data;
31     }
32 };
33
34 Vector createVector(int n) {
35     Vector v(n);
36     // Initialize...
37     return v; // Move constructor called, not copy
38 }
39
40 Vector v = createVector(100); // Efficient! Only one construction

```

25.12.4 std::move

Use `std::move()` to force move semantics on lvalues.

```

1 #include <utility>
2
3 Vector v1(100);
4 Vector v2(200);
5
6 // Copy
7 v2 = v1; // Copy assignment (v1 still valid)
8
9 // Move
10 v2 = move(v1); // Move assignment (v1 becomes invalid)
11
12 // After move, v1 is valid but empty
13 cout << v1.size() << "\n"; // 0 (moved from)
14
15 // Perfect for returning from function
16 Vector getVector() {
17     Vector v(100);
18     // Initialize...
19     return v; // Compiler elides copy, uses move anyway
20 }
21
22 // Container operations use move
23 vector<Vector> vv;
24 Vector v3(50);
25 vv.push_back(move(v3)); // Move, not copy
26
27 // Array operations
28 string s1 = "hello";
29 string s2 = move(s1); // Move string data
30 cout << s1 << "\n"; // Empty (s1 moved from)

```

25.13 3.3 Perfect Forwarding

Pass parameters to another function preserving their value category (lvalue/rvalue).

25.13.1 The Problem

```

1 // Without perfect forwarding, you lose the value category
2 template<typename T>
3 void wrapper(T& arg) {
4     // arg is always lvalue
5     process(arg); // Can't call rvalue version
6 }
7
8 // Solution: Perfect forwarding with std::forward
9 template<typename T>
10 void perfectWrapper(T&& arg) {
11     // arg is universal reference
12     // std::forward preserves whether arg was lvalue or rvalue
13     process(forward<T>(arg));
14 }

```

25.13.2 Implementation

```

1 #include <utility>
2
3 void process(int& x) {
4     cout << "Lvalue reference\n";
5 }
6
7 void process(int&& x) {
8     cout << "Rvalue reference\n";
9 }
10
11 // WITHOUT perfect forwarding
12 template<typename T>
13 void wrapper1(T arg) {
14     process(arg); // arg is lvalue, always calls lvalue version
15 }
16
17 // WITH perfect forwarding
18 template<typename T>
19 void wrapper2(T&& arg) {
20     process(forward<T>(arg)); // Forwards correctly
21 }
22
23 int main() {
24     int x = 5;
25
26     wrapper1(x);           // Lvalue reference (correct)
27     wrapper1(10);         // Lvalue reference (WRONG! Should be rvalue)
28
29     wrapper2(x);           // Lvalue reference (correct)
30     wrapper2(10);         // Rvalue reference (correct!)
31
32     return 0;
33 }

```

25.13.3 Perfect Forwarding with Multiple Parameters

```

1 template<typename T1, typename T2>
2 void forward_to_process(T1&& arg1, T2&& arg2) {
3     process(forward<T1>(arg1), forward<T2>(arg2));
4 }
5
6 // Works correctly with all combinations:
7 forward_to_process(lval1, lval2); // Both forwarded as lvalues
8 forward_to_process(rval1, rval2); // Both forwarded as rvalues
9 forward_to_process(lval1, rval2); // Mixed - each forwarded correctly

```

25.13.4 Universal Reference

The parameter `T&&` is called a **universal reference** (or forwarding reference) when `T` is a template parameter.

```

1 // Universal reference - can be lvalue or rvalue
2 template<typename T>
3 void universal(T&& param) { }
4
5 // Rvalue reference - always rvalue
6 void notUniversal(int&& param) { } // Not template
7
8 class X {
9     // This is rvalue reference (not universal, because class is known)

```

```

10     void method(int&& x) { }
11 };
12
13 // When you see T&&:
14 // 1. If T is deduced from template parameter Universal reference
15 // 2. Otherwise Rvalue reference

```

25.14 LAMBDA FUNCTIONS

25.15 4.1 Lambda Basics

Lambda functions are anonymous functions that can capture variables.

```

1 #include <iostream>
2 using namespace std;
3
4 // Simplest lambda - no parameters, no capture
5 auto greet = []() {
6     cout << "Hello!\n";
7 };
8 greet();
9
10 // Lambda with parameters
11 auto add = [](int a, int b) {
12     return a + b;
13 };
14 cout << add(5, 3) << "\n"; // 8
15
16 // Lambda with return type (usually not needed)
17 auto multiply = [](int a, int b) -> int {
18     return a * b;
19 };
20
21 // Lambda with auto parameter (C++14)
22 auto square = [](auto x) {
23     return x * x;
24 };
25 cout << square(5) << "\n"; // 25
26 cout << square(3.14) << "\n"; // 9.8596

```

25.15.1 Lambda with Capture

Capture variables from enclosing scope.

```

1 int global_x = 10;
2
3 // Capture by value [=]
4 auto cap_value = [global_x]() {
5     cout << global_x << "\n"; // Captures value at creation
6     // global_x = 20; // ERROR - can't modify captured value
7 };
8
9 // Capture by reference [&]
10 auto cap_ref = [&global_x]() {
11     cout << global_x << "\n"; // Uses current value
12     global_x = 20; // OK - modifies original
13 };
14

```

```

15 // Capture by value with default [=, &x]
16 int x = 5, y = 10;
17 auto mixed1 = [=, &x]() {
18     // y captured by value, x by reference
19 };
20
21 // Capture by reference with default [&, =x]
22 auto mixed2 = [&, x]() {
23     // x captured by value, others by reference
24 };
25
26 // Specific captures
27 auto specific = [x, y, &global_x]() {
28     // x, y by value; global_x by reference
29 };

```

25.15.2 Lambda Capture Examples

```

1 #include <vector>
2
3 vector<int> v = {1, 2, 3, 4, 5};
4 int multiplier = 2;
5
6 // Capture multiplier for use in lambda
7 vector<int> result;
8
9 // Method 1: Capture by value
10 transform(v.begin(), v.end(), back_inserter(result),
11     [multiplier](int x) {
12         return x * multiplier; // multiplier captured
13     });
14
15 // Method 2: Capture by reference
16 int sum = 0;
17 for_each(v.begin(), v.end(),
18     [&sum](int x) {
19         sum += x; // Modifies original sum
20     });
21
22 // Method 3: Implicit capture
23 int factor = 3;
24 auto calc = [=]() {
25     return v[0] * factor; // factor automatically captured
26 };
27
28 // Method 4: Mutable lambda
29 auto counter = [count = 0]() mutable {
30     return ++count;
31 };
32 cout << counter() << "\n"; // 1
33 cout << counter() << "\n"; // 2
34 cout << counter() << "\n"; // 3

```

25.16 4.2 Lambda with Algorithms

Lambdas are perfect for use with STL algorithms.

```

1 #include <algorithm>
2 #include <vector>
3
4 vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
5
6 // Find first even number
7 auto it = find_if(v.begin(), v.end(),
8     [](int x) { return x % 2 == 0; });
9 cout << *it << "\n"; // 2
10
11 // Count numbers > 5
12 int count = count_if(v.begin(), v.end(),
13     [](int x) { return x > 5; });
14 cout << count << "\n"; // 5
15
16 // Transform: square each element
17 transform(v.begin(), v.end(), v.begin(),
18     [](int x) { return x * x; });
19
20 // Sort with custom comparator
21 sort(v.begin(), v.end(),
22     [](int a, int b) { return a > b; }); // Descending
23
24 // Remove if
25 v.erase(remove_if(v.begin(), v.end(),
26     [](int x) { return x % 2 == 0; }), v.end());

```

25.17 4.3 Advanced Lambda Features

25.17.1 Init Capture (Generalized Capture - C++14)

```

1 auto ptr = make_unique<int>(42);
2
3 // Can't capture unique_ptr directly (move-only)
4 // auto lambda = [ptr = move(ptr)]() { }; // ERROR in C++11
5
6 // C++14 - init capture allows move
7 auto lambda = [ptr = move(ptr)]() {
8     cout << *ptr << "\n";
9 };
10 // Now lambda owns the unique_ptr!
11
12 // Other init capture examples
13 auto value_capture = [x = 5]() { return x; }; // Initializes x to 5
14 auto copy_capture = [v = original_vector]() { }; // Copies vector

```

25.17.2 Recursive Lambdas

```

1 // Lambdas can call themselves if captured
2 function<int(int)> factorial; // Forward declare
3 factorial = [&factorial](int n) {
4     return n <= 1 ? 1 : n * factorial(n - 1);
5 };
6
7 cout << factorial(5) << "\n"; // 120

```


25.17.3 Const and Mutable Lambdas

```

1 // By default, lambdas are const
2 auto const_lambda = [x = 0]() {
3     // x = 1; // ERROR - can't modify captured variables
4 };
5
6 // Mutable lambda can modify captured variables
7 auto mutable_lambda = [x = 0]() mutable {
8     x++; // OK - modifies copy of x
9     return x;
10 };
11
12 cout << mutable_lambda() << "\n"; // 1
13 cout << mutable_lambda() << "\n"; // 2 (x persists)

```

25.18 4.4 Lambda Under the Hood

When you write a lambda, the compiler generates a class (functor) for you.

Source Code:

```

1 int factor = 10;
2 auto lambda = [factor](int x) { return x * factor; };

```

Compiler Generated Code (Approximate):

```

1 class __Lambda_1 {
2 private:
3     int m_factor; // Captured variable
4 public:
5     __Lambda_1(int factor) : m_factor(factor) {}
6
7     // operator() is const by default!
8     int operator()(int x) const {
9         return x * m_factor;
10    }
11 };
12
13 // Usage
14 __Lambda_1 lambda(factor);

```

Mutable Lambda: If you use mutable, the operator() is NOT const, allowing modification of m_factor.

Capture by Reference:

```

1 class __Lambda_Ref {
2     int& m_ref; // Pointer under the hood
3 public:
4     __Lambda_Ref(int& ref) : m_ref(ref) {}
5     int operator()(int x) const { return x * m_ref; }
6 };

```

25.19 VARIADIC TEMPLATES

25.20 5.1 Parameter Packs

Templates that accept variable number of parameters.

25.20.1 Basic Variadic Template

```

1 #include <iostream>
2 using namespace std;
3
4 // Base case (template specialization for no parameters)
5 template<typename T>
6 T sum(T val) {
7     return val;
8 }
9
10 // Recursive case (typename... is parameter pack)
11 template<typename T, typename... Rest>
12 T sum(T first, Rest... rest) {
13     return first + sum(rest...);
14 }
15
16 int main() {
17     cout << sum(1, 2, 3, 4, 5) << "\n";           // 15
18     cout << sum(1.5, 2.5, 3.0) << "\n";           // 7.0
19     cout << sum("hello") << "\n";                 // "hello"
20
21     return 0;
22 }

```

25.20.2 Understanding Parameter Packs

```

1 // template<typename... Args>
2 //      ^           ^           ^
3 //      |           |           |
4 //      keyword  pattern  name
5
6 template<typename... Types>
7 void printTypes() {
8     // Types is a parameter pack
9 }
10
11 // Calling with different numbers of parameters
12 printTypes<int>();                // Types = <int>
13 printTypes<int, double>();        // Types = <int, double>
14 printTypes<int, double, string>(); // Types = <int, double, string>

```

25.20.3 Pack Expansion

```

1 // sizeof... operator
2 template<typename... Args>
3 void printCount(Args... args) {
4     cout << "Number of arguments: " << sizeof...(Args) << "\n";
5     cout << "Number of values: " << sizeof...(args) << "\n";
6 }
7
8 printCount(1, 2, 3);              // Both print: 3
9 printCount("a", 1.5);             // Both print: 2
10 printCount();                     // Both print: 0

```

25.20.4 Variadic Function Print Example

```

1 #include <iostream>
2 using namespace std;
3
4 // Base case - end recursion
5 void print() { }
6
7 // Recursive case
8 template<typename T, typename... Rest>
9 void print(T first, Rest... rest) {
10     cout << first << " ";
11     print(rest...); // Pack expansion - recursively process rest
12 }
13
14 int main() {
15     print(1, 2, 3, 4, 5);           // 1 2 3 4 5
16     print("hello", 3.14, 42);      // hello 3.14 42
17     print();                       // (nothing)
18
19     return 0;
20 }

```

25.20.5 Fold Expressions (C++17)

Modern way to process parameter packs.

```

1 // C++17 - Fold expressions (much cleaner!)
2 template<typename... Args>
3 auto sum(Args... args) {
4     return (... + args); // Left fold: ((a + b) + c) + d
5 }
6
7 template<typename... Args>
8 auto product(Args... args) {
9     return (... * args); // Left fold: ((a * b) * c) * d
10 }
11
12 cout << sum(1, 2, 3, 4) << "\n"; // 10
13 cout << product(2, 3, 4) << "\n"; // 24
14
15 // Other fold directions
16 // (args + ...) is right fold
17 // (args * ...) with no operator is right fold

```

25.21 RANGE-BASED FOR LOOPS

25.22 6.1 Iterating Over Containers

```

1 #include <vector>
2 #include <string>
3
4 vector<int> v = {1, 2, 3, 4, 5};
5
6 // Old way (C++98)
7 for (int i = 0; i < v.size(); i++) {
8     cout << v[i] << " ";
9 }

```

```

10
11 // Range-based for (C++11)
12 for (int val : v) {
13     cout << val << " ";
14 }
15
16 // With auto (cleaner)
17 for (auto val : v) {
18     cout << val << " ";
19 }
20
21 // By reference (to modify)
22 for (auto& val : v) {
23     val *= 2; // Modifies original
24 }
25
26 // Const reference (efficient, can't modify)
27 for (const auto& val : v) {
28     cout << val << " ";
29 }

```

25.22.1 Range-Based For with Different Containers

```

1 // Vector
2 vector<int> vec = {1, 2, 3};
3 for (auto x : vec) { cout << x << " "; }
4
5 // Array
6 int arr[] = {4, 5, 6};
7 for (auto x : arr) { cout << x << " "; }
8
9 // String
10 string s = "hello";
11 for (auto c : s) { cout << c << " "; } // h e l l o
12
13 // Map
14 map<string, int> m = {{ "a", 1 }, { "b", 2 }};
15 for (auto [key, value] : m) { // C++17 structured binding
16     cout << key << ": " << value << " ";
17 }
18
19 // Set
20 set<int> s = {10, 20, 30};
21 for (auto x : s) { cout << x << " "; }
22
23 // String iteration
24 string text = "hello";
25 for (char c : text) {
26     cout << c << " "; // h e l l o
27 }

```

25.22.2 Custom Range-Based For

```

1 // Custom container with iterator support
2 class MyContainer {
3 private:
4     int data[5] = {1, 2, 3, 4, 5};
5
6 public:
7     int* begin() { return data; }

```

```

8     int* end() { return data + 5; }
9 };
10
11 MyContainer container;
12 for (auto val : container) {
13     cout << val << " "; // Works with range-based for!
14 }
15
16 // Or with const
17 class MyList {
18 private:
19     vector<int> items;
20
21 public:
22     MyList() : items({10, 20, 30}) {}
23
24     auto begin() { return items.begin(); }
25     auto end() { return items.end(); }
26     auto begin() const { return items.cbegin(); }
27     auto end() const { return items.cend(); }
28 };

```

25.23 UNIFORM INITIALIZATION

25.24 7.1 Brace Initialization

```

1 #include <vector>
2 #include <string>
3
4 // Before C++11
5 int a = 5;
6 vector<int> v;
7 v.push_back(1);
8 v.push_back(2);
9 v.push_back(3);
10
11 // C++11 Uniform Initialization
12 int b{5}; // Direct initialization
13 vector<int> v2{1, 2, 3, 4, 5}; // List initialization
14 string s{"hello"}; // Constructor call
15
16 // Arrays
17 int arr[5] = {1, 2, 3}; // Old way
18 int arr2[5]{1, 2, 3}; // New way
19
20 // Structs
21 struct Point {
22     int x, y;
23 };
24 Point p{10, 20}; // Uniform initialization
25
26 // Classes
27 class Rectangle {
28 public:
29     Rectangle(int w, int h) {}
30 };
31 Rectangle r{100, 50}; // Uniform initialization

```

25.24.1 Initializer Lists

```

1 #include <initializer_list>
2
3 // Container initialization
4 vector<int> v{1, 2, 3, 4, 5};           // Built-in support
5 set<int> s{5, 3, 1, 4, 2};
6
7 // Custom class with initializer_list
8 class Numbers {
9 private:
10     vector<int> data;
11
12 public:
13     Numbers(initializer_list<int> init) : data(init) {}
14 };
15
16 Numbers nums{1, 2, 3, 4, 5};
17
18 // Function with initializer_list
19 void printNumbers(initializer_list<int> nums) {
20     for (int n : nums) {
21         cout << n << " ";
22     }
23 }
24
25 printNumbers({10, 20, 30});

```

25.25 NULLPTR & STRONGLY TYPED ENUMS

25.26 8.1 nullptr

```

1 // Before C++11 - NULL was problematic
2 #define NULL 0
3 int* ptr = NULL;           // Actually 0, not a pointer!
4 void func(int x) { }
5 void func(int* ptr) { }
6 func(NULL);               // Ambiguous! Which func?
7
8 // C++11 - nullptr
9 int* ptr = nullptr;       // Actual null pointer type
10 void func(int* ptr) { }
11 func(nullptr);           // Clear - calls pointer version
12
13 // nullptr conversions
14 bool b = nullptr;        // ERROR - can't convert to bool
15 if (ptr == nullptr) { }  // OK
16 if (!ptr) { }            // OK - nullptr is falsy
17 if (ptr) { }             // OK - checks if not null
18
19 // nullptr type
20 nullptr_t null_val = nullptr;

```

25.27 8.2 Strongly Typed Enums

```

1 // Old enum (C++98) - pollutes namespace
2 enum Color {
3     RED,          // Color::RED? RED?
4     GREEN,
5     BLUE
6 };
7 int x = RED;          // Implicit conversion to int!
8
9 // New scoped enum (C++11)
10 enum class Status {
11     OK,            // Must use Status::OK
12     ERROR,
13     PENDING
14 };
15
16 Status s = Status::OK;
17 // int i = Status::OK;    // ERROR - no implicit conversion
18
19 // Enum with specific type
20 enum class Priority : unsigned char {
21     LOW = 1,
22     MEDIUM = 2,
23     HIGH = 3
24 };
25
26 Priority p = Priority::HIGH;
27 unsigned char value = (unsigned char)p; // Explicit cast

```

25.28 TUPLE, ARRAY, & UNORDERED CONTAINERS

25.29 9.1 Tuple

```

1 #include <tuple>
2
3 // Creating tuple
4 tuple<int, string, double> t1(42, "hello", 3.14);
5 auto t2 = make_tuple(100, "world", 2.71);
6
7 // Accessing elements
8 cout << get<0>(t1) << "\n";    // 42
9 cout << get<1>(t1) << "\n";    // "hello"
10 cout << get<2>(t1) << "\n";    // 3.14
11
12 // Unpacking (C++17)
13 auto [num, str, dec] = t1;
14 cout << num << ", " << str << ", " << dec << "\n";
15
16 // Tuple size
17 cout << tuple_size<decltype(t1)>::value << "\n"; // 3
18
19 // Compare tuples
20 auto t3 = make_tuple(42, "hello", 3.14);
21 cout << (t1 == t3) << "\n";    // 1 (true)
22
23 // Tuple of references
24 int a = 5, b = 10;
25 auto t4 = tie(a, b);            // Tuple of references

```

```

26 get<0>(t4) = 20;
27 cout << a << "\n";           // 20

```

25.30 9.2 Array

```

1  #include <array>
2
3  // std::array - type-safe, fixed-size array
4  array<int, 5> arr{1, 2, 3, 4, 5};
5
6  // Access elements
7  cout << arr[0] << "\n";       // 1
8  cout << arr.at(1) << "\n";   // 2
9
10 // Iteration
11 for (auto x : arr) {
12     cout << x << " ";
13 }
14
15 // Size operations
16 cout << arr.size() << "\n";    // 5
17 cout << arr.empty() << "\n";  // false
18
19 // Use with algorithms
20 sort(arr.begin(), arr.end());
21 reverse(arr.begin(), arr.end());
22
23 // Compare arrays
24 array<int, 5> arr2{1, 2, 3, 4, 5};
25 cout << (arr == arr2) << "\n"; // false (different order after sort)
26
27 // Multi-dimensional
28 array<array<int, 3>, 3> matrix{
29     {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
30 };
31 cout << matrix[0][0] << "\n";  // 1

```

25.31 9.3 Unordered Containers

```

1  #include <unordered_map>
2  #include <unordered_set>
3
4  // Unordered Map (hash table)
5  unordered_map<string, int> map{
6      {"one", 1}, {"two", 2}, {"three", 3}
7  };
8
9  map["four"] = 4;           // Insert
10 cout << map["one"] << "\n"; // 1
11 map.erase("two");          // Remove
12
13 // Iteration (order is arbitrary)
14 for (const auto& [key, value] : map) {
15     cout << key << ": " << value << "\n";
16 }
17
18 // Hash info
19 cout << map.bucket_count() << "\n"; // Number of buckets

```



```

20 cout << map.load_factor() << "\n"; // Load factor
21 cout << map.max_load_factor() << "\n"; // Max load factor
22
23 // Unordered Set
24 unordered_set<int> set{5, 3, 1, 4, 2};
25
26 if (set.count(3)) {
27     cout << "3 found\n";
28 }
29
30 // Custom hash function
31 struct StringHash {
32     size_t operator()(const string& s) const {
33         hash<string> h;
34         return h(s);
35     }
36 };
37
38 unordered_set<string, StringHash> customSet{"a", "b", "c"};

```

25.32 DECLTYPE & TYPE TRAITS

25.33 10.1 decltype

Already covered in detail in Section 1.2, but here are more examples:

```

1 int x = 5;
2 decltype(x) y = 10; // int
3
4 vector<int> v;
5 decltype(v.begin()) it = v.begin(); // vector<int>::iterator
6
7 // In template context
8 template<typename T, typename U>
9 auto divide(T a, U b) -> decltype(a / b) {
10     return a / b;
11 }
12
13 // Type checking
14 cout << is_same<decltype(x), int>::value << "\n"; // true
15 cout << is_same<decltype(3.14), double>::value << "\n"; // true

```

25.34 10.2 Type Traits

```

1 #include <type_traits>
2
3 // Checking types
4 cout << is_integral<int>::value << "\n"; // true
5 cout << is_integral<double>::value << "\n"; // false
6 cout << is_floating_point<double>::value << "\n"; // true
7 cout << is_pointer<int*>::value << "\n"; // true
8 cout << is_reference<int&>::value << "\n"; // true
9
10 // Removing qualifiers
11 remove_const<const int>::type a = 5; // int
12 remove_reference<int&>::type b = 10; // int

```

```

13
14 // Adding qualifiers
15 add_const<int>::type c = 5;           // const int
16 add_pointer<int>::type d = &a;       // int*
17
18 // Checking relationships
19 cout << is_same<int, int>::value << "\n";           // true
20 cout << is_same<int, double>::value << "\n";         // false
21
22 // Conditional types
23 conditional<true, int, double>::type e = 5; // int
24 conditional<false, int, double>::type f = 3.14; // double
25
26 // Function traits
27 template<typename T>
28 struct is_iterable {
29     static constexpr bool value =
30         is_integral_v<T> || is_floating_point_v<T>;
31 };
32
33 // SFINAE (Substitution Failure Is Not An Error)
34 template<typename T>
35 enable_if_t<is_integral<T>::value>
36 process(T value) {
37     cout << "Processing integer: " << value << "\n";
38 }
39
40 template<typename T>
41 enable_if_t<is_floating_point<T>::value>
42 process(T value) {
43     cout << "Processing float: " << value << "\n";
44 }
45
46 process(42);           // "Processing integer: 42"
47 process(3.14);         // "Processing float: 3.14"

```

25.35 CONCURRENCY & THREADING

25.36 11.1 Threads

```

1 #include <thread>
2 #include <iostream>
3
4 // Simple thread
5 void workerFunction() {
6     cout << "Worker thread executing\n";
7 }
8
9 int main() {
10     // Create and launch thread
11     thread t(workerFunction);
12
13     // Wait for thread to finish
14     t.join();
15
16     cout << "Main thread continues\n";
17
18     return 0;

```

```
19 }
```

25.36.1 Threading with Parameters

```
1 #include <thread>
2 #include <iostream>
3
4 void add(int a, int b) {
5     cout << a << " + " << b << " = " << (a + b) << "\n";
6 }
7
8 int main() {
9     // Pass parameters to thread function
10    thread t1(add, 5, 3);
11    thread t2(add, 10, 20);
12
13    t1.join();
14    t2.join();
15
16    return 0;
17 }
```

25.36.2 Thread with Lambda

```
1 #include <thread>
2
3 int main() {
4     int value = 42;
5
6     thread t([value]() {
7         cout << "Lambda captured: " << value << "\n";
8     });
9
10    t.join();
11
12    return 0;
13 }
```

25.36.3 Multiple Threads

```
1 #include <thread>
2 #include <vector>
3
4 int main() {
5     vector<thread> threads;
6
7     // Create multiple threads
8     for (int i = 0; i < 5; i++) {
9         threads.emplace_back([i]() {
10             cout << "Thread " << i << " executing\n";
11         });
12     }
13
14     // Wait for all
15     for (auto& t : threads) {
16         t.join();
17     }
18 }
```

```
19     return 0;
20 }
```

25.37 11.2 Mutex & Locks

```
1  #include <thread>
2  #include <mutex>
3
4  int counter = 0;
5  mutex mtx;
6
7  void incrementCounter() {
8      for (int i = 0; i < 100000; i++) {
9          lock_guard<mutex> lock(mtx); // RAII locking
10         counter++;
11     }
12 }
13
14 int main() {
15     thread t1(incrementCounter);
16     thread t2(incrementCounter);
17
18     t1.join();
19     t2.join();
20
21     cout << "Counter: " << counter << "\n"; // 200000
22
23     return 0;
24 }
```

25.38 11.3 Atomic Operations

```
1  #include <atomic>
2  #include <thread>
3
4  atomic<int> counter(0);
5
6  void incrementAtomic() {
7      for (int i = 0; i < 100000; i++) {
8          counter++; // Atomic operation, thread-safe
9      }
10 }
11
12 int main() {
13     thread t1(incrementAtomic);
14     thread t2(incrementAtomic);
15
16     t1.join();
17     t2.join();
18
19     cout << "Counter: " << counter << "\n"; // 200000
20
21     return 0;
22 }
```

25.39 REGULAR EXPRESSIONS

25.40 12.1 Basic Regex

```
1 #include <regex>
2 #include <string>
3 #include <iostream>
4
5 // Create regex pattern
6 regex pattern("\\d+"); // Match one or more digits
7
8 string text = "The number is 42";
9
10 // Check if matches
11 if (regex_search(text, pattern)) {
12     cout << "Found digits\n";
13 }
14
15 // Extract matches
16 smatch match;
17 if (regex_search(text, match, pattern)) {
18     cout << "Matched: " << match[0] << "\n"; // "42"
19 }
20
21 // Replace
22 string result = regex_replace(text, pattern, "XXX");
23 cout << result << "\n"; // "The number is XXX"
24
25 // Multiple matches
26 pattern = regex("\\d+");
27 string text2 = "Numbers: 10, 20, 30";
28
29 sregex_iterator iter(text2.begin(), text2.end(), pattern);
30 sregex_iterator end;
31
32 while (iter != end) {
33     cout << iter->str() << "\n"; // 10, 20, 30
34     ++iter;
35 }
```

25.41 NEW LIBRARY FEATURES

25.42 13.1 Standard Library Additions

25.42.1 Chrono Library

```
1 #include <chrono>
2 #include <thread>
3
4 using namespace std::chrono;
5
6 // Time point
7 auto start = high_resolution_clock::now();
8
```

```

9 // Simulate work
10 this_thread::sleep_for(milliseconds(100));
11
12 auto end = high_resolution_clock::now();
13
14 // Duration
15 auto elapsed = duration_cast<milliseconds>(end - start);
16 cout << "Elapsed: " << elapsed.count() << " ms\n";

```

25.42.2 Random Number Generation

```

1 #include <random>
2 #include <iostream>
3
4 // Seed
5 random_device rd;
6 mt19937 gen(rd());
7
8 // Distribution
9 uniform_int_distribution<> dis(1, 6); // Roll dice
10
11 // Generate numbers
12 for (int i = 0; i < 10; i++) {
13     cout << dis(gen) << " "; // 1-6
14 }
15
16 // Different distributions
17 normal_distribution<> normal(0, 1); // Mean 0, stddev 1
18 uniform_real_distribution<> real(0.0, 1.0); // 0.0-1.0
19
20 cout << normal(gen) << "\n";
21 cout << real(gen) << "\n";

```

25.42.3 Function

```

1 #include <functional>
2
3 // Store any callable
4 function<int(int, int)> f1 = [](int a, int b) { return a + b; };
5 function<int(int, int)> f2 = plus<int>();
6
7 cout << f1(5, 3) << "\n"; // 8
8 cout << f2(5, 3) << "\n"; // 8
9
10 // Function vector
11 vector<function<void()>> tasks;
12
13 tasks.push_back([]() { cout << "Task 1\n"; });
14 tasks.push_back([]() { cout << "Task 2\n"; });
15
16 for (auto& task : tasks) {
17     task();
18 }

```

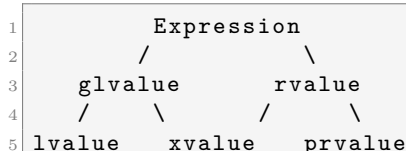
Chapter 26

ADVANCED MOVE SEMANTICS & VALUE CATEGORIES

“Move Semantics” is often misunderstood. It’s not magic; it’s type casting.

26.0.1 4.5.1 The C++17 Value Category Taxonomy

Everything in C++ is an Expression, and every expression has a **Type** and a **Value Category**.



1. **lvalue (Identity + Movable? No)**: Has a name, persists beyond expression.

- `int x`; `x` is an lvalue.
- `std::string s`; `s` is an lvalue.

2. **prvalue (Pure Rvalue)**: No name, temporary, initializes an object.

- `10`, `true`, `nullptr`.
- `std::string("hello")` (constructor call).

3. **xvalue (eXpiring Value)**: Has identity, but can be moved from.

- Result of `std::move(x)`.
- Rvalue reference cast `static_cast<T&&>(x)`.

glvalue = lvalue + xvalue (Has Identity) **rvalue** = prvalue + xvalue (Can be moved from)

26.0.2 4.5.2 `std::move` and `std::forward` Internals

`std::move`: Does NOT move. It unconditionally casts to rvalue reference.

```
1 template<typename T>
2 typename remove_reference<T>::type&& move(T&& t) noexcept {
3     return static_cast<typename remove_reference<T>::type&&>(t);
4 }
```

`std::forward`: Conditionally casts to rvalue reference *only if* the argument was initialized with an rvalue. Used for **Perfect Forwarding**.

```
1 template<typename T>
2 T&& forward(typename remove_reference<T>::type& t) noexcept {
3     return static_cast<T&&>(t);
4 }
```

26.0.3 4.5.3 Reference Collapsing Rules

When templates meet references, types collapse:

- $T\& \& \rightarrow T\&$
- $T\& \&\& \rightarrow T\&$
- $T\&\& \& \rightarrow T\&$
- $T\&\& \&\& \rightarrow T\&\&$ (The only way to get an rvalue reference)

This is why $T\&\&$ in a template is a **Universal Reference** (Forwarding Reference). It can become $T\&$ (lvalue) or $T\&\&$ (rvalue).

Part III

Volume III: C++14 - Refinement & Generics

Chapter 27

C++14 ENHANCEMENTS

27.1 C++14 Overview & Philosophy

C++14 (finalized in 2014) is a **refinement and maintenance release** of C++11.

27.1.1 Timeline & Context

- **2011:** C++11 released (revolutionary)
- **2014:** C++14 released (refinement + useful features)
- **2017:** C++17 released (significant improvements)
- **2020:** C++20 released (revolutionary again)

27.1.2 C++14 Philosophy

- **Smaller, focused** improvements rather than revolution
- **Fix** C++11 issues and limitations
- **Enhance** usability and convenience
- **Add** frequently-requested features
- **Simplify** compile-time computation

27.1.3 Key Features

1. Generic lambdas with `auto` parameters
2. Return type deduction for all functions
3. Binary literals and digit separators
4. `std::make_unique`
5. Relaxed `constexpr` rules
6. Variable templates
7. Library improvements

27.1.4 Why C++14 Matters

While smaller than C++11, C++14 makes C++11 more practical: - Fixes usability issues - Adds convenient features - Improves compile-time computation - Better template support - More standard library features

27.2 GENERIC LAMBDAS

27.3 1.1 Auto Parameters in Lambdas

C++14 allows auto as lambda parameters, creating **generic lambdas**.

27.3.1 Basic Generic Lambda

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // C++11: Type-specific lambda
6 auto add11 = [](int a, int b) { return a + b; };
7 cout << add11(5, 3) << "\n";           // 8
8 // cout << add11(2.5, 3.5) << "\n";    // ERROR - int only
9
10 // C++14: Generic lambda with auto
11 auto add14 = [](auto a, auto b) { return a + b; };
12 cout << add14(5, 3) << "\n";           // 8 (int)
13 cout << add14(2.5, 3.5) << "\n";       // 6 (double)
14 cout << add14(string("Hello"), string(" World")) << "\n"; // "Hello World"

```

27.3.2 Generic Lambda Deduction

```

1 // Each auto parameter is independently deduced
2 auto process = [](auto x, auto y) {
3     // x and y can be different types
4     cout << x << ", " << y << "\n";
5 };
6
7 process(5, 3.14);           // int, double
8 process("hello", 42);       // const char*, int
9 process(3.14, "world");     // double, const char*

```

27.3.3 Generic Lambdas with std::vector

```

1 vector<int> vi = {1, 2, 3};
2 vector<double> vd = {1.1, 2.2, 3.3};
3 vector<string> vs = {"a", "b", "c"};
4
5 // Single generic lambda works with all containers
6 auto print = [](auto val) {
7     cout << val << " ";
8 };
9
10 for_each(vi.begin(), vi.end(), print);
11 cout << "\n";

```

```

12
13 for_each(vd.begin(), vd.end(), print);
14 cout << "\n";
15
16 for_each(vs.begin(), vs.end(), print);
17 cout << "\n";

```

27.3.4 Generic Lambdas with Algorithms

```

1 // Works with any type supporting operator*
2 auto square = [](auto x) { return x * x; };
3
4 vector<int> vi = {1, 2, 3};
5 vector<double> vd = {1.5, 2.5, 3.5};
6
7 transform(vi.begin(), vi.end(), vi.begin(), square);
8 // vi: {1, 4, 9}
9
10 transform(vd.begin(), vd.end(), vd.begin(), square);
11 // vd: {2.25, 6.25, 12.25}

```

27.3.5 Generic Lambda Compile-Time Behavior

```

1 // Type checking still happens at compile time
2 auto multiply = [](auto a, auto b) { return a * b; };
3
4 cout << multiply(5, 3) << "\n"; // 15 (int)
5 cout << multiply(2.5, 3.0) << "\n"; // 7.5 (double)
6
7 // This would compile-time error if * not defined:
8 // multiply("a", "b"); // ERROR - string doesn't support *

```

27.3.6 When to Use Generic Lambdas

```

1 // Good use case: Works with any comparable type
2 auto find_min = [](auto a, auto b) { return a < b ? a : b; };
3
4 int min_int = find_min(5, 3); // 3
5 double min_double = find_min(2.5, 1.5); // 1.5
6 string min_str = find_min("cat", "apple"); // "apple"
7
8 // Bad use case: Type-specific logic
9 auto process = [](auto x) {
10     if (is_integral_v<decltype(x)>) {
11         cout << "Integer\n";
12     } else if (is_floating_point_v<decltype(x)>) {
13         cout << "Float\n";
14     }
15     // Too complex - use template or function overloads instead
16 };

```

27.4 RETURN TYPE DEDUCTION FOR ALL FUNCTIONS

27.5 2.1 Return Type Deduction (C++14 Enhancement)

C++11 allowed return type deduction with `-> auto`, but C++14 simplifies it.

27.5.1 Basic Return Type Deduction

```

1 // C++11: Must use trailing return type
2 auto add_11(int a, int b) -> int { return a + b; }
3 auto divide_11(double a, double b) -> double { return a / b; }
4
5 // C++14: Can deduce from return statement
6 auto add_14(int a, int b) { return a + b; }           // Returns int
7 auto divide_14(double a, double b) { return a / b; } // Returns double
8
9 auto get_string() { return string("hello"); }         // Returns string
10 auto get_vector() { return vector<int>{1, 2, 3}; }    // Returns vector<int>

```

27.5.2 Multiple Return Statements

```

1 // C++14: All returns must be consistent type
2 auto absolute(int x) {
3     if (x >= 0) {
4         return x;           // int
5     } else {
6         return -x;         // Must also be int
7     }
8 }
9
10 // ERROR: Different return types
11 // auto mixed(int x) {
12 //     if (x > 0) {
13 //         return x;         // int
14 //     } else {
15 //         return 3.14;     // double - ERROR!
16 //     }
17 // }

```

27.5.3 Return Type Deduction with Complex Types

```

1 #include <vector>
2 using namespace std;
3
4 // Deduce vector
5 auto get_data() {
6     return vector<int>{1, 2, 3, 4, 5};
7 }
8
9 // Deduce map
10 auto get_map() {
11     return map<string, int>{{"a", 1}, {"b", 2}};
12 }
13
14 // Deduce function
15 auto get_comparator() {
16     return [](int a, int b) { return a > b; };
17 }

```

```

18
19 // Works seamlessly
20 vector<int> v = get_data();
21 map<string, int> m = get_map();
22 auto cmp = get_comparator();

```

27.5.4 Return Type Deduction in Templates

```

1 template<typename T, typename U>
2 auto add(T a, U b) {
3     return a + b; // Type deduced from a + b
4 }
5
6 cout << add(5, 3) << "\n"; // int
7 cout << add(2.5, 3.0) << "\n"; // double
8 cout << add(5, 3.14) << "\n"; // double (int + double = double)
9
10 // Return type varies by input types
11 static_assert(is_same_v<decltype(add(5, 3)), int>);
12 static_assert(is_same_v<decltype(add(5.0, 3)), double>);

```

27.5.5 Recursion with Return Type Deduction

```

1 // C++14: Recursive functions can use auto return type
2 // (But compiler may need hints for some cases)
3
4 auto factorial(int n) {
5     if (n <= 1) return 1;
6     return n * factorial(n - 1);
7 }
8
9 cout << factorial(5) << "\n"; // 120
10
11 // More complex recursion
12 auto fibonacci(int n) {
13     if (n <= 1) return n;
14     return fibonacci(n - 1) + fibonacci(n - 2);
15 }
16
17 cout << fibonacci(10) << "\n"; // 55

```

27.5.6 Benefits of Return Type Deduction

```

1 // Less redundant code
2 // Before: Must specify return type
3 int add_old(int a, int b) -> int { return a + b; }
4
5 // After: Auto-deduced
6 auto add_new(int a, int b) { return a + b; }
7
8 // Refactoring friendly - type changes automatically
9 auto get_value() { return 42; } // int
10 // Later: auto get_value() { return 3.14; } // Changes to double - easy!

```

27.6 AUTO FOR VARIABLES IN LAMBDAS

27.7 3.1 Init Capture with Auto (C++14)

Lambda capture allows variables to be initialized inside the capture list.

27.7.1 Basic Init Capture

```
1 #include <memory>
2 #include <iostream>
3 using namespace std;
4
5 // Create a unique_ptr (move-only type)
6 auto ptr = make_unique<int>(42);
7
8 // C++11: Can't capture unique_ptr (can't copy)
9 // auto lambda11 = [ptr]() { }; // ERROR - can't copy unique_ptr
10
11 // C++14: Init capture allows move
12 auto lambda = [ptr = move(ptr)]() {
13     cout << *ptr << "\n";
14 };
15
16 lambda(); // 42
17 // ptr is now nullptr (moved into lambda)
```

27.7.2 Init Capture with Values

```
1 int original = 10;
2
3 // Capture with transformation
4 auto lambda = [copy = original * 2]() {
5     return copy; // 20
6 };
7
8 cout << lambda() << "\n"; // 20
9 cout << original << "\n"; // Still 10 (original unchanged)
10
11 // Useful for expensive copies
12 vector<int> large_vector = {1, 2, 3, 4, 5};
13 auto process = [copy = vector<int>(large_vector)]() {
14     // Use copy (independent of large_vector)
15 };
16
```

27.7.3 Init Capture with Move

```
1 class Resource {
2 public:
3     Resource() { cout << "Created\n"; }
4     ~Resource() { cout << "Destroyed\n"; }
5     void use() { cout << "Using\n"; }
6 };
7
8 auto res = make_unique<Resource>();
9
10 // Move resource into lambda
11 auto lambda = [res = move(res)]() {
12     if (res) {
```

```
13     res->use();
14 }
15 };
16
17 lambda(); // Using, Destroyed
18 // res is now nullptr
```

27.7.4 Init Capture with Complex Types

```
1 #include <map>
2
3 map<string, int> data{{"a", 1}, {"b", 2}};
4
5 // Copy map into lambda
6 auto process = [data_copy = data]() {
7     for (const auto& [k, v] : data_copy) {
8         cout << k << ": " << v << "\n";
9     }
10 };
11
12 // Modify copy without affecting original
13 auto modify = [data_copy = move(data)]() {
14     data_copy["c"] = 3;
15     // data is moved
16 };
17
18 modify();
```

27.7.5 Init Capture Patterns

```
1 // Pattern 1: Capture with computation
2 auto compute = [val = 2 + 3]() { return val; }; // val = 5
3
4 // Pattern 2: Capture with function call
5 auto get_timestamp = [time = chrono::high_resolution_clock::now]() {
6     return time;
7 };
8
9 // Pattern 3: Capture with complex initialization
10 auto setup = [config = []() {
11     map<string, string> m;
12     m["key"] = "value";
13     return m;
14 }()]() {
15     // config is initialized with map
16 };
17
18 // Pattern 4: Move-only types
19 auto factory = [ptr = make_unique<int>(42)]() {
20     return *ptr;
21 };
```


27.8 BINARY LITERALS & DIGIT SEPARATORS

27.9 4.1 Binary Literals

C++14 introduces `0b` prefix for binary literals.

27.9.1 Binary Literal Syntax

```
1 #include <iostream>
2 using namespace std;
3
4 // Decimal (C++98)
5 int dec = 42;
6
7 // Hexadecimal (C++98)
8 int hex = 0x2A;
9
10 // Octal (C++98)
11 int oct = 052;
12
13 // Binary (C++14)
14 int bin = 0b101010;
15
16 cout << dec << "\n"; // 42
17 cout << hex << "\n"; // 42
18 cout << oct << "\n"; // 42
19 cout << bin << "\n"; // 42
```

27.9.2 Binary Literals Use Cases

```
1 // Bitwise operations are clearer with binary
2 unsigned char flags = 0b11010110;
3 unsigned char mask = 0b00001111;
4
5 unsigned char result = flags & mask; // Much clearer than 0xD6 & 0x0F
6
7 // Single bit operations
8 unsigned int option1 = 0b00000001;
9 unsigned int option2 = 0b00000010;
10 unsigned int option3 = 0b00000100;
11
12 unsigned int enabled = option1 | option3; // 0b00000101
13
14 // Permission bits
15 unsigned char read = 0b100; // 4
16 unsigned char write = 0b010; // 2
17 unsigned char execute = 0b001; // 1
18
19 unsigned char permissions = read | write;
```

27.10 4.2 Digit Separators

C++14 allows single quotes `'` as digit separators for readability.

27.10.1 Digit Separator Examples

```
1 // Large numbers are clearer with separators
2 long large = 1'000'000'000; // One billion
3 double pi = 3.141'592'653; // Pi
4
5 // Binary with separators (very clear)
6 unsigned char bits = 0b1111'0000;
7 unsigned short value = 0xDEAD'BEEF;
8
9 // All numeric literals support separators
10 int decimal = 123'456'789;
11 long long big = 9'223'372'036'854'775'807; // Max int64
12
13 double d = 1'000.123'456; // Works with decimals
14 double e = 1.234'567e3; // Works with exponents
```

27.10.2 Readability Improvement

```
1 // Before (hard to count zeros)
2 unsigned int ip = 192168001001; // What is this?
3
4 // After (clear structure)
5 unsigned int ip = 192'168'001'001; // IP address: 192.168.1.1
6
7 // Before (hard to verify)
8 long big = 9223372036854775807;
9
10 // After (easy to verify)
11 long big = 9'223'372'036'854'775'807; // Max int64
12
13 // Before (unclear magnitude)
14 double money = 1000000000;
15
16 // After (clear)
17 double money = 1'000'000'000; // One billion
```

27.10.3 Digit Separator Rules

```
1 // Valid usage
2 int a = 1'000'000;
3 int b = 0xDEAD'BEEF;
4 int c = 0b1111'0000;
5
6 // NOT at start or end
7 // int bad1 = '123; // ERROR
8 // int bad2 = 123'; // ERROR
9
10 // NOT adjacent to decimal point or exponent
11 // double bad3 = 1.'5; // ERROR
12 // double bad4 = 1e'10; // ERROR
13
14 // Multiple separators are OK
15 int d = 1'000'000'000;
16 int e = 0xFF'FF'FF'FF;
```

27.11 STD::MAKE_UNIQUE

27.12 5.1 std::make_unique (C++14)

std::make_unique creates unique_ptr safely and efficiently.

27.12.1 Before C++14

```

1 #include <memory>
2 using namespace std;
3
4 // C++11: Two-step process
5 unique_ptr<int> ptr1(new int(42));
6 unique_ptr<string> ptr2(new string("hello"));
7 unique_ptr<vector<int>> ptr3(new vector<int>{1, 2, 3});
8
9 // Problem: New and unique_ptr are separate
10 // If exception between new and unique_ptr, memory leaks

```

27.12.2 With std::make_unique

```

1 #include <memory>
2 using namespace std;
3
4 // C++14: One-step, exception-safe
5 auto ptr1 = make_unique<int>(42);
6 auto ptr2 = make_unique<string>("hello");
7 auto ptr3 = make_unique<vector<int>>(initializer_list<int>{1, 2, 3});
8
9 // Automatically determines type
10 // Exception-safe: if constructor throws, no memory leak

```

27.12.3 make_unique with Classes

```

1 class Person {
2 public:
3     string name;
4     int age;
5
6     Person(string n, int a) : name(n), age(a) {
7         cout << "Person created\n";
8     }
9     ~Person() {
10         cout << "Person destroyed\n";
11     }
12 };
13
14 // Create unique_ptr with constructor arguments
15 auto person = make_unique<Person>("Alice", 30);
16 cout << person->name << " is " << person->age << "\n";
17
18 // Automatic cleanup when going out of scope

```

27.12.4 make_unique with Arrays (C++20)

```

1 // C++14: Dynamic sized arrays need manual approach
2 unique_ptr<int[]> arr1(new int[10]);
3
4 // C++20: make_unique supports arrays
5 // auto arr2 = make_unique<int[]>(10); // C++20 only
6
7 // For C++14, use the manual approach:
8 auto arr3 = make_unique<int[]>(); // C++20

```

27.12.5 make_unique vs new

```

1 // Old way (manual, error-prone)
2 function<unique_ptr<int>>()> factory_old = []() {
3     return unique_ptr<int>(new int(42));
4 };
5
6 // New way (cleaner, safer)
7 function<unique_ptr<int>>()> factory_new = []() {
8     return make_unique<int>(42);
9 };
10
11 // Exception safety benefit:
12 class Dangerous {
13 public:
14     Dangerous(unique_ptr<Resource> r) : resource(move(r)) { }
15 private:
16     unique_ptr<Resource> resource;
17 };
18
19 // Safe: If Dangerous constructor throws, r is still managed
20 auto danger = make_unique<Dangerous>(make_unique<Resource>());
21
22 // Unsafe: If Dangerous constructor throws, new Resource() leaks
23 // auto danger = unique_ptr<Dangerous>(
24 //     new Dangerous(unique_ptr<Resource>(new Resource())));

```

27.12.6 make_unique Best Practices

```

1 // Prefer make_unique over new + unique_ptr
2 // Exception-safe
3 auto ptr1 = make_unique<MyClass>(arg1, arg2);
4
5 // More concise
6 auto ptr2 = make_unique<MyClass>();
7
8 // Automatic type deduction
9 auto ptr3 = make_unique<string>("hello");
10
11 // Use in containers
12 vector<unique_ptr<Resource>> resources;
13 resources.push_back(make_unique<Resource>());
14 resources.push_back(make_unique<Resource>());
15 // Automatic cleanup when vector destroyed

```

27.13 RELAXED CONSTEXPR RESTRICTIONS

27.14 6.1 Enhanced constexpr Functions

C++14 relaxes constexpr restrictions, allowing more complex compile-time computation.

27.14.1 C++11 constexpr Limitations

```

1 // C++11: constexpr function must have exactly one statement
2 constexpr int square_11(int x) {
3     return x * x; // Only return statement allowed
4 }
5
6 // C++11: Can't use local variables or loops
7 // constexpr int factorial_11(int n) {
8 //     int result = 1; // ERROR - variable not allowed
9 //     for (int i = 2; i <= n; i++) { // ERROR - loops not allowed
10 //         result *= i;
11 //     }
12 //     return result;
13 // }
```

27.14.2 C++14 constexpr Enhancements

```

1 // C++14: Local variables allowed
2 constexpr int factorial(int n) {
3     int result = 1;
4     for (int i = 2; i <= n; i++) {
5         result *= i;
6     }
7     return result;
8 }
9
10 cout << factorial(5) << "\n"; // Computed at compile-time if possible
11
12 // Compile-time constant
13 int arr[factorial(5)]; // Array of size 120
14
15 // C++14: More complex logic allowed
16 constexpr bool is_prime(int n) {
17     if (n < 2) return false;
18     for (int i = 2; i * i <= n; i++) {
19         if (n % i == 0) return false;
20     }
21     return true;
22 }
23
24 static_assert(is_prime(7)); // Compile-time check
25 static_assert(!is_prime(4)); // Compile-time check
```

27.14.3 C++14 constexpr Features

```

1 // Control flow statements
2 constexpr int abs_diff(int a, int b) {
3     if (a > b) {
4         return a - b;
5     } else {
6         return b - a;
```

```

7     }
8 }
9
10 // Multiple return points
11 constexpr int sign(int x) {
12     if (x > 0) return 1;
13     if (x < 0) return -1;
14     return 0;
15 }
16
17 // Loops
18 constexpr int sum_range(int start, int end) {
19     int total = 0;
20     for (int i = start; i < end; i++) {
21         total += i;
22     }
23     return total;
24 }
25
26 cout << sum_range(1, 10) << "\n"; // 45
27
28 // Fibonacci with better performance
29 constexpr int fib(int n) {
30     if (n <= 1) return n;
31     int a = 0, b = 1;
32     for (int i = 2; i <= n; i++) {
33         int next = a + b;
34         a = b;
35         b = next;
36     }
37     return b;
38 }
39
40 cout << fib(20) << "\n"; // 6765

```

27.14.4 constexpr Still Has Limitations

```

1 // C++14: Still can't do
2 // - Dynamic memory allocation (new/delete)
3 // - Floating-point in some contexts (limited)
4 // - Most library functions
5
6 constexpr void* bad() {
7     // return new int(42); // ERROR
8 }
9
10 // But can call other constexpr functions
11 constexpr int helper() { return 42; }
12 constexpr int caller() {
13     return helper() * 2; // OK
14 }

```

27.14.5 Practical constexpr Uses

```

1 // Compile-time lookup table
2 constexpr int digit_to_value(char d) {
3     if (d >= '0' && d <= '9') return d - '0';
4     if (d >= 'a' && d <= 'f') return d - 'a' + 10;
5     if (d >= 'A' && d <= 'F') return d - 'A' + 10;
6     return -1;

```

```

7 }
8
9 // Compile-time string parsing
10 constexpr int hex_to_int(const char* str) {
11     int result = 0;
12     for (int i = 0; str[i]; i++) {
13         int digit = digit_to_value(str[i]);
14         if (digit < 0) break;
15         result = result * 16 + digit;
16     }
17     return result;
18 }
19
20 constexpr int hex_value = hex_to_int("FF"); // 255, computed at compile-time
21
22 // Compile-time array generation
23 constexpr int powers[10] = {
24     1, 10, 100, 1000, 10000, 100000, 1000000, 10000000, 100000000, 1000000000
25 };
26
27 // Compile-time validation
28 constexpr bool validate_date(int year, int month, int day) {
29     if (month < 1 || month > 12) return false;
30     if (day < 1 || day > 31) return false;
31     if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30)
32         return false;
33     return true;
34 }
35 static_assert(validate_date(2024, 12, 25));

```

27.15 VARIABLE TEMPLATES

27.16 7.1 Template Variables (C++14)

Variables can be templates, not just functions and classes.

27.16.1 Basic Variable Template

```

1 #include <iostream>
2 using namespace std;
3
4 // Template variable
5 template<typename T>
6 constexpr T pi = T(3.141592653589793);
7
8 // Use with different types
9 cout << pi<double> << "\n"; // 3.14159 (double)
10 cout << pi<float> << "\n"; // 3.14159 (float)
11 cout << pi<int> << "\n"; // 3 (int)
12
13 // Can use in computations
14 double circle_area = pi<double> * 5 * 5; // Area of circle with radius 5
15 float sphere_volume = (4.0/3.0) * pi<float> * 3 * 3 * 3;

```

27.16.2 Variable Template with Type Traits

```

1 #include <type_traits>
2
3 // Type trait as variable template
4 template<typename T>
5 constexpr bool is_integral_v = is_integral<T>::value;
6
7 template<typename T>
8 constexpr bool is_floating_point_v = is_floating_point<T>::value;
9
10 // Usage (cleaner than ::value)
11 if (is_integral_v<int>) { }           // true
12 if (is_integral_v<double>) { }       // false
13 if (is_floating_point_v<double>) { } // true

```

27.16.3 Useful Variable Templates

```

1 // Concept-like variable template
2 template<typename T>
3 constexpr bool is_arithmetic_type =
4     is_integral_v<T> || is_floating_point_v<T>;
5
6 static_assert(is_arithmetic_type<int>);
7 static_assert(is_arithmetic_type<double>);
8 // static_assert(is_arithmetic_type<string>); // false
9
10 // Size information
11 template<typename T>
12 constexpr size_t sizeof_v = sizeof(T);
13
14 cout << sizeof_v<int> << "\n";      // 4
15 cout << sizeof_v<double> << "\n";  // 8
16
17 // Min/max values
18 template<typename T>
19 constexpr T max_value = numeric_limits<T>::max();
20
21 template<typename T>
22 constexpr T min_value = numeric_limits<T>::min();
23
24 cout << max_value<int> << "\n";
25 cout << max_value<unsigned char> << "\n";

```

27.16.4 C++17 Standard Variable Templates

```

1 // C++17 standard library additions
2 #include <type_traits>
3
4 // These are now variable templates in C++17
5 is_integral_v<int>;           // true
6 is_floating_point_v<double>;  // true
7 is_same_v<int, int>;          // true
8 remove_const_v<const int>;    // int
9 is_pointer_v<int*>;           // true
10
11 // Work like the old ::value but more concise
12 // is_integral<int>::value;    // Old way
13 is_integral_v<int>;           // New way (C++17)

```


27.17 AGGREGATE MEMBER INITIALIZATION

27.18 8.1 Extended Aggregate Initialization

C++14 extends what can be aggregate-initialized.

27.18.1 Basic Aggregate Initialization (C++98)

```
1 #include <iostream>
2 using namespace std;
3
4 struct Point {
5     int x;
6     int y;
7 };
8
9 // C++98: Brace initialization
10 Point p1 = {10, 20};
11 Point p2{30, 40};
12
13 cout << p1.x << ", " << p1.y << "\n"; // 10, 20
```

27.18.2 With Base Classes (C++14)

```
1 struct Base {
2     int b;
3 };
4
5 struct Derived : Base {
6     int d;
7 };
8
9 // C++14: Can initialize base class members
10 Derived obj{1, 2}; // b=1, d=2
11 cout << obj.b << ", " << obj.d << "\n"; // 1, 2
```

27.18.3 Nested Aggregates

```
1 struct Address {
2     string street;
3     string city;
4 };
5
6 struct Person {
7     string name;
8     int age;
9     Address address;
10 };
11
12 // Nested initialization
13 Person p{"Alice", 30, {"123 Main St", "NYC"}};
14
15 cout << p.name << " lives at " << p.address.street << "\n";
```

27.18.4 C++14 vs C++11 Differences

```

1 struct Point {
2     int x = 0;    // Default member initializer (C++11)
3     int y = 0;
4 };
5
6 // C++11: Default initializer sets x=0, y=0
7 Point p1;        // x=0, y=0
8 Point p2{};      // x=0, y=0
9
10 // Explicit initialization
11 Point p3{10, 20}; // x=10, y=20
12
13 // Partial initialization with defaults
14 // Behavior similar between C++11 and C++14

```

27.19 MEMBER FUNCTION REF/CONST-REF QUALIFIERS

27.20 9.1 Lvalue vs Rvalue Member Functions

C++11 introduced, C++14 standardized: member functions can be qualified as `&` or `&&`.

27.20.1 Member Function Overloading

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Text {
6 private:
7     string data;
8
9 public:
10    Text(string s) : data(s) {}
11
12    // For lvalue (normal objects)
13    string& get_data() & {
14        cout << "Lvalue version\n";
15        return data;
16    }
17
18    // For rvalue (temporaries)
19    string get_data() && {
20        cout << "Rvalue version\n";
21        return move(data);
22    }
23
24    // Const lvalue
25    const string& get_data() const& {
26        cout << "Const lvalue version\n";
27        return data;
28    }
29 };
30
31 int main() {
32    Text t("hello");

```

```

33
34 // Calls lvalue version
35 auto& result1 = t.get_data(); // "Lvalue version"
36
37 // Calls const lvalue version
38 const auto& result2 = t.get_data(); // "Const lvalue version"
39
40 // Calls rvalue version
41 auto result3 = Text("world").get_data(); // "Rvalue version"
42
43 return 0;
44 }

```

27.20.2 Practical Use Case

```

1 class Vector {
2 private:
3     int* data;
4     int size;
5
6 public:
7     Vector() : data(nullptr), size(0) {}
8     Vector(int n) : data(new int[n]), size(n) {}
9
10    // Cheap copy for lvalue - return reference
11    int* get_data() & {
12        return data;
13    }
14
15    // Cheap move for rvalue - return by value
16    int* get_data() && {
17        int* tmp = data;
18        data = nullptr;
19        return tmp;
20    }
21
22    ~Vector() { delete[] data; }
23 };
24
25 Vector createVector(int n) {
26     return Vector(n);
27 }
28
29 int main() {
30     Vector v(100);
31
32     // Lvalue: efficient reference
33     int* ptr1 = v.get_data();
34
35     // Rvalue: efficient move
36     int* ptr2 = createVector(100).get_data();
37
38     return 0;
39 }

```

27.20.3 Const/Volatile Combinations

```

1 class Object {
2 public:
3     // All combinations possible:

```

```

4 void method() & { }           // Lvalue
5 void method() const& { }      // Const lvalue
6 void method() && { }          // Rvalue
7 void method() const&& { }     // Const rvalue
8 void method() volatile& { }   // Volatile lvalue
9 // ... more combinations
10 };

```

27.21 STD::INTEGER_SEQUENCE

27.22 10.1 Compile-Time Integer Sequences

`std::integer_sequence` provides compile-time sequences of integers.

27.22.1 Basic Usage

```

1 #include <utility>
2 #include <iostream>
3 using namespace std;
4
5 // Create a sequence 0, 1, 2, 3, 4
6 using seq = integer_sequence<int, 0, 1, 2, 3, 4>;
7
8 // More practical: Generate sequence
9 using seq5 = make_integer_sequence<int, 5>; // 0, 1, 2, 3, 4
10
11 // Use with function
12 template<typename T, T... Is>
13 void print_sequence(integer_sequence<T, Is...>) {
14     ((cout << Is << " "), ...); // C++17 fold expression
15     cout << "\n";
16 }
17
18 print_sequence(make_integer_sequence<int, 10>()); // 0 1 2 3 4 5 6 7 8 9

```

27.22.2 Unpacking Tuple

```

1 #include <tuple>
2
3 // Convert tuple to function arguments
4 template<typename F, typename Tuple, size_t... Is>
5 auto apply_impl(F&& f, Tuple&& t, index_sequence<Is...>) {
6     return forward<F>(f)(get<Is>(forward<Tuple>(t))...);
7 }
8
9 template<typename F, typename Tuple>
10 auto apply(F&& f, Tuple&& t) {
11     return apply_impl(
12         forward<F>(f),
13         forward<Tuple>(t),
14         make_index_sequence<tuple_size_v<decay_t<Tuple>>>()
15     );
16 }
17
18 // Usage
19 auto add = [](int a, int b, int c) { return a + b + c; };

```

```
20 auto result = apply(add, make_tuple(1, 2, 3)); // 6
```

27.22.3 Array Initialization

```
1 template<typename T, size_t N, size_t... Is>
2 void fill_array_impl(array<T, N>& arr, index_sequence<Is...>) {
3     (... , (arr[Is] = Is * Is)); // Fill with squares
4 }
5
6 template<typename T, size_t N>
7 void fill_array(array<T, N>& arr) {
8     fill_array_impl(arr, make_index_sequence<N>());
9 }
10
11 array<int, 5> arr;
12 fill_array(arr);
13 // arr: {0, 1, 4, 9, 16}
```

27.23 LIBRARY IMPROVEMENTS

27.24 11.1 STL Enhancements in C++14

27.24.1 std::quoted for String I/O

```
1 #include <iostream>
2 #include <iomanip>
3 #include <string>
4 using namespace std;
5
6 string text = "Hello \"World\"";
7
8 // Without quoted
9 cout << text << "\n";
10 // Output: Hello "World"
11
12 // With quoted (C++14)
13 cout << quoted(text) << "\n";
14 // Output: "Hello \"World\""
15
16 // Useful for CSV and JSON
17 cout << quoted("value with spaces") << "\n";
```

27.24.2 std::less and Comparators

```
1 // Transparent comparators (C++14)
2 set<int, less<>> s; // Uses operator< for any comparable types
3
4 s.insert(5);
5 cout << s.count(5) << "\n"; // 1
6
7 // Can search with different type
8 cout << s.count(5.0) << "\n"; // Works with double too
```

27.24.3 Algorithms Returning Pair

```

1 #include <algorithm>
2
3 vector<int> v = {1, 2, 3, 4, 5};
4
5 // Functions returning pairs of iterators
6 auto [first, last] = equal_range(v.begin(), v.end(), 3);
7 // C++17: structured binding to unpack pair
8
9 // Alternative (C++14)
10 auto range = equal_range(v.begin(), v.end(), 3);
11 auto first_elem = range.first;
12 auto last_elem = range.second;

```

27.24.4 std::exchange

```

1 #include <utility>
2
3 int x = 5;
4 int old_value = exchange(x, 10);
5
6 cout << x << "\n";           // 10
7 cout << old_value << "\n";    // 5
8
9 // Useful for swapping
10 struct Object {
11     Data data;
12     Object& operator=(Object&& other) noexcept {
13         data = exchange(other.data, Data());
14         return *this;
15     }
16 };

```

27.24.5 std::get with Type

```

1 #include <tuple>
2
3 tuple<int, double, string> t{42, 3.14, "hello"};
4
5 // Get by index (C++11)
6 auto a = get<0>(t); // 42
7
8 // Get by type (C++14) - must be unique type
9 auto b = get<double>(t); // 3.14
10 auto c = get<string>(t); // "hello"
11
12 // ERROR if type appears twice
13 // tuple<int, int, string> t2;
14 // get<int>(t2); // Ambiguous!

```

27.25 DEPRECATED FEATURES & REMOVALS

27.26 12.1 Features Deprecated in C++14

```

1 // 1. std::auto_ptr (deprecated)
2 // Use unique_ptr instead
3 // auto_ptr<int> old_ptr(new int(42)); // Deprecated
4 auto new_ptr = make_unique<int>(42); // Modern
5
6 // 2. std::binary_function, unary_function
7 // No longer needed with lambdas
8 // struct Plus : binary_function<int, int, int> {
9 //     int operator()(int a, int b) const { return a + b; }
10 // };
11
12 auto plus = [](int a, int b) { return a + b; }; // Modern
13
14 // 3. std::bind1st, bind2nd
15 // Use std::bind or lambdas
16 // auto partial = bind1st(plus(), 5); // Deprecated
17
18 auto partial = [](int x) { return 5 + x; }; // Modern

```

27.26.1 12.5 Shared Locks (Reader-Writer Mutex)

C++14 introduces `shared<_timed>_mutex` allowing multiple readers but exclusive writers.

```

1 #include <shared_mutex>
2 #include <mutex>
3 #include <map>
4
5 class ThreadSafeCache {
6     std::map<int, int> data;
7     mutable std::shared_timed_mutex mtx; // C++14 (use shared_mutex in C++17)
8
9 public:
10    // Reader: Multiple threads can hold shared_lock
11    int get(int key) const {
12        std::shared_lock<std::shared_timed_mutex> lock(mtx);
13        if (data.find(key) != data.end()) {
14            return data.at(key);
15        }
16        return -1;
17    }
18
19    // Writer: Only one thread can hold unique_lock
20    void put(int key, int value) {
21        std::unique_lock<std::shared_timed_mutex> lock(mtx);
22        data[key] = value;
23    }
24 };

```

27.27 C++14 BEST PRACTICES

27.28 What's Better with C++14

```

1 // 1. Use generic lambdas for flexibility
2 auto process = [](auto x) { cout << x << "\n"; };
3 process(42);
4 process("hello");

```

```
5 process(3.14);
6
7 // 2. Use auto return types to avoid redundancy
8 auto add(int a, int b) { return a + b; }
9
10 // 3. Use make_unique for safety
11 auto ptr = make_unique<MyClass>(arg1, arg2);
12
13 // 4. Use binary literals for clarity
14 unsigned char mask = 0b11110000;
15
16 // 5. Use digit separators for readability
17 long big = 1'000'000'000'000;
18
19 // 6. Use init capture for move-only types
20 auto lambda = [ptr = move(ptr)]() { };
21
22 // 7. Use relaxed constexpr for compile-time computation
23 constexpr int factorial(int n) {
24     int result = 1;
25     for (int i = 2; i <= n; i++) result *= i;
26     return result;
27 }
28
29 // 8. Use variable templates for type information
30 template<typename T>
31 constexpr bool is_integral_v = is_integral<T>::value;
```

Part IV

Volume IV: C++17 - Simplification & Modernization

Chapter 28

C++17 MODERN FEATURES

28.1 C++17 Overview & Significance

C++17 (finalized in 2017) is a **major language update** rivaling C++11 in scope.

28.1.1 Timeline & Context

- **2011:** C++11 released (revolutionary)
- **2014:** C++14 released (refinement)
- **2017:** C++17 released (major overhaul)
- **2020:** C++20 released (revolutionary again)

28.1.2 C++17 Philosophy

- **Fix** fundamental issues with C++11/14
- **Add** major features requested by industry
- **Improve** performance and expressivity
- **Simplify** complex code patterns
- **Standardize** common idioms

28.1.3 Key Themes

1. **Pattern Matching** - Structured bindings
2. **Null Safety** - optional, variant
3. **Performance** - string_view, if constexpr
4. **Expressivity** - Fold expressions
5. **Safety** - Filesystem library
6. **Parallelism** - Parallel algorithms
7. **Type Deduction** - CTAD
8. **Flexibility** - std::any

28.1.4 Why C++17 Matters

C++17 addresses real pain points: - Safer null handling (optional) - Type-safe unions (variant) - Zero-copy string operations (string_view) - Compile-time branching (if constexpr) - Pattern matching (structured bindings) - Safe filesystem access - Automatic template deduction - Flexible type storage (any)

28.2 STRUCTURED BINDINGS

28.3 1.1 Introduction to Structured Bindings

Structured bindings allow unpacking objects into individual variables.

28.3.1 Basic Structured Bindings

```
1 #include <tuple>
2 #include <iostream>
3 using namespace std;
4
5 // Before C++17: Verbose
6 tuple<int, double, string> t = {42, 3.14, "hello"};
7 int a = get<0>(t);
8 double b = get<1>(t);
9 string c = get<2>(t);
10
11 // C++17: Clean and simple
12 auto [x, y, z] = t;
13 cout << x << ", " << y << ", " << z << "\n"; // 42, 3.14, hello
```

28.3.2 Structured Bindings with Pairs

```
1 #include <map>
2
3 map<string, int> ages{{"Alice", 30}, {"Bob", 25}};
4
5 // Before C++17: Awkward
6 for (const auto& pair : ages) {
7     const string& name = pair.first;
8     int age = pair.second;
9     cout << name << " is " << age << "\n";
10 }
11
12 // C++17: Natural
13 for (const auto& [name, age] : ages) {
14     cout << name << " is " << age << "\n";
15 }
```

28.3.3 Structured Bindings with Arrays

```
1 // Arrays
2 int arr[3] = {1, 2, 3};
3 auto [a, b, c] = arr;
4 cout << a << ", " << b << ", " << c << "\n"; // 1, 2, 3
5
```

```
6 // Fixed-size arrays
7 array<double, 4> coords = {1.0, 2.0, 3.0, 4.0};
8 auto [x, y, z, w] = coords;
```

28.3.4 Structured Bindings with Structs

```
1 struct Person {
2     string name;
3     int age;
4     string city;
5 };
6
7 Person p{"Alice", 30, "NYC"};
8
9 // Before C++17
10 string name = p.name;
11 int age = p.age;
12 string city = p.city;
13
14 // C++17
15 auto [name, age, city] = p;
16 cout << name << " is " << age << " from " << city << "\n";
```

28.3.5 Structured Bindings with Return Values

```
1 pair<bool, int> divide(int a, int b) {
2     if (b == 0) return {false, 0};
3     return {true, a / b};
4 }
5
6 // Before C++17: Awkward
7 auto result = divide(10, 2);
8 if (result.first) {
9     cout << "Result: " << result.second << "\n";
10 }
11
12 // C++17: Clear
13 auto [success, value] = divide(10, 2);
14 if (success) {
15     cout << "Result: " << value << "\n";
16 }
```

28.3.6 Structured Bindings with References

```
1 int x = 5, y = 10;
2
3 // Modify through references
4 auto& [rx, ry] = tuple<int&, int&>(x, y);
5 rx = 20;
6 cout << x << "\n"; // 20 (modified)
7
8 // Const references
9 const auto& [cx, cy] = tuple<int, int>(5, 10);
10 // cx = 100; // ERROR - const
```

28.3.7 Practical Use Cases

```

1 // Function returning multiple values
2 tuple<bool, string, int> parse_config(const string& path) {
3     // Parse and return success, name, port
4     return {true, "server", 8080};
5 }
6
7 auto [ok, name, port] = parse_config("/etc/config");
8 if (ok) {
9     cout << "Server " << name << " on port " << port << "\n";
10 }
11
12 // Iterating over map with unpacking
13 map<string, vector<int>> data{
14     {"a", {1, 2, 3}},
15     {"b", {4, 5, 6}}
16 };
17
18 for (auto [key, values] : data) {
19     cout << key << ": ";
20     for (int v : values) cout << v << " ";
21     cout << "\n";
22 }
23
24 // Database-like access
25 vector<tuple<int, string, double>> records{
26     {1, "Alice", 95.5},
27     {2, "Bob", 87.0},
28     {3, "Carol", 92.5}
29 };
30
31 for (auto [id, name, score] : records) {
32     cout << id << ": " << name << " scored " << score << "\n";
33 }

```

28.3.8 Structured Bindings Rules

```

1 // Auto deduction (copies)
2 tuple<int, int> t{1, 2};
3 auto [a, b] = t;           // a, b are copies
4
5 // References
6 auto& [x, y] = t;          // x, y reference t's elements
7 const auto& [cx, cy] = t;  // const references
8
9 // Move
10 auto&& [mx, my] = move(t); // rvalue references
11
12 // Partial binding (with operator[])
13 struct Container {
14     int& operator[](size_t i); // Must return reference
15 };
16
17 Container c;
18 auto [elem] = c;           // Gets copy via operator[]
19
20 // Multiple items
21 auto [a, b, c, d] = tuple{1, 2, 3, 4};
22 auto [x, y, z] = array{10, 20, 30};

```

28.4 1.2 Structured Bindings Under the Hood

The compiler generates a hidden variable.

Code:

```
1 auto [x, y] = my_pair;
```

Compiler Logic:

```
1 auto __hidden = my_pair;
2 auto& x = __hidden.first; // Aliases
3 auto& y = __hidden.second;
```

Implication: * x and y are NOT variables; they are names referring to subobjects of the hidden variable. * If you use `auto& [x, y]`, the hidden variable is a reference.

28.5 OPTIONAL & VARIANT

28.6 2.1 std::optional - Safe Nullable Values

`std::optional` represents a value that may or may not be present.

28.6.1 Basic optional Usage

```
1 #include <optional>
2 #include <iostream>
3 using namespace std;
4
5 // Function that might not return a value
6 optional<int> parse_int(const string& s) {
7     try {
8         return stoi(s);
9     } catch (...) {
10         return nullopt; // No value
11     }
12 }
13
14 // Usage
15 auto result1 = parse_int("42");
16 if (result1.has_value()) {
17     cout << "Value: " << result1.value() << "\n";
18 }
19
20 // Or using operator*
21 if (result1) {
22     cout << "Value: " << *result1 << "\n";
23 }
24
25 // Default value
26 auto value = result1.value_or(0); // 0 if no value
```

28.6.2 optional with Complex Types

```
1 struct User {
2     int id;
3     string name;
```

```

4 };
5
6 optional<User> find_user(int id) {
7     if (id < 0) return nullopt;
8     return User{id, "User" + to_string(id)};
9 }
10
11 // Usage
12 if (auto user = find_user(42)) {
13     cout << user->name << "\n";
14 } else {
15     cout << "User not found\n";
16 }

```

28.6.3 optional Operations

```

1 optional<int> opt(42);
2
3 // Check
4 if (opt) cout << "Has value\n";           // true
5 if (opt.has_value()) cout << "Has\n";     // true
6
7 // Access
8 cout << opt.value() << "\n";              // 42
9 cout << *opt << "\n";                    // 42
10 cout << opt.value_or(0) << "\n";         // 42
11
12 // Modify
13 opt.value() = 100;
14 cout << *opt << "\n";                    // 100
15
16 // Reset
17 opt = nullopt;
18 if (opt) cout << "Has value\n";         // false
19
20 // Or assign
21 opt = 99;
22 cout << *opt << "\n";                    // 99

```

28.6.4 optional with Chaining

```

1 optional<int> process(optional<int> input) {
2     if (!input) return nullopt;
3     return input.value() * 2;
4 }
5
6 auto result = process(optional<int>(5));
7 if (result) {
8     cout << result.value() << "\n"; // 10
9 }
10
11 // Chain operations
12 auto chain = parse_int("42")
13     .and_then([](int x) -> optional<int> { return x * 2; })
14     .or_else([]() { return optional<int>(0); });

```

28.7 2.2 std::variant - Type-Safe Union

std::variant is a type-safe union that holds one of several types.

28.7.1 Basic variant Usage

```

1 #include <variant>
2 #include <iostream>
3 using namespace std;
4
5 // Can hold int, double, or string
6 variant<int, double, string> value;
7
8 // Store int
9 value = 42;
10 cout << get<int>(value) << "\n"; // 42
11
12 // Store double
13 value = 3.14;
14 cout << get<double>(value) << "\n"; // 3.14
15
16 // Store string
17 value = string("hello");
18 cout << get<string>(value) << "\n"; // hello
19
20 // Check type
21 cout << value.index() << "\n"; // 2 (string is third)

```

28.7.2 variant with Type Checking

```

1 void process(variant<int, double, string> value) {
2     if (holds_alternative<int>(value)) {
3         cout << "Integer: " << get<int>(value) << "\n";
4     } else if (holds_alternative<double>(value)) {
5         cout << "Double: " << get<double>(value) << "\n";
6     } else if (holds_alternative<string>(value)) {
7         cout << "String: " << get<string>(value) << "\n";
8     }
9 }
10
11 process(42); // "Integer: 42"
12 process(3.14); // "Double: 3.14"
13 process("hello"); // "String: hello"

```

28.7.3 variant with Visitor Pattern

```

1 struct Visitor {
2     void operator()(int i) const {
3         cout << "Integer: " << i << "\n";
4     }
5
6     void operator()(double d) const {
7         cout << "Double: " << d << "\n";
8     }
9
10    void operator()(const string& s) const {
11        cout << "String: " << s << "\n";
12    }
13 };

```



```

14
15 variant<int, double, string> v = 42;
16 visit(Visitor(), v); // "Integer: 42"
17
18 v = 3.14;
19 visit(Visitor(), v); // "Double: 3.14"
20
21 v = string("hello");
22 visit(Visitor(), v); // "String: hello"

```

28.7.4 variant with Lambdas (C++20 or using overload trick)

```

1 // C++17 overload trick
2 template<typename... Ts>
3 struct overload : Ts... { using Ts::operator()...; };
4 template<typename... Ts>
5 overload(Ts...) -> overload<Ts...>;
6
7 variant<int, double, string> v = 42;
8
9 // Visit with lambdas
10 visit(overload{
11     [](int i) { cout << "Integer: " << i << "\n"; },
12     [](double d) { cout << "Double: " << d << "\n"; },
13     [](const string& s) { cout << "String: " << s << "\n"; }
14 }, v); // "Integer: 42"

```

28.7.5 Practical variant Example

```

1 variant<int, string> parse_value(const string& input) {
2     try {
3         return stoi(input); // Try as int
4     } catch (...) {
5         return input; // Return as string
6     }
7 }
8
9 auto result = parse_value("42");
10 if (holds_alternative<int>(result)) {
11     cout << "Parsed as int: " << get<int>(result) << "\n";
12 } else {
13     cout << "Parsed as string: " << get<string>(result) << "\n";
14 }

```

28.8 STD::ANY

28.9 3.1 Type-Erased Storage with std::any

std::any can hold any copyable type.

28.9.1 Basic any Usage

```

1 #include <any>
2 #include <iostream>
3 using namespace std;
4
5 any value;
6
7 // Store different types
8 value = 42;
9 cout << any_cast<int>(value) << "\n"; // 42
10
11 value = 3.14;
12 cout << any_cast<double>(value) << "\n"; // 3.14
13
14 value = string("hello");
15 cout << any_cast<string>(value) << "\n"; // hello
16
17 // Check type
18 if (value.type() == typeid(string)) {
19     cout << "It's a string\n";
20 }

```

28.9.2 any with Type Checking

```

1 any value = 42;
2
3 if (value.type() == typeid(int)) {
4     cout << "Value: " << any_cast<int>(value) << "\n";
5 }
6
7 // Safe cast (throws if wrong type)
8 try {
9     double d = any_cast<double>(value); // Wrong type
10 } catch (const bad_any_cast& e) {
11     cout << "Type mismatch: " << e.what() << "\n";
12 }
13
14 // Check before casting
15 if (value.type() == typeid(int)) {
16     int i = any_cast<int>(value);
17 }

```

28.9.3 any in Collections

```

1 vector<any> data;
2 data.push_back(42);
3 data.push_back(3.14);
4 data.push_back(string("hello"));
5 data.push_back(vector<int>{1, 2, 3});
6
7 // Process
8 for (auto& item : data) {
9     if (item.type() == typeid(int)) {
10         cout << "Int: " << any_cast<int>(item) << "\n";
11     } else if (item.type() == typeid(double)) {
12         cout << "Double: " << any_cast<double>(item) << "\n";
13     } else if (item.type() == typeid(string)) {
14         cout << "String: " << any_cast<string>(item) << "\n";
15     }
16 }

```

28.9.4 any vs variant

```

1 // variant<int, double, string>: Fixed types, type-safe
2 variant<int, double, string> v = 42;
3 cout << get<int>(v) << "\n"; // Type-safe, no runtime check needed
4
5 // any: Any type, runtime type checking
6 any a = 42;
7 cout << any_cast<int>(a) << "\n"; // Runtime check, potential exception
8
9 // Use variant when types are known
10 // Use any when types are truly dynamic

```

28.10 STD::STRING_VIEW

28.11 4.1 Non-Owning String References

`std::string_view` provides efficient, zero-copy string operations.

28.11.1 Basic string_view

```

1 #include <string_view>
2 #include <iostream>
3 using namespace std;
4
5 string s = "Hello, World!";
6 string_view sv = s;
7
8 cout << sv << "\n"; // "Hello, World!"
9 cout << sv.length() << "\n"; // 13
10 cout << sv[0] << "\n"; // 'H'
11 cout << sv.data() << "\n"; // "Hello, World!"

```

28.11.2 string_view from Different Sources

```

1 // From std::string
2 string s = "hello";
3 string_view sv1 = s;
4
5 // From C-string
6 const char* cstr = "world";
7 string_view sv2 = cstr;
8
9 // From string literal
10 string_view sv3 = "test";
11
12 // Substring
13 string_view sv4 = sv1.substr(1, 3); // "ell"

```

28.11.3 string_view Operations

```

1 string_view sv = "Hello, World!";
2
3 // Searching
4 cout << sv.find("World") << "\n";           // 7
5 cout << sv.find(',') << "\n";               // 5
6
7 // Comparing
8 cout << sv.compare("Hello, World!") << "\n"; // 0 (equal)
9
10 // Prefix/suffix
11 cout << sv.starts_with("Hello") << "\n";    // true
12 cout << sv.ends_with("!") << "\n";          // true
13
14 // Substrings
15 auto hello = sv.substr(0, 5);                 // "Hello"
16 auto world = sv.substr(7);                    // "World!"
17
18 // Remove prefix/suffix (C++20)
19 // sv.remove_prefix(7);
20 // sv.remove_suffix(1);

```

28.11.4 Performance Benefits of string_view

```

1 // OLD: Copy string
2 void process_old(const string& s) {
3     // s might be copied
4 }
5
6 // NEW: No copy
7 void process_new(string_view sv) {
8     // sv references the string, no copy
9 }
10
11 // Usage
12 string data = "important";
13 process_old(data); // Might copy
14 process_new(data); // No copy!
15
16 // Works with literals too
17 process_new("temporary"); // No copy, no allocation

```

28.11.5 string_view Limitations

```

1 string_view sv = "hello";
2
3 // What you CAN'T do:
4 // sv[0] = 'H';           // ERROR - can't modify
5 // sv.resize(3);          // ERROR - can't resize
6 // string s(sv);          // OK - explicit conversion needed
7
8 // Works only while source exists
9 string_view sv2;
10 {
11     string temp = "danger";
12     sv2 = temp;
13     // temp destroyed here
14 }
15 // sv2 now points to destroyed string - DANGER!
16

```

```

17 // Safe way: Make a copy if needed
18 string copy(sv2);

```

28.11.6 string_view Use Cases

```

1 // Parse tokens without copying
2 string_view parse_token(string_view& input) {
3     size_t pos = input.find(' ');
4     if (pos == string_view::npos) {
5         string_view token = input;
6         input = "";
7         return token;
8     }
9     string_view token = input.substr(0, pos);
10    input = input.substr(pos + 1);
11    return token;
12 }
13
14 // Check file extensions efficiently
15 bool is_cpp_file(string_view filename) {
16     return filename.ends_with(".cpp") ||
17            filename.ends_with(".h") ||
18            filename.ends_with(".hpp");
19 }
20
21 // Efficient URL parsing
22 void parse_url(string_view url) {
23     size_t protocol_end = url.find("://");
24     string_view protocol = url.substr(0, protocol_end);
25     // ... continue parsing
26 }

```

28.12 IF CONSTEXPR

28.13 5.1 Compile-Time Conditional Code

if constexpr allows branching at compile-time.

28.13.1 Basic if constexpr

```

1 #include <type_traits>
2
3 template<typename T>
4 void process(T value) {
5     if constexpr (is_integral_v<T>) {
6         cout << "Integer: " << value << "\n";
7     } else if constexpr (is_floating_point_v<T>) {
8         cout << "Float: " << value << "\n";
9     } else {
10        cout << "Other type\n";
11    }
12 }
13
14 process(42);           // "Integer: 42" - int branch only compiled
15 process(3.14);        // "Float: 3.14" - double branch only compiled
16 process("hello");     // "Other type"

```

28.13.2 if constexpr Advantages

```

1 // Before C++17: SFINAE (complex, verbose)
2 template<typename T>
3 enable_if_t<is_integral_v<T>>
4 old_way(T x) { cout << "Int\n"; }
5
6 template<typename T>
7 enable_if_t<is_floating_point_v<T>>
8 old_way(T x) { cout << "Float\n"; }
9
10 // After C++17: if constexpr (clean, readable)
11 template<typename T>
12 void new_way(T x) {
13     if constexpr (is_integral_v<T>) {
14         cout << "Int\n";
15     } else if constexpr (is_floating_point_v<T>) {
16         cout << "Float\n";
17     }
18 }

```

28.13.3 if constexpr with Complex Logic

```

1 template<typename T>
2 void serialize(const T& value) {
3     if constexpr (is_arithmetic_v<T>) {
4         // Serialize numbers efficiently
5         write_binary(value);
6     } else if constexpr (is_same_v<T, string>) {
7         // Serialize strings
8         write_string(value);
9     } else if constexpr (requires { value.size(); }) {
10        // Serialize containers
11        for (const auto& item : value) {
12            serialize(item);
13        }
14    }
15 }

```

28.13.4 if constexpr with Concepts (C++20-like)

```

1 template<typename T>
2 void print_info(const T& value) {
3     cout << "Type: " << typeid(T).name() << "\n";
4
5     if constexpr (requires { value.size(); }) {
6         cout << "Size: " << value.size() << "\n";
7     }
8
9     if constexpr (requires { value.data(); }) {
10        cout << "Data pointer available\n";
11    }
12
13    if constexpr (is_default_constructible_v<T>) {
14        T temp; // Can create temporary
15    }
16 }

```

28.14 5.2 The Death of SFINAE?

if constexpr replaces SFINAE for *implementation details*, but not for *overload resolution*.

SFINAE (Old Way - C++11/14):

```

1 template <typename T>
2 typename std::enable_if<std::is_integral<T>::value, T>::type
3 gcd(T a, T b) { return b == 0 ? a : gcd(b, a % b); }
4
5 template <typename T>
6 typename std::enable_if<!std::is_integral<T>::value, T>::type
7 gcd(T a, T b) { static_assert(std::is_integral<T>::value, "GCD not for floats")
8 ; }

```

if constexpr (New Way - C++17):

```

1 template <typename T>
2 T gcd(T a, T b) {
3     if constexpr (std::is_integral_v<T>) {
4         return b == 0 ? a : gcd(b, a % b);
5     } else {
6         static_assert(always_false<T>, "GCD not for floats");
7     }
8 }

```

Result: Much cleaner, easier to debug errors.

28.15 FOLD EXPRESSIONS

28.16 6.1 Operating on Parameter Packs

Fold expressions simplify operations on variadic templates.

28.16.1 Basic Fold Expressions

```

1 #include <iostream>
2 using namespace std;
3
4 // Sum with fold
5 template<typename... Args>
6 int sum(Args... args) {
7     return (... + args); // Left fold: ((a + b) + c) + d
8 }
9
10 cout << sum(1, 2, 3, 4) << "\n"; // 10
11
12 // Product with fold
13 template<typename... Args>
14 int product(Args... args) {
15     return (... * args); // Left fold: ((a * b) * c) * d
16 }
17
18 cout << product(2, 3, 4) << "\n"; // 24
19
20 // Logical AND
21 template<typename... Args>
22 bool all_true(Args... args) {
23     return (... && args);

```

```

24 }
25
26 cout << all_true(true, true, true) << "\n";      // true
27 cout << all_true(true, false, true) << "\n";     // false

```

28.16.2 Fold Directions

```

1 // Left fold: (... + args) = ((a + b) + c) + d
2 template<typename... Args>
3 int left_fold(Args... args) {
4     return (... + args);
5 }
6
7 // Right fold: (args + ...) = a + (b + (c + d))
8 template<typename... Args>
9 int right_fold(Args... args) {
10    return (args + ...);
11 }
12
13 // For addition, result is the same
14 left_fold(1, 2, 3);    // ((1 + 2) + 3) = 6
15 right_fold(1, 2, 3);   // (1 + (2 + 3)) = 6
16
17 // For subtraction, different!
18 // left: ((1 - 2) - 3) = -4
19 // right: (1 - (2 - 3)) = 2

```

28.16.3 Fold with Default Value

```

1 // No default value
2 template<typename... Args>
3 int sum_no_default(Args... args) {
4     return (... + args); // Error if no args
5 }
6
7 // With default value
8 template<typename... Args>
9 int sum_default(Args... args) {
10    return (args + ... + 0); // 0 if no args
11 }
12
13 sum_default();           // 0
14 sum_default(1, 2, 3);    // 6

```

28.16.4 Practical Fold Examples

```

1 // Print all arguments
2 template<typename... Args>
3 void print_all(Args... args) {
4     ((cout << args << " "), ...);
5 }
6
7 print_all(1, "hello", 3.14, "world");
8 // Output: 1 hello 3.14 world
9
10 // Check if any true
11 template<typename... Args>
12 bool any_true(Args... args) {

```



```

13     return (... || args);
14 }
15
16 any_true(false, false, true, false); // true
17
18 // Maximum of values
19 template<typename... Args>
20 int maximum(Args... args) {
21     return max({args...}); // Fold into initializer
22 }
23
24 cout << maximum(3, 1, 4, 1, 5, 9) << "\n"; // 9

```

28.17 6.2 Advanced Fold Expressions

The comma operator `,` is the most powerful fold operator. It evaluates LHS, discards it, then RHS.

28.17.1 Calling Function on Pack

```

1 template<typename... Args>
2 void log_all(Args... args) {
3     (... , (cout << "[LOG] " << args << "\n"));
4 }

```

28.17.2 Validating All Arguments

```

1 template<typename... Args>
2 bool validate_all(Args... args) {
3     // Returns true if ALL args are valid
4     return (... && (args.is_valid()));
5 }

```

28.18 CLASS TEMPLATE ARGUMENT DEDUCTION (CTAD)

28.19 7.1 Automatic Template Type Deduction

CTAD allows deduction of class template parameters from constructor arguments.

28.19.1 Before CTAD (C++14)

```

1 #include <vector>
2 #include <map>
3 using namespace std;
4
5 // Must specify types explicitly
6 vector<int> v{1, 2, 3};
7 pair<string, int> p("hello", 42);
8 map<string, int> m{{"a", 1}, {"b", 2}};

```

28.19.2 With CTAD (C++17)

```
1 // Types deduced automatically!
2 vector v{1, 2, 3}; // Deduced as vector<int>
3 pair p("hello", 42); // Deduced as pair<string, int>
4 map m{pair("a", 1), pair("b", 2)}; // Deduced as map<string, int>
5
6 // Works with custom classes too
7 template<typename T, typename U>
8 struct Pair {
9     T first;
10    U second;
11 };
12
13 Pair p{42, 3.14}; // Deduced as Pair<int, double>
```

28.19.3 CTAD with Custom Deduction Guides

```
1 template<typename T>
2 struct Container {
3     vector<T> data;
4 };
5
6 // Without guide: Would fail
7 // Container c{1, 2, 3}; // ERROR - can't deduce T
8
9 // With deduction guide
10 template<typename T>
11 Container(initializer_list<T>) -> Container<T>;
12
13 // Now it works!
14 Container c{1, 2, 3}; // Deduced as Container<int>
```

28.19.4 Practical CTAD Examples

```
1 // Multiple parameters
2 template<typename T, typename U>
3 class Pair {
4 public:
5     Pair(T first, U second) : first(first), second(second) {}
6     T first;
7     U second;
8 };
9
10 Pair p{1, "hello"}; // Deduced as Pair<int, const char*>
11
12 // Arrays
13 array a{1, 2, 3, 4, 5}; // Deduced as array<int, 5>
14
15 // Aggregates
16 struct Point {
17     int x, y;
18 };
19
20 Point pt{10, 20}; // No deduction needed (aggregate), but works
```

28.19.5 CTAD Benefits

```

1 // Reduces verbosity
2 auto v1 = vector<int>{1, 2, 3}; // C++11 way
3 auto v2 = vector{1, 2, 3};      // C++17 way
4
5 // More readable with complex types
6 map m1{pair<const int, string>{1, "a"}}; // Verbose
7 map m2{pair{1, "a"}};                  // Clean
8
9 // Especially useful with templates
10 template<typename Iter>
11 auto process(Iter first, Iter last) {
12     vector v(first, last); // Deduced type!
13     return v;
14 }

```

28.20 STD::FILESYSTEM

28.21 8.1 Portable File System Operations

`std::filesystem` provides safe, portable file system access.

28.21.1 Basic Path Operations

```

1 #include <filesystem>
2 #include <iostream>
3 using namespace std;
4 namespace fs = filesystem;
5
6 // Create path
7 fs::path p1 = "/home/user/file.txt";
8 fs::path p2 = "relative/path/file.txt";
9
10 // Query components
11 cout << p1.filename() << "\n"; // "file.txt"
12 cout << p1.parent_path() << "\n"; // "/home/user"
13 cout << p1.extension() << "\n"; // ".txt"
14 cout << p1.stem() << "\n"; // "file"
15
16 // Normalize
17 cout << p1.lexically_normal() << "\n"; // Remove .. and .

```

28.21.2 File Existence & Type Checking

```

1 namespace fs = filesystem;
2
3 fs::path p = "/path/to/file";
4
5 if (fs::exists(p)) {
6     cout << "Path exists\n";
7 }
8
9 if (fs::is_regular_file(p)) {
10     cout << "Is a regular file\n";
11     cout << "Size: " << fs::file_size(p) << " bytes\n";
12 }

```

```
13
14 if (fs::is_directory(p)) {
15     cout << "Is a directory\n";
16 }
17
18 if (fs::is_symlink(p)) {
19     cout << "Is a symbolic link\n";
20 }
```

28.21.3 Directory Operations

```
1 namespace fs = filesystem;
2
3 fs::path dir = "/path/to/directory";
4
5 // List directory contents
6 for (const auto& entry : fs::directory_iterator(dir)) {
7     cout << entry.path().filename() << "\n";
8     if (entry.is_directory()) {
9         cout << "    (directory)\n";
10    }
11 }
12
13 // Recursive directory listing
14 for (const auto& entry : fs::recursive_directory_iterator(dir)) {
15     cout << entry.path().relative_path(dir) << "\n";
16 }
```

28.21.4 File Operations

```
1 namespace fs = filesystem;
2
3 fs::path src = "original.txt";
4 fs::path dst = "copy.txt";
5
6 // Copy file
7 fs::copy_file(src, dst);
8
9 // Move/rename
10 fs::rename(src, dst);
11
12 // Delete file
13 fs::remove(dst);
14
15 // Delete directory (must be empty)
16 fs::path dir = "empty_directory";
17 fs::remove(dir);
18
19 // Create directory
20 fs::path newdir = "new_directory";
21 fs::create_directory(newdir);
22
23 // Create nested directories
24 fs::create_directories("a/b/c/d");
```

28.21.5 Practical filesystem Example

```

1 #include <filesystem>
2 #include <fstream>
3 using namespace std;
4 namespace fs = filesystem;
5
6 // Find all C++ files in directory
7 void find_cpp_files(const fs::path& dir) {
8     for (const auto& entry : fs::recursive_directory_iterator(dir)) {
9         if (entry.is_regular_file()) {
10             auto ext = entry.path().extension();
11             if (ext == ".cpp" || ext == ".h" || ext == ".hpp") {
12                 cout << entry.path() << "\n";
13             }
14         }
15     }
16 }
17
18 // Count lines in a file
19 int count_lines(const fs::path& file) {
20     ifstream f(file);
21     int count = 0;
22     string line;
23     while (getline(f, line)) count++;
24     return count;
25 }
26
27 // Safe file backup
28 void backup_file(const fs::path& original) {
29     if (!fs::exists(original)) {
30         throw runtime_error("File not found");
31     }
32
33     fs::path backup = original;
34     backup.replace_extension(
35         backup.extension().string() + ".bak"
36     );
37
38     fs::copy_file(original, backup,
39                 fs::copy_options::overwrite_existing);
40 }

```

28.22 8.5 Filesystem Deep Dive

28.22.1 Exception-Free API

Most `std::filesystem` functions have an overload taking `std::error_code&` to avoid exceptions.

```

1 std::error_code ec;
2 if (fs::exists("/tmp/ghost", ec)) {
3     // ...
4 }
5 if (ec) {
6     std::cerr << "Error: " << ec.message() << "\n";
7 }

```

28.22.2 Space & Permissions

```

1 auto space = fs::space("/");
2 cout << "Free: " << space.free / 1024 / 1024 << " MB\n";

```

```

3
4 fs::permissions("file.txt",
5     fs::perms::owner_read | fs::perms::owner_write,
6     fs::perm_options::add);

```

28.23 STD::INVOKE

28.24 9.1 Uniform Callable Invocation

`std::invoke` provides uniform way to call functions, methods, and functors.

28.24.1 Basic invoke

```

1 #include <functional>
2 #include <iostream>
3 using namespace std;
4
5 // Regular function
6 int add(int a, int b) { return a + b; }
7
8 // Invoke function
9 cout << invoke(add, 5, 3) << "\n"; // 8
10
11 // Function pointer
12 int (*fp)(int, int) = add;
13 cout << invoke(fp, 5, 3) << "\n"; // 8
14
15 // Lambda
16 auto lambda = [](int a, int b) { return a * b; };
17 cout << invoke(lambda, 5, 3) << "\n"; // 15
18
19 // Functor
20 struct Multiplier {
21     int operator()(int a, int b) const { return a * b; }
22 };
23
24 Multiplier m;
25 cout << invoke(m, 5, 3) << "\n"; // 15

```

28.24.2 invoke with Methods

```

1 struct Calculator {
2     int value = 0;
3
4     int add(int x) { return value + x; }
5     int multiply(int x) const { return value * x; }
6 };
7
8 Calculator calc{10};
9
10 // Invoke member function
11 cout << invoke(&Calculator::add, calc, 5) << "\n"; // 15
12 cout << invoke(&Calculator::multiply, calc, 3) << "\n"; // 30
13
14 // With pointer
15 Calculator* ptr = &calc;

```

```

16 cout << invoke(&Calculator::add, ptr, 5) << "\n";           // 15
17
18 // With shared_ptr, unique_ptr
19 auto up = make_unique<Calculator>();
20 up->value = 20;
21 cout << invoke(&Calculator::add, up.get(), 5) << "\n"; // 25

```

28.24.3 invoke with Data Members

```

1 struct Person {
2     string name;
3     int age;
4 };
5
6 Person p{"Alice", 30};
7
8 // Invoke data member access
9 cout << invoke(&Person::name, p) << "\n"; // "Alice"
10 cout << invoke(&Person::age, p) << "\n"; // 30
11
12 // Modify through invoke
13 invoke(&Person::age, p) = 31;
14 cout << p.age << "\n"; // 31

```

28.24.4 Practical invoke Pattern

```

1 // Create callable wrapper that works with anything
2 template<typename Func, typename... Args>
3 auto call_wrapper(Func&& f, Args&&... args) {
4     return invoke(forward<Func>(f), forward<Args>(args)...);
5 }
6
7 // Use with different callables
8 call_wrapper(add, 5, 3);           // Function
9 call_wrapper(lambda, 5, 3);       // Lambda
10 call_wrapper(&Calculator::add, calc, 5); // Member function

```

28.25 PARALLEL ALGORITHMS

28.26 10.1 Parallel Algorithm Execution

C++17 adds parallel execution to standard algorithms.

28.26.1 Execution Policies

```

1 #include <algorithm>
2 #include <execution>
3 #include <vector>
4 using namespace std;
5
6 vector<int> v = {5, 2, 8, 1, 9, 3};
7
8 // Sequential (traditional)
9 sort(v.begin(), v.end());

```

```

10
11 // Parallel (may use multiple threads)
12 sort(execution::par, v.begin(), v.end());
13
14 // Parallel unsequenced (even less ordering guarantee)
15 sort(execution::par_unseq, v.begin(), v.end());
16
17 // Sequenced unsequenced (no vectorization)
18 sort(execution::unseq, v.begin(), v.end());

```

28.26.2 Parallel Algorithm Examples

```

1 #include <algorithm>
2 #include <execution>
3 #include <numeric>
4 #include <vector>
5
6 vector<int> v = {1, 2, 3, 4, 5};
7
8 // Parallel transform
9 transform(execution::par, v.begin(), v.end(), v.begin(),
10          [](int x) { return x * 2; });
11
12 // Parallel accumulate
13 int sum = reduce(execution::par, v.begin(), v.end());
14
15 // Parallel find_if
16 auto it = find_if(execution::par, v.begin(), v.end(),
17                  [](int x) { return x > 5; });
18
19 // Parallel count_if
20 int even_count = count_if(execution::par, v.begin(), v.end(),
21                           [](int x) { return x % 2 == 0; });

```

28.26.3 Performance Considerations

```

1 vector<int> large(1000000);
2
3 // Sequential: Simple, predictable
4 sort(large.begin(), large.end());
5
6 // Parallel: Faster for large data (overhead for small)
7 sort(execution::par, large.begin(), large.end());
8
9 // Parallel unsequenced: Allows compiler optimizations
10 sort(execution::par_unseq, large.begin(), large.end());

```

28.27 CORE LANGUAGE FEATURES

28.28 11.1 Additional C++17 Features

28.28.1 Nested namespaces


```

1 // C++14
2 namespace A {
3     namespace B {
4         namespace C {
5             int x = 42;
6         }
7     }
8 }
9
10 // C++17
11 namespace A::B::C {
12     int x = 42;
13 }

```

28.28.2 Inline Variables

```

1 // Header file
2 inline int global_var = 42; // Definition, can be in header
3 inline vector<int> global_vec;
4
5 // No ODR (One Definition Rule) violation
6 // Can include this header in multiple translation units

```

28.28.3 Structured Exception Handling (still mostly the same, but with improvements)

```

1 try {
2     // Code
3 } catch (const exception& e) {
4     cout << e.what() << "\n";
5 }

```

28.28.4 Deduced Return Types in Lambdas

```

1 auto lambda = [](int x) -> auto { // C++17
2     return x * 2; // Return type deduced
3 };

```

28.28.5 std::byte

```

1 #include <cstdint>
2
3 byte b1{42};
4 byte b2 = 0xAB_B; // Literal
5 cout << (int)b1 << "\n"; // Cast to see value

```

28.29 LIBRARY IMPROVEMENTS

28.30 12.1 STL Enhancements in C++17

28.30.1 std::optional Algorithms

```

1 optional<int> opt{42};
2
3 opt.and_then([](int x) -> optional<int> {
4     return x * 2;
5 }).or_else([]() { return optional<int>(0); });

```

28.30.2 Improved Algorithms

```

1 vector<int> v = {1, 2, 3, 4, 5};
2
3 // uninitialized operations now work with ranges
4 vector<int> v2(5);
5 uninitialized_copy(v.begin(), v.end(), v2.begin());
6
7 // reduce (like accumulate but parallel-friendly)
8 int sum = reduce(v.begin(), v.end());

```

28.30.3 std::charconv

```

1 #include <charconv>
2
3 // Fast, exception-safe number conversion
4 int x = 42;
5 char buffer[100];
6 auto result = to_chars(buffer, buffer + 100, x);
7
8 string str("123");
9 int value;
10 auto [ptr, ec] = from_chars(str.data(), str.data() + str.size(), value);
11 if (ec == errc()) {
12     cout << value << "\n"; // 123
13 }

```

28.31 POLYMORPHIC MEMORY RESOURCES (PMR)

28.32 13.1 Introduction to std::pmr

C++17 introduces `std::pmr` (Polymorphic Memory Resources) in `<memory_resource>`, enabling efficient, customizable memory management without changing container types.

28.32.1 The Problem with Traditional Allocators

Traditional allocators are part of the type signature: `std::vector<int, MyAlloc<int>>` is a different type from `std::vector<int>`.

`std::pmr` erases the allocator type, allowing containers with different allocation strategies to be used interchangeably.

```

1 #include <vector>
2 #include <memory_resource>
3 #include <iostream>
4
5 // Use pmr namespace
6 namespace pmr = std::pmr;

```

```

7
8 void process_data(const pmr::vector<int>& data) {
9     // Works with ANY allocator (stack, heap, pool)
10    for (int x : data) std::cout << x << " ";
11}
12
13 int main() {
14     // 1. Default allocator (Heap)
15    pmr::vector<int> heap_vec = {1, 2, 3};
16    process_data(heap_vec);
17
18    // 2. Stack allocator (Monotonic Buffer)
19    std::array<std::byte, 1024> buffer;
20    pmr::monotonic_buffer_resource pool{
21        buffer.data(), buffer.size(), pmr::null_memory_resource()
22    };
23
24    pmr::vector<int> stack_vec(&pool);
25    stack_vec.push_back(4);
26    stack_vec.push_back(5);
27    process_data(stack_vec);
28
29    return 0;
30}

```

28.33 13.2 Memory Resources

28.33.1 Standard Memory Resources

```

1 #include <memory_resource>
2
3 // 1. new_delete_resource (Global Heap)
4 auto* heap = std::pmr::new_delete_resource();
5
6 // 2. null_memory_resource (Throws bad_alloc)
7 auto* null = std::pmr::null_memory_resource();
8
9 // 3. monotonic_buffer_resource (Fast, no deallocation)
10 // Very fast for building complex structures, deallocates all at once
11 std::pmr::monotonic_buffer_resource fast_pool(heap);
12
13 // 4. synchronized_pool_resource (Thread-safe pool)
14 // Good for many small allocations of same size
15 std::pmr::synchronized_pool_resource thread_safe_pool(heap);
16
17 // 5. unsynchronized_pool_resource (Single-thread pool)
18 // Fastest for single-threaded small allocations
19 std::pmr::unsynchronized_pool_resource local_pool(heap);

```

28.33.2 Chaining Resources

Memory resources can be chained. If a pool runs out of memory, it requests more from an “upstream” resource.

```

1 #include <memory_resource>
2 #include <vector>
3
4 void chaining_example() {
5     // Buffer on stack
6     std::array<std::byte, 256> buffer;

```

```

7
8 // Primary: Use stack buffer
9 // Upstream: If buffer full, go to heap
10 std::pmr::monotonic_buffer_resource mem_res(
11     buffer.data(), buffer.size(), std::pmr::new_delete_resource()
12 );
13
14 std::pmr::vector<int> vec(&mem_res);
15
16 // These go to stack buffer
17 for (int i = 0; i < 50; i++) vec.push_back(i);
18
19 // If we exceed buffer, it silently falls back to heap
20 }

```

28.33.3 Performance Benefits

std::pmr allows easy implementation of **Arena Allocation** or **Stack Allocation** for standard containers, which can provide massive performance gains (cache locality, no malloc overhead) for short-lived complex data structures.

28.34 C++17 BEST PRACTICES

28.35 What's Better with C++17

```

1 // 1. Use structured bindings to unpack
2 auto [x, y, z] = tuple{1, 2, 3};
3
4 // 2. Use optional for nullable values
5 optional<int> result = process();
6 if (result) { cout << result.value() << "\n"; }
7
8 // 3. Use variant for type-safe unions
9 variant<int, string> value = 42;
10
11 // 4. Use string_view for non-owning strings
12 void process(string_view sv); // No copy!
13
14 // 5. Use if constexpr for compile-time branching
15 if constexpr (is_integral_v<T>) { }
16
17 // 6. Use fold expressions for parameter packs
18 auto sum = (... + args);
19
20 // 7. Use filesystem for file operations
21 for (const auto& entry : fs::directory_iterator(dir)) { }
22
23 // 8. Use invoke for uniform callable invocation
24 invoke(func, args...);
25
26 // 9. Use parallel algorithms for performance
27 sort(execution::par, v.begin(), v.end());
28
29 // 10. Use CTAD for cleaner code
30 vector v{1, 2, 3}; // Not vector<int>{...}

```

Part V

Volume V: C++20 - The Gigantic Leap

Chapter 29

C++20 REVOLUTIONARY FEATURES

29.1 C++20 Overview & Revolutionary Scope

C++20 (finalized in 2020) is a **revolutionary language update** rivaling C++11 in magnitude.

29.1.1 Timeline & Context

- **2011:** C++11 (first modern standard)
- **2014:** C++14 (refinement)
- **2017:** C++17 (major improvements)
- **2020:** C++20 (revolutionary leap)
- **2023:** C++23 (latest)

29.1.2 C++20 Philosophy

- **Revolutionize** generic programming with concepts
- **Simplify** iteration with ranges
- **Empower** asynchronous programming with coroutines
- **Standardize** previously non-standard patterns
- **Address** fundamental C++ limitations
- **Enable** modern programming paradigms

29.1.3 Key Themes

1. **Concepts** - Readable, constrained templates
2. **Ranges** - Composable, lazy evaluation
3. **Coroutines** - Asynchronous, generator patterns
4. **Spaceship** - Three-way comparison
5. **Modules** - Modularity & faster compilation

- 6. **Designated Initializers** - Named struct initialization
- 7. **Format** - Type-safe string formatting
- 8. **Constraints** - Compile-time validation

29.1.4 Why C++20 Matters

C++20 addresses fundamental limitations: - Readable generic programming (concepts) - Composable iteration (ranges) - Async/await patterns (coroutines) - Lazy evaluation (ranges with coroutines) - Modular code (modules) - Type-safe formatting (std::format) - Compile-time validation (constexpr) - Powerful iteration patterns

29.2 CONCEPTS & CONSTRAINTS

29.3 1.1 Introduction to Concepts

Concepts are constraints on template parameters that make templates readable and enable better error messages.

29.3.1 Basic Concept Definition

```

1 #include <concepts>
2 using namespace std;
3
4 // Define a concept
5 template<typename T>
6 concept Integral = is_integral_v<T>;
7
8 // Use concept as constraint
9 template<Integral T>
10 void process(T value) {
11     cout << "Integer: " << value << "\n";
12 }
13
14 process(42);           // OK - int satisfies Integral
15 process(3.14);         // ERROR - double doesn't satisfy Integral
16 // Error message is clear: doesn't satisfy Integral concept

```

29.3.2 Standard Library Concepts

```

1 #include <concepts>
2
3 // Predefined concepts
4 template<typename T>
5 concept Integer = integral<T>; // std::integral
6
7 template<typename T>
8 concept Floating = floating_point<T>; // std::floating_point
9
10 template<typename T>
11 concept Numeric = integral<T> || floating_point<T>;
12
13 template<typename T>
14 concept Comparable = requires(T a, T b) {

```

```

15     { a < b } -> convertible_to<bool>;
16     { a == b } -> convertible_to<bool>;
17 };
18
19 // Usage
20 template<Comparable T>
21 T find_min(T a, T b) {
22     return a < b ? a : b;
23 }
24
25 find_min(5, 3);           // OK
26 find_min("a", "b");       // OK - strings are comparable
27 // find_min(complex(1,2), complex(3,4)); // ERROR - complex not comparable

```

29.3.3 Complex Concept Definition

```

1 #include <concepts>
2 #include <ranges>
3
4 // Concept with multiple requirements
5 template<typename T>
6 concept Container = requires(T c) {
7     typename T::value_type;
8     typename T::iterator;
9     typename T::const_iterator;
10    { c.begin() } -> convertible_to<typename T::iterator>;
11    { c.end() } -> convertible_to<typename T::iterator>;
12    { c.size() } -> convertible_to<size_t>;
13    { c.empty() } -> convertible_to<bool>;
14 };
15
16 // Use in function
17 template<Container C>
18 void print_container(const C& c) {
19     for (const auto& elem : c) {
20         cout << elem << " ";
21     }
22     cout << "\n";
23 }
24
25 vector<int> v{1, 2, 3};
26 print_container(v); // OK
27
28 // Custom type
29 struct MyContainer {
30     vector<int> data;
31     using value_type = int;
32     using iterator = vector<int>::iterator;
33     using const_iterator = vector<int>::const_iterator;
34
35     iterator begin() { return data.begin(); }
36     iterator end() { return data.end(); }
37     const_iterator begin() const { return data.begin(); }
38     const_iterator end() const { return data.end(); }
39     size_t size() const { return data.size(); }
40     bool empty() const { return data.empty(); }
41 };
42
43 MyContainer mc;
44 print_container(mc); // OK

```


29.3.4 Concept Benefits

```

1 // Before C++20: Complex error messages
2 template<typename T>
3 void process_old(T x) {
4     // If T doesn't have operator+, error is confusing
5     auto result = x + 5;
6 }
7
8 process_old("string"); // ERROR - cryptic, long error message
9
10 // After C++20: Clear error messages
11 template<typename T>
12 requires requires(T x) { x + 5; }
13 void process_new(T x) {
14     auto result = x + 5;
15 }
16
17 process_new("string"); // ERROR - "string" doesn't satisfy concept

```

29.4 1.2 Requires Expressions

Requires expressions test compile-time properties of types.

29.4.1 Basic Requires Expression

```

1 #include <concepts>
2
3 template<typename T>
4 requires requires(T x) {
5     x + 1;           // Must support addition
6     x.size();        // Must have size() member
7     { x == x };      // Must support equality
8 }
9 void process(T x);
10
11 // Can also write as concept
12 template<typename T>
13 concept Processable = requires(T x) {
14     x + 1;
15     x.size();
16     { x == x };
17 };
18
19 template<Processable T>
20 void process2(T x);

```

29.4.2 Requires with Return Type Checking

```

1 #include <concepts>
2
3 template<typename T>
4 concept Addable = requires(T a, T b) {
5     { a + b } -> convertible_to<T>;
6 };
7

```

```

8 template<typename T>
9 concept Multipliable = requires(T a, T b) {
10     { a * b } -> convertible_to<T>;
11 };
12
13 template<typename T>
14 concept Arithmetic = Addable<T> && Multipliable<T>;
15
16 template<Arithmetic T>
17 T compute(T x, T y) {
18     return (x + y) * (x - y);
19 }
20
21 cout << compute(5, 3) << "\n";           // OK - int is Arithmetic
22 cout << compute(2.5, 1.5) << "\n";       // OK - double is Arithmetic

```

29.4.3 Practical Requires Examples

```

1 // Check for operator[] and size()
2 template<typename T>
3 concept Indexable = requires(T t, size_t i) {
4     { t[i] };
5     { t.size() } -> convertible_to<size_t>;
6 };
7
8 // Check for specific method
9 template<typename T>
10 concept HasValue = requires(T t) {
11     { t.value() };
12 };
13
14 // Check for const and non-const versions
15 template<typename T>
16 concept ConstIterable = requires(const T& t) {
17     t.begin();
18     t.end();
19 };
20
21 // Multi-type concepts
22 template<typename Iter, typename Sentinel>
23 concept SentinelFor = requires(Iter it, Sentinel s) {
24     { it == s } -> convertible_to<bool>;
25 };

```

29.5 1.3 Concepts & Overload Resolution

Concepts participate in overload resolution. The compiler selects the **most constrained** template.

```

1 template<typename T>
2 void process(T x) {
3     cout << "Generic\n";
4 }
5
6 template<typename T> requires std::integral<T>
7 void process(T x) {
8     cout << "Integral\n";
9 }
10
11 template<typename T> requires (std::integral<T> && sizeof(T) >= 4)

```

```

12 void process(T x) {
13     cout << "Large Integral\n";
14 }
15
16 process(3.14);           // "Generic"
17 process((short)10);      // "Integral"
18 process(100);            // "Large Integral" (int is >= 4 bytes)

```

29.6 RANGES LIBRARY

29.7 2.1 Introduction to Ranges

Ranges provide a composable, lazy way to work with sequences.

29.7.1 Basic Range Operations

```

1 #include <ranges>
2 #include <vector>
3 #include <iostream>
4 using namespace std;
5
6 vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
7
8 // Traditional algorithm
9 vector<int> result;
10 for (int x : v) {
11     if (x % 2 == 0) {
12         result.push_back(x * 2);
13     }
14 }
15
16 // With ranges (composable, lazy)
17 auto result = v
18     | ranges::views::filter([](int x) { return x % 2 == 0; })
19     | ranges::views::transform([](int x) { return x * 2; });
20
21 // result is lazy - computation happens on iteration
22 for (int x : result) {
23     cout << x << " "; // 4 8 12 16 20
24 }

```

29.7.2 Range Views

```

1 #include <ranges>
2 #include <vector>
3 using namespace std;
4
5 vector<int> v = {1, 2, 3, 4, 5};
6
7 // filter view
8 auto evens = v | ranges::views::filter([](int x) { return x % 2 == 0; });
9
10 // transform view
11 auto doubled = v | ranges::views::transform([](int x) { return x * 2; });
12
13 // take view (first N elements)

```

```

14 auto first3 = v | ranges::views::take(3);
15
16 // drop view (skip first N elements)
17 auto skip2 = v | ranges::views::drop(2);
18
19 // reverse view
20 auto reversed = v | ranges::views::reverse;
21
22 // iota view (generate sequence)
23 auto seq = ranges::views::iota(1, 11); // 1..10
24
25 // join view (flatten nested ranges)
26 vector<vector<int>> matrix = {{1, 2}, {3, 4}, {5, 6}};
27 auto flattened = matrix | ranges::views::join;
28
29 // zip view (pair elements from two ranges)
30 vector<int> a = {1, 2, 3};
31 vector<string> b = {"a", "b", "c"};
32 auto zipped = ranges::views::zip(a, b);
33
34 for (auto [num, str] : zipped) {
35     cout << num << ":" << str << " "; // 1:a 2:b 3:c
36 }

```

29.7.3 Range Algorithms

```

1 #include <ranges>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 vector<int> v = {3, 1, 4, 1, 5, 9};
7
8 // Range algorithms (work with ranges, not iterators)
9 ranges::sort(v); // In-place sort
10 ranges::reverse(v); // In-place reverse
11 ranges::fill(v, 0); // Fill with value
12
13 // Range algorithms with predicates
14 ranges::sort(v, ranges::greater{}); // Sort descending
15 auto it = ranges::find(v, 5); // Find element
16 auto count = ranges::count_if(v, [](int x) { return x > 3; });
17
18 // Range operations
19 ranges::rotate(v.begin(), v.begin() + 2, v.end());
20 ranges::partition(v, [](int x) { return x % 2 == 0; });

```

29.7.4 Composing Multiple Views

```

1 #include <ranges>
2 #include <vector>
3 #include <iostream>
4 using namespace std;
5
6 vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
7
8 // Chain multiple operations
9 auto result = v
10     | ranges::views::filter([](int x) { return x > 2; }) // > 2
11     | ranges::views::transform([](int x) { return x * x; }) // Square

```

```

12 | ranges::views::take(4); // First 4
13
14 // Process
15 for (int x : result) {
16     cout << x << " "; // 9 16 25 36
17 }
18
19 // All operations are lazy - no temporary vectors created
20 // Composition is clear and readable

```

29.8 2.2 Ranges Deep Dive

29.8.1 Projections

Most range algorithms accept a “projection” argument to transform data *before* comparison.

```

1 struct User { int id; string name; };
2 vector<User> users = {{2, "Bob"}, {1, "Alice"}};
3
4 // Sort by ID
5 ranges::sort(users, {}, &User::id);
6
7 // Sort by Name (descending)
8 ranges::sort(users, ranges::greater{}, &User::name);

```

29.8.2 Dangling Iterators

Algorithms return `std::ranges::dangling` if the range is an rvalue (temporary) to prevent use-after-free.

```

1 auto get_vector() { return vector{1, 2, 3}; }
2
3 auto it = ranges::find(get_vector(), 2);
4 // Compile Error! 'it' would be dangling.
5 // The vector is destroyed at the end of the statement.

```

29.9 COROUTINES

29.10 3.1 Introduction to Coroutines

Coroutines enable asynchronous, generator, and lazy evaluation patterns.

29.10.1 Basic Generator Coroutine

```

1 #include <coroutine>
2 #include <iostream>
3 using namespace std;
4
5 template<typename T>
6 class Generator {
7 public:
8     struct promise_type {
9         T current_value;
10

```

```

11     Generator get_return_object() {
12         return Generator{coroutine_handle<promise_type>::from_promise(*this
13     });
14     }
15
16     suspend_never initial_suspend() { return {}; }
17     suspend_always final_suspend() noexcept { return {}; }
18
19     suspend_always yield_value(T value) {
20         current_value = value;
21         return {};
22     }
23
24     void return_void() {}
25     void unhandled_exception() {}
26 };
27
28 struct iterator {
29     coroutine_handle<promise_type> handle;
30
31     iterator(coroutine_handle<promise_type> h, bool done)
32         : handle(h) {
33         if (done) {
34             handle = nullptr;
35         }
36     }
37
38     iterator& operator++() {
39         handle.resume();
40         if (handle.done()) {
41             handle = nullptr;
42         }
43         return *this;
44     }
45
46     bool operator==(const iterator& other) const {
47         return handle == other.handle;
48     }
49
50     bool operator!=(const iterator& other) const {
51         return !(*this == other);
52     }
53
54     T operator*() const {
55         return handle.promise().current_value;
56     }
57 };
58
59 iterator begin() {
60     if (handle) {
61         handle.resume();
62     }
63     return iterator{handle, !handle || handle.done()};
64 }
65
66 iterator end() {
67     return iterator{nullptr, true};
68 }
69 private:
70     coroutine_handle<promise_type> handle;
71
72     Generator(coroutine_handle<promise_type> h) : handle(h) {}

```

```

73 };
74
75 // Generator coroutine
76 Generator<int> count_up(int max) {
77     for (int i = 1; i <= max; i++) {
78         co_yield i; // Yield value and suspend
79     }
80 }
81
82 // Usage
83 int main() {
84     for (int i : count_up(5)) {
85         cout << i << " "; // 1 2 3 4 5
86     }
87     return 0;
88 }

```

29.10.2 Async Coroutine

```

1  #include <coroutine>
2  #include <iostream>
3  #include <chrono>
4  using namespace std;
5
6  class Task {
7  public:
8      struct promise_type {
9          Task get_return_object() {
10             return Task{coroutine_handle<promise_type>::from_promise(*this)};
11          }
12
13          suspend_never initial_suspend() { return {}; }
14          suspend_always final_suspend() noexcept { return {}; }
15
16          void return_void() {}
17          void unhandled_exception() {}
18      };
19
20      coroutine_handle<promise_type> handle;
21
22      Task(coroutine_handle<promise_type> h) : handle(h) {}
23
24      ~Task() {
25          if (handle) {
26              handle.destroy();
27          }
28      }
29 };
30
31 // Async coroutine
32 Task async_work() {
33     cout << "Starting work\n";
34     co_await std::suspend_always{}; // Suspend and resume later
35     cout << "Continuing work\n";
36 }
37
38 int main() {
39     auto task = async_work(); // Starts coroutine
40     // Coroutine is suspended
41     task.handle.resume();     // Resume execution
42     return 0;
43 }

```

29.10.3 Practical Coroutine: Fibonacci Generator

```

1 #include <coroutine>
2 #include <iostream>
3 using namespace std;
4
5 template<typename T>
6 class Generator { /* ... implementation ... */ };
7
8 Generator<int> fibonacci(int limit) {
9     int a = 0, b = 1;
10    while (a < limit) {
11        co_yield a;
12        int next = a + b;
13        a = b;
14        b = next;
15    }
16 }
17
18 int main() {
19     for (int i : fibonacci(100)) {
20         cout << i << " "; // 0 1 1 2 3 5 8 13 21 34 55 89
21     }
22     return 0;
23 }

```

29.11 3.2 Coroutines Deep Dive

A coroutine is a function that can suspend and resume.

29.11.1 The Awaitable Interface

To `co_await x`, `x` must be an Awaitable.

```

1 struct Awaiter {
2     bool await_ready() { return false; } // Always suspend?
3
4     void await_suspend(std::coroutine_handle<> h) {
5         // Schedule resumption (e.g., on a thread pool)
6         // h.resume();
7     }
8
9     int await_resume() { return 42; } // Result of co_await
10 };
11
12 Task coroutine() {
13     int result = co_await Awaiter{}; // result = 42
14 }

```

29.11.2 Symmetric Transfer

Returning a `coroutine_handle` from `await_suspend` performs a “tail-call” to resume another coroutine without consuming stack space.

```

1 std::coroutine_handle<> await_suspend(std::coroutine_handle<> h) {
2     return other_handle; // Switch to other coroutine immediately
3 }

```


29.12 SPACESHIP OPERATOR (THREE-WAY COMPARISON)

29.13 4.1 The Spaceship Operator <=>

The spaceship operator performs three-way comparison and returns comparison category.

29.13.1 Basic Spaceship Usage

```
1 #include <compare>
2 #include <iostream>
3 using namespace std;
4
5 int a = 5, b = 10;
6
7 // Spaceship operator returns ordering
8 auto cmp = a <=> b;
9
10 if (cmp < 0) {
11     cout << "a < b\n";
12 } else if (cmp > 0) {
13     cout << "a > b\n";
14 } else {
15     cout << "a == b\n";
16 }
```

29.13.2 Spaceship with Custom Types

```
1 #include <compare>
2
3 struct Person {
4     string name;
5     int age;
6
7     // Default spaceship (compares as tuple)
8     auto operator<=>(const Person&) const = default;
9 };
10
11 Person p1{"Alice", 30};
12 Person p2{"Bob", 25};
13
14 auto cmp = p1 <=> p2;
15 if (cmp < 0) cout << "p1 < p2\n";
16 if (cmp > 0) cout << "p1 > p2\n";
```

29.13.3 Defaulted Comparison

```
1 #include <compare>
2
3 struct Point {
4     int x, y;
5
6     // Default spaceship - compares lexicographically
7     auto operator<=>(const Point&) const = default;
```

```

8 };
9
10 Point p1{1, 2};
11 Point p2{1, 2};
12 Point p3{2, 1};
13
14 cout << (p1 <=> p2 == 0) << "\n";    // true (equal)
15 cout << (p1 <=> p3 < 0) << "\n";    // true (p1 < p3)

```

29.13.4 Comparison Categories

```

1 #include <compare>
2 #include <iostream>
3
4 // Different comparison categories
5 struct Comparable {
6     int value;
7
8     // Returns std::strong_ordering (can do all operations)
9     strong_ordering operator<=>(const Comparable& other) const {
10         return value <=> other.value;
11     }
12 };
13
14 struct PartiallyComparable {
15     double value;
16
17     // Returns std::partial_ordering (NaN is not comparable)
18     partial_ordering operator<=>(const PartiallyComparable& other) const {
19         return value <=> other.value;
20     }
21 };
22
23 Comparable c1{5}, c2{10};
24 cout << (c1 <=> c2 < 0) << "\n";    // true
25
26 PartiallyComparable p1{1.5}, p2{2.5};
27 cout << (p1 <=> p2 < 0) << "\n";    // true

```

29.13.5 Spaceship Benefits

```

1 // Before C++20: Must define all comparison operators
2 struct Person {
3     string name;
4     int age;
5
6     bool operator<(const Person& other) const {
7         if (name != other.name) return name < other.name;
8         return age < other.age;
9     }
10
11     bool operator<=(const Person& other) const { /* ... */ }
12     bool operator>(const Person& other) const { /* ... */ }
13     bool operator>=(const Person& other) const { /* ... */ }
14     bool operator==(const Person& other) const { /* ... */ }
15     bool operator!=(const Person& other) const { /* ... */ }
16 };
17
18 // After C++20: Default spaceship does all of it
19 struct Person {

```

```

20     string name;
21     int age;
22
23     auto operator<=> (const Person&) const = default;
24 };

```

29.14 MODULES

29.15 5.1 Introduction to Modules

Modules provide better code organization and faster compilation.

29.15.1 Module Definition

```

1  // math_module.cppm (module interface unit)
2  export module math;
3
4  export int add(int a, int b) {
5      return a + b;
6  }
7
8  export int multiply(int a, int b) {
9      return a * b;
10 }
11
12 // Helper function (not exported)
13 int helper(int x) {
14     return x * 2;
15 }

```

29.15.2 Using Modules

```

1  // main.cpp
2  import math;
3  #include <iostream>
4  using namespace std;
5
6  int main() {
7      cout << add(5, 3) << "\n";           // OK - exported
8      cout << multiply(5, 3) << "\n";       // OK - exported
9      // cout << helper(5) << "\n";         // ERROR - not exported
10
11     return 0;
12 }

```

29.15.3 Module Partitions

```

1  // math.cppm (main interface)
2  export module math;
3  export import :impl;
4
5  // math-impl.cppm (partition)
6  export module math:impl;
7

```

```

8 export struct Complex {
9     double real, imag;
10
11     Complex operator+(const Complex& other) const {
12         return {real + other.real, imag + other.imag};
13     }
14 };

```

29.15.4 Module Benefits

```

1 // Before modules (header files):
2 // - Recompile overhead
3 // - Macro pollution
4 // - Circular dependencies
5 // - Header guards boilerplate
6
7 // After modules:
8 // - Faster compilation (parse once)
9 // - No macro pollution
10 // - No circular dependency issues
11 // - Clean interface definition

```

29.16 5.2 Modules Deep Dive

29.16.1 Global Module Fragment

For legacy headers that must be included before the module declaration.

```

1 module; // Start fragment
2 #include <vector>
3 #include <string>
4
5 export module my_app; // End fragment, start module
6
7 export void process(std::vector<int>& v);

```

29.16.2 Private Module Partition

Hiding implementation details within the same file.

```

1 export module calculator;
2
3 export int add(int a, int b);
4
5 module :private; // Start private implementation
6
7 int helper(int x) { return x + 1; }
8
9 int add(int a, int b) {
10     return helper(a) + helper(b) - 2;
11 }

```

29.17 DESIGNATED INITIALIZERS

29.18 6.1 Named Member Initialization

Designated initializers allow initializing struct/class members by name.

29.18.1 Basic Designated Initializers

```
1 #include <iostream>
2 using namespace std;
3
4 struct Point {
5     int x;
6     int y;
7     int z;
8 };
9
10 // Before C++20: Order matters
11 Point p1{1, 2, 3}; // x=1, y=2, z=3
12
13 // After C++20: Can specify by name
14 Point p2{.x = 10, .y = 20, .z = 30};
15 Point p3{.y = 20, .x = 10, .z = 30}; // Order doesn't matter
16 Point p4{.x = 5, .z = 15};           // y defaults to 0
17
18 cout << p2.x << " " << p2.y << " " << p2.z << "\n"; // 10 20 30
```

29.18.2 With Classes and Inheritance

```
1 struct Base {
2     int a;
3 };
4
5 struct Derived : Base {
6     int b;
7     int c;
8 };
9
10 // Designators for base and derived members
11 Derived d{.a = 1, .b = 2, .c = 3};
12
13 cout << d.a << " " << d.b << " " << d.c << "\n"; // 1 2 3
```

29.18.3 Practical Designated Initializers

```
1 struct Config {
2     string name;
3     int port;
4     string host;
5     bool ssl;
6     int timeout;
7 };
8
9 // Clear intent - parameters obvious
10 Config cfg{
11     .name = "server",
12     .port = 8080,
13     .host = "localhost",
```

```
14     .ssl = true,
15     .timeout = 30
16 };
17
18 // Much better than:
19 // Config cfg{"server", 8080, "localhost", true, 30};
```

29.19 CALENDAR & TIME ZONES

29.20 7.1 Advanced Chrono Features

C++20 adds comprehensive calendar and timezone support.

29.20.1 Calendar Types

```
1 #include <chrono>
2 #include <iostream>
3 using namespace std;
4 using namespace chrono;
5
6 // Year, month, day
7 year y{2024};
8 month m{12};
9 day d{25};
10
11 // Construct date
12 auto date = y / m / d; // 2024-12-25
13 cout << date << "\n";
14
15 // Current date
16 auto today = floor<days>(system_clock::now());
17 cout << "Today: " << today << "\n";
18
19 // Date arithmetic
20 auto tomorrow = date + days(1);
21 auto next_month = date + months(1);
22 auto next_year = date + years(1);
```

29.20.2 Time Zones

```
1 #include <chrono>
2 #include <iostream>
3 using namespace std;
4 using namespace chrono;
5
6 // Get timezone
7 const auto& tz = locate_zone("America/New_York");
8
9 // Current time in timezone
10 auto now = system_clock::now();
11 auto zoned_time = make_zoned(tz, now);
12
13 cout << "UTC: " << now << "\n";
14 cout << "NY: " << zoned_time << "\n";
```

29.20.3 Formatted Time Output

```

1 #include <chrono>
2 #include <format>
3 #include <iostream>
4 using namespace std;
5 using namespace chrono;
6
7 auto now = system_clock::now();
8
9 // Format with pattern
10 cout << format("{:%Y-%m-%d %H:%M:%S}", now) << "\n";
11 // Output: 2024-12-25 15:30:45

```

29.21 STD::FORMAT

29.22 8.1 Type-Safe String Formatting

`std::format` provides Python-like formatting without type unsafety.

29.22.1 Basic format Usage

```

1 #include <format>
2 #include <iostream>
3 using namespace std;
4
5 // Simple substitution
6 cout << format("Hello {}, you are {} years old", "Alice", 30) << "\n";
7 // Output: Hello Alice, you are 30 years old
8
9 // Positional arguments
10 cout << format("{1} {0}", "World", "Hello") << "\n";
11 // Output: Hello World
12
13 // Argument access by index
14 cout << format("{0} + {0} = {}", 5, 10) << "\n";
15 // Output: 5 + 5 = 10

```

29.22.2 Formatting Specifications

```

1 #include <format>
2 #include <iostream>
3 using namespace std;
4
5 int num = 255;
6 double pi = 3.14159;
7
8 // Hex, binary, octal
9 cout << format("{:x}", num) << "\n";           // ff (hex)
10 cout << format("{:b}", num) << "\n";          // 11111111 (binary)
11 cout << format("{:o}", num) << "\n";          // 377 (octal)
12
13 // Floating point precision
14 cout << format("{:.2f}", pi) << "\n";         // 3.14
15 cout << format("{:.5f}", pi) << "\n";         // 3.14159

```

```

16
17 // Padding and alignment
18 cout << format("{:>10}", "hello") << "\n";      // "      hello" (right)
19 cout << format("{:<10}", "hello") << "\n";      // "hello      " (left)
20 cout << format("{:^10}", "hello") << "\n";      // "  hello  " (center)
21
22 // Number formatting
23 cout << format("{:,}", 1234567) << "\n";        // 1,234,567 (with separator)
24 cout << format("{:e}", pi) << "\n";            // 3.14e+00 (scientific)

```

29.22.3 Format with Custom Types

```

1 #include <format>
2
3 struct Point {
4     int x, y;
5 };
6
7 // Define formatter for Point
8 template<>
9 struct format_traits<Point> {
10     static auto format(const Point& p) {
11         return format_string("({}, {})", p.x, p.y);
12     }
13 };
14
15 Point pt{10, 20};
16 cout << format("Point: {}", pt) << "\n"; // Point: (10, 20)

```

29.23 CONSTEVAL & CONSTINIT

29.24 9.1 Immediate Functions and Constants

29.24.1 consteval - Immediate Functions

```

1 #include <iostream>
2 using namespace std;
3
4 // Must be evaluated at compile-time
5 consteval int square(int x) {
6     return x * x;
7 }
8
9 int main() {
10     int arr[square(5)]; // OK - computed at compile-time
11     cout << square(10) << "\n"; // OK - 100
12
13     int x = 5;
14     // cout << square(x) << "\n"; // ERROR - x is not compile-time constant
15
16     return 0;
17 }

```


29.24.2 constinit - Compile-Time Initialization

```
1 #include <iostream>
2 using namespace std;
3
4 // Thread-local with compile-time initialization
5 thread_local constinit int counter = 0;
6
7 int main() {
8     counter = 10; // Can be modified at runtime
9     cout << counter << "\n"; // 10
10
11     return 0;
12 }
```

29.24.3 Difference: constexpr vs consteval

```
1 // constexpr: Can be evaluated at compile-time OR runtime
2 constexpr int add_constexpr(int a, int b) {
3     return a + b;
4 }
5
6 // consteval: MUST be evaluated at compile-time
7 consteval int add_consteval(int a, int b) {
8     return a + b;
9 }
10
11 int main() {
12     int x = 5, y = 10;
13
14     int c1 = add_constexpr(x, y); // Runtime evaluation
15     int c2 = add_constexpr(5, 10); // Compile-time evaluation
16
17     // int d1 = add_consteval(x, y); // ERROR - must be compile-time
18     int d2 = add_consteval(5, 10); // OK - compile-time
19
20     return 0;
21 }
```

29.25 LAMBDA ENHANCEMENTS

29.26 10.1 C++20 Lambda Improvements

29.26.1 Default Constructible Lambdas

```
1 #include <iostream>
2 using namespace std;
3
4 // C++20: Lambdas without captures can be default constructed
5 auto counter = [count = 0]() mutable { return ++count; };
6
7 // Can be default constructed
8 decltype(counter) c1; // Default construct
9 c1();
10
11 // But lambdas with captures still can't
```

```

12 // auto [x] = 5;
13 // decltype([x]() {}) bad; // ERROR

```

29.26.2 Stateless Lambda as Template Parameter

```

1 #include <iostream>
2 using namespace std;
3
4 template<auto F>
5 void call_func() {
6     F();
7 }
8
9 // Stateless lambda as template argument
10 call_func<[]() { cout << "Hello\n"; }>(); // OK
11
12 // Stateful lambda (captures) can't be template argument
13 // auto y = 5;
14 // call_func<[y]() { cout << y; }>(); // ERROR

```

29.27 ADVANCED FEATURES

29.28 11.1 Additional C++20 Features

29.28.1 Spaceship Operator with Library Support

```

1 #include <compare>
2 #include <vector>
3
4 // All standard library types support spaceship
5 vector<int> v1{1, 2, 3};
6 vector<int> v2{1, 2, 4};
7
8 auto cmp = v1 <=> v2;
9 if (cmp < 0) cout << "v1 < v2\n";

```

29.28.2 Bit Operations

```

1 #include <bit>
2 #include <iostream>
3 using namespace std;
4
5 unsigned int x = 12; // 0b1100
6
7 cout << bit_width(x) << "\n"; // 4 (bits needed)
8 cout << popcount(x) << "\n"; // 2 (number of 1s)
9 cout << countl_zero(x) << "\n"; // 28 (leading zeros on 32-bit)
10 cout << rotl(x, 2) << "\n"; // Rotate left
11 cout << rotr(x, 2) << "\n"; // Rotate right
12 cout << (x & ~(x - 1)) << "\n"; // Lowest set bit
13
14 // std::bit_cast (Safe type punning)
15 float f = 3.14f;
16 auto i = std::bit_cast<uint32_t>(f); // Safe reinterpretation of bits
17 cout << std::hex << i << "\n";

```

29.28.3 std::atomic_ref

std::atomic_ref allows atomic operations on non-atomic objects.

```

1 #include <atomic>
2 #include <thread>
3 #include <vector>
4
5 void process(int& counter) {
6     // Treat 'counter' as atomic for this scope
7     std::atomic_ref<int> atomic_counter(counter);
8     atomic_counter++;
9 }
10
11 int main() {
12     int val = 0;
13     std::vector<std::thread> threads;
14     for(int i=0; i<10; ++i) threads.emplace_back(process, std::ref(val));
15     for(auto& t : threads) t.join();
16     return 0;
17 }

```

29.28.4 Concepts in Standard Library

```

1 #include <concepts>
2 #include <iostream>
3
4 // Standard concepts
5 static_assert(integral<int>);
6 static_assert(floating_point<double>);
7 static_assert(invocable<int(*)>(int), int);
8 static_assert(copyable<int>);
9 static_assert(assignable_from<int&, int>);
10
11 template<typename T>
12 requires copyable<T>
13 void copy_safe(const T& src, T& dst) {
14     dst = src;
15 }

```

29.29 LIBRARY IMPROVEMENTS

29.30 12.1 STL Enhancements in C++20

29.30.1 std::span (Non-owning Array View)

std::span provides a lightweight, non-owning view over a contiguous sequence of objects (like array, vector, or C-array).

```

1 #include <span>
2 #include <vector>
3 #include <iostream>
4 #include <array>
5
6 void print_values(std::span<int> data) {
7     for (int x : data) {
8         std::cout << x << " ";
9     }
10 }

```

```

9     }
10    std::cout << "\n";
11 }
12
13 int main() {
14     int arr[] = {1, 2, 3};
15     std::vector<int> vec = {4, 5, 6};
16     std::array<int, 3> std_arr = {7, 8, 9};
17
18     // Works with all contiguous containers
19     print_values(arr);           // 1 2 3
20     print_values(vec);           // 4 5 6
21     print_values(std_arr);       // 7 8 9
22
23     // Sub-span (slicing)
24     print_values(std::span(vec).subspan(1)); // 5 6
25
26     return 0;
27 }

```

29.30.2 std::semaphore

```

1 #include <semaphore>
2 #include <thread>
3
4 counting_semaphore<3> sem(3); // Max 3 concurrent
5
6 void worker() {
7     sem.acquire();
8     // Critical section (at most 3 threads)
9     // Do work
10    sem.release();
11 }

```

29.30.3 std::latch & std::barrier

```

1 #include <latch>
2 #include <barrier>
3 #include <thread>
4 #include <vector>
5
6 // Latch: one-time synchronization
7 latch finish(3);
8
9 void worker(latch& l) {
10    // Do work
11    l.count_down();
12    l.wait(); // Wait for all to finish
13 }
14
15 // Barrier: reusable synchronization
16 barrier sync(3);
17
18 void barrier_worker(barrier& b) {
19     while (true) {
20         // Do work
21         b.arrive_and_wait(); // Synchronize every iteration
22     }
23 }

```

29.30.4 `std::source_location` (Reflection for Logging)

```

1 #include <source_location>
2 #include <iostream>
3
4 void log(const char* message,
5         const std::source_location location = std::source_location::current())
6 {
7     std::cout << "Info: " << message << "\n"
8               << "File: " << location.file_name() << "("
9               << location.line() << ":" << location.column() << ")\n"
10            << "Func: " << location.function_name() << "\n";
11 }
12
13 int main() {
14     log("Something happened");
15     return 0;
16 }

```

29.30.5 `std::osyncstream` (Synchronized Output)

Prevents interleaved output from multiple threads.

```

1 #include <syncstream>
2 #include <iostream>
3 #include <thread>
4
5 void worker(int id) {
6     std::osyncstream(std::cout) << "Worker " << id << " is running\n";
7 }
8
9 int main() {
10     std::thread t1(worker, 1);
11     std::thread t2(worker, 2);
12     t1.join(); t2.join();
13     return 0;
14 }

```

29.30.6 Ranges with Algorithms

```

1 #include <ranges>
2 #include <vector>
3 #include <algorithm>
4
5 vector<int> v = {3, 1, 4, 1, 5};
6
7 // Ranges algorithms with pipes
8 v | ranges::views::sort
9   | ranges::views::unique
10  | ranges::views::take(3)
11
12 ### std::jthread (Auto-joining Thread)
13
14 ```cpp
15 #include <thread>
16 #include <iostream>
17 using namespace std;
18
19 void worker(std::stop_token st) {
20     while (!st.stop_requested()) {

```

```

21     cout << "Working...\n";
22     std::this_thread::sleep_for(std::chrono::milliseconds(100));
23 }
24 cout << "Worker stopped\n";
25 }
26
27 int main() {
28     // jthread automatically joins on destruction
29     // and supports stop_token
30     std::jthread t(worker);
31
32     std::this_thread::sleep_for(std::chrono::milliseconds(300));
33     // t.request_stop() called automatically or manually
34     return 0;
35 }

```

```

1 ---
2
3 ## C++20 BEST PRACTICES
4
5 ## What's Better with C++20
6
7 ```cpp
8 // 1. Use concepts for readable templates
9 template<integral T>
10 void process(T x);
11
12 // 2. Use ranges for composable operations
13 auto result = v
14     | ranges::views::filter([](int x) { return x > 0; })
15     | ranges::views::transform([](int x) { return x * 2; });
16
17 // 3. Use coroutines for generators
18 Generator<int> count(int n) {
19     for (int i = 0; i < n; i++) {
20         co_yield i;
21     }
22 }
23
24 // 4. Use spaceship for comparisons
25 auto cmp = a <=> b;
26
27 // 5. Use designated initializers
28 Config cfg{.host = "localhost", .port = 8080};
29
30 // 6. Use std::format for formatting
31 cout << format("Value: {:.2f}", value);
32
33 // 7. Use constexpr for compile-time guarantees
34 constexpr int compile_time_only(int x);
35
36 // 8. Use modules for better organization
37 export module app;

```

Part VI

Volume VI: C++23/26 - The Future

Chapter 30

C++23 LATEST FEATURES

30.1 C++23 Overview & Direction

C++23 (finalized in 2023) is a **refinement and enhancement** of C++20 with practical improvements.

30.1.1 Timeline & Context

- **2011:** C++11 (revolutionary)
- **2014:** C++14 (refinement)
- **2017:** C++17 (major improvements)
- **2020:** C++20 (revolutionary leap)
- **2023:** C++23 (practical enhancements)

30.1.2 C++23 Philosophy

- **Enhance** existing C++20 features
- **Fill gaps** in C++20 design
- **Improve** convenience and usability
- **Optimize** common patterns
- **Standardize** frequently-requested features
- **Fix** issues discovered in C++20

30.1.3 Key Themes

1. **Output & Formatting** - `std::print` for easy output
2. **Error Handling** - `std::expected` for results
3. **Loop Control** - Enhanced for loops with ranges
4. **Memory Safety** - Better pointer/array handling
5. **Debugging** - Stack traces
6. **Templates** - Deducing this improvements

7. **Constexpr** - More compile-time power
8. **Library** - Quality of life improvements

30.1.4 Why C++23 Matters

C++23 builds on C++20 strengths: - Easier output without iostream overhead - Type-safe error handling (`std::expected`) - Better for loop control - Debugging support (stack traces) - More flexible subscript operator - Improved constexpr capabilities - More convenient library features - Better optional support

30.2 STD::PRINT & FORMATTED OUTPUT

30.3 1.1 std::print - Simple Output

`std::print` provides easy, fast output without iostream overhead.

30.3.1 Basic print Usage

```
1 #include <print>
2 #include <iostream>
3
4 // Simple output (no newline by default)
5 std::print("Hello, World!");
6
7 // With newline
8 std::println("Hello, World!");
9
10 // With format
11 std::println("Number: {}, Float: {:.2f}", 42, 3.14159);
12
13 // To stderr
14 std::print(std::cerr, "Error: {}\n", "something went wrong");
15 std::println(std::cerr, "Error: {}", "something went wrong");
```

30.3.2 print vs format vs iostream

```
1 #include <print>
2 #include <format>
3 #include <iostream>
4
5 std::string msg = "Hello";
6 int value = 42;
7
8 // iostream (slow, verbose)
9 std::cout << msg << ": " << value << "\n";
10
11 // format (creates string, then print)
12 std::cout << std::format("{}: {}\n", msg, value);
13
14 // print (direct output, fast)
15 std::println("{}: {}", msg, value);
```

30.3.3 print with File Streams

```

1 #include <print>
2 #include <fstream>
3
4 std::ofstream file("output.txt");
5
6 // Direct to file
7 std::println(file, "Line 1: {}", 42);
8 std::println(file, "Line 2: {}", "test");
9
10 // Simpler than:
11 // file << "Line 1: " << 42 << "\n";

```

30.4 1.2 std::format Enhancements

30.4.1 Format Improvements in C++23

```

1 #include <format>
2
3 // More format options
4 double pi = 3.14159;
5
6 std::format("{:.2%}", 0.25);           // "25.00%" (percentage)
7 std::format("{:g}", 0.0001);           // General format
8 std::format("{:#x}", 255);             // "0xff" (with prefix)
9 std::format("{:_^10}", "test");        // "____test" (custom fill)

```

30.5 DEDUCING THIS

30.6 2.1 Explicit Member Function Parameters

Deducing this allows capturing the type and constness of the object.

30.6.1 Basic Deducing This

```

1 #include <iostream>
2 using namespace std;
3
4 struct Counter {
5     int count = 0;
6
7     // Traditional
8     void increment() {
9         count++;
10    }
11
12    // With deducing this
13    void increment_new(this auto& self) {
14        self.count++;
15    }
16

```

```

17 // Value vs reference overloads now simple
18 auto get_data(this auto& self) {
19     return self.count;
20 }
21 };
22
23 Counter c;
24 c.increment_new();
25 cout << c.get_data() << "\n"; // 1

```

30.6.2 Deducing This for Const/Non-Const

```

1 struct Data {
2     int value = 42;
3
4     // Single function handles both const and non-const
5     auto& get(this auto& self) {
6         return self.value;
7     }
8
9     // Before C++23: Need two overloads
10    // int& get() { return value; }
11    // const int& get() const { return value; }
12 };
13
14 Data d;
15 d.get() = 100; // Mutable reference
16 cout << d.get() << "\n"; // 100
17
18 const Data cd;
19 cout << cd.get() << "\n"; // 100 (const reference)

```

30.6.3 Practical Deducing This

```

1 template<typename T>
2 struct Optional {
3     T value;
4     bool has_value_flag = false;
5
6     // Works for both optional<T> and optional<const T>
7     auto* get_if_value(this auto* self) {
8         return self->has_value_flag ? &self->value : nullptr;
9     }
10 };
11
12 Optional<int> opt;
13 opt.value = 42;
14 opt.has_value_flag = true;
15
16 if (auto* ptr = opt.get_if_value()) {
17     cout << *ptr << "\n"; // 42
18 }

```

30.7 2.2 Deducing This - Beyond the Basics

30.7.1 Recursive Lambdas

Previously, lambdas couldn't easily call themselves. Now they can via the explicit object parameter.

```
1 auto fib = [](this auto&& self, int n) {
2     if (n <= 1) return n;
3     return self(n - 1) + self(n - 2);
4 };
5
6 cout << fib(10) << "\n"; // 55
```

30.7.2 Replacing CRTP

The Curiously Recurring Template Pattern (CRTP) was used to inject functionality into derived classes. Deducing this simplifies it.

Old CRTP:

```
1 template <typename Derived>
2 struct Addable {
3     Derived& operator+=(const Derived& other) {
4         static_cast<Derived*>(this)->value += other.value;
5         return *static_cast<Derived*>(this);
6     }
7 };
8 struct Int : Addable<Int> { int value; };
```

New C++23 Way:

```
1 struct Addable {
2     template <typename Self>
3     auto& operator+=(this Self&& self, const Self& other) {
4         self.value += other.value;
5         return self;
6     }
7 };
8 struct Int : Addable { int value; }; // No template parameter needed!
```

30.8 RANGE-BASED FOR LOOP ENHANCEMENTS

30.9 3.1 For Loop Initializers

C++23 allows initialization in range-based for loops.

30.9.1 For Loop with Init

```
1 #include <vector>
2 #include <iostream>
3 using namespace std;
4
5 vector<int> v1 = {1, 2, 3};
6 vector<int> v2 = {4, 5, 6};
7
8 // Traditional
```

```

9 vector<int> combined;
10 for (int x : v1) combined.push_back(x);
11 for (int x : v2) combined.push_back(x);
12
13 // C++23: Initialize in loop
14 for (auto v = vector<int>{1, 2, 3, 4, 5, 6}; int x : v) {
15     cout << x << " ";
16 }
17
18 // More practical
19 for (auto file = open_file("data.txt"); auto line : file.lines()) {
20     cout << line << "\n";
21 }

```

30.9.2 For Loop with Init and Structured Binding

```

1 map<string, vector<int>> data{
2     {"a", {1, 2, 3}},
3     {"b", {4, 5, 6}}
4 };
5
6 // Initialize and use with structured binding
7 for (auto it = data.begin(); auto [key, values] : data) {
8     cout << key << ": ";
9     for (int v : values) cout << v << " ";
10    cout << "\n";
11 }

```

30.10 STD::EXPECTED

30.11 4.1 Result Type for Error Handling

`std::expected` represents either a value or an error.

30.11.1 Basic expected Usage

```

1 #include <expected>
2 #include <string>
3 #include <iostream>
4 using namespace std;
5
6 enum class ParseError { InvalidFormat, OutOfRange };
7
8 // Function returning expected
9 expected<int, ParseError> parse_int(const string& s) {
10     try {
11         return stoi(s);
12     } catch (const invalid_argument&) {
13         return unexpected(ParseError::InvalidFormat);
14     } catch (const out_of_range&) {
15         return unexpected(ParseError::OutOfRange);
16     }
17 }
18
19 // Usage
20 auto result = parse_int("42");

```

```

21
22 if (result) {
23     cout << "Value: " << result.value() << "\n";
24 } else {
25     cout << "Error\n";
26 }

```

30.11.2 expected with Transform

```

1 #include <expected>
2
3 expected<int, string> get_value();
4
5 // Chain operations with transform
6 auto result = get_value()
7     .transform([](int x) { return x * 2; })
8     .transform_error([](const string& e) { return "Failed: " + e; });
9
10 if (result) {
11     cout << result.value() << "\n";
12 } else {
13     cout << result.error() << "\n";
14 }

```

30.11.3 expected vs optional

```

1 // optional: Has value or nothing
2 optional<int> opt = parse_int("42");
3 if (!opt) {
4     // But why did it fail?
5 }
6
7 // expected: Has value or specific error
8 expected<int, ParseError> exp = parse_int("42");
9 if (!exp) {
10     cout << "Error: " << static_cast<int>(exp.error()) << "\n";
11 }

```

30.12 STD::OPTIONAL IMPROVEMENTS

30.13 5.1 Enhanced optional Operations

30.13.1 optional with Deref Operator

```

1 #include <optional>
2
3 optional<int> opt{42};
4
5 // C++23: Monadic operations
6 auto result = opt
7     .and_then([](int x) -> optional<int> { return x * 2; })
8     .or_else([]() { return optional<int>(0); });
9
10 // C++23: Chaining

```

```

11 opt.transform([](int x) { return x + 10; })
12   .and_then([](int x) -> optional<int> {
13       if (x > 50) return x;
14       return nullopt;
15   });

```

30.13.2 optional::value_or_else

```

1 #include <optional>
2
3 optional<int> opt;
4
5 // Get value or call function to generate default
6 int value = opt.value_or_else([]() { return compute_default(); });
7
8 // More flexible than value_or
9 // value_or: int value = opt.value_or(0);
10 // value_or_else: int value = opt.value_or_else([]() { return
    expensive_computation(); });

```

30.14 MULTIDIMENSIONAL SUBSCRIPT OPERATOR

30.15 6.1 Multiple Index Support

C++23 allows multiple indices in subscript operator.

30.15.1 Multi-Index Subscript

```

1 #include <iostream>
2 using namespace std;
3
4 struct Matrix {
5     int data[10][10];
6
7     // C++23: Multi-index operator
8     int& operator[](int row, int col) {
9         return data[row][col];
10    }
11
12    int& operator[](int row, int col) const {
13        return data[row][col];
14    }
15 };
16
17 Matrix m;
18 m[2, 3] = 42; // Two indices
19 cout << m[2, 3] << "\n"; // 42

```

30.15.2 Dynamic 2D Array Wrapper

```

1 template<typename T>
2 struct Array2D {
3     vector<T> data;
4     size_t width, height;

```

```

5   Array2D(size_t w, size_t h) : width(w), height(h), data(w * h) {}
6
7   T& operator[](size_t row, size_t col) {
8       return data[row * width + col];
9   }
10
11   const T& operator[](size_t row, size_t col) const {
12       return data[row * width + col];
13   }
14 };
15
16
17 Array2D<int> grid(3, 3);
18 grid[1, 1] = 5;
19 cout << grid[1, 1] << "\n"; // 5

```

30.16 6.2 std::mdspan (Multidimensional View)

std::mdspan provides a non-owning multidimensional view of contiguous data.

30.16.1 Basic mdspan Usage

```

1 #include <mdspan>
2 #include <vector>
3 #include <iostream>
4
5 int main() {
6     std::vector<int> data = {
7         1, 2, 3, 4,
8         5, 6, 7, 8,
9         9, 10, 11, 12
10    };
11
12    // Create a 3x4 view over the data (C++23)
13    // using MatrixView = std::mdspan<int, std::dextents<size_t, 2>>;
14    // MatrixView m(data.data(), 3, 4);
15
16    // m[1, 2] == 7 (row 1, col 2)
17    // No copying of data involved!
18
19    return 0;
20 }

```

30.17 6.3 mdspan Layouts

You can control how 2D indices map to the 1D memory.

- std::layout_right: Row-major (default in C++). Index (i, j) is $i * N + j$.
- std::layout_left: Column-major (Fortran/MATLAB). Index (i, j) is $j * M + i$.

```

1 using RowMajor = std::mdspan<double, std::dextents<size_t, 2>, std::
    layout_right>;
2 using ColMajor = std::mdspan<double, std::dextents<size_t, 2>, std::layout_left
    >;

```



```

3
4 // Same data, different interpretation
5 std::vector<double> v = {1, 2, 3, 4}; // 2x2 matrix
6 RowMajor m_row(v.data(), 2, 2); // [[1, 2], [3, 4]]
7 ColMajor m_col(v.data(), 2, 2); // [[1, 3], [2, 4]]

```

30.18 STD::STACKTRACE

30.19 7.1 Runtime Stack Trace Capture

`std::stacktrace` provides runtime stack trace information.

30.19.1 Basic Stacktrace

```

1 #include <stacktrace>
2 #include <print>
3 #include <iostream>
4
5 void deep_function() {
6     // Capture current stack trace
7     auto trace = std::stacktrace::current();
8
9     // Print trace
10    std::println("Stack trace:");
11    for (const auto& entry : trace) {
12        std::println("  {}", entry);
13    }
14 }
15
16 void middle_function() {
17     deep_function();
18 }
19
20 void top_function() {
21     middle_function();
22 }
23
24 int main() {
25     top_function();
26     return 0;
27 }
28
29 // Output:
30 // Stack trace:
31 //   top_function()
32 //   middle_function()
33 //   deep_function()

```

30.19.2 Stacktrace in Error Handling

```

1 #include <stacktrace>
2 #include <exception>
3
4 class TracedException : public std::exception {
5     std::stacktrace trace;
6     std::string message;

```

```

7
8 public:
9     TracedException(const std::string& msg)
10         : trace(std::stacktrace::current()), message(msg) {}
11
12     const char* what() const noexcept override {
13         return message.c_str();
14     }
15
16     const std::stacktrace& get_trace() const {
17         return trace;
18     }
19 };
20
21 // Usage
22 void operation() {
23     if (error_condition) {
24         throw TracedException("Operation failed");
25     }
26 }
27
28 try {
29     operation();
30 } catch (const TracedException& e) {
31     std::println("Error: {}", e.what());
32     std::println("Trace: {}", e.get_trace());
33 }

```

30.20 CONSTEXPR ENHANCEMENTS

30.21 8.1 More Compile-Time Capabilities

30.21.1 constexpr std::string

```

1 #include <string>
2
3 // C++23: Can use std::string in constexpr context
4 constexpr std::string concat(const char* a, const char* b) {
5     std::string result = a;
6     result += b;
7     return result;
8 }
9
10 constexpr auto msg = concat("Hello", " World");
11 // msg = "Hello World" at compile-time

```

30.21.2 constexpr vector Operations

```

1 #include <vector>
2
3 // C++23: vector works in constexpr
4 constexpr std::vector<int> make_sequence(int n) {
5     std::vector<int> result;
6     for (int i = 0; i < n; i++) {
7         result.push_back(i);
8     }
9 }

```

```

9     return result;
10 }
11
12 constexpr auto seq = make_sequence(5);
13 // seq = {0, 1, 2, 3, 4} at compile-time

```

30.21.3 constexpr Algorithms

```

1 #include <algorithm>
2
3 // C++23: Algorithms work in constexpr
4 constexpr int compute() {
5     std::vector<int> v{3, 1, 4, 1, 5, 9};
6     std::sort(v.begin(), v.end());
7     int sum = 0;
8     for (int x : v) sum += x;
9     return sum;
10 }
11
12 constexpr int result = compute(); // Computed at compile-time

```

30.22 ADAPTOR IMPROVEMENTS

30.23 9.1 Ranges::to Conversion

`std::ranges::to` converts ranges to containers.

30.23.1 Basic ranges::to

```

1 #include <ranges>
2 #include <vector>
3 #include <string>
4
5 vector<int> v = {1, 2, 3, 4, 5};
6
7 // Convert filtered range to vector
8 auto evens = v
9     | std::ranges::views::filter([](int x) { return x % 2 == 0; })
10    | std::ranges::to<std::vector>();
11
12 // Or with map
13 map<int, int> m{{1, 10}, {2, 20}, {3, 30}};
14 auto keys = m
15     | std::ranges::views::keys
16    | std::ranges::to<std::vector>();

```

30.23.2 ranges::to with Construction Args

```

1 #include <ranges>
2 #include <set>
3
4 vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};
5
6 // Convert to sorted unique set

```

```

7 auto unique_sorted = v
8   | std::ranges::to<std::set>();
9
10 // Or to deque
11 auto d = v | std::ranges::to<std::deque>();
12
13 // With custom construction
14 auto s = v
15   | std::ranges::to<std::set>(std::greater{}); // Descending

```

30.24 LIBRARY IMPROVEMENTS

30.25 10.1 Utility Improvements

30.25.1 std::out_ptr & std::inout_ptr

```

1 #include <memory>
2
3 // For converting unique_ptr to C API
4 unique_ptr<int> ptr;
5
6 // Before C++23: Complicated
7 // int* tmp = ptr.release();
8 // legacy_function(&tmp);
9 // ptr.reset(tmp);
10
11 // C++23: Simple
12 legacy_c_function(std::out_ptr(ptr));
13
14 // For bidirectional
15 inout_ptr(ptr);

```

30.25.2 std::move_iterator Improvements

```

1 #include <iterator>
2 #include <algorithm>
3
4 vector<string> v = {"a", "b", "c"};
5
6 // C++23: Cleaner move semantics
7 auto result = v
8   | std::views::transform([](auto& s) { return std::move(s); });

```

30.25.3 Bit Manipulation Improvements

```

1 #include <bit>
2
3 unsigned int x = 5; // 0b0101
4
5 // C++23 additions
6 std::byteswap(x);           // Byte swap
7 std::has_single_bit(x);     // Check if power of 2
8 std::bit_width(x);          // Bits needed
9 std::popcount(x);           // Count 1 bits

```

30.26 ATTRIBUTES & FEATURES

30.27 11.1 `[[assume]]` Attribute

`[[assume]]` allows providing hints to optimizer.

30.27.1 Using `assume`

```
1 void process(int x) {
2     // Tell compiler to assume x > 0 (for optimization)
3     [[assume(x > 0)]];
4
5     // Compiler can optimize based on this
6     if (x < 0) {
7         // This branch won't be taken
8         std::cout << "Negative\n";
9     }
10 }
11
12 // Useful for performance-critical code
13 int* find_first(int* arr, int size) {
14     [[assume(size > 0)]]; // Array has elements
15
16     for (int i = 0; i < size; i++) {
17         if (arr[i] == target) return &arr[i];
18     }
19     return nullptr;
20 }
```

30.28 11.2 `[[stdcall]]` and ABI Attributes

```
1 // Platform-specific calling conventions
2 #ifdef _WIN32
3 void __stdcall legacy_function() { }
4 [[gnu::stdcall]] void c_function();
5 #endif
```

30.29 STANDARD LIBRARY ADDITIONS

30.30 12.1 Container & Utility Additions

30.30.1 `std::debug_assert` (Conditional Assertion)

```
1 void function(int x) {
2     // Debug assertion (disabled in release)
3     _ASSERT(x > 0, "x must be positive");
4
5     // Work with x
6 }
```

30.30.2 std::repeat_view

```

1 #include <ranges>
2
3 // Repeat a value
4 auto repeated = std::views::repeat(42, 5);
5 for (int x : repeated) {
6     cout << x << " "; // 42 42 42 42 42
7 }

```

30.30.3 std::stride_view

```

1 #include <ranges>
2
3 vector<int> v = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
4
5 // Take every Nth element
6 auto every_other = v | std::views::stride(2);
7 for (int x : every_other) {
8     cout << x << " "; // 0 2 4 6 8
9 }

```

30.30.4 std::chunk_by

```

1 #include <ranges>
2
3 vector<int> v = {1, 2, 2, 3, 3, 3, 4, 4};
4
5 // Group by predicate
6 auto chunks = v | std::views::chunk_by(std::equal_to{});
7 for (auto chunk : chunks) {
8     cout << "[";
9     for (int x : chunk) cout << x << " ";
10    cout << "]" ";
11 }
12 // Output: [1 ] [2 2 ] [3 3 3 ] [4 4 ]

```

30.30.5 std::flat_map and std::flat_set

`std::flat_map` is a container adaptor that stores elements in sorted order in contiguous memory (like a vector).

```

1 #include <flat_map>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 int main() {
7     // Stores keys and values in separate vectors
8     // Cache-friendly, fast iteration, binary search
9     std::flat_map<int, std::string> map;
10
11     map[1] = "one";
12     map[3] = "three";
13     map[2] = "two"; // Inserted in correct sorted position
14
15     for (const auto& [key, val] : map) {
16         std::cout << key << ": " << val << "\n";
17     }
18 }

```

```

17     }
18     // Output: 1: one, 2: two, 3: three
19
20     return 0;
21 }

```

30.30.6 std::generator (Synchronous Coroutine)

`std::generator` is the standard coroutine generator for synchronous sequences.

```

1  #include <generator>
2  #include <iostream>
3  #include <ranges>
4
5  std::generator<int> fib(int n) {
6      int a = 0, b = 1;
7      while (n-- > 0) {
8          co_yield a;
9          auto next = a + b;
10         a = b;
11         b = next;
12     }
13 }
14
15 int main() {
16     for (int x : fib(10)) {
17         std::cout << x << " ";
18     }
19     // Output: 0 1 1 2 3 5 8 13 21 34
20
21     // Composable with ranges
22     auto gen = fib(100) | std::views::filter([](int x) { return x % 2 == 0; });
23
24     return 0;
25 }

```

30.30.7 12.3 Generator Internals

`std::generator` is a coroutine that:

1. **Suspends on yield:** `co_yield` suspends execution and returns value to caller.
2. **Promise Type:** Handles the `yield_value` call.
3. **Recursive:** Can `co_yield` another generator (unlike basic coroutines).

```

1  // Pseudocode of how it works
2  struct promise_type {
3      int current_value;
4      std::suspend_always yield_value(int value) {
5          current_value = value;
6          return {}; // Suspend
7      }
8  };

```

30.31 C++23 BEST PRACTICES

30.32 What's Better with C++23

```
1 // 1. Use std::print for simple output
2 std::println("Value: {}", value);
3
4 // 2. Use std::expected for error handling
5 expected<int, Error> result = operation();
6
7 // 3. Use deducing this for simpler overloads
8 auto get(this auto& self) { return self.value; }
9
10 // 4. Use range-based for with init
11 for (auto data = load_data(); auto item : data) { }
12
13 // 5. Use multi-index subscript
14 matrix[row, col] = value;
15
16 // 6. Use std::stacktrace for debugging
17 auto trace = std::stacktrace::current();
18
19 // 7. Use constexpr string/vector
20 constexpr auto msg = std::string("hello");
21
22 // 8. Use std::ranges::to for conversions
23 auto set_result = v | std::ranges::to<std::set>();
24
25 // 9. Use [[assume]] for optimization hints
26 [[assume(pointer != nullptr)]];
27
28 // 10. Use monadic operations on optional
29 opt.transform([](int x) { return x * 2; });
```

Chapter 31

THE FUTURE - C++26 PREVIEW

As of 2026, the C++26 standard is nearing finalization. Here are the transformative features likely to be included.

31.0.1 13.1 Static Reflection (std::meta)

Reflection allows a program to inspect and modify itself at compile-time. This eliminates the need for external code generators or macros for serialization, ORMs, and enum-to-string conversions.

```
1 #include <meta>
2 #include <iostream>
3 #include <string_view>
4
5 struct Person {
6     std::string name;
7     int age;
8     double salary;
9 };
10
11 // Generic serialization using C++26 Reflection
12 template<typename T>
13 void serialize(const T& obj) {
14     constexpr auto type_info = ^T; // Reflection operator
15
16     template for (constexpr auto member : std::meta::members_of(type_info)) {
17         std::cout << std::meta::name_of(member) << ": "
18             << obj.[member:] << "\n"; // Splicing
19     }
20 }
21
22 int main() {
23     Person p{"Alice", 30, 95000.0};
24     serialize(p);
25     // Output:
26     // name: Alice
27     // age: 30
28     // salary: 95000
29 }
```

31.0.2 13.2 Contracts

Contracts provide a standardized way to specify preconditions, postconditions, and assertions, improving safety and optimizer information.

```
1 // pre: Precondition (Caller must ensure)
```

```

2 // post: Postcondition (Function ensures upon return)
3 // assert: Internal check
4
5 int safe_divide(int a, int b)
6     pre { b != 0 }           // Contract: b must not be zero
7     post(r) { r * b == a }   // Contract: result * divisor equals dividend
8 {
9     return a / b;
10 }
11
12 // Modes:
13 // - enforce: Terminate if violated
14 // - observe: Log/Debug but continue
15 // - ignore: Optimizer hint (assume true)

```

31.0.3 13.3 Senders & Receivers (std::execution)

A unified framework for asynchronous execution, replacing raw threads, futures, and callbacks with a composable pipeline model.

```

1 #include <execution>
2 #include <iostream>
3
4 using namespace std::execution;
5
6 int main() {
7     scheduler auto sch = thread_pool_scheduler{};
8
9     sender auto work = schedule(sch)
10         | then([]{ return 42; })
11         | then([](int i){ return i * 2; })
12         | then([](int i){ std::cout << "Result: " << i << "\n"; });
13
14     // Launch execution
15     std::this_thread::sync_wait(std::move(work));
16
17     return 0;
18 }

```

31.0.4 13.4 Linear Algebra (std::linalg)

Standardized BLAS (Basic Linear Algebra Subprograms) support for high-performance math.

```

1 #include <linalg>
2 #include <mdspan>
3 #include <vector>
4
5 int main() {
6     std::vector<double> A_vec(9), B_vec(3), C_vec(3);
7     // ... fill vectors ...
8
9     std::mdspan A(A_vec.data(), 3, 3);
10    std::mdspan B(B_vec.data(), 3);
11    std::mdspan C(C_vec.data(), 3);
12
13    // Matrix-Vector Multiplication: C = A * B
14    std::linalg::matrix_vector_product(A, B, C);
15
16    return 0;
17 }

```


Part VII

Volume VII: Advanced Topics & Systems Architecture

Chapter 32

ADVANCED TOPICS

32.1 TEMPLATE METAPROGRAMMING

32.2 1.1 Compile-Time Computation

Template metaprogramming enables computation at compile-time.

32.2.1 Factorial at Compile-Time

```
1 #include <iostream>
2 using namespace std;
3
4 // Base case
5 template<int N>
6 struct Factorial {
7     static constexpr int value = N * Factorial<N-1>::value;
8 };
9
10 // Specialization (base case)
11 template<>
12 struct Factorial<0> {
13     static constexpr int value = 1;
14 };
15
16 int arr[Factorial<5>::value]; // Array of size 120!
17
18 cout << Factorial<10>::value << "\n"; // 3628800
```

32.2.2 Fibonacci at Compile-Time

```
1 template<int N>
2 struct Fib {
3     static constexpr int value = Fib<N-1>::value + Fib<N-2>::value;
4 };
5
6 template<>
7 struct Fib<0> {
8     static constexpr int value = 0;
9 };
10
11 template<>
12 struct Fib<1> {
13     static constexpr int value = 1;
14 };
```

```

15
16 // Memoization to avoid exponential time
17 template<int N>
18 struct FibMemo {
19     static constexpr int value = FibMemo<N-1>::value + FibMemo<N-2>::value;
20 };
21
22 static constexpr int fib20 = FibMemo<20>::value; // Computed at compile-time

```

32.2.3 Compile-Time Power Check

```

1 template<unsigned int N>
2 struct IsPowerOfTwo {
3     static constexpr bool value = (N > 0) && ((N & (N - 1)) == 0);
4 };
5
6 static_assert(IsPowerOfTwo<16>::value);
7 static_assert(!IsPowerOfTwo<7>::value);

```

32.3 1.2 Template Specialization

32.3.1 Full Specialization

```

1 // Primary template
2 template<typename T, typename U>
3 struct Pair {
4     static void info() { cout << "Generic pair\n"; }
5 };
6
7 // Full specialization
8 template<>
9 struct Pair<int, string> {
10     static void info() { cout << "int-string pair\n"; }
11 };
12
13 // Full specialization for pointers
14 template<typename T>
15 struct Pair<T*, T*> {
16     static void info() { cout << "Two pointers\n"; }
17 };
18
19 Pair<int, string>::info(); // "int-string pair"
20 Pair<double, string>::info(); // "Generic pair"
21 Pair<int*, int*>::info(); // "Two pointers"

```

32.3.2 Partial Specialization

```

1 // Primary
2 template<typename T, typename U>
3 struct Container {
4     static void type() { cout << "Generic\n"; }
5 };
6
7 // Partial specialization - same types
8 template<typename T>

```

```

9 struct Container<T, T> {
10     static void type() { cout << "Same type\n"; }
11 };
12
13 // Partial specialization - pointers
14 template<typename T, typename U>
15 struct Container<T*, U*> {
16     static void type() { cout << "Two pointers\n"; }
17 };
18
19 // Partial specialization - array
20 template<typename T, int N>
21 struct Container<T[N], T> {
22     static void type() { cout << "Array and element\n"; }
23 };
24
25 Container<int, int>::type();           // "Same type"
26 Container<int*, double*>::type();     // "Two pointers"
27 Container<int[5], int>::type();       // "Array and element"

```

32.4 1.3 Advanced Metaprogramming Patterns

32.4.1 Typelists

A list of types at compile-time, essential for ECS and Variant implementation.

```

1 template<typename... Ts>
2 struct TypeList {};
3
4 // Length of list
5 template<typename List> struct Length;
6
7 template<typename... Ts>
8 struct Length<TypeList<Ts...>> {
9     static constexpr size_t value = sizeof...(Ts);
10 };
11
12 // Access type at index
13 template<size_t N, typename List> struct At;
14
15 template<typename Head, typename... Tail>
16 struct At<0, TypeList<Head, Tail...>> {
17     using type = Head;
18 };
19
20 template<size_t N, typename Head, typename... Tail>
21 struct At<N, TypeList<Head, Tail...>> {
22     using type = typename At<N-1, TypeList<Tail...>>::type;
23 };
24
25 // Usage
26 using MyTypes = TypeList<int, float, double>;
27 static_assert(Length<MyTypes>::value == 3);
28 static_assert(std::is_same_v<At<1, MyTypes>::type, float>);

```

32.5 SFINAE & TYPE TRAITS

32.6 2.1 SFINAE - Substitution Failure Is Not An Error

SFINAE enables overload resolution based on template parameter substitution.

32.6.1 Basic SFINAE

```
1 #include <type_traits>
2
3 // Enable if T is integral
4 template<typename T>
5 enable_if_t<is_integral_v<T>>
6 process(T x) {
7     cout << "Integer: " << x << "\n";
8 }
9
10 // Enable if T is floating point
11 template<typename T>
12 enable_if_t<is_floating_point_v<T>>
13 process(T x) {
14     cout << "Float: " << x << "\n";
15 }
16
17 process(42);           // "Integer: 42"
18 process(3.14);        // "Float: 3.14"
```

32.6.2 Detector Pattern

```
1 // Detect if type has value_type
2 template<typename T, typename = void>
3 struct HasValueType : false_type {};
4
5 template<typename T>
6 struct HasValueType<T, void_t<typename T::value_type>> : true_type {};
7
8 // Usage
9 static_assert(HasValueType<vector<int>>::value);
10 static_assert(!HasValueType<int>::value);
```

32.6.3 Advanced SFINAE with Multiple Conditions

```
1 template<typename T>
2 enable_if_t<
3     is_copy_constructible_v<T> &&
4     is_move_constructible_v<T> &&
5     is_equality_comparable_v<T>
6 >
7 smart_copy(const T& src, T& dst) {
8     dst = src;
9 }
```


32.7 2.2 Type Traits Mastery

32.7.1 Custom Type Traits

```

1 // Check if type has a begin() and end()
2 template<typename T, typename = void>
3 struct IsContainer : false_type {};
4
5 template<typename T>
6 struct IsContainer<T, void_t<
7     decltype(declval<T>().begin()),
8     decltype(declval<T>().end())
9 >> : true_type {};
10
11 // Check if callable with specific signature
12 template<typename F, typename... Args>
13 struct IsCallable : false_type {};
14
15 template<typename F, typename... Args>
16 struct IsCallable<F,
17     void_t<decltype(declval<F>()(declval<Args>()...))>
18 > : true_type {};
19
20 // Usage
21 static_assert(IsContainer<vector<int>>::value);
22 static_assert(!IsContainer<int>::value);
23
24 auto lambda = [](int x) { return x; };
25 static_assert(IsCallable<decltype(lambda), int>::value);

```

32.8 EXPRESSION TEMPLATES

32.9 3.1 Lazy Evaluation Pattern

Expression templates defer computation until needed.

32.9.1 Vector Operations

```

1 #include <vector>
2 #include <iostream>
3
4 // Expression template for vector operations
5 template<typename Expr>
6 class VectorExpr {
7 public:
8     double operator[](int i) const {
9         return static_cast<const Expr&>(*this)[i];
10    }
11
12    int size() const {
13        return static_cast<const Expr&>(*this).size();
14    }
15 };
16
17 class Vector : public VectorExpr<Vector> {
18 private:
19     std::vector<double> data;

```

```

20
21 public:
22     Vector(int n) : data(n) {}
23
24     double operator[](int i) const { return data[i]; }
25     int size() const { return data.size(); }
26     double& operator[](int i) { return data[i]; }
27 };
28
29 // Addition expression (no computation yet)
30 template<typename L, typename R>
31 class VectorSum : public VectorExpr<VectorSum<L, R>> {
32 private:
33     const L& lhs;
34     const R& rhs;
35
36 public:
37     VectorSum(const L& l, const R& r) : lhs(l), rhs(r) {}
38
39     double operator[](int i) const {
40         return lhs[i] + rhs[i];
41     }
42
43     int size() const { return lhs.size(); }
44 };
45
46 // Operator overloading creates expression tree
47 template<typename L, typename R>
48 VectorSum<L, R> operator+(const VectorExpr<L>& l, const VectorExpr<R>& r) {
49     return VectorSum<L, R>(static_cast<const L>(l), static_cast<const R>(r));
50 }
51
52 // Scalar multiplication
53 template<typename Expr>
54 class VectorScale : public VectorExpr<VectorScale<Expr>> {
55 private:
56     const Expr& expr;
57     double scale;
58
59 public:
60     VectorScale(const Expr& e, double s) : expr(e), scale(s) {}
61
62     double operator[](int i) const {
63         return expr[i] * scale;
64     }
65
66     int size() const { return expr.size(); }
67 };
68
69 template<typename Expr>
70 VectorScale<Expr> operator*(double s, const VectorExpr<Expr>& e) {
71     return VectorScale<Expr>(static_cast<const Expr>(e), s);
72 }
73
74 // Usage
75 int main() {
76     Vector v1(3), v2(3), v3(3);
77     v1[0] = 1; v1[1] = 2; v1[2] = 3;
78     v2[0] = 4; v2[1] = 5; v2[2] = 6;
79
80     // No computation yet!
81     auto expr = v1 + v2 + 2.0 * v3;
82

```

```

83 // Computation happens here
84 for (int i = 0; i < 3; i++) {
85     cout << expr[i] << " ";
86 }
87
88 return 0;
89 }

```

32.10 3.2 Triggering Computation (Assignment)

To make `v3 = v1 + v2` work efficiently, we add an assignment operator to `Vector` that takes any `VectorExpr`.

```

1 // Inside class Vector
2 template <typename Expr>
3 Vector& operator=(const VectorExpr<Expr>& expr) {
4     if (size() != expr.size()) {
5         // resize...
6     }
7     for (int i = 0; i < size(); ++i) {
8         data[i] = expr[i]; // Evaluates tree at index i
9     }
10    return *this;
11 }

```

Why is this fast? It expands to: `v3[i] = v1[i] + v2[i]`. There are **zero temporary vectors** created. No allocations. Just one loop.

32.11 POLICY-BASED DESIGN

32.12 4.1 Template Policies

Policy-based design separates concerns using template parameters.

32.12.1 Thread Safety Policy

```

1 #include <mutex>
2 #include <memory>
3
4 // Policy: No synchronization
5 struct NoSync {
6     void lock() {}
7     void unlock() {}
8 };
9
10 // Policy: Mutex-based
11 struct MutexSync {
12 private:
13     mutable std::mutex m;
14
15 public:
16     void lock() { m.lock(); }
17     void unlock() { m.unlock(); }
18 };
19
20 // Generic container with synchronization policy

```

```

21 template<typename T, typename SyncPolicy = NoSync>
22 class SafeVector : private SyncPolicy {
23 private:
24     std::vector<T> data;
25
26 public:
27     void push_back(const T& value) {
28         this->lock();
29         data.push_back(value);
30         this->unlock();
31     }
32
33     T pop_back() {
34         this->lock();
35         T value = data.back();
36         data.pop_back();
37         this->unlock();
38         return value;
39     }
40 };
41
42 // Usage
43 SafeVector<int> single_threaded; // No synchronization
44 SafeVector<int, MutexSync> multi_threaded; // Thread-safe

```

32.12.2 Storage Policy

```

1 // Policy: Dynamic allocation
2 struct DynamicStorage {
3     template<typename T>
4     using Allocator = std::allocator<T>;
5 };
6
7 // Policy: Static allocation
8 template<int MaxSize>
9 struct StaticStorage {
10     // Allocate from stack
11 };
12
13 template<typename T, typename StoragePolicy>
14 class Container : private StoragePolicy {
15 private:
16     std::vector<T> data;
17 };

```

32.13 4.2 Policy-Based Smart Pointer (Advanced Example)

A smart pointer is defined by: 1. **Storage**: How it stores the pointer (raw vs compressed). 2. **Ownership**: Ref-counted vs Unique vs Linked. 3. **Checking**: Assert on access vs No check.

```

1 template <
2     class T,
3     template <class> class CheckingPolicy,
4     template <class> class ThreadingPolicy
5 >
6 class SmartPtr : public CheckingPolicy<T>, public ThreadingPolicy<T> {
7     T* ptr;
8 public:
9     T* operator->() {
10         CheckingPolicy<T>::check(ptr);

```

```

11     ThreadingPolicy<T>::lock(); // Fake lock example
12     return ptr;
13 }
14 };

```

This approach allows generating thousands of smart pointer variants with zero runtime overhead (all resolved at compile-time).

32.14 MEMORY MANAGEMENT & OPTIMIZATION

32.15 5.1 Custom Allocators

```

1 #include <memory>
2
3 template<typename T>
4 class ArenaAllocator {
5 private:
6     static constexpr size_t ARENA_SIZE = 10000;
7     char arena[ARENA_SIZE];
8     char* current;
9
10 public:
11     ArenaAllocator() : current(arena) {}
12
13     T* allocate(size_t n) {
14         if (current + n * sizeof(T) > arena + ARENA_SIZE) {
15             throw std::bad_alloc();
16         }
17         T* ptr = reinterpret_cast<T*>(current);
18         current += n * sizeof(T);
19         return ptr;
20     }
21
22     void deallocate(T* p, size_t n) {
23         // No deallocation for arena allocator
24     }
25 };
26
27 // Usage
28 vector<int, ArenaAllocator<int>> fast_vector;

```

32.16 5.2 Small Object Optimization (SOO)

```

1 template<typename T, size_t SmallSize = 32>
2 class SmallVector {
3 private:
4     union {
5         T* ptr; // Dynamic allocation
6         std::aligned_storage_t<SmallSize, alignof(T)> small;
7     };
8
9     size_t size_val;
10    bool is_small;
11
12 public:
13    SmallVector() : size_val(0), is_small(true) {}

```

```

14
15 void push_back(const T& value) {
16     if (size_val < SmallSize / sizeof(T)) {
17         // Use small storage
18         new (&small) T(value);
19         is_small = true;
20     } else {
21         // Switch to dynamic
22         if (is_small) {
23             // Copy small to dynamic
24             ptr = new T[size_val + 1];
25             is_small = false;
26         }
27         ptr[size_val] = value;
28     }
29     size_val++;
30 }
31
32 ~SmallVector() {
33     if (!is_small) {
34         delete[] ptr;
35     }
36 }
37 };

```

32.17 CONCURRENCY & PARALLELISM

32.18 6.1 Advanced Threading Patterns

32.18.1 Thread Pool

```

1 #include <thread>
2 #include <queue>
3 #include <mutex>
4 #include <condition_variable>
5 #include <functional>
6
7 class ThreadPool {
8 private:
9     vector<std::thread> workers;
10    queue<std::function<void()>> tasks;
11    mutex task_mutex;
12    condition_variable cv;
13    bool stop = false;
14
15 public:
16    ThreadPool(size_t num_threads) {
17        for (size_t i = 0; i < num_threads; i++) {
18            workers.emplace_back([this] {
19                while (true) {
20                    std::function<void()> task;
21
22                    {
23                        unique_lock<mutex> lock(task_mutex);
24                        cv.wait(lock, [this] { return !tasks.empty() || stop;
25
26                        if (stop && tasks.empty()) return;

```

```

27         task = std::move(tasks.front());
28         tasks.pop();
29     }
30
31     task();
32 }
33
34 });
35 }
36 }
37
38 template<typename F, typename... Args>
39 auto enqueue(F&& f, Args&&... args) {
40     using return_type = std::invoke_result_t<F, Args...>;
41
42     auto task = make_shared<std::packaged_task<return_type()>>(
43         bind(forward<F>(f), forward<Args>(args)...)
44     );
45
46     auto result = task->get_future();
47
48     {
49         unique_lock<mutex> lock(task_mutex);
50         tasks.emplace([task] { (*task)(); });
51     }
52
53     cv.notify_one();
54     return result;
55 }
56
57 ~ThreadPool() {
58     {
59         unique_lock<mutex> lock(task_mutex);
60         stop = true;
61     }
62     cv.notify_all();
63     for (auto& worker : workers) {
64         worker.join();
65     }
66 }
67 };
68
69 // Usage
70 ThreadPool pool(4);
71
72 auto future = pool.enqueue([](int x) { return x * 2; }, 42);
73 cout << future.get() << "\n"; // 84

```

32.18.2 Lock-Free Queue

```

1 #include <atomic>
2
3 template<typename T>
4 class LockFreeQueue {
5 private:
6     struct Node {
7         T value;
8         atomic<Node*> next{nullptr};
9     };
10
11     atomic<Node*> head;
12     atomic<Node*> tail;

```

```

13
14 public:
15     LockFreeQueue() {
16         auto dummy = new Node();
17         head.store(dummy, memory_order_relaxed);
18         tail.store(dummy, memory_order_relaxed);
19     }
20
21     void push(const T& value) {
22         auto new_node = new Node{value};
23         Node* old_tail = tail.load(memory_order_acquire);
24
25         old_tail->next.store(new_node, memory_order_release);
26         tail.store(new_node, memory_order_release);
27     }
28
29     bool pop(T& value) {
30         Node* old_head = head.load(memory_order_acquire);
31         Node* next = old_head->next.load(memory_order_acquire);
32
33         if (next == nullptr) return false;
34
35         value = next->value;
36         head.store(next, memory_order_release);
37         delete old_head;
38
39         return true;
40     }
41 };

```

32.19 TYPE ERASURE PATTERNS

32.20 7.1 Virtual Function-Based Type Erasure

```

1 class AnyCallable {
2 private:
3     struct Base {
4         virtual ~Base() = default;
5         virtual void call() = 0;
6     };
7
8     template<typename F>
9     struct Derived : Base {
10         F func;
11         Derived(F f) : func(f) {}
12         void call() override { func(); }
13     };
14
15     unique_ptr<Base> impl;
16
17 public:
18     template<typename F>
19     AnyCallable(F f) : impl(make_unique<Derived<F>>(f)) {}
20
21     void operator()() {
22         impl->call();
23     }
24 };

```



```

25
26 // Usage
27 AnyCallable c1([](){ cout << "Lambda\n"; });
28 AnyCallable c2(std::bind(...));
29 AnyCallable c3(&function);
30
31 c1(); // "Lambda"

```

32.21 7.2 std::function Implementation

```

1 #include <memory>
2
3 template<typename>
4 class Function;
5
6 template<typename R, typename... Args>
7 class Function<R(Args...)> {
8 private:
9     struct Base {
10         virtual ~Base() = default;
11         virtual R call(Args...) = 0;
12         virtual unique_ptr<Base> clone() = 0;
13     };
14
15     template<typename F>
16     struct Derived : Base {
17         F func;
18         Derived(F f) : func(f) {}
19         R call(Args... args) override {
20             return func(args...);
21         }
22         unique_ptr<Base> clone() override {
23             return make_unique<Derived>(*this);
24         }
25     };
26
27     unique_ptr<Base> impl;
28
29 public:
30     template<typename F>
31     Function(F f) : impl(make_unique<Derived<F>>(f)) {}
32
33     R operator()(Args... args) {
34         return impl->call(args...);
35     }
36
37     Function(const Function& other) : impl(other.impl->clone()) {}
38 };

```

32.22 7.3 Concept-Based Type Erasure (C++20)

Instead of inheritance, we can erase types that satisfy a concept.

```

1 #include <concepts>
2 #include <memory>
3
4 template<typename T>
5 concept Drawable = requires(T x) { x.draw(); };
6

```

```

7  class AnyDrawable {
8      struct Concept {
9          virtual ~Concept() = default;
10         virtual void draw() = 0;
11         virtual std::unique_ptr<Concept> clone() = 0;
12     };
13
14     template<Drawable T>
15     struct Model : Concept {
16         T data;
17         Model(T x) : data(std::move(x)) {}
18         void draw() override { data.draw(); }
19         std::unique_ptr<Concept> clone() override { return std::make_unique<
Model>(data); }
20     };
21
22     std::unique_ptr<Concept> pimpl;
23
24 public:
25     template<Drawable T>
26     AnyDrawable(T x) : pimpl(std::make_unique<Model<T>>(std::move(x))) {}
27
28     AnyDrawable(const AnyDrawable& other) : pimpl(other.pimpl->clone()) {}
29
30     void draw() { pimpl->draw(); }
31 };

```

32.23 CRTP (CURIOSLY RECURRING TEMPLATE PATTERN)

32.24 8.1 Static Polymorphism

```

1  // Base class template
2  template<typename Derived>
3  class Shape {
4  public:
5      void draw() {
6          static_cast<Derived*>(this)->draw_impl();
7      }
8
9      double area() {
10         return static_cast<Derived*>(this)->area_impl();
11     }
12 };
13
14 // Derived classes
15 class Circle : public Shape<Circle> {
16 private:
17     double radius;
18
19 public:
20     Circle(double r) : radius(r) {}
21
22     void draw_impl() {
23         cout << "Drawing circle\n";
24     }
25

```

```

26     double area_impl() {
27         return 3.14159 * radius * radius;
28     }
29 };
30
31 class Square : public Shape<Square> {
32 private:
33     double side;
34
35 public:
36     Square(double s) : side(s) {}
37
38     void draw_impl() {
39         cout << "Drawing square\n";
40     }
41
42     double area_impl() {
43         return side * side;
44     }
45 };
46
47 // Usage - no virtual function overhead!
48 Circle c(5);
49 c.draw();
50 cout << c.area() << "\n";
51
52 Square s(4);
53 s.draw();
54 cout << s.area() << "\n";

```

32.25 8.2 CRTP for Comparisons

```

1  template<typename Derived>
2  class Comparable {
3  public:
4      bool operator<(const Comparable& other) const {
5          return static_cast<const Derived*>(this)->compare(
6              static_cast<const Derived&>(other)
7              ) < 0;
8      }
9
10     bool operator>(const Comparable& other) const {
11         return other < *this;
12     }
13
14     bool operator<=(const Comparable& other) const {
15         return !(other < *this);
16     }
17
18     bool operator>=(const Comparable& other) const {
19         return !(*this < other);
20     }
21
22     bool operator==(const Comparable& other) const {
23         return !(*this < other) && !(other < *this);
24     }
25
26     bool operator!=(const Comparable& other) const {
27         return !(*this == other);
28     }
29 };

```

```

30
31 class Value : public Comparable<Value> {
32 private:
33     int val;
34
35 public:
36     Value(int v) : val(v) {}
37
38     int compare(const Value& other) const {
39         return val - other.val;
40     }
41 };
42
43 // Usage
44 Value v1(5), v2(10);
45 cout << (v1 < v2) << "\n";    // true
46 cout << (v1 == v2) << "\n";   // false

```

32.26 PERFECT FORWARDING & MOVE SEMANTICS

32.27 9.1 Forwarding Problems

```

1 // The problem: losing information about lvalue/rvalue
2 template<typename T>
3 void bad_forward(T arg) {
4     // arg is always lvalue
5     sink(arg); // Loses rvalue status
6 }
7
8 // Solution: universal references + std::forward
9 template<typename T>
10 void good_forward(T&& arg) {
11     // Preserves lvalue/rvalue nature
12     sink(std::forward<T>(arg));
13 }
14
15 // Double forwarding
16 template<typename T>
17 void wrapper(T&& arg) {
18     process(std::forward<T>(arg));
19 }
20
21 template<typename T>
22 void double_wrapper(T&& arg) {
23     wrapper(std::forward<T>(arg));
24 }

```

32.28 9.2 Move Semantics Implementation

```

1 class String {
2 private:
3     char* data;
4     size_t size;
5
6 public:

```

```

7 // Move constructor
8 String(String&& other) noexcept
9     : data(other.data), size(other.size) {
10     other.data = nullptr;
11     other.size = 0;
12 }
13
14 // Move assignment
15 String& operator=(String&& other) noexcept {
16     if (this != &other) {
17         delete[] data;
18         data = other.data;
19         size = other.size;
20         other.data = nullptr;
21         other.size = 0;
22     }
23     return *this;
24 }
25
26 // Copy constructor (for lvalues)
27 String(const String& other)
28     : size(other.size), data(new char[size + 1]) {
29     strcpy(data, other.data);
30 }
31
32 // Copy assignment (for lvalues)
33 String& operator=(const String& other) {
34     if (this != &other) {
35         delete[] data;
36         size = other.size;
37         data = new char[size + 1];
38         strcpy(data, other.data);
39     }
40     return *this;
41 }
42 };

```

32.29 COMPILE-TIME PROGRAMMING

32.30 10.1 Tuple Operations at Compile-Time

```

1 #include <tuple>
2 #include <iostream>
3
4 // Compile-time tuple iteration
5 namespace detail {
6     template<typename F, typename Tuple, size_t... Is>
7     void for_each_impl(F&& f, Tuple&& t, index_sequence<Is...>) {
8         (... , f(get<Is>(forward<Tuple>(t))));
9     }
10 }
11
12 template<typename F, typename Tuple>
13 void for_each(F&& f, Tuple&& t) {
14     constexpr auto size = tuple_size_v<decay_t<Tuple>>;
15     detail::for_each_impl(
16         forward<F>(f),
17         forward<Tuple>(t),

```

```

18     make_index_sequence<size>{}
19     );
20 }
21
22 // Usage
23 auto t = make_tuple(1, 2.5, "hello");
24 for_each([](auto x) { cout << x << " "; }, t); // 1 2.5 hello

```

32.31 10.2 Compile-Time String Processing

```

1 #include <array>
2 #include <string_view>
3
4 // Compile-time string hashing
5 constexpr size_t hash_compile_time(string_view s) {
6     size_t h = 0;
7     for (char c : s) {
8         h = h * 31 + c;
9     }
10    return h;
11 }
12
13 // Compile-time string search
14 constexpr bool contains_compile_time(string_view s, string_view sub) {
15     return s.find(sub) != string_view::npos;
16 }
17
18 // Usage in compile-time context
19 static_assert(contains_compile_time("hello world", "world"));
20 constexpr auto hash = hash_compile_time("key");

```

32.32 META-OBJECT PROTOCOL

32.33 11.1 Reflection-Like Patterns

```

1 // Manual reflection without std::meta
2 struct Member {
3     string_view name;
4     string_view type;
5     size_t offset;
6 };
7
8 template<typename T>
9 constexpr auto get_members() {
10     // Specialize for each type
11     return array<Member, 0>{};
12 }
13
14 template<>
15 constexpr auto get_members<Person>() {
16     return array<Member, 3>{{
17         {"name", "string", offsetof(Person, name)},
18         {"age", "int", offsetof(Person, age)},
19         {"email", "string", offsetof(Person, email)}
20     }};

```

```

21 }
22
23 // Serialize any type
24 template<typename T>
25 string serialize(const T& obj) {
26     string result;
27     for (const auto& member : get_members<T>()) {
28         result += member.name;
29         result += ": ";
30         // Serialize value...
31     }
32     return result;
33 }

```

32.34 11.2 Magic Enum (Reflection Hack)

Before C++26 Reflection, we use compiler intrinsics to get enum names.

```

1  template <auto V>
2  constexpr std::string_view get_enum_name() {
3      // Compiler-specific macros
4      #ifdef __clang__
5          return __PRETTY_FUNCTION__;
6      #elif defined(__GNUC__)
7          return __PRETTY_FUNCTION__;
8      #elif defined(_MSC_VER)
9          return __FUNCSIG__;
10     #endif
11 }
12
13 enum class Color { Red, Green, Blue };
14
15 int main() {
16     // Output contains "Color::Red" buried in the string
17     std::cout << get_enum_name<Color::Red>() << "\n";
18 }

```

Note: Libraries like `magic_enum` automate parsing this string.

32.35 ADVANCED CONTAINER TECHNIQUES

32.36 12.1 COW (Copy-On-Write) String

```

1  class CowString {
2  private:
3      struct Buffer : public enable_shared_from_this<Buffer> {
4          string data;
5
6          Buffer(const string& s) : data(s) {}
7      };
8
9      shared_ptr<Buffer> buffer;
10
11     void ensure_unique() {
12         if (!buffer.unique()) {
13             buffer = make_shared<Buffer>(*buffer);
14         }
15     }
16 }

```

```

15     }
16
17 public:
18     CowString(const string& s = "")
19         : buffer(make_shared<Buffer>(s)) {}
20
21     const char* data() const {
22         return buffer->data.c_str();
23     }
24
25     char& operator[](size_t i) {
26         ensure_unique();
27         return buffer->data[i];
28     }
29 };

```

32.37 12.2 Intrusive Containers

```

1  #include <list>
2
3  // Node that contains its own link
4  class IntrusiveNode {
5  public:
6      IntrusiveNode* next = nullptr;
7      IntrusiveNode* prev = nullptr;
8  };
9
10 template<typename T>
11 class IntrusiveList {
12 private:
13     IntrusiveNode* head;
14     IntrusiveNode* tail;
15
16 public:
17     void push_back(T* node) {
18         if (!head) {
19             head = tail = node;
20         } else {
21             tail->next = node;
22             node->prev = tail;
23             tail = node;
24         }
25     }
26 };

```

32.38 ABI & BINARY COMPATIBILITY

32.39 13.1 Stable ABI Design

```

1  // Version-stable interface
2  class StableObject {
3  private:
4      // Use void* for opaque pointer
5      void* impl;
6

```



```

7 public:
8     StableObject();
9     ~StableObject();
10
11     // Stable functions
12     void do_something(int x);
13     int get_value() const;
14
15     // Virtual table for future extensions
16     virtual ~StableObject() = default;
17 };
18
19 // Implementation in separate library
20 class StableObjectImpl {
21 public:
22     int value = 0;
23     void do_something(int x);
24     int get_value() const { return value; }
25 };
26
27 StableObject::StableObject()
28     : impl(new StableObjectImpl()) {}
29
30 StableObject::~~StableObject() {
31     delete static_cast<StableObjectImpl*>(impl);
32 }
33
34 void StableObject::do_something(int x) {
35     static_cast<StableObjectImpl*>(impl)->do_something(x);
36 }
37
38 int StableObject::get_value() const {
39     return static_cast<StableObjectImpl*>(impl)->get_value();
40 }

```

32.40 PERFORMANCE PROFILING & OPTIMIZATION

32.41 14.1 Benchmarking Framework

```

1 #include <chrono>
2 #include <vector>
3
4 class Benchmark {
5 private:
6     using Clock = chrono::high_resolution_clock;
7     vector<chrono::nanoseconds> times;
8
9 public:
10     template<typename F>
11     void run(F func, int iterations = 1000) {
12         for (int i = 0; i < iterations; i++) {
13             auto start = Clock::now();
14             func();
15             auto end = Clock::now();
16             times.push_back(chrono::duration_cast<chrono::nanoseconds>(end -
17 start));
18         }
19     }
20 }

```

```

19
20 void report() {
21     if (times.empty()) return;
22
23     sort(times.begin(), times.end());
24
25     auto sum = 0LL;
26     for (auto t : times) sum += t.count();
27
28     cout << "Min: " << times.front().count() << " ns\n";
29     cout << "Max: " << times.back().count() << " ns\n";
30     cout << "Avg: " << (sum / times.size()) << " ns\n";
31     cout << "Median: " << times[times.size() / 2].count() << " ns\n";
32 }
33 };
34
35 // Usage
36 Benchmark bm;
37 bm.run([]() { /* test code */ }, 10000);
38 bm.report();

```

32.42 14.2 Cache-Friendly Algorithms

```

1 // Cache-friendly data access pattern
2 template<typename T>
3 void cache_friendly_process(vector<T>& data) {
4     // Process in cache-line aligned chunks
5     const size_t CACHE_LINE = 64; // 64 bytes typical
6     const size_t CHUNK_SIZE = CACHE_LINE / sizeof(T);
7
8     for (size_t i = 0; i < data.size(); i += CHUNK_SIZE) {
9         // Process one cache line worth of data
10        for (size_t j = i; j < min(i + CHUNK_SIZE, data.size()); j++) {
11            process_element(data[j]);
12        }
13    }
14 }
15
16 // SIMD-friendly loop
17 void simd_process(int* data, size_t size) {
18     // Align data for SIMD operations
19     alignas(32) int buffer[32];
20
21     for (size_t i = 0; i < size; i += 32) {
22         // Process 32-byte chunks (8 ints)
23         for (int j = 0; j < 8; j++) {
24             data[i + j] = process(data[i + j]);
25         }
26     }
27 }

```

32.43 DOMAIN-SPECIFIC LANGUAGE DESIGN

32.44 15.1 Expression DSL

```

1 // Domain-specific language for math expressions
2 class Expr {
3 public:
4     virtual ~Expr() = default;
5     virtual double evaluate() = 0;
6 };
7
8 class Constant : public Expr {
9 private:
10     double value;
11 public:
12     Constant(double v) : value(v) {}
13     double evaluate() override { return value; }
14 };
15
16 class BinaryOp : public Expr {
17 private:
18     shared_ptr<Expr> left, right;
19     function<double(double, double)> op;
20
21 public:
22     BinaryOp(shared_ptr<Expr> l, shared_ptr<Expr> r,
23             function<double(double, double)> op)
24         : left(l), right(r), op(op) {}
25
26     double evaluate() override {
27         return op(left->evaluate(), right->evaluate());
28     }
29 };
30
31 // Operator overloading for DSL
32 auto operator+(shared_ptr<Expr> l, shared_ptr<Expr> r) {
33     return make_shared<BinaryOp>(l, r, [](double a, double b) { return a + b;
34 });
35
36 // Usage
37 auto expr = make_shared<Constant>(3) + make_shared<Constant>(4);
38 cout << expr->evaluate() << "\n"; // 7

```

32.45 MODERN DESIGN PATTERNS

Traditional GoF patterns often use inheritance. Modern C++ favors composition, templates, and lambdas.

32.46 16.1 Strategy Pattern (Functional Approach)

Instead of a class hierarchy, use `std::function` or templates.

```

1 #include <functional>
2 #include <iostream>
3 #include <vector>
4
5 // Traditional: abstract base class Strategy
6 // Modern: std::function
7 using SortStrategy = std::function<void(std::vector<int>&)>;
8

```

```

9  class Sorter {
10     SortStrategy strategy;
11 public:
12     Sorter(SortStrategy s) : strategy(s) {}
13
14     void sort(std::vector<int>& data) {
15         if (strategy) strategy(data);
16     }
17 };
18
19 int main() {
20     std::vector<int> data = {5, 2, 9, 1};
21
22     // Strategy 1: Lambda
23     Sorter s1([](auto& v) { std::sort(v.begin(), v.end()); });
24
25     // Strategy 2: Different logic
26     Sorter s2([](auto& v) { std::sort(v.rbegin(), v.rend()); });
27
28     s1.sort(data);
29     return 0;
30 }

```

32.47 16.2 Visitor Pattern (std::variant)

Replace virtual functions with `std::variant` and `std::visit` for closed sets of types.

```

1  #include <variant>
2  #include <iostream>
3  #include <vector>
4
5  struct Circle { double radius; };
6  struct Square { double side; };
7  using Shape = std::variant<Circle, Square>;
8
9  // Visitor
10 struct AreaVisitor {
11     double operator()(const Circle& c) { return 3.14159 * c.radius * c.radius; }
12     double operator()(const Square& s) { return s.side * s.side; }
13 };
14
15 int main() {
16     std::vector<Shape> shapes = { Circle{2.0}, Square{3.0} };
17
18     for (const auto& s : shapes) {
19         // Apply visitor
20         double area = std::visit(AreaVisitor{}, s);
21         std::cout << "Area: " << area << "\n";
22     }
23
24     // With lambda (overloaded pattern)
25     // See "Helper for std::visit" in many codebases
26     return 0;
27 }

```

32.48 HARDWARE SYMPATHY

32.49 17.1 Cache Locality & False Sharing

CPUs load data in cache lines (typically 64 bytes).

32.49.1 False Sharing

When two threads modify independent variables that sit on the *same* cache line, they invalidate each other's cache, destroying performance.

```

1 #include <new>
2 #include <atomic>
3
4 struct BadCounter {
5     std::atomic<int> a; // Thread 1 modifies
6     std::atomic<int> b; // Thread 2 modifies
7     // Likely on same cache line -> ping-pong effect
8 };
9
10 struct GoodCounter {
11     alignas(64) std::atomic<int> a; // Forced to own cache line
12     alignas(64) std::atomic<int> b;
13 };

```

32.50 17.2 Branch Prediction

CPUs try to guess which way an if will go. Modern C++20 provides attributes to help.

```

1 void process(int* ptr) {
2     if (!ptr) [[unlikely]] {
3         // Compiler optimizes this block to be "cold"
4         // CPU assumes this won't happen
5         throw std::runtime_error("Null pointer");
6     }
7
8     // This "hot" path is optimized for fall-through
9     if (ptr) [[likely]] {
10         *ptr = 42;
11     }
12 }

```

32.51 17.3 SIMD (Single Instruction, Multiple Data)

Using intrinsics (or libraries like `std::simd` in future) to process data in parallel lanes.

```

1 // Example: Manual unrolling for auto-vectorization
2 void add_arrays(float* a, float* b, float* c, int n) {
3     // Tell compiler pointers don't alias (C99 restrict, or implementation
4     // specific)
5     // #pragma omp simd
6     for (int i = 0; i < n; ++i) {
7         c[i] = a[i] + b[i];
8     }
9 }

```

Chapter 33

PRODUCTION & PROFESSIONAL

33.1 LARGE-SCALE PROJECT ARCHITECTURE

33.2 1.1 Layered Architecture

```
1 // src/layers/presentation/controller.h
2 #ifndef PRESENTATION_CONTROLLER_H
3 #define PRESENTATION_CONTROLLER_H
4
5 #include "../domain/user.h"
6 #include "../application/user_service.h"
7
8 namespace presentation {
9     class UserController {
10     private:
11         application::UserService& service;
12
13     public:
14         UserController(application::UserService& s) : service(s) {}
15
16         void create_user(const std::string& name, const std::string& email) {
17             auto user = service.create(name, email);
18             display_result(user);
19         }
20
21     private:
22         void display_result(const domain::User& user);
23     };
24 }
25
26 #endif
```

```
1 // src/layers/application/user_service.h
2 #ifndef APPLICATION_USER_SERVICE_H
3 #define APPLICATION_USER_SERVICE_H
4
5 #include "../domain/user.h"
6 #include "../infrastructure/user_repository.h"
7
8 namespace application {
9     class UserService {
10     private:
11         infrastructure::UserRepository& repo;
```

```

12
13     public:
14         UserService(infrastructure::UserRepository& r) : repo(r) {}
15
16         domain::User create(const std::string& name, const std::string& email)
17     {
18         // Business logic
19         domain::User user(name, email);
20         validate_user(user);
21         return repo.save(user);
22     }
23
24     private:
25         void validate_user(const domain::User& user);
26 };
27
28 #endif

```

```

1 // src/layers/domain/user.h
2 #ifndef DOMAIN_USER_H
3 #define DOMAIN_USER_H
4
5 namespace domain {
6     class User {
7     private:
8         int id;
9         std::string name;
10        std::string email;
11
12    public:
13        User(const std::string& n, const std::string& e)
14            : id(0), name(n), email(e) {}
15
16        // Domain methods
17        bool is_valid() const;
18        void update_email(const std::string& new_email);
19    };
20 }
21
22 #endif

```

```

1 // src/layers/infrastructure/user_repository.h
2 #ifndef INFRASTRUCTURE_USER_REPOSITORY_H
3 #define INFRASTRUCTURE_USER_REPOSITORY_H
4
5 #include "../domain/user.h"
6 #include "database_connection.h"
7
8 namespace infrastructure {
9     class UserRepository {
10    private:
11        DatabaseConnection& db;
12
13    public:
14        UserRepository(DatabaseConnection& d) : db(d) {}
15
16        domain::User save(const domain::User& user);
17        std::optional<domain::User> find_by_id(int id);
18        std::vector<domain::User> find_all();
19    };
20 }
21

```

```
22 #endif
```

33.3 1.2 Microservices Architecture

```
1 // Service 1: User Service
2 namespace user_service {
3     class UserAPI {
4     private:
5         application::UserService& service;
6         http::Server& server;
7
8     public:
9         void setup_routes() {
10             server.post("/users", [this](const auto& req) {
11                 auto user = service.create(req.name, req.email);
12                 return http::Response::ok(user.to_json());
13             });
14
15             server.get("/users/:id", [this](const auto& req) {
16                 auto user = service.find(req.id);
17                 return http::Response::ok(user.to_json());
18             });
19         }
20     };
21 }
22
23 // Service 2: Order Service
24 namespace order_service {
25     class OrderAPI {
26     private:
27         application::OrderService& service;
28         http::Client& http_client;
29
30     public:
31         void create_order(int user_id, const Order& order) {
32             // Call user service
33             auto user = http_client.get("http://user-service/users/" + std::
34 to_string(user_id));
35
36             // Create order
37             service.create(user_id, order);
38         }
39     };
40 }
```

33.4 CODE ORGANIZATION & PROJECT STRUCTURE

33.5 2.1 Modern CMake Project Structure

```
1 # CMakeLists.txt - Project root
2 cmake_minimum_required(VERSION 3.20)
3 project(MyProject VERSION 1.0.0 LANGUAGES CXX)
4
5 # C++ standard
6 set(CMAKE_CXX_STANDARD 23)
```



```

7 set(CMAKE_CXX_STANDARD_REQUIRED ON)
8
9 # Project structure
10 add_subdirectory(src)
11 add_subdirectory(tests)
12 add_subdirectory(docs)
13
14 # Compiler flags
15 if(MSVC)
16     add_compile_options(/W4 /WX)
17 else()
18     add_compile_options(-Wall -Wextra -Wpedantic -Werror)
19 endif()
20
21 # Find dependencies
22 find_package(Boost REQUIRED)
23 find_package(Catch2 REQUIRED)

```

```

1 # src/CMakeLists.txt - Main library
2 add_library(mylib
3     domain/user.cpp
4     domain/order.cpp
5     application/user_service.cpp
6     infrastructure/user_repository.cpp
7 )
8
9 target_include_directories(mylib PUBLIC
10     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}>
11     $<INSTALL_INTERFACE:include>
12 )
13
14 target_link_libraries(mylib
15     PUBLIC Boost::system
16     PRIVATE Boost::thread
17 )
18
19 # Executable
20 add_executable(myapp main.cpp)
21 target_link_libraries(myapp PRIVATE mylib)

```

```

1 # tests/CMakeLists.txt - Test suite
2 add_executable(tests
3     test_user_service.cpp
4     test_user_repository.cpp
5 )
6
7 target_link_libraries(tests
8     PRIVATE mylib Catch2::Catch2WithMain
9 )
10
11 add_test(NAME AllTests COMMAND tests)

```

33.6 2.2 Header Organization

```

1 // include/mylib/version.h
2 #ifndef MYLIB_VERSION_H
3 #define MYLIB_VERSION_H
4
5 #define MYLIB_VERSION_MAJOR 1
6 #define MYLIB_VERSION_MINOR 0

```

```

7 #define MYLIB_VERSION_PATCH 0
8
9 namespace mylib {
10     struct Version {
11         static constexpr int major = MYLIB_VERSION_MAJOR;
12         static constexpr int minor = MYLIB_VERSION_MINOR;
13         static constexpr int patch = MYLIB_VERSION_PATCH;
14     };
15 }
16
17 #endif

```

```

1 // include/mylib/mylib.h - Main header
2 #ifndef MYLIB_H
3 #define MYLIB_H
4
5 // Version
6 #include "mylib/version.h"
7
8 // Core components
9 #include "mylib/domain/user.h"
10 #include "mylib/domain/order.h"
11
12 // Services
13 #include "mylib/application/user_service.h"
14 #include "mylib/application/order_service.h"
15
16 // Infrastructure
17 #include "mylib/infrastructure/database.h"
18
19 // Re-export main classes
20 namespace mylib {
21     using domain::User;
22     using domain::Order;
23     using application::UserService;
24 }
25
26 #endif

```

33.7 2.3 Modern CMake with Modules (C++20)

Using C++20 Modules requires CMake 3.28+.

```

1 # CMakeLists.txt
2 cmake_minimum_required(VERSION 3.28)
3 project(ModulesDemo LANGUAGES CXX)
4
5 set(CMAKE_CXX_STANDARD 20)
6 set(CMAKE_CXX_STANDARD_REQUIRED ON)
7 set(CMAKE_CXX_EXTENSIONS OFF)
8
9 # Library with modules
10 add_library(math_engine)
11 target_sources(math_engine
12     PUBLIC
13     FILE_SET CXX_MODULES FILES
14         src/math.cppm
15         src/vector.cppm
16 )
17
18 # Executable consuming modules

```

```

19 add_executable(app main.cpp)
20 target_link_libraries(app PRIVATE math_engine)

```

33.8 BUILD SYSTEMS & COMPILATION

33.9 3.1 Conan Package Manager

```

1 # conanfile.txt
2 [requires]
3 boost/1.81.0
4 fmt/9.1.0
5 nlohmann_json/3.11.2
6 catch2/3.3.2
7
8 [generators]
9 CMakeDeps
10 CMakeToolchain
11
12 [options]
13 boost/*:shared=False

```

```

1 # conanfile.py - Advanced
2 from conan import ConanFile
3 from conan.tools.cmake import CMakeToolchain, CMake
4
5 class MyProjectConan(ConanFile):
6     name = "myproject"
7     version = "1.0.0"
8     settings = "os", "compiler", "build_type", "arch"
9     options = {"shared": [True, False], "fPIC": [True, False]}
10    default_options = {"shared": False, "fPIC": True}
11
12    requires = "boost/1.81.0", "fmt/9.1.0"
13
14    def generate(self):
15        tc = CMakeToolchain(self)
16        tc.generate()
17
18    def build(self):
19        cmake = CMake(self)
20        cmake.configure()
21        cmake.build()
22
23    def package(self):
24        cmake = CMake(self)
25        cmake.install()

```

33.10 3.2 Incremental Build Optimization

```

1 # Enable ccache for faster rebuilds
2 find_program(CCACHE_PROGRAM ccache)
3 if(CCACHE_PROGRAM)
4     set_property(GLOBAL PROPERTY RULE_LAUNCH_COMPILE "${CCACHE_PROGRAM}")
5     set_property(GLOBAL PROPERTY RULE_LAUNCH_LINK "${CCACHE_PROGRAM}")
6 endif()

```

```

7
8 # Unity builds for faster compilation
9 set_target_properties(mylib PROPERTIES
10     UNITY_BUILD_ON
11     UNITY_BUILD_BATCH_SIZE 8
12 )
13
14 # Precompiled headers
15 target_precompile_headers(mylib PRIVATE
16     <vector>
17     <string>
18     <memory>
19     <boost/asio.hpp>
20 )

```

33.11 TESTING STRATEGIES

33.12 4.1 Unit Testing with Catch2

```

1 #include <catch2/catch_all.hpp>
2 #include "mylib/application/user_service.h"
3 #include "test_fixtures.h"
4
5 TEST_CASE("UserService creates users correctly", "[user_service]") {
6     auto repo = MockUserRepository();
7     UserService service(repo);
8
9     SECTION("Valid user creation") {
10         auto user = service.create("John Doe", "john@example.com");
11
12         REQUIRE(user.name == "John Doe");
13         REQUIRE(user.email == "john@example.com");
14         REQUIRE(repo.save_called == true);
15     }
16
17     SECTION("Invalid email rejects") {
18         REQUIRE_THROWS_AS(
19             service.create("John", "invalid-email"),
20             InvalidEmailException
21         );
22     }
23 }
24
25 TEST_CASE("UserService handles duplicates", "[user_service]") {
26     auto repo = MockUserRepository();
27     UserService service(repo);
28
29     service.create("John", "john@example.com");
30
31     REQUIRE_THROWS_AS(
32         service.create("Jane", "john@example.com"),
33         DuplicateEmailException
34     );
35 }

```

33.13 4.2 Integration Testing

```

1 TEST_CASE("User creation workflow", "[integration]") {
2     // Setup
3     Database db = setup_test_database();
4     UserRepository repo(db);
5     UserService service(repo);
6
7     // Execute
8     auto user = service.create("John Doe", "john@example.com");
9     auto fetched = repo.find_by_id(user.id);
10
11    // Verify
12    REQUIRE(fetched.has_value());
13    REQUIRE(fetched->name == "John Doe");
14
15    // Cleanup
16    cleanup_test_database(db);
17 }

```

33.14 4.3 Test Fixtures & Mocks

```

1 class UserRepositoryMock : public UserRepository {
2 public:
3     bool save_called = false;
4     std::vector<User> saved_users;
5
6     User save(const User& user) override {
7         save_called = true;
8         User u = user;
9         u.id = ++last_id;
10        saved_users.push_back(u);
11        return u;
12    }
13
14 private:
15     static int last_id;
16 };
17
18 class UserServiceTest {
19 protected:
20     UserRepositoryMock repo;
21     UserService service{repo};
22 };
23
24 TEST_CASE_METHOD(UserServiceTest, "Multiple users") {
25     auto u1 = service.create("John", "john@example.com");
26     auto u2 = service.create("Jane", "jane@example.com");
27
28     REQUIRE(repo.saved_users.size() == 2);
29 }

```

33.15 4.4 Advanced Mocking with Google Mock (GMock)

For complex interactions, use GMock.

```

1 #include <gmock/gmock.h>
2
3 class MockDB : public Database {
4 public:

```

```

5     MOCK_METHOD(bool, connect, (string), (override));
6     MOCK_METHOD(void, query, (string), (override));
7 };
8
9 TEST(DBTest, LoginSequence) {
10     MockDB db;
11
12     // Expect connect called once with "admin"
13     EXPECT_CALL(db, connect("admin"))
14         .Times(1)
15         .WillOnce(testing::Return(true));
16
17     // Expect query called any number of times
18     EXPECT_CALL(db, query(testing::_))
19         .Times(testing::AtLeast(0));
20
21     UserManager mgr(&db);
22     mgr.login("admin");
23 }

```

33.16 DEBUGGING & PROFILING

33.17 5.1 Debug Utilities

```

1 // include/mylib/debug.h
2 #ifndef MYLIB_DEBUG_H
3 #define MYLIB_DEBUG_H
4
5 #include <iostream>
6 #include <source_location>
7
8 namespace mylib::debug {
9     enum class Level { Debug, Info, Warning, Error };
10
11     class Logger {
12     private:
13         static Level current_level;
14
15     public:
16         template<typename... Args>
17         static void log(Level level, std::format_string<Args...> fmt, Args...
18         args) {
19             if (level < current_level) return;
20
21             auto loc = std::source_location::current();
22             std::cerr << std::format("[{}: {}: {}] {}",
23                                     loc.file_name(), loc.line(), loc.column(),
24                                     std::format(fmt, args...))
25             << "\n";
26         }
27
28         static void set_level(Level l) { current_level = l; }
29     };
30
31 #ifdef DEBUG
32     #define LOG_DEBUG(...) debug::Logger::log(debug::Level::Debug,
33     __VA_ARGS__)
34 #endif
35 }

```

```

32     #define LOG_INFO(...) debug::Logger::log(debug::Level::Info,
    __VA_ARGS__)
33     #else
34     #define LOG_DEBUG(...)
35     #define LOG_INFO(...)
36     #endif
37 }
38
39 #endif

```

33.18 5.2 Performance Profiling

```

1  #include <chrono>
2  #include <map>
3
4  class PerformanceProfiler {
5  private:
6      struct Measurement {
7          std::chrono::nanoseconds total{0};
8          int count = 0;
9          std::chrono::nanoseconds min{LLONG_MAX};
10         std::chrono::nanoseconds max{0};
11     };
12
13     std::map<std::string, Measurement> measurements;
14
15 public:
16     class Scope {
17     private:
18         std::string name;
19         PerformanceProfiler& profiler;
20         std::chrono::high_resolution_clock::time_point start;
21
22     public:
23         Scope(std::string n, PerformanceProfiler& p)
24             : name(n), profiler(p), start(std::chrono::high_resolution_clock::
25               now()) {}
26
27         ~Scope() {
28             auto end = std::chrono::high_resolution_clock::now();
29             auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>
30               (end - start);
31             profiler.record(name, duration);
32         }
33
34     void record(const std::string& name, std::chrono::nanoseconds duration) {
35         auto& m = measurements[name];
36         m.total += duration;
37         m.count++;
38         m.min = std::min(m.min, duration);
39         m.max = std::max(m.max, duration);
40     }
41
42     void report() const {
43         std::cout << "Performance Report:\n";
44         std::cout << std::string(60, '=') << "\n";
45
46         for (const auto& [name, m] : measurements) {
47             auto avg = m.total.count() / m.count;

```

```

47         std::cout << std::format("{:<30} | Count: {:>5} | Avg: {:>10}ns |
      Min: {:>10}ns | Max: {:>10}ns\n",
48             name, m.count, avg, m.min.count(), m.max.count());
49     }
50 }
51 };
52
53 // Usage
54 #define PROFILE(name) PerformanceProfiler::Scope _scope(name, get_profiler())
55
56 void process_data(const std::vector<int>& data) {
57     PROFILE("process_data");
58
59     {
60         PROFILE("sort");
61         std::sort(data.begin(), data.end());
62     }
63
64     {
65         PROFILE("filter");
66         // Filter implementation
67     }
68 }

```

33.19 5.3 Memory Profiling with AddressSanitizer

```

1 # CMakeLists.txt - AddressSanitizer configuration
2 option(ENABLE_SANITIZER "Enable AddressSanitizer" OFF)
3
4 if(ENABLE_SANITIZER)
5     add_compile_options(-fsanitize=address -fno-omit-frame-pointer)
6     add_link_options(-fsanitize=address)
7 endif()

```

33.20 VERSION CONTROL & COLLABORATION

33.21 6.1 Git Workflow

```

1 # .gitignore - Standard C++ project
2 build/
3 dist/
4 *.o
5 *.a
6 *.so
7 *.exe
8 .vscode/
9 .idea/
10 *.swp
11 CMakeLists.txt.user
12 *.qbs.user
13
14 # Git configuration - .git/config
15 [user]
16     name = Developer
17     email = dev@company.com

```



```
18
19 [core]
20     editor = vim
21     autocrlf = input
22
23 [pull]
24     rebase = true
25
26 [branch]
27     autosetuprebase = always
```

33.22 6.2 Feature Branch Workflow

```
1 # Main branch is production-ready
2 git checkout main
3 git pull origin main
4
5 # Create feature branch
6 git checkout -b feature/user-authentication
7
8 # Commit with conventional commits
9 git add src/
10 git commit -m "feat: implement JWT authentication"
11
12 # Rebase and squash commits
13 git rebase -i main
14
15 # Create PR and request review
16 git push origin feature/user-authentication
```

33.23 DOCUMENTATION & KNOWLEDGE TRANSFER

33.24 7.1 Code Documentation with Doxygen

```
1 /**
2  * @file user_service.h
3  * @brief User service implementation for managing user lifecycle
4  * @author John Doe
5  * @version 1.0.0
6  * @date 2024-01-15
7  */
8
9 namespace application {
10     /**
11      * @class UserService
12      * @brief Service for managing user operations
13      *
14      * UserService provides business logic for user management including
15      * creation, deletion, and modification. It validates input and
16      * persists data through the UserRepository.
17      *
18      * @example
19      * @code
20      * UserService service(repository);
21      * auto user = service.create("John Doe", "john@example.com");
```

```

22     * @endcode
23     */
24     class UserService {
25     public:
26         /**
27          * @brief Creates a new user
28          *
29          * @param name The user's full name
30          * @param email The user's email address
31          *
32          * @return User The created user object with assigned ID
33          *
34          * @throws InvalidNameException if name is empty
35          * @throws InvalidEmailException if email is invalid
36          * @throws DuplicateEmailException if email already exists
37          *
38          * @complexity O(n) where n is the number of existing users
39          */
40         User create(const std::string& name, const std::string& email);
41
42         /**
43          * @brief Updates user information
44          *
45          * @param id User ID
46          * @param name New name
47          * @param email New email
48          *
49          * @return User Updated user object
50          *
51          * @throws UserNotFoundException if user doesn't exist
52          * @throws InvalidEmailException if email is invalid
53          */
54         User update(int id, const std::string& name, const std::string& email);
55     };
56 }

```

33.25 7.2 Architecture Decision Records (ADR)

```

1 # ADR-001: Use Layered Architecture
2
3 ## Status
4 Accepted
5
6 ## Context
7 The system needs to be scalable, testable, and maintainable.
8 Different concerns (UI, business logic, data access) must be separated.
9
10 ## Decision
11 We will implement a layered architecture with:
12 - Presentation Layer (Controllers, Views)
13 - Application Layer (Services, Use Cases)
14 - Domain Layer (Business Logic)
15 - Infrastructure Layer (Database, External Services)
16
17 ## Consequences
18 ### Positive
19 - Clear separation of concerns
20 - Easy to test (mock layers)
21 - Scalable architecture
22
23 ### Negative

```

```

24 - More files to navigate
25 - Increased complexity for simple features
26 - Potential performance overhead from layer crossing
27
28 ## Alternatives Considered
29 - Hexagonal (Ports & Adapters) - More complex, chosen layered for simplicity
30 - Microservices - Future consideration for scalability

```

33.26 SECURITY & SAFETY

33.27 8.1 Input Validation

```

1 #include <regex>
2 #include <stdexcept>
3
4 namespace security {
5     class InputValidator {
6     public:
7         static void validate_email(const std::string& email) {
8             static const std::regex email_regex(
9                 R"([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$)"
10            );
11
12            if (!std::regex_match(email, email_regex)) {
13                throw std::invalid_argument("Invalid email format");
14            }
15        }
16
17        static void validate_length(const std::string& str,
18                                   size_t min_length, size_t max_length) {
19            if (str.length() < min_length || str.length() > max_length) {
20                throw std::invalid_argument(
21                    std::format("String length must be between {} and {}",
22                               min_length, max_length)
23                );
24            }
25        }
26
27        static void validate_integer_range(int value, int min, int max) {
28            if (value < min || value > max) {
29                throw std::out_of_range(
30                    std::format("Value must be between {} and {}", min, max)
31                );
32            }
33        }
34    };
35 }

```

33.28 8.2 SQL Injection Prevention

```

1 #include <sqlite3.h>
2
3 namespace database {
4     class SafeQuery {
5     private:

```

```

6     sqlite3* db;
7
8     public:
9         std::vector<User> find_by_email_safe(const std::string& email) {
10             const char* sql = "SELECT * FROM users WHERE email = ?";
11             sqlite3_stmt* stmt;
12
13             // Prepare statement (prevents SQL injection)
14             int rc = sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr);
15             if (rc != SQLITE_OK) {
16                 throw std::runtime_error("SQL error");
17             }
18
19             // Bind parameters safely
20             sqlite3_bind_text(stmt, 1, email.c_str(), -1, SQLITE_STATIC);
21
22             std::vector<User> results;
23             while (sqlite3_step(stmt) == SQLITE_ROW) {
24                 // Extract results
25                 int id = sqlite3_column_int(stmt, 0);
26                 const char* name = (const char*)sqlite3_column_text(stmt, 1);
27
28                 results.emplace_back(id, name, email);
29             }
30
31             sqlite3_finalize(stmt);
32             return results;
33         }
34     };
35 }

```

33.29 PERFORMANCE ENGINEERING

33.30 9.1 Benchmarking Framework

```

1 #include <benchmark/benchmark.h>
2
3 static void BM_StringCreation(benchmark::State& state) {
4     for (auto _ : state) {
5         std::string s = "hello world";
6         benchmark::DoNotOptimize(s);
7     }
8 }
9 BENCHMARK(BM_StringCreation);
10
11 static void BM_VectorOperations(benchmark::State& state) {
12     for (auto _ : state) {
13         std::vector<int> v;
14         for (int i = 0; i < 1000; i++) {
15             v.push_back(i);
16         }
17         benchmark::DoNotOptimize(v);
18     }
19 }
20 BENCHMARK(BM_VectorOperations);
21
22 BENCHMARK_MAIN();

```

33.31 9.2 Load Testing

```

1  class LoadTester {
2  public:
3      struct Result {
4          int total_requests;
5          std::chrono::milliseconds total_time;
6          double requests_per_second;
7          double avg_latency_ms;
8          double p99_latency_ms;
9      };
10
11     Result run_load_test(
12         std::function<void()> operation,
13         int num_threads,
14         int requests_per_thread
15     ) {
16         std::vector<std::thread> threads;
17         std::vector<std::chrono::nanoseconds> latencies;
18         std::mutex latencies_mutex;
19
20         auto start = std::chrono::high_resolution_clock::now();
21
22         for (int t = 0; t < num_threads; t++) {
23             threads.emplace_back([&, t] {
24                 for (int r = 0; r < requests_per_thread; r++) {
25                     auto op_start = std::chrono::high_resolution_clock::now();
26                     operation();
27                     auto op_end = std::chrono::high_resolution_clock::now();
28
29                     auto latency = std::chrono::duration_cast<std::chrono::
nanoseconds>(
30                         op_end - op_start
31                     );
32
33                     {
34                         std::lock_guard lock(latencies_mutex);
35                         latencies.push_back(latency);
36                     }
37                 }
38             });
39         }
40
41         for (auto& t : threads) t.join();
42
43         auto end = std::chrono::high_resolution_clock::now();
44         auto total_time = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
45
46         // Calculate percentiles
47         std::sort(latencies.begin(), latencies.end());
48         int p99_idx = (latencies.size() * 99) / 100;
49
50         Result result;
51         result.total_requests = num_threads * requests_per_thread;
52         result.total_time = total_time;
53         result.requests_per_second = result.total_requests / (total_time.count() / 1000.0);
54
55         long long sum = 0;
56         for (auto l : latencies) sum += l.count();
57         result.avg_latency_ms = (sum / latencies.size()) / 1e6;
58         result.p99_latency_ms = latencies[p99_idx].count() / 1e6;

```

```

59         return result;
60     }
61 };

```

33.32 ERROR HANDLING & RECOVERY

33.33 10.1 Exception Hierarchy

```

1  #include <stdexcept>
2
3  namespace application {
4      // Base exception
5      class ApplicationError : public std::runtime_error {
6      protected:
7          int error_code;
8          std::string error_context;
9
10     public:
11         ApplicationError(const std::string& msg, int code = -1)
12             : std::runtime_error(msg), error_code(code) {}
13
14         int get_error_code() const { return error_code; }
15         void set_context(const std::string& ctx) { error_context = ctx; }
16     };
17
18     // Domain exceptions
19     class DomainError : public ApplicationError {
20     public:
21         DomainError(const std::string& msg)
22             : ApplicationError(msg, 1001) {}
23     };
24
25     class ValidationError : public DomainError {
26     public:
27         ValidationError(const std::string& msg)
28             : DomainError("Validation: " + msg) {}
29     };
30
31     class InvalidEmailException : public ValidationError {
32     public:
33         InvalidEmailException(const std::string& email)
34             : ValidationError("Invalid email: " + email) {}
35     };
36
37     // Repository exceptions
38     class RepositoryError : public ApplicationError {
39     public:
40         RepositoryError(const std::string& msg)
41             : ApplicationError(msg, 2001) {}
42     };
43
44     class UserNotFoundException : public RepositoryError {
45     public:
46         UserNotFoundException(int id)
47             : RepositoryError("User not found: " + std::to_string(id)) {}
48     };
49 }

```

33.34 10.2 Error Recovery Patterns

```

1 class RetryPolicy {
2 public:
3     struct Config {
4         int max_retries = 3;
5         std::chrono::milliseconds initial_delay{100};
6         double backoff_multiplier = 2.0;
7         int max_delay_ms = 10000;
8     };
9
10    template<typename F>
11    static auto execute_with_retry(F operation, const Config& config)
12        -> std::invoke_result_t<F> {
13
14        std::exception_ptr last_exception;
15        auto delay = config.initial_delay;
16
17        for (int attempt = 0; attempt <= config.max_retries; attempt++) {
18            try {
19                return operation();
20            } catch (const std::exception& e) {
21                last_exception = std::current_exception();
22
23                if (attempt < config.max_retries) {
24                    LOG_INFO("Retry attempt {} after {}ms",
25                        attempt + 1, delay.count());
26                    std::this_thread::sleep_for(delay);
27
28                    delay = std::chrono::milliseconds(
29                        std::min(static_cast<int>(delay.count() * config.
30                            backoff_multiplier),
31                                config.max_delay_ms)
32                            );
33                }
34            }
35
36            std::rethrow_exception(last_exception);
37        }
38    };
39
40    // Usage
41    auto user = RetryPolicy::execute_with_retry(
42        [&]() { return repository.find_user(id); },
43        RetryPolicy::Config{.max_retries = 5}
44    );

```

33.35 DEPLOYMENT & DEVOPS

33.36 11.1 Docker Containerization

```

1 # Dockerfile - Multi-stage build
2 FROM ubuntu:22.04 AS builder
3
4 RUN apt-get update && apt-get install -y \
5     cmake \

```

```

6      g++ \
7      git \
8      libboost-all-dev
9
10 WORKDIR /build
11 COPY . .
12
13 RUN cmake -B build -DCMAKE_BUILD_TYPE=Release
14 RUN cmake --build build -j$(nproc)
15 RUN cmake --install build --prefix /install
16
17 # Runtime stage
18 FROM ubuntu:22.04
19
20 RUN apt-get update && apt-get install -y \
21     libboost-system1.74.0 \
22     ca-certificates
23
24 COPY --from=builder /install /usr/local
25
26 EXPOSE 8080
27 CMD ["/usr/local/bin/myapp"]

```

33.37 11.2 Kubernetes Deployment

```

1 # deployment.yaml
2 apiVersion: apps/v1
3 kind: Deployment
4 metadata:
5   name: myapp
6   labels:
7     app: myapp
8 spec:
9   replicas: 3
10  selector:
11    matchLabels:
12      app: myapp
13  template:
14    metadata:
15      labels:
16        app: myapp
17    spec:
18      containers:
19        - name: myapp
20          image: myapp:1.0.0
21          ports:
22            - containerPort: 8080
23          resources:
24            requests:
25              memory: "256Mi"
26              cpu: "250m"
27            limits:
28              memory: "512Mi"
29              cpu: "500m"
30          livenessProbe:
31            httpGet:
32              path: /health
33              port: 8080
34            initialDelaySeconds: 30
35            periodSeconds: 10
36          readinessProbe:

```



```

37     httpGet:
38         path: /ready
39         port: 8080
40         initialDelaySeconds: 5
41         periodSeconds: 5

```

33.38 11.3 CI/CD Pipeline (GitHub Actions)

Automate building and testing.

```

1 name: C++ CI
2
3 on: [push, pull_request]
4
5 jobs:
6     build:
7         runs-on: ubuntu-latest
8
9         steps:
10            - uses: actions/checkout@v3
11
12            - name: Install Dependencies
13              run: sudo apt-get install -y libboost-dev cmake
14
15            - name: Configure CMake
16              run: cmake -B build -DCMAKE_BUILD_TYPE=Release
17
18            - name: Build
19              run: cmake --build build
20
21            - name: Test
22              run: cd build && ctest --output-on-failure

```

33.39 CODE REVIEW & QUALITY

33.40 12.1 Code Review Checklist

```

1 # Code Review Checklist
2
3 ## Functionality
4 - [ ] Code implements the required feature
5 - [ ] Code handles all edge cases
6 - [ ] Error handling is appropriate
7 - [ ] Tests cover the implementation
8
9 ## Code Quality
10 - [ ] Variable/function names are clear
11 - [ ] Code is DRY (Don't Repeat Yourself)
12 - [ ] Functions are appropriately sized
13 - [ ] Complexity is reasonable
14
15 ## Performance
16 - [ ] No obvious performance issues
17 - [ ] Memory usage is appropriate
18 - [ ] Algorithms are efficient
19 - [ ] No memory leaks

```

```

20
21 ## Security
22 - [ ] Input validation is present
23 - [ ] No SQL injection vulnerabilities
24 - [ ] No buffer overflows
25 - [ ] Sensitive data is protected
26
27 ## Testing
28 - [ ] Unit tests are present
29 - [ ] Integration tests pass
30 - [ ] Test coverage is adequate
31 - [ ] Edge cases are tested
32
33 ## Documentation
34 - [ ] Code is well-commented
35 - [ ] Public APIs are documented
36 - [ ] Changes are documented
37 - [ ] Architecture decisions are recorded

```

33.41 12.2 Static Code Analysis

```

1 # CMakeLists.txt - Clang-Tidy integration
2 find_program(CLANG_TIDY clang-tidy)
3
4 if(CLANG_TIDY)
5     set(CMAKE_CXX_CLANG_TIDY "${CLANG_TIDY}"
6         "-checks="
7         "-header-filter=.*"
8         "-fix"
9     )
10 endif()
11
12 # Cppcheck integration
13 find_program(CPPCHECK cppcheck)
14
15 if(CPPCHECK)
16     add_custom_target(cppcheck
17         COMMAND ${CPPCHECK}
18             --enable=all
19             --suppress=missingIncludeSystem
20             ${CMAKE_SOURCE_DIR}/src
21     )
22 endif()

```

33.42 TECHNICAL DEBT MANAGEMENT

33.43 13.1 Tracking Technical Debt

```

1 // Technical debt marker
2 namespace technical_debt {
3     /**
4      * @deprecated Refactor this when performance is not critical.
5      * Use optimized_algorithm_v2 instead.
6      *
7      * @todo Refactor by Q2 2024

```

```

8      * @complexity O(n) - needs optimization
9      */
10     void inefficient_sort(std::vector<int>& data) {
11         // Bubble sort - inefficient but simple
12         for (size_t i = 0; i < data.size(); i++) {
13             for (size_t j = 0; j < data.size() - 1; j++) {
14                 if (data[j] > data[j + 1]) {
15                     std::swap(data[j], data[j + 1]);
16                 }
17             }
18         }
19     }
20
21     /**
22      * @deprecated Temporary workaround for issue #123.
23      * Remove when backend API is fixed.
24      * @todo Track: https://github.com/team/project/issues/123
25      */
26     std::string get_user_name(int id) {
27         // Workaround: return hardcoded values until backend is fixed
28         static const std::map<int, std::string> workaround{
29             {1, "John Doe"},
30             {2, "Jane Smith"}
31         };
32         return workaround.at(id);
33     }
34 }

```

33.44 13.2 Refactoring Strategy

```

1 // Old code
2 class User {
3 public:
4     void save_to_db(const std::string& connection_string) {
5         // Direct database access - tightly coupled
6     }
7
8     void send_email(const std::string& subject, const std::string& body) {
9         // Email sending logic mixed with domain logic
10    }
11};
12
13 // Refactored code
14 class User {
15 private:
16     int id;
17     std::string name;
18     std::string email;
19
20 public:
21     int get_id() const { return id; }
22     const std::string& get_name() const { return name; }
23     const std::string& get_email() const { return email; }
24};
25
26 // Separated concerns
27 class UserRepository {
28 public:
29     void save(const User& user);
30};
31

```

```

32 class UserEmailService {
33 public:
34     void send_notification(const User& user, const std::string& message);
35 };

```

33.45 LEGACY CODE MODERNIZATION

33.46 14.1 Incremental Modernization

```

1  // Legacy code (C++98 style)
2  class LegacyUser {
3  public:
4      char* name;           // Raw pointer
5      char* email;          // Raw pointer
6
7      LegacyUser(const char* n, const char* e) {
8          name = new char[strlen(n) + 1];
9          strcpy(name, n); // Unsafe
10         email = new char[strlen(e) + 1];
11         strcpy(email, e); // Unsafe
12     }
13
14     ~LegacyUser() {
15         delete[] name;
16         delete[] email;
17     }
18 };
19
20 // Step 1: Add modern wrapper
21 class ModernUserWrapper {
22 private:
23     LegacyUser* legacy;
24
25 public:
26     ModernUserWrapper(const std::string& name, const std::string& email) {
27         legacy = new LegacyUser(name.c_str(), email.c_str());
28     }
29
30     ~ModernUserWrapper() { delete legacy; }
31
32     std::string_view get_name() const { return legacy->name; }
33     std::string_view get_email() const { return legacy->email; }
34 };
35
36 // Step 2: Full modernization
37 class ModernUser {
38 private:
39     std::string name;
40     std::string email;
41
42 public:
43     ModernUser(std::string n, std::string e)
44         : name(std::move(n)), email(std::move(e)) {}
45
46     const std::string& get_name() const { return name; }
47     const std::string& get_email() const { return email; }
48 };

```

33.47 LEADERSHIP & TEAM MANAGEMENT

33.48 15.1 Mentoring Framework

```
1 # Mentoring Guidelines for C++ Teams
2
3 ## Levels
4
5 ### Junior Developer (0-1 year)
6 - Focus: Understanding fundamentals
7 - Guidance: Pair programming, code reviews, architecture training
8 - Goals: Contribute to features, improve C++ knowledge
9
10 ### Mid-Level Developer (1-3 years)
11 - Focus: Mastering patterns and best practices
12 - Guidance: Design reviews, mentoring juniors, taking ownership
13 - Goals: Lead features, improve design decisions
14
15 ### Senior Developer (3+ years)
16 - Focus: Architecture, leadership, knowledge sharing
17 - Guidance: Strategic decisions, cross-team collaboration
18 - Goals: Shape team direction, mentor other seniors
19
20 ## Mentoring Actions
21 1. Code review with detailed feedback
22 2. Design discussions before implementation
23 3. Pair programming sessions
24 4. Knowledge sharing sessions
25 5. Challenge with progressively harder problems
```

33.49 15.2 Technical Decision Making

```
1 # RFC (Request For Comments) Template
2
3 ## Title
4 Brief description of the proposal
5
6 ## Motivation
7 Why this change is needed
8
9 ## Detailed Design
10 Technical approach and architecture
11
12 ## Trade-offs
13 What we're giving up
14
15 ## Alternatives
16 Other approaches considered
17
18 ## Implementation Plan
19 Steps to implement
20
21 ## Timeline
22 Expected completion
23
24 ## Success Metrics
```

```
25 How we measure success
26
27 ## Discussion
28 Team feedback period (1 week)
29
30 ## Decision
31 Final decision and rationale
```

Final decision and rationale

““

Chapter 34

SYSTEM DESIGN CASE STUDIES (C++ EDITION)

Solving common interview system design problems using C++ primitives.

34.0.1 10.5.1 LRU Cache

Problem: Design a Least Recently Used cache with $O(1)$ get and put. **Solution:** Combine `std::list` (ordering) and `std::unordered_map` (lookup).

```
1 #include <list>
2 #include <unordered_map>
3 #include <iostream>
4
5 template<typename Key, typename Value>
6 class LRUCache {
7     size_t capacity;
8     std::list<std::pair<Key, Value>> items;
9     std::unordered_map<Key, typename std::list<std::pair<Key, Value>>::iterator>
10     > lookup;
11
12 public:
13     LRUCache(size_t cap) : capacity(cap) {}
14
15     void put(Key key, Value val) {
16         if (lookup.find(key) != lookup.end()) {
17             // Update: Move to front, update value
18             items.splice(items.begin(), items, lookup[key]);
19             lookup[key]->second = val;
20             return;
21         }
22
23         if (items.size() == capacity) {
24             // Evict: Remove back
25             lookup.erase(items.back().first);
26             items.pop_back();
27         }
28
29         // Insert: Push front
30         items.emplace_front(key, val);
31         lookup[key] = items.begin();
32     }
33
34     std::optional<Value> get(Key key) {
35         if (lookup.find(key) == lookup.end()) return std::nullopt;
36         // Access: Move to front
37         items.splice(items.begin(), items, lookup[key]);
38     }
39 }
```

```

37     return lookup[key]->second;
38 }
39 };

```

34.0.2 10.5.2 Token Bucket Rate Limiter

Problem: Limit requests to N per second. **Solution:** Refill tokens based on time elapsed.

```

1  #include <chrono>
2  #include <mutex>
3
4  class TokenBucket {
5      const long long capacity;
6      const long long rate_per_sec;
7
8      double tokens;
9      std::chrono::steady_clock::time_point last_refill;
10     std::mutex mtx;
11
12 public:
13     TokenBucket(long long cap, long long rate)
14         : capacity(cap), rate_per_sec(rate), tokens(cap),
15           last_refill(std::chrono::steady_clock::now()) {}
16
17     bool allow_request(int cost = 1) {
18         std::lock_guard<std::mutex> lock(mtx);
19         refill();
20
21         if (tokens >= cost) {
22             tokens -= cost;
23             return true;
24         }
25         return false;
26     }
27
28 private:
29     void refill() {
30         auto now = std::chrono::steady_clock::now();
31         auto duration = std::chrono::duration_cast<std::chrono::microseconds>(
32             now - last_refill).count();
33
34         double new_tokens = (duration * rate_per_sec) / 1000000.0;
35         tokens = std::min((double)capacity, tokens + new_tokens);
36         last_refill = now;
37     }
38 };

```


Chapter 35

CONCURRENCY DESIGN PATTERNS

35.0.1 10.6.1 Active Object Pattern

Decouples method execution from invocation. The object owns a thread and a message queue.

```
1 #include <queue>
2 #include <functional>
3 #include <thread>
4 #include <mutex>
5 #include <condition_variable>
6
7 class ActiveObject {
8     std::queue<std::function<void()>> tasks;
9     std::mutex mtx;
10    std::condition_variable cv;
11    std::thread worker;
12    bool done = false;
13
14 public:
15     ActiveObject() {
16         worker = std::thread([this] { run(); });
17     }
18
19     ~ActiveObject() {
20         { std::lock_guard lock(mtx); done = true; }
21         cv.notify_one();
22         worker.join();
23     }
24
25     void invoke(std::function<void()> task) {
26         std::lock_guard lock(mtx);
27         tasks.push(std::move(task));
28         cv.notify_one();
29     }
30
31 private:
32     void run() {
33         while (true) {
34             std::unique_lock lock(mtx);
35             cv.wait(lock, [this] { return !tasks.empty() || done; });
36
37             if (done && tasks.empty()) return;
38
39             auto task = std::move(tasks.front());
40             tasks.pop();
41             lock.unlock();
```

```
42
43         task(); // Execute
44     }
45 }
46};
```

35.0.2 10.6.2 Monitor Object (Thread-Safe Interface)

Ensure thread safety by locking in public methods and calling private implementation methods.

```
1  class Monitor {
2      mutable std::mutex mtx;
3      int state = 0;
4
5  public:
6      void update(int val) {
7          std::lock_guard lock(mtx); // Lock here
8          update_impl(val);
9      }
10
11 private:
12     // Expects lock to be held
13     void update_impl(int val) {
14         state = val;
15     }
16};
```

Chapter 36

THE C++ BUILD ECOSYSTEM MASTERY

Writing code is half the battle. Building and debugging it is the rest.

36.0.1 30.1 Package Managers Deep Dive

vcpkg (Manifest Mode)

Create `vcpkg.json` in your root:

```
1 {  
2   "name": "my-app",  
3   "version": "1.0.0",  
4   "dependencies": [  
5     "fmt",  
6     "nlohmann-json"  
7   ]  
8 }
```

CMake integration:

```
1 cmake -B build -DCMAKE_TOOLCHAIN_FILE=.../vcpkg.cmake
```

Conan (conanfile.txt)

```
1 [requires]  
2 fmt/9.1.0  
3 nlohmann_json/3.11.2  
4  
5 [generators]  
6 CMakeDeps  
7 CMakeToolchain
```

36.0.2 30.2 Sanitizers: The Developer's Best Friend

AddressSanitizer (ASan)

Detects out-of-bounds, use-after-free. `clang++ -fsanitize=address -g main.cpp`

Example: Use-After-Free

```
1 int* p = new int(5);  
2 delete p;  
3 *p = 10; // ASan catches this instantly!
```

ThreadSanitizer (TSan)

Detects data races. `clang++ -fsanitize=thread -g main.cpp`

Example: Data Race

```
1 int counter = 0;
2 std::thread t1([&]{ counter++; });
3 std::thread t2([&]{ counter++; }); // TSan catches this race
4 t1.join(); t2.join();
```

UndefinedBehaviorSanitizer (UBSan)

Detects overflow, null dereference, alignment issues. `clang++ -fsanitize=undefined -g main.cpp`

36.0.3 30.3 Profiling Tools

- **perf (Linux):** `perf record -g ./app -> perf report.`
 - **Valgrind (Massif):** Heap profiler. `valgrind --tool=massif ./app.`
 - **Hotspot:** UI for perf.
-

Chapter 37

LOW-LATENCY C++ OPTIMIZATION

For HFT, Game Engines, and Real-Time Systems, every nanosecond counts.

37.0.1 17.1 CPU Pipelines & Branch Prediction

Modern CPUs are pipelined. A branch misprediction flushes the pipeline, costing 10-20 cycles.

Optimization: Branchless Programming

```
1 // Branchy (Slow if unpredictable)
2 if (val > 100) val = 100;
3
4 // Branchless (Fast)
5 // Compiler might generate 'cmov' (Conditional Move) instruction
6 val = (val > 100) ? 100 : val;
```

Benchmark: Sorted vs Unsorted Array Processing Processing a sorted array is faster due to successful branch prediction.

37.0.2 17.2 Data-Oriented Design (DoD)

Stop thinking in “Objects”. Think in “Data Transforms”.

OOP (Array of Structures - AoS):

```
1 struct Entity {
2     float x, y, z;
3     int hp;
4     // ...
5 };
6 vector<Entity> entities;
7 // Updating 'x' loads 'hp' into cache (waste)
```

DoD (Structure of Arrays - SoA):

```
1 struct Entities {
2     vector<float> x, y, z;
3     vector<int> hp;
4 };
5 // Updating 'x' loads only 'x' data (SIMD friendly, cache friendly)
```

37.0.3 17.3 Prefetching

Use `__builtin_prefetch` (GCC/Clang) or `_mm_prefetch` (Intel) to load data into L1 cache before it's needed.

```

1 for (int i = 0; i < N; ++i) {
2     __builtin_prefetch(&data[i + 16]); // Lookahead
3     process(data[i]);
4 }

```

37.0.4 17.4 Micro-Benchmarking (Google Benchmark)

Don't guess; measure. `std::chrono` is often too noisy for nanosecond-scale operations.

```

1 #include <benchmark/benchmark.h>
2
3 static void BM_StringCopy(benchmark::State& state) {
4     std::string x = "hello";
5     for (auto _ : state) {
6         std::string copy = x;
7         benchmark::DoNotOptimize(copy); // Prevent optimizing away
8     }
9 }
10 BENCHMARK(BM_StringCopy);

```

37.0.5 17.5 System Warm-up

The first few thousand iterations of code are slow due to: 1. **Instruction Cache Misses:** Code not yet in CPU cache. 2. **Data Cache Misses:** Data not yet in L1/L2. 3. **Branch Predictor:** Hasn't learned the patterns yet. 4. **OS Page Faults:** Memory pages not yet committed.

Strategy: Run a “dummy” loop of your critical path 10,000 times before enabling the network listener or trading signal.

37.0.6 17.6 False Sharing Prevention

When two threads modify variables on the same cache line (64 bytes), they invalidate each other's L1 cache.

```

1 #include <new>
2
3 struct SharedData {
4     // Bad: a and b likely share a cache line
5     std::atomic<int> a;
6     std::atomic<int> b;
7 };
8
9 struct PaddedData {
10     alignas(std::hardware_destructive_interference_size) std::atomic<int> a;
11     alignas(std::hardware_destructive_interference_size) std::atomic<int> b;
12 };

```

Chapter 38

LOW-LATENCY SYSTEM ARCHITECTURE

Designing systems where microseconds matter (Trading, Real-time AdTech).

38.0.1 24.1 The Disruptor Pattern (C++ Implementation)

A high-performance inter-thread messaging library. Key concept: **Single-Writer Ring Buffer** with no locks.

```
1 template<typename T, size_t Size>
2 class Disruptor {
3     std::array<T, Size> ring_buffer;
4     alignas(64) std::atomic<int64_t> cursor{-1}; // Cache line padded
5
6 public:
7     template<typename F>
8     void publish(F&& factory) {
9         int64_t current = cursor.load(std::memory_order_relaxed);
10        int64_t next = current + 1;
11
12        // Write data (no contention for single writer)
13        factory(ring_buffer[next & (Size - 1)]);
14
15        // Commit
16        cursor.store(next, std::memory_order_release);
17    }
18
19    // Consumer tracks its own sequence...
20};
```

38.0.2 24.2 Kernel Bypass Networking (Concept)

Standard OS networking (interrupts, context switches) adds 10-50us latency. **Solution:** Map the NIC (Network Interface Card) directly to user-space memory (DPDK, Solarflare OpenOn-load).

- **Zero Copy:** Packet data goes from NIC -> CPU L3 Cache -> User Buffer.
- **Polling:** Instead of interrupts, one CPU core spins (`while(true)`) checking the NIC ring.

38.0.3 24.3 OS Tuning for C++

Your code is only as fast as the OS allows.

1. **CPU Isolation** (`isolcpus`): Isolate cores from the OS scheduler so your thread never gets preempted.
2. **Huge Pages**: Use 2MB or 1GB pages to reduce TLB (Translation Lookaside Buffer) misses. `cpp` `void* ptr = mmap(NULL, size, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_HUGETLB, -1, 0);`
3. **Disable C-States**: Prevent CPU from going to sleep (power save) which causes wake-up latency.

38.0.4 24.4 Zero-Copy Serialization (Cap'n Proto / FlatBuffers)

Avoid parsing JSON/XML. Access data directly from the binary buffer.

```
1 // FlatBuffers schema compiled to C++ header
2 // No parsing step! Pointers just point to the right offsets.
3 auto monster = GetMonster(buffer_pointer);
4 auto hp = monster->hp(); // Immediate access
5 auto pos = monster->pos();
```

38.0.5 24.5 LMAX Disruptor Internals

The key to Disruptor's speed is the **Sequence Barrier**.

1. **Cursor**: Monotonically increasing number (atomic).
2. **Barrier**: Consumers wait until `cursor >= my\_sequence`.
3. **Wait Strategy**:
 - `BusySpinWaitStrategy`: Loops while(`cursor < seq`). 100% CPU, 0ns latency.
 - `YieldingWaitStrategy`: Loops but calls `std::this_thread::yield()`.
 - `BlockingWaitStrategy`: Uses `std::condition_variable` (slowest).

Chapter 39

EXTREME LOW LATENCY & HARDWARE MASTERY

To achieve sub-microsecond latency, you must program the hardware, not just the language.

39.0.1 31.1 CPU Architecture & Cache Topology

- **L1 Cache:** ~32KB, 3-4 cycles. Per core.
- **L2 Cache:** ~256KB-1MB, 10-12 cycles. Per core.
- **L3 Cache:** ~10MB+, 40-70 cycles. Shared across cores.
- **RAM:** 100+ cycles.

Optimization Goal: Stay in L1/L2. **Technique:** Minimize object size, use contiguous memory (arrays), align data to cache lines (64 bytes).

39.0.2 31.2 NUMA (Non-Uniform Memory Access)

On multi-socket servers, accessing RAM attached to another CPU socket is slow. * **Solution:** Pin threads to cores. Allocate memory on the local node. * **Tool:** `numactl --cpunodebind=0 --membind=0 ./app`

39.0.3 31.3 Compiler Optimizations (The “Free Lunch”)

- `-O3`: Aggressive optimization.
- `-march=native`: Use instructions available on the build machine (AVX2, AVX-512).
- `-flto` (Link Time Optimization): Optimize across translation units (inlining across .cpp files).
- **PGO (Profile Guided Optimization):**
 1. Compile with `-fprofile-generate`.
 2. Run the app (training run).
 3. Recompile with `-fprofile-use`.

39.0.4 31.4 Lock-Free Stack Implementation (Wait-Free Push)

A classic interview and system component.

```

1  template<typename T>
2  struct Node {
3      T data;
4      Node* next;
5      Node(const T& d) : data(d), next(nullptr) {}
6  };
7
8  template<typename T>
9  class LockFreeStack {
10     std::atomic<Node<T>*> head{nullptr};
11
12 public:
13     void push(const T& data) {
14         Node<T>* new_node = new Node<T>(data);
15         new_node->next = head.load(std::memory_order_relaxed);
16
17         // CAS Loop
18         while (!head.compare_exchange_weak(
19             new_node->next,
20             new_node,
21             std::memory_order_release,
22             std::memory_order_relaxed));
23     }
24
25     bool pop(T& result) {
26         Node<T>* old_head = head.load(std::memory_order_acquire);
27
28         while (old_head && !head.compare_exchange_weak(
29             old_head,
30             old_head->next,
31             std::memory_order_acquire,
32             std::memory_order_relaxed));
33
34         if (!old_head) return false;
35
36         result = old_head->data;
37         // Note: Deletion in lock-free requires Hazard Pointers or RCU!
38         // Leaking here for simplicity of example.
39         return true;
40     }
41 };

```

39.0.5 31.5 Measurable Performance Targets

Define Service Level Objectives (SLOs) in percentiles. * **p50 (Median)**: Typical case. * **p99**: The “slow” case (1 in 100). * **p99.9**: The tail latency (1 in 1000). Crucial for HFT.

Example Target: “Order processing must have p99 latency < 5 microseconds.”

Chapter 40

ADVANCED SIMD (AVX2 & AVX-512)

Data Parallelism: Processing 8 or 16 numbers in a single CPU cycle.

40.0.1 32.1 SIMD Basics & Registers

- **SSE:** 128-bit (4 floats). XMM registers.
- **AVX2:** 256-bit (8 floats). YMM registers.
- **AVX-512:** 512-bit (16 floats). ZMM registers.

40.0.2 32.2 Intrinsics Example (Vector Addition)

Using <immintrin.h>.

```
1 #include <immintrin.h>
2
3 void add_avx2(float* a, float* b, float* c, int N) {
4     // Process 8 floats at a time
5     for (int i = 0; i < N; i += 8) {
6         // Load
7         __m256 va = _mm256_loadu_ps(&a[i]);
8         __m256 vb = _mm256_loadu_ps(&b[i]);
9
10        // Operation
11        __m256 vc = _mm256_add_ps(va, vb);
12
13        // Store
14        _mm256_storeu_ps(&c[i], vc);
15    }
16 }
```

- `_mm256_loadu_ps`: Load Unaligned Packed Single-precision.
- `_mm256_add_ps`: Add packed singles.

40.0.3 32.3 Measurable Outcome

- **Objective:** Convert a scalar loop to AVX2.
- **Success Metric:** 4x-8x speedup on large arrays (memory bandwidth permitting).

Chapter 41

CUSTOM MEMORY ALLOCATORS

`malloc` and `new` are general-purpose and slow (locks, fragmentation). Real-time systems use custom allocators.

41.0.1 33.1 Linear Allocator (Arena)

The absolute fastest allocator. $O(1)$. Zero overhead.

```
1 class LinearAllocator {
2     char* start;
3     char* current;
4     size_t size;
5 public:
6     LinearAllocator(size_t s) : size(s) {
7         start = new char[s];
8         current = start;
9     }
10
11     void* allocate(size_t n) {
12         if (current + n > start + size) return nullptr;
13         void* ptr = current;
14         current += n;
15         return ptr;
16     }
17
18     void reset() { current = start; } // Free ALL at once
19 };
```

- Use Case: Per-frame game memory, Request-scoped web server memory.

41.0.2 33.2 Pool Allocator

Fixed-size blocks. No external fragmentation. $O(1)$ `malloc/free`.

```
1 struct Chunk { Chunk* next; };
2
3 class PoolAllocator {
4     Chunk* head = nullptr;
5 public:
6     void* allocate() {
7         if (!head) return ::operator new(sizeof(Chunk)); // Or expand pool
8         Chunk* ptr = head;
9         head = head->next;
10        return ptr;
11    }
```

```
11     }
12
13     void deallocate(void* ptr) {
14         Chunk* chunk = static_cast<Chunk*>(ptr);
15         chunk->next = head;
16         head = chunk;
17     }
18 };
```

Chapter 42

APPENDICES

Chapter 43

Appendix A: C++ Keywords & Operators Reference

43.0.1 Essential Keywords (Non-Exhaustive)

- **alignas** / **alignof**: Memory alignment queries and specifications.
- **asm**: Inline assembly block.
- **auto**: Type deduction (C++11).
- **const** / **volatile**: cv-qualifiers for type safety and hardware access.
- **constexpr** / **consteval** / **constinit**: Compile-time constant specifications.
- **decltype**: Inspect declared type of an entity.
- **explicit**: Prevent implicit conversions in constructors.
- **export**: Module interface export (C++20).
- **friend**: Allow access to private members.
- **inline**: Suggest inlining to compiler; allow definition in header.
- **mutable**: Allow modification of member in const object.
- **noexcept**: Specifier for functions that don't throw.
- **nullptr**: Null pointer literal (C++11).
- **operator**: Overload operators.
- **requires**: Constraint clause for Concepts (C++20).
- **static_assert**: Compile-time assertion.
- **template**: Define generic classes/functions.
- **thread_local**: Storage duration specifier.
- **typeid**: Runtime type identification (RTTI).
- **typename**: Declare a type parameter in templates.
- **virtual**: Declare virtual function for polymorphism.

43.0.2 Special Operators

- `::` Scope resolution
 - `->*` Pointer to member selection
 - `<=>` Three-way comparison (Spaceship) (C++20)
 - `co\_await`, `co\_yield`, `co\_return` Coroutine operators (C++20)
-

Chapter 44

Appendix B: Common Acronyms

- **ABI**: Application Binary Interface.
 - **API**: Application Programming Interface.
 - **COW**: Copy On Write.
 - **CRTP**: Curiously Recurring Template Pattern.
 - **CTAD**: Class Template Argument Deduction.
 - **UB**: Undefined Behavior (Avoid at all costs!).
 - **IB**: Implementation-defined Behavior.
 - **IIFE**: Immediately Invoked Function Expression (often with lambdas).
 - **NrvO** / **RVO**: (Named) Return Value Optimization.
 - **ODR**: One Definition Rule.
 - **PIMPL**: Pointer to Implementation (Opaque Pointer).
 - **RAII**: Resource Acquisition Is Initialization.
 - **RTTI**: Run-Time Type Information.
 - **SFINAE**: Substitution Failure Is Not An Error.
 - **SOO** / **SSO**: Small Object/String Optimization.
 - **STL**: Standard Template Library.
 - **TMP**: Template Metaprogramming.
 - **TU**: Translation Unit.
-

Chapter 45

Appendix C: Recommended Tooling

45.0.1 Compilers

- **GCC (GNU Compiler Collection)**: Standard on Linux.
- **Clang/LLVM**: Excellent error messages, widely used on macOS/Linux.
- **MSVC (Microsoft Visual C++)**: Standard on Windows.

45.0.2 Build Systems

- **CMake**: The industry standard meta-build system.
- **Meson**: Modern, fast, Python-based.
- **Bazel**: Google's build system, good for monorepos.

45.0.3 Package Managers

- **Conan**: Decentralized package manager for C/C++.
- **vcpkg**: Microsoft's C++ library manager.

45.0.4 Static Analysis & Sanitizers

- **AddressSanitizer (ASan)**: Detects memory errors (buffer overflows, use-after-free).
 - **UndefinedBehaviorSanitizer (UBSan)**: Detects undefined behavior.
 - **ThreadSanitizer (TSan)**: Detects data races.
 - **Clang-Tidy**: Linter and static analysis tool.
 - **Cppcheck**: Static analysis tool.
-

Chapter 46

Appendix D: Common C++ Traps & Pitfalls

46.0.1 I. General & Syntax Traps

1. Most Vexing Parse

- *Issue:* `MyClass obj();` declares a function returning `MyClass`, not a default-constructed object.
- *Fix:* Use brace initialization: `MyClass obj{\{\}};`.

2. The “dangling else” Problem

- *Issue:* Nested `if-else` without braces can associate `else` with the wrong `if`.
- *Fix:* Always use braces `\{\}` for control structures.

3. Integer Division

- *Issue:* `1/2` results in 0 (integer), not 0.5.
- *Fix:* Cast one operand to float/double: `1.0/2` or `static\_cast<double>(1)/2`.

4. Loop Variable Type Mismatch

- *Issue:* `for (unsigned i = v.size() - 1; i >= 0; --i)` causes an infinite loop because `unsigned` is never negative.
- *Fix:* Use `int` (and cast size) or standard iterators/ranges.

5. Shadowing Variables

- *Issue:* Declaring a local variable with the same name as a member or outer variable hides the outer one.
- *Fix:* Enable compiler warnings (`-Wshadow`) and use `this->member` if necessary.

46.0.2 II. Pointers, References & Memory

6. Object Slicing

- *Issue:* Assigning a `Derived` object to a `Base` value slices off the derived part.
- *Fix:* Use pointers `Base*` or references `Base\&` for polymorphism.

7. Dangling References

- *Issue*: Returning a reference to a local stack variable.
- *Fix*: Return by value or use smart pointers/dynamic allocation.

8. Iterator Invalidation

- *Issue*: Adding elements to a `std::vector` may reallocate memory, invalidating all pointers/iterators to elements.
- *Fix*: Don't cache iterators across mutating operations; use `reserve()` if possible.

9. `delete` VS `delete[]`

- *Issue*: Mismatching `new` with `delete[]` or `new[]` with `delete` causes undefined behavior.
- *Fix*: Use `std::vector` or `std::unique_ptr` instead of manual management.

10. Use-After-Move

- *Issue*: Accessing an object after `std::move()` (except for reassignment/destruction).
- *Fix*: Treat moved-from objects as empty; do not read their state.

46.0.3 III. Classes & OOP

11. Virtual Destructor Missing

- *Issue*: Deleting a derived class via a base pointer when the base destructor is not `virtual` leaks derived resources.
- *Fix*: Always mark base class destructors `virtual` (or `protected` if non-polymorphic).

12. Calling Virtual Functions in Constructor/Destructor

- *Issue*: Calls the *base* class version, not the derived one, because the derived part isn't initialized/is already destroyed.
- *Fix*: Use two-phase initialization or factory methods.

13. Copy Constructor/Assignment Missing

- *Issue*: Classes managing raw pointers will default to shallow copy (double free error).
- *Fix*: Follow the **Rule of Three/Five/Zero**.

14. Initialization Order

- *Issue*: Members are initialized in *declaration order*, not initializer list order.
- *Fix*: Keep initializer list order identical to member declaration order to avoid warnings.

46.0.4 IV. Concurrency

15. Data Races

- *Issue:* Multiple threads accessing shared memory without synchronization (at least one writer).
- *Fix:* Use `std::mutex`, `std::atomic`, or `std::shared_mutex`.

16. Deadlocks

- *Issue:* Two threads waiting on each other's locks.
- *Fix:* Acquire locks in a consistent global order; use `std::scoped_lock` (C++17) to lock multiple mutexes safely.

17. False Sharing

- *Issue:* Independent atomic variables on the same cache line degrade performance due to cache coherency protocols.
- *Fix:* Use `alignas(hardware_destructive_interference_size)` to pad variables.

46.0.5 V. Modern C++ & Macros

18. `std::vector<bool>` Weirdness

- *Issue:* It's a template specialization (bitfield), not a vector of bools. Returns a proxy object, not `bool&`.
- *Fix:* Use `std::deque<bool>` or `std::vector<char>` if you need real references.

19. Auto Type Deduction

- *Issue:* `auto` drops references and `const`.
- *Fix:* Use `auto&` or `const auto&` explicitly when needed.

20. Macro Side Effects

- *Issue:* `#define MAX(a,b) ((a) > (b) ? (a) : (b))` evaluates arguments twice. `MAX(x++, y)` increments `x` twice.
- *Fix:* Use `inline` functions or templates instead of macros.

21. Static Initialization Order Fiasco

- *Issue:* Global objects in different files have undefined initialization order.
 - *Fix:* Use the “Construct On First Use” idiom (Meyers Singleton).
-

Chapter 47

Appendix H: Professional C++ Idioms

47.0.1 1. RAII (Resource Acquisition Is Initialization)

- **Concept:** Bind resource lifecycle to object lifecycle. Constructor acquires, destructor releases.
- **Use Case:** Memory, file handles, mutex locks, sockets.
- **Example:** `std::lock_guard`, `std::unique_ptr`.

47.0.2 2. Pimpl (Pointer to Implementation)

- **Concept:** Hide private members in a separate class/struct, accessed via a pointer.
- **Benefit:** ABI stability, reduced compilation times (header dependency changes don't trigger rebuilds of clients).
- **Pattern:**

```
cpp      class Widget \{          struct Impl;          std::unique_ptr<Impl> pImpl
;      public:          Widget();          ~Widget(); // Defined in .cpp where Impl is visible
          \};
```

47.0.3 3. Copy-and-Swap

- **Concept:** Implement assignment operator in terms of copy constructor and swap.
- **Benefit:** Strong Exception Safety guarantee; removes code duplication.
- **Pattern:**

```
cpp      T& operator=(T other) \{ // Pass by value (copy)          swap(*this, other
);          return *this;          \}
```

47.0.4 4. NVI (Non-Virtual Interface)

- **Concept:** Public interface is non-virtual; virtual functions are private/protected.
- **Benefit:** Separation of interface (pre/post-conditions) from implementation.
- **Pattern:**

```
cpp      class Base \{      public:          void doWork() \{          // Pre-condition
      logic          doWorkImpl();          // Post-condition logic          \}      private
:          virtual void doWorkImpl() = 0;          \};
```

47.0.5 5. Erase-Remove Idiom

- **Concept:** Standard way to remove elements from a `std::vector` (before C++20 `std::erase`).
- **Pattern:** `v.erase(std::remove(v.begin(), v.end(), value), v.end());`

47.0.6 6. SFINAE (Substitution Failure Is Not An Error)

- **Concept:** Remove functions from overload resolution set if types don't match constraints.
- **Modern Replacement:** C++20 Concepts (`requires`).

47.0.7 7. CRTP (Curiously Recurring Template Pattern)

- **Concept:** Class `Derived` inherits from `Base<Derived>`.
 - **Use Case:** Static polymorphism (compile-time), adding functionality (mixins) without vtable overhead.
 - **Example:** `std::enable_shared_from_this`.
-

Chapter 48

Appendix E: C++ Interview Cheat Sheet

48.0.1 Core Concepts

1. **Virtual Functions:** Enable runtime polymorphism via `vtable/vptr`. Destructors must be `virtual` in base classes.
2. **Smart Pointers:**
 - `unique_ptr`: Exclusive ownership, no overhead.
 - `shared_ptr`: Shared ownership, ref-counted (atomic), control block overhead.
 - `weak_ptr`: Non-owning reference to `shared_ptr` (breaks cycles).
3. **Move Semantics:** Transfers resources (pointers) instead of deep copying. Enabled by rvalue references (`&&`) and `std::move`.
4. **RAII:** Resource Acquisition Is Initialization. Constructor acquires, destructor releases. Core to C++ safety.
5. **Cast Types:**
 - `static_cast`: Compile-time safe conversions.
 - `dynamic_cast`: Runtime checked downcasting (requires RTTI).
 - `reinterpret_cast`: Bitwise reinterpretation (unsafe).
 - `const_cast`: Remove/add constness.

48.0.2 Modern C++ (C++11/14/17/20)

1. **Lambdas:** Anonymous function objects. Capture `[=]`, `[&]`, or move-only `[x = std::move(y)]`.
2. **Auto:** Type deduction. Always initialize.
3. **Structured Bindings (C++17):** `auto [x, y] = pair;`
4. **Concepts (C++20):** Constrain templates for better errors/readability.
5. **Coroutines (C++20):** Functions that can suspend/resume.

48.0.3 System Design Questions

1. **Vector vs List:** Vector (contiguous, cache-friendly) is almost always better than List (node-based, cache misses) unless aggressive splicing is needed.
2. **Map vs Unordered Map:** Map (BST, $O(\log n)$, sorted) vs Unordered Map (Hash Table, $O(1)$ avg, unsorted).
3. **Handling 1M connections:** Use non-blocking I/O (epoll/kqueue) or `io_uring`, not one thread per connection.
4. **Memory Layout:** Stack (local vars) vs Heap (dynamic) vs Data (globals) vs Text (code).

48.0.4 Quick Coding

- **Implement Singleton:** Use static local variable (Thread-safe in C++11+).
 - **Implement String Class:** Handle deep copy, move semantics, and destructor.
 - **Reverse Linked List:** Classic pointer manipulation.
-

Chapter 49

Appendix F: The C++ Standard Evolution Matrix

49.0.1 1. Versioned Changelog

C++98 (ISO/IEC 14882:1998) - *The Foundation*

Released: 1998 * **Core:** Templates, Exceptions, Namespaces, bool type, cast operators (`static_cast`, etc.), `mutable`, `explicit`. * **STL:** Containers (`vector`, `list`, `map`, `set`, `deque`), Algorithms (`sort`, `find`, `transform`), Iterators, Strings (`std::string`), I/O Streams (`iostream`). * **Memory:** `std::auto_ptr` (Deprecated in C++11).

C++03 (ISO/IEC 14882:2003) - *The Bug Fix*

Released: 2003 * **Focus:** Defect Report (DR) fixes for C++98 to ensure consistency across compilers. * **Features:** Value initialization `T()`, fixes to `std::vector` contiguous memory guarantee.

C++11 (ISO/IEC 14882:2011) - *The Modern Revolution*

Released: September 2011 * **Language:** `auto`, `nullptr`, Range-based `for`, Lambda expressions, Rvalue references (`&&`) & Move semantics, Variadic templates, `constexpr` (limited), `decltype`, Uniform initialization `{}`, `static_assert`, `override`, `final`, `enum class`. * **Concurrency:** `std::thread`, `std::mutex`, `std::atomic`, `std::future`, `std::async`. * **Library:** Smart pointers (`unique_ptr`, `shared_ptr`, `weak_ptr`), `std::array`, `std::tuple`, `std::unordered_map/set`, `std::regex`, `std::chrono`.

C++14 (ISO/IEC 14882:2014) - *The Refinement*

Released: December 2014 * **Language:** Generic lambdas (auto params), Relaxed `constexpr` (loops/variables allowed), Binary literals (`0b1010`), Digit separators (`1'000`), Variable templates, Return type deduction. * **Library:** `std::make_unique`, `std::shared_timed_mutex`, `std::integer_sequence`, `std::exchange`, `std::quoted`.

C++17 (ISO/IEC 14882:2017) - *The Major Update*

Released: December 2017 * **Language:** Structured bindings `auto [x,y] = p;`, `if constexpr`, Fold expressions (`... + args`), Class Template Argument Deduction (CTAD), Inline variables, `__has_include`. * **Library:** `std::filesystem`, `std::optional`, `std::variant`, `std::any`, `std::string_view`, Parallel Algorithms (`std::execution::par`), `std::invoke`, `std::byte`, `std::pmr` (Polymorphic Memory Resources).

C++20 (ISO/IEC 14882:2020) - *The Gigantic Leap*

Released: December 2020 * **Language:** Concepts (Constraints), Modules (`import/export`), Coroutines (`co_await`), Three-way comparison (`<=>`), Designated initializers `{.x=1}`, `constexpr` (Immediate functions), `constexpr`, Range-based `for` with `init`. * **Library:** Ranges (`std::ranges`), `std::span`, `std::format`, `std::jthread`, `std::stop_token`, `std::barrier`, `std::latch`, `std::semaphore`, `std::bit_cast`, `std::source_location`, Calendars & Timezones.

C++23 (ISO/IEC 14882:2023) - *The Completion*

Released: October 2023 * **Language:** Deducing `this` (Explicit object parameter), `if constexpr`, Multidimensional subscript `m[1,2]`, Static `operator()`, `auto(x)` decay copy. * **Library:** `std::print`, `std::println`, `std::expected` (Error handling), `std::mdspan`, `std::flat_map`, `std::flat_set`, `std::generator` (Synchronous coroutines), `std::stacktrace`, `std::stdatomic.h`.

49.0.2 2. Feature Matrix

Feature	C++98	C++11	C++14	C++17	C++20	C++23
Memory Variables	<code>auto_ptr</code> Type req.	<code>unique_ptr</code> <code>auto</code>	<code>make_unique</code> Var templates	<code>pmr</code> Structured Bindings	<code>shared_ptr</code> <code>atomic</code> <code>constexpr</code>	<code>weak_ptr</code> <code>weak_ptr_of</code> <code>weak_ptr_of</code>
Loops	<code>for(;;)</code>	Range- <code>for</code>	-	-	Range- <code>for</code> <code>init</code>	Range- <code>for</code> <code>init</code>
Templates	Basic	Variadic	Variable	Fold Expressions	Concepts	Concepts
Lambdas	-	Basic	Generic	<code>constexpr</code>	Template	Template
Concurrency	-	<code>thread</code>	<code>shared_lock</code>	Parallel Algos	<code>jthread</code> , Latches	<code>jthread</code> , Latches
String	<code>string</code>	<code>to_string</code>	<code>quoted</code>	<code>string_view</code>	<code>format</code>	<code>format</code>
Metaprog.	Traits	<code>static_assert</code>	<code>integer_seq</code>	<code>if constexpr</code>	<code>constexpr</code>	<code>constexpr</code>
Modules	-	-	-	-	Modules	Modules
Coroutines	-	-	-	-	Async	Async

49.0.3 3. Timeline & Release Accuracy

Standard	ISO Publication	Codename	Compiler Flag (GCC/Clang)
C++98	1998-09	C++98	<code>-std=c++98</code>
C++03	2003-10	C++03	<code>-std=c++03</code>
C++11	2011-09	C++0x	<code>-std=c++11</code>
C++14	2014-12	C++1y	<code>-std=c++14</code>
C++17	2017-12	C++1z	<code>-std=c++17</code>
C++20	2020-12	C++2a	<code>-std=c++20</code>
C++23	2023-10	C++2b	<code>-std=c++23</code>
C++26	<i>Expected 2026</i>	C++2c	<code>-std=c++26</code> / <code>-std=c++2c</code>

Chapter 50

Appendix G: C++ Standard Library Headers Reference

50.0.1 Concepts & Utilities

- `<concepts>` (C++20): Fundamental concepts library.
- `<coroutine>` (C++20): Coroutine support library.
- `<functional>`: Function objects, binder, and reference wrappers.
- `<memory>`: Smart pointers and allocators.
- `<tuple>`: Tuple library.
- `<type_traits>`: Compile-time type information.
- `<utility>`: Utility components (`std::pair`, `std::move`).

50.0.2 Containers

- `<array>` (C++11): Fixed-size array class.
- `<deque>`: Double-ended queue.
- `<list>`: Doubly-linked list.
- `<map>`: Associative containers (Red-Black Tree).
- `<queue>`: Queue adapter.
- `<set>`: Associative containers (Red-Black Tree).
- `<stack>`: Stack adapter.
- `<unordered_map>` (C++11): Hash map.
- `<vector>`: Dynamic array.
- `<span >` (C++20): Non-owning view of contiguous memory.

50.0.3 Algorithms & Iterators

- `<algorithm>`: Algorithms that operate on ranges.
- `<execution>` (C++17): Parallel algorithms.
- `<iterator>`: Iterator primitives.
- `<numeric>`: Numeric operations (`accumulate`, `reduce`).
- `<ranges>` (C++20): Range primitives and views.

50.0.4 Concurrency

- `<atomic>` (C++11): Atomic operations.
- `<barrier>` (C++20): Barriers.
- `<condition_variable>` (C++11): Condition variables.
- `<future>` (C++11): Futures and promises.
- `<latch>` (C++20): Latches.
- `<mutex>` (C++11): Mutual exclusion primitives.
- `<semaphore>` (C++20): Semaphores.
- `<shared_mutex>` (C++14): Shared mutexes.
- `<thread>` (C++11): Thread class.

50.0.5 Input/Output

- `<filesystem>` (C++17): File system operations.
- `<format>` (C++20): Formatting library.
- `<fstream>`: File stream classes.
- `<iostream>`: Standard I/O stream objects.
- `<print>` (C++23): Print functions.
- `<sstream>`: String stream classes.

50.0.6 Numerics & Math

- `<bit>` (C++20): Bit manipulation.
- `<complex>`: Complex number arithmetic.
- `<random>` (C++11): Random number generation.
- `<ratio>` (C++11): Compile-time rational arithmetic.
- `<valarray>`: Class for representing and manipulating arrays of values.
- `<numbers>` (C++20): Mathematical constants.

Chapter 51

C++ UNDER THE HOOD

To truly master C++, you must understand what the compiler generates.

51.0.1 14.1 Object Layout & ABI (Itanium C++ ABI)

How does virtual work?

```
1 class Base {
2     int64_t id;
3 public:
4     virtual void func() {}
5 };
6
7 class Derived : public Base {
8     int64_t data;
9 public:
10    void func() override {}
11};
```

Memory Layout (64-bit system):

```
1 [ vptr (8 bytes) ] -> [ vtable for Base ]
2 [ id (8 bytes) ]
```

For Derived:

```
1 [ vptr (8 bytes) ] -> [ vtable for Derived ]
2 [ id (8 bytes) ]
3 [ data (8 bytes) ]
```

- **vptr**: Hidden pointer added to classes with virtual functions.
- **vtable**: Static table of function pointers.
- **Alignment**: Data is padded to align with word boundaries.

51.0.2 14.2 Small String Optimization (SSO)

`std::string` doesn't always allocate heap memory.

```
1 std::string s = "Hello"; // 5 chars
2 // Layout typically (24-32 bytes):
3 // [ size (8) ] [ capacity (8) ] [ pointer (8) ] <-- Normal mode
4 // [ size (1) ] [ ... chars 22 bytes ... ] <-- SSO mode (Union)
```

Strings shorter than 15-22 chars (depending on libc++) live entirely on the stack.

51.0.3 14.3 Return Value Optimization (RVO)

Copy elision is mandatory in C++17.

```
1 struct BigObject { int data[1000]; };
2
3 BigObject create() {
4     BigObject obj;
5     // ... fill obj ...
6     return obj; // No copy, no move. Constructed directly in caller's stack
7                 frame.
8 }
9 BigObject x = create();
```

Chapter 52

MASTERING THE MEMORY MODEL

The C++ Memory Model defines how threads interact through memory.

52.0.1 15.1 Atomicity vs Ordering

- **Atomicity:** An operation is indivisible (all or nothing).
- **Ordering:** The order in which operations are observed by other threads.

`std::atomic<int>` guarantees atomicity, but `memory_order` controls ordering.

52.0.2 15.2 Memory Orders Deep Dive

1. **memory_order_relaxed:** No ordering constraints. Only atomicity.

- Use for: Incrementing stats counters.

```
1 cnt.fetch_add(1, std::memory_order_relaxed);
```

2. **memory_order_acquire:** Read operation.

- Guarantee: No reads/writes in the current thread can be reordered *before* this load.
- Use with: Release.

3. **memory_order_release:** Write operation.

- Guarantee: No reads/writes in the current thread can be reordered *after* this store.
- Use for: Publishing data.

4. **memory_order_seq_cst** (Default): Sequentially Consistent.

- Guarantee: A total global ordering exists. Expensive.

52.0.3 15.3 The Happens-Before Relationship

If Operation A *happens-before* Operation B: 1. A is sequenced before B (same thread). 2. A *synchronizes-with* B (inter-thread, e.g., A releases, B acquires).

Example: Lock-Free Flag

```
1 std::atomic<int> data = 0;
2 std::atomic<bool> ready = false;
3
4 void producer() {
5     data.store(42, std::memory_order_relaxed);
6     ready.store(true, std::memory_order_release); // "Publish"
7 }
8
9 void consumer() {
10    while (!ready.load(std::memory_order_acquire)); // "Acquire"
11    assert(data.load(std::memory_order_relaxed) == 42); // Guaranteed 42
12 }
```

Chapter 53

WRITING A C++ COMPILER (BASICS)

To understand C++, build a toy compiler.

53.0.1 18.1 Lexical Analysis (Tokenizer)

Converting source code into tokens.

```
1 enum class TokenType { Int, Identifier, Plus, Minus, End };
2
3 struct Token {
4     TokenType type;
5     std::string text;
6 };
7
8 std::vector<Token> tokenize(std::string_view source) {
9     std::vector<Token> tokens;
10    // ... implementation ...
11    return tokens;
12 }
```

53.0.2 18.2 Parsing (Recursive Descent)

Building an Abstract Syntax Tree (AST).

```
1 struct ASTNode { virtual ~ASTNode() = default; };
2 struct BinaryExpr : ASTNode {
3     std::unique_ptr<ASTNode> left, right;
4     char op;
5 };
6
7 // parseExpression() calls parseTerm(), etc.
```

53.0.3 18.3 Semantic Analysis (Types & Scopes)

Before generating code, we must validate it.

Symbol Table:

```
1 struct Symbol { string type; };
2 using Scope = map<string, Symbol>;
3 vector<Scope> scopes; // Stack of scopes
4
5 void enter_scope() { scopes.push_back({}); }
6 void exit_scope() { scopes.pop_back(); }
```

Type Checking: Recursively visit the AST. * `BinaryExpr`: Check `left.type == right.type`.
* `Variable`: Check if exists in symbol table.

Chapter 54

WRITING A GARBAGE COLLECTOR

C++ has RAI, but implementing a GC teaches you about the stack and object graph.

54.0.1 29.1 Mark-and-Sweep Basics

1. **Roots:** Pointers on the stack/globals.
2. **Mark:** Traverse object graph from roots, marking reachable objects.
3. **Sweep:** Iterate heap, free unmarked objects.

```
1 struct GCObject {
2     bool marked = false;
3     virtual ~GCObject() = default;
4 };
5
6 class VM {
7     std::vector<GCObject*> heap;
8     std::vector<GCObject*> roots; // Pointers currently on stack
9
10 public:
11     void mark() {
12         for (auto* obj : roots) mark_object(obj);
13     }
14
15     void mark_object(GCObject* obj) {
16         if (!obj || obj->marked) return;
17         obj->marked = true;
18         // ... traverse children ...
19     }
20
21     void sweep() {
22         auto it = std::remove_if(heap.begin(), heap.end(), [](GCObject* obj) {
23             if (!obj->marked) {
24                 delete obj;
25                 return true;
26             }
27             obj->marked = false; // Reset for next cycle
28             return false;
29         });
30         heap.erase(it, heap.end());
31     }
32 };
```


Chapter 55

THE STANDARD LIBRARY FROM SCRATCH

Implementing core STL components to understand their cost.

55.0.1 19.1 Implementing my::vector

Managing raw memory, growth, and construction.

```
1 template<typename T>
2 class Vector {
3     T* data = nullptr;
4     size_t sz = 0;
5     size_t cap = 0;
6
7 public:
8     void push_back(const T& val) {
9         if (sz == cap) {
10             reallocate(cap == 0 ? 1 : cap * 2);
11         }
12         new (data + sz) T(val); // Placement new
13         sz++;
14     }
15
16 private:
17     void reallocate(size_t new_cap) {
18         T* new_data = static_cast<T*>(::operator new(new_cap * sizeof(T)));
19         // Move old elements...
20         // Delete old memory...
21         data = new_data;
22         cap = new_cap;
23     }
24 };
```

55.0.2 19.2 Implementing my::shared_ptr

Understanding the Control Block.

```
1 template<typename T>
2 class SharedPtr {
3     T* ptr;
4     struct ControlBlock {
5         std::atomic<int> ref_count{1};
6     } *cb;
7
8 public:
```

```
9      SharedPtr(T* p) : ptr(p), cb(new ControlBlock()) {}
10
11      SharedPtr(const SharedPtr& other) {
12          ptr = other.ptr;
13          cb = other.cb;
14          if (cb) cb->ref_count++;
15      }
16
17      ~SharedPtr() {
18          if (cb && --cb->ref_count == 0) {
19              delete ptr;
20              delete cb;
21          }
22      }
23  };
```

Part VIII

Volume VIII: Specialized Domains & Expert Mastery

Chapter 56

DISTRIBUTED C++

Moving beyond a single process: Networking, RPC, and Consensus.

56.0.1 16.1 Serialization (Binary Protocols)

Efficiently packing data for network transmission.

```
1 #include <vector>
2 #include <cstring>
3 #include <string>
4
5 // Simple Binary Serializer
6 class Buffer {
7     std::vector<uint8_t> data;
8 public:
9     template<typename T>
10    void write(const T& val) {
11        static_assert(std::is_trivially_copyable_v<T>);
12        const uint8_t* ptr = reinterpret_cast<const uint8_t*>(&val);
13        data.insert(data.end(), ptr, ptr + sizeof(T));
14    }
15
16    void write_string(const std::string& s) {
17        write<uint32_t>(s.size());
18        const uint8_t* ptr = reinterpret_cast<const uint8_t*>(s.data());
19        data.insert(data.end(), ptr, ptr + s.size());
20    }
21
22    const uint8_t* begin() const { return data.data(); }
23    size_t size() const { return data.size(); }
24 };
```

56.0.2 16.2 RPC (Remote Procedure Call) Concept

Calling a function on another machine.

Stub Interface:

```
1 // User Code
2 // auto result = service.Add(5, 3);
3
4 // Generated Stub
5 int Add(int a, int b) {
6     Buffer buf;
7     buf.write(101); // Function ID for 'Add'
8     buf.write(a);
9     buf.write(b);
10    return network.send_and_wait(buf); // Blocks
}
```

```
11 }
```

56.0.3 16.3 Consensus (Raft Basics)

Distributed systems need to agree on state.

Raft State Machine:

```
1 enum class State { Follower, Candidate, Leader };
2
3 struct Node {
4     State state = State::Follower;
5     int current_term = 0;
6     int voted_for = -1;
7
8     void on_timeout() {
9         if (state == State::Follower) {
10             state = State::Candidate;
11             current_term++;
12             voted_for = my_id;
13             request_votes();
14         }
15     }
16 };
```

Chapter 57

NETWORKING FROM SCRATCH

Understanding `asio` requires understanding BSD Sockets.

57.0.1 28.1 Berkeley Sockets API

The foundation of the Internet.

```
1 #include <sys/socket.h>
2 #include <netinet/in.h>
3 #include <unistd.h>
4
5 int main() {
6     int server_fd = socket(AF_INET, SOCK_STREAM, 0);
7
8     sockaddr_in address;
9     address.sin_family = AF_INET;
10    address.sin_addr.s_addr = INADDR_ANY;
11    address.sin_port = htons(8080);
12
13    bind(server_fd, (struct sockaddr*)&address, sizeof(address));
14    listen(server_fd, 3);
15
16    int new_socket = accept(server_fd, nullptr, nullptr);
17    char buffer[1024] = {0};
18    read(new_socket, buffer, 1024);
19
20    // Send HTTP response
21    const char* hello = "HTTP/1.1 200 OK\nContent-Type: text/plain\n\nHello!";
22    write(new_socket, hello, strlen(hello));
23
24    close(new_socket);
25    close(server_fd);
26    return 0;
27 }
```

57.0.2 28.2 Non-Blocking I/O & Epoll (Linux)

How Nginx/Node.js handle 10k connections.

```
1 // 1. Create epoll instance
2 int epoll_fd = epoll_create1(0);
3
4 // 2. Add server socket
5 epoll_event event;
6 event.events = EPOLLIN; // Read available
7 event.data.fd = server_fd;
8 epoll_ctl(epoll_fd, EPOLL_CTL_ADD, server_fd, &event);
```

```
9
10 // 3. Event Loop
11 while (true) {
12     epoll_event events[10];
13     int event_count = epoll_wait(epoll_fd, events, 10, -1);
14     for (int i = 0; i < event_count; i++) {
15         if (events[i].data.fd == server_fd) {
16             // Accept new connection...
17         } else {
18             // Read data...
19         }
20     }
21 }
```

Chapter 58

C++ IN THE CLOUD

Modern C++ is a first-class citizen in Cloud Native architectures.

58.0.1 20.1 Microservices with C++

Using frameworks like **Drogon** or **Userver** (Yandex) for high-throughput services.

Example: Simple HTTP Endpoint (Drogon style)

```
1 // Controller
2 void Handler::get(const HttpRequestPtr& req, std::function<void (const
  HttpResponsePtr &)> &&callback) {
3     auto resp = HttpResponse::newHttpResponse();
4     resp->setBody("Hello from High-Performance Microservice!");
5     callback(resp);
6 }
```

58.0.2 20.2 Serverless C++ (AWS Lambda)

Using the AWS Lambda C++ Runtime to run native binaries. * **Cold Start:** < 5ms (vs 100ms+ for Java/Node). * **Cost:** Lower duration due to speed.

```
1 #include <aws/lambda-runtime/runtime.h>
2
3 aws::lambda_runtime::invocation_response handler(aws::lambda_runtime::
  invocation_request const& req) {
4     return aws::lambda_runtime::invocation_response::success("Processed!", "
  application/json");
5 }
6
7 int main() {
8     aws::lambda_runtime::run_handler(handler);
9     return 0;
10 }
```

Chapter 59

CROSS-PLATFORM DEVELOPMENT

Write once, run everywhere (Desktop, Web, Mobile).

59.0.1 21.1 WebAssembly (Wasm) with Emscripten

Compiling C++ to run in the browser.

```
1 emcc main.cpp -o index.html -s WASM=1
```

```
1 #include <emscripten/emscripten.h>
2
3 extern "C" {
4     EMSCRIPTEN_KEEPALIVE
5     int add(int a, int b) {
6         return a + b; // callable from JavaScript
7     }
8 }
```

59.0.2 21.2 Mobile C++ (Android NDK & JNI)

Integrating C++ with Java/Kotlin.

```
1 #include <jni.h>
2
3 extern "C" JNIEXPORT jstring JNICALL
4 Java_com_example_myapp_MainActivity_stringFromJNI(JNIEnv* env, jobject /* this
5     */) {
6     return env->NewStringUTF("Hello from C++");
7 }
```

Chapter 60

GUI DEVELOPMENT WITH C++

Building desktop applications and tools.

60.0.1 22.1 Qt Framework (Retained Mode)

Qt uses a unique Signal/Slot mechanism (via MOC - Meta-Object Compiler).

```
1 // MainWindow.h
2 class MainWindow : public QMainWindow {
3     Q_OBJECT // Macro for MOC
4 public:
5     MainWindow(QWidget *parent = nullptr);
6 public slots:
7     void handleButton(); // Slot
8 };
9
10 // MainWindow.cpp
11 MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
12     QPushButton *button = new QPushButton("Click me", this);
13     connect(button, &QPushButton::clicked, this, &MainWindow::handleButton);
14 }
```

60.0.2 22.2 Dear ImGui (Immediate Mode)

Ideal for game engines and internal tools. Re-renders UI every frame.

```
1 // Main Loop
2 void Render() {
3     ImGui::Begin("Debug Tools");
4     static float col[3] = { 0.0f, 0.0f, 0.0f };
5     ImGui::ColorEdit3("Background Color", col);
6     if (ImGui::Button("Reset")) {
7         col[0] = col[1] = col[2] = 0.0f;
8     }
9     ImGui::End();
10 }
```

Chapter 61

SCIENTIFIC COMPUTING & GPU

C++ is the language of high-performance math.

61.0.1 23.1 Eigen (Linear Algebra)

Template-heavy library that avoids temporaries using Expression Templates.

```
1 #include <Eigen/Dense>
2 using Eigen::MatrixXd;
3
4 void solve_system() {
5     MatrixXd A(3, 3);
6     A << 1, 2, 3,
7         4, 5, 6,
8         7, 8, 10;
9
10    Eigen::VectorXd b(3);
11    b << 3, 3, 4;
12
13    Eigen::VectorXd x = A.colPivHouseholderQr().solve(b);
14 }
```

61.0.2 23.2 CUDA (GPU Programming)

Running C++ directly on NVIDIA GPUs.

```
1 // Kernel (runs on GPU)
2 __global__ void vectorAdd(float* A, float* B, float* C, int N) {
3     int i = blockDim.x * blockIdx.x + threadIdx.x;
4     if (i < N) C[i] = A[i] + B[i];
5 }
6
7 // Host (runs on CPU)
8 void launch_kernel(float* d_A, float* d_B, float* d_C, int N) {
9     int threadsPerBlock = 256;
10    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
11    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
12 }
```


Chapter 62

INTEROPERABILITY

C++ is the dark matter of the software universe: it binds everything together.

62.0.1 35.1 Python Bindings (pybind11)

Bridging the gap between Python's ease of use and C++'s raw power. * **The Problem:** Python objects are ref-counted (PyObject*). C++ has RAIL. Who owns the pointer? * **Return Value Policies:** * `py::return_value_policy::copy`: Python gets a copy (Safe). * `py::return_value_policy::reference`: Python references C++ memory (Dangous if C++ deletes it). * `py::return_value_policy::take_ownership`: Python takes over delete.

```
1 #include <pybind11/pybind11.h>
2 namespace py = pybind11;
3
4 struct Pet {
5     std::string name;
6     Pet(const std::string &name) : name(name) { }
7 };
8
9 PYBIND11_MODULE(example, m) {
10     py::class_<Pet>(m, "Pet")
11         .def(py::init<const std::string &>())
12         .def_readwrite("name", &Pet::name);
13 }
```

62.0.2 35.2 Java Native Interface (JNI)

The bridge to the Enterprise (and Android). * **Cost:** Crossing the JVM boundary is expensive (pointer chasing, pinning objects). * **Pitfall:** `JNIEnv*` is thread-local. Do not share it between threads. * **Local References:** JNI creates local refs for every object returned. If you don't `DeleteLocalRef` in a loop, the JVM OOMs.

```
1 extern "C" JNIEXPORT jstring JNICALL
2 Java_com_example_MyClass_nativeMethod(JNIEnv *env, jobject thiz) {
3     return env->NewStringUTF("Hello from C++");
4 }
```

62.0.3 35.3 Stable C ABI

To speak to Rust, C#, or Go, use the lingua franca: C. * `extern "C"`: Disables C++ name mangling (e.g., `_Z3foo` becomes `foo`). * **Struct Layout:** Use `StandardLayoutType` structs (no virtuals, all public).

Chapter 63

SECURITY ENGINEERING

63.0.1 36.1 Fuzzing (libFuzzer / AFL++)

Unit tests test what you *know*. Fuzzing tests what you *don't*. * **Coverage-Guided:** The fuzzer instruments binaries to see which inputs explore new code paths. * **Sanitizers:** Always fuzz with AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) enabled.

63.0.2 36.2 Cryptography & Timing Attacks

NEVER write your own crypto. Use libsodium or BoringSSL. * **The Trap:** `memcmp(hash1, hash2, 32)` exits early if the first byte differs. * **The Attack:** Attacker measures time. If it takes longer, they guessed the first byte right. * **The Fix:** Constant-time comparison. `cpp // libsodium`
`'s constant time check if (crypto\_verify\_32(hash1, hash2) != 0) \{ /* Error */ \}`

63.0.3 36.3 Side-Channel Mitigations (Spectre)

Modern CPUs execute instructions speculatively. * **Scenario:** `if (x < array\_len) \{ val = array[x]; \}` * **Attack:** CPU predicts true, loads `array[x]` (where `x` is out of bounds secret). Even if checks fail, `array[x]` is now in L1 cache. * **Mitigation:** `LFENCE` (Load Fence) or `std::clamp` indices to 0 on failure (masking).

Chapter 64

SPECIALIZED DOMAINS

64.0.1 37.1 Game Development (Data-Oriented Design)

OOP is cache-poison. Games use **Entity-Component-Systems (ECS)**. * **AoS (Array of Structs)**: [Pos, Vel], [Pos, Vel] -> Bad stride. * **SoA (Structure of Arrays)**: [Pos, Pos], [Vel, Vel] -> SIMD friendly.

64.0.2 37.2 Embedded Systems

- **Memory-Mapped I/O**: Casting integer addresses to pointers.
- `volatile`: Tells compiler “Hardware changes this value, do not optimize reads”.
- **Freestanding Implementation**: No OS, no heap (`malloc` is a myth).

64.0.3 37.3 High-Frequency Trading (HFT)

- **Kernel Bypass**: Solarflare `OpenOnload` maps NIC ring buffers directly to userspace, skipping the Linux Kernel (save ~2-3 microseconds).
- **Warm-up**: Pinging order gateways to ensure the TCP congestion window is open and CPU is in C0 state (max turbo).

64.0.4 37.4 Automotive (MISRA C++ / AUTOSAR)

- **No Dynamic Memory**: All containers have fixed capacity (`etl::vector` or `boost::static_vector`).
- **No Exceptions**: `try-catch` adds binary size and non-deterministic unwind paths. Use `std::expected` or error codes.
- **Static Analysis**: Code must pass MISRA-2008 rules (e.g., “Rule 5-0-15: Array indexing shall be checked”).

```
1 // Stack-only pattern (Automotive safe)
2 template <typename T, size_t N>
3 class FixedVector {
4     T data[N];
5     size_t count = 0;
6 public:
7     void push_back(const T& val) {
8         if (count < N) data[count++] = val;
9     }
10 };
```

64.1 FINAL COMPREHENSIVE CHECKLIST

64.1.1 C++98/03 Foundation

- Variables and basic types
- Operators and control flow
- Functions and overloading
- Arrays and pointers
- Classes and objects
- Constructors and destructors
- Inheritance
- Virtual functions and polymorphism
- STL containers (vector, list, map, set)
- Algorithms
- Strings and I/O

64.1.2 C++11 Major Features

- Auto type deduction
- `nullptr` and `nullptr_t`
- Uniform initialization `{}`
- Range-based for loops
- Smart pointers (`unique_ptr`, `shared_ptr`)
- Move semantics
- Rvalue references
- Lambda functions
- Variadic templates
- `std::array` and `std::tuple`
- `std::unordered_map` and `std::unordered_set`

64.1.3 C++14 Improvements

- Generic lambdas
- Return type deduction
- `std::make_unique`
- Digit separators (`1'000'000`)
- `decltype(auto)`

64.1.4 C++17 Modern Features

- Structured bindings
- `std::optional`
- `std::variant`
- `std::any`
- `if constexpr`
- Fold expressions
- Filesystem library
- Parallel algorithms
- `std::string_view`

64.1.5 C++20 Revolutionary

- Concepts and constraints
- Ranges
- Coroutines
- Modules
- Spaceship operator `<=>`
- Designated initializers
- Requires expressions

64.1.6 C++23 Latest

- Deducing `this`
- `std::expected`
- Literal classes in `constexpr`

64.1.7 Advanced Concepts

- Template metaprogramming
- CRTP (Curiously Recurring Template Pattern)
- SFINAE (Substitution Failure Is Not An Error)
- Type traits
- Memory management (stack vs heap)
- Smart pointers (`unique_ptr`, `shared_ptr`, `weak_ptr`)
- Move semantics and forwarding
- Perfect forwarding with `std::forward`

64.1.8 Concurrency

- Threading basics
- Mutexes and locks
- Condition variables
- Atomic operations
- Memory ordering
- Lock-free programming

64.1.9 Performance & Optimization

- Memory profiling
- Cache optimization
- SIMD and vectorization
- Compiler flags (-O2, -O3)
- Profiling tools (perf, valgrind)

64.1.10 STL Mastery

- All container types
- All algorithms
- Iterators
- Custom comparators
- Ranges (C++20)

64.1.11 Design Patterns

- Singleton
- Factory
- Observer
- Strategy
- CRTP

64.1.12 Professional Development

- CMake build system
- Testing frameworks (Google Test)
- Debugging (gdb)
- Profiling
- Code organization

- RAII pattern
 - Error handling
 - Memory leak detection
-

64.2 Key Insights for C++ Mastery

1. **RAII** = Guaranteed resource cleanup
 2. **Smart Pointers** = No manual memory management
 3. **Move Semantics** = Zero-copy optimization
 4. **Const Correctness** = Prevents bugs, enables optimizations
 5. **Templates** = Compile-time computation
 6. **Concepts** (C++20) = Type-safe constraints
 7. **Ranges** (C++20) = Composable algorithms
 8. **Coroutines** (C++20) = Async programming made easy
 9. **Atomic Operations** = Safe concurrent access
 10. **Performance** = Measure, profile, then optimize
-

64.3 Learning Path

1. **Week 1-2:** C++98 basics (variables, control flow, functions)
 2. **Week 3-4:** Classes and OOP (constructors, inheritance, polymorphism)
 3. **Week 5:** STL containers and algorithms
 4. **Week 6-7:** C++11 (smart pointers, lambdas, move semantics)
 5. **Week 8:** C++14 and C++17 features
 6. **Week 9:** C++20 features (concepts, ranges)
 7. **Week 10+:** Advanced topics (metaprogramming, concurrency, optimization)
-

64.4 Resources

64.4.1 Official Documentation

- cppreference.com - C++ standard library reference
- en.cppreference.com - Excellent resource
- isocpp.org - C++ standards committee

64.4.2 Books

- “A Tour of C++” by Bjarne Stroustrup
- “Effective Modern C++” by Scott Meyers
- “C++ Concurrency in Action” by Anthony Williams

64.4.3 Practice

- LeetCode.com
- HackerRank.com
- Codeforces.com
- ProjectEuler.net

You are now equipped to master C++ from absolute zero to expert level!

Last Updated: December 2025 C++ Versions Covered: C++98 through C++23

Chapter 65

ABA PROBLEM & MEMORY RECLAMATION

In the rarefied air of lock-free programming, the **ABA Problem** is the dragon that guards the gate. Conquering it requires understanding the very fabric of memory lifecycles.

65.0.1 1. The ABA Problem Explained

The Compare-And-Swap (CAS) primitive (`std::atomic::compare_exchange_weak`) checks if a value is *equal* to an expected value. It does **not** check if it is the *same* object.

The Scenario: 1. Thread 1 reads pointer **A** from a lock-free stack top. 2. Thread 1 is preempted. 3. Thread 2 pops **A**, frees it, then pushes **B**, then pushes a *new* object allocated at address **A** (recycled memory). 4. Thread 1 wakes up, performs CAS. The address is still **A**. CAS succeeds. 5. **Catastrophe:** Thread 1 has popped the *new* **A**, but its local logic assumes it's the *old* **A** (e.g., pointing to **B** as the next node). The stack is now corrupted.

65.0.2 2. Solution I: Tagged Pointers (Version Counters)

Pack a version counter into the unused bits of a pointer (usually top 16 bits on 64-bit systems). * **Mechanism:** Every modification increments the counter. `Ptr(A, v1) != Ptr(A, v2)`. * **Limitation:** Reduces addressable memory space; requires platform-specific bit manipulation.

```
1 // Example of a 64-bit tagged pointer
2 struct TaggedPtr {
3     uint64_t data; // 48 bits pointer, 16 bits tag
4
5     TaggedPtr(void* ptr, uint16_t tag) {
6         data = (reinterpret_cast<uint64_t>(ptr) & 0x0000FFFFFFFFFFFF) | (
7             static_cast<uint64_t>(tag) << 48);
8     }
9
10    void* get_ptr() const { return reinterpret_cast<void*>(data & 0
11        x0000FFFFFFFFFFFF); }
12    uint16_t get_tag() const { return static_cast<uint16_t>(data >> 48); }
```

65.0.3 3. Solution II: Hazard Pointers (The Gold Standard)

A **Hazard Pointer (HP)** is a thread-local signal saying “I am reading this object, do not delete it.”

The Protocol: 1. **Reader:** publish the pointer **P** to a thread-local HP slot. 2. **Reader:** Verify **P** is still in the data structure. If not, retry. 3. **Writer (Deleter):** Unlink **P** from the

structure. 4. **Writer:** Check all other threads' HPs. * If `p` is found in any HP, add `p` to a “Retire List” (do not `delete` yet). * If `p` is not found, `delete` immediately. 5. **Cleanup:** Periodically scan the Retire List and free objects no longer protected by HPs.

- **Pros:** Wait-free readers, deterministic memory bound.
- **Cons:** Heavy memory barrier usage (Store-Load fence needed after publishing HP).

65.0.4 4. Solution III: Epoch-Based Reclamation (EBR)

Used by `malloc` implementations and databases (like Silo). * **Concept:** A Global Epoch counter (`E`) and per-thread Local Epochs (`e_t`). * **Operation:** 1. Global Epoch `E` increments periodically. 2. Threads update `e_t = E` when entering a critical section. 3. Objects retired in Epoch `E` can be safely deleted when all threads have reached Epoch `E+1` or higher. * **Pros:** Extremely fast (just checking integers). * **Cons:** One stalled thread prevents *all* memory reclamation (OOM risk).

Chapter 66

TEMPLATE METAPROGRAMMING PATTERNS

Moving computation from runtime to compile-time saves cycles and enables zero-cost abstractions.

66.0.1 1. Expression Templates (Lazy Evaluation)

Avoid temporary objects in math operations. **Naive:** `Vector sum = A + B + C;` creates `tmp = A+B`, then `sum = tmp+C`. Allocations! **Expression Template:** `A + B` returns a lightweight `Sum<Vec, Vec>` object.

```
1 template <typename L, typename R>
2 struct Sum {
3     const L& l; const R& r;
4     auto operator[](size_t i) const { return l[i] + r[i]; }
5 };
6
7 template <typename L, typename R>
8 auto operator+(const L& l, const R& r) {
9     return Sum<L, R>{l, r};
10 }
11
12 // Usage
13 // Vector result = A + B + C;
14 // Becomes: result[i] = A[i] + B[i] + C[i] in a single loop!
```

66.0.2 2. Type Erasure (The `std::any` Pattern)

Polymorphism without inheritance. * **Technique:** Hold a `void*` or a `unique_ptr<Base>`, where `Base` is an abstract class inside a templated wrapper. * **Example:** `std::function`, `std::any`.

66.0.3 3. The Detection Idiom (`void_t`)

Check if a type has a member function or typedef at compile time.

```
1 template <typename, typename = std::void_t<>>
2 struct has_serialize : std::false_type {};
3
4 template <typename T>
5 struct has_serialize<T, std::void_t<decltype(std::declval<T>().serialize())>> :
6     std::true_type {};
```

```
6
7 static_assert(has_serialize<MyClass>::value, "MyClass must implement serialize  
    ( )");
```

Note: In C++20, simply use Concepts.

66.0.4 4. Policy-Based Design

Compose behavior via template arguments.

```
1 template <typename T, typename CheckingPolicy, typename ThreadingPolicy>
2 class SmartPtr : public CheckingPolicy, public ThreadingPolicy {
3     T* ptr;
4     // ...
5 };
6 // User chooses: SmartPtr<int, NoCheck, MultiThreaded>
```

Chapter 67

HIGH-PERFORMANCE DATA STRUCTURES

When `std::unordered_map` is too slow, we descend into the hardware.

67.0.1 1. The Disruptor (Ring Buffer on Steroids)

A lock-free ring buffer designed for high-throughput messaging (LMAX Trading). * **Key Concept:** Pre-allocated memory, sequence numbers, and “barriers”. * **False Sharing Prevention:** Padding sequence counters to 64 bytes (cache line). * **Batching:** Consumers process up to the known “published” sequence.

67.0.2 2. Swiss Table (Open Addressing + Metadata)

Used in `absl::flat_hash_map`. * **Structure:** Arrays of control bytes (metadata) and data slots. * **Control Byte:** 7 bits of hash + 1 bit for empty/deleted. * **SIMD Probing:** Load 16 control bytes into a vector register (SSE/AVX). Compare all 16 tags in parallel to find the slot. * **Result:** Drastically fewer cache misses than chaining.

67.0.3 3. Burst Tries / Judy Arrays

Cache-efficient digital trees (tries) for integer keys. * **Idea:** Nodes dynamically change type based on population (Linear list -> Bitmap -> Sub-trie).

67.0.4 4. Slot Map

O(1) insertion, deletion, and access with stable “handles” (indices) instead of pointers. * **Generational Indices:** Handle = [Index | Generation]. Prevents “Dangling Reference” equivalent (accessing a slot that was re-used for a new object).

Chapter 68

REAL-TIME AUDIO & SIGNAL PROCESSING

The Golden Rule: In the audio callback, thou shalt not: 1. **Block** (No Mutexes). 2. **Allocate** (No `malloc/new`). 3. **Perform I/O** (No file reads, no `printf`).

68.0.1 1. Lock-Free IPC (SPSC Queue)

Communication between the UI Thread (writes parameters) and Audio Thread (reads parameters) must be wait-free. * **Structure:** Single-Producer Single-Consumer Circular Buffer. * **Atomic Indices:** `head` (write) and `tail` (read) are `std::atomic<size_t>`.

68.0.2 2. SIMD for DSP

Digital Signal Processing (Filters, FFT) is purely math-bound. * **Auto-vectorization:** Help the compiler (restrict pointers, fixed loop bounds). * **Intrinsics:** Manually using `<immintrin.h>` for AVX-512 processing of 16 samples per cycle. * **Biquad Filter:** The workhorse of EQ.

```
cpp // Processing 4 stereo channels (8 floats) at once with AVX
_mm256 samples =
_mm256_load_ps(input_ptr);
_mm256 result = _mm256_add_ps(_mm256_mul_ps(samples,
b0), ...);
```

68.0.3 3. Double Buffering

For spectral analysis (FFT) which requires blocks (e.g., 1024 samples), while audio comes in small chunks (e.g., 64 samples). * **Technique:** Fill Buffer A. When full, signal worker thread to process A, swap to filling Buffer B.

Chapter 69

ROBOTICS & ROS2 DEVELOPMENT

Robotics is where Soft Real-Time (Navigation) meets Hard Real-Time (Motor Control).

69.0.1 1. ROS2 Architecture & DDS

Robot Operating System 2 (ROS2) runs on top of DDS (Data Distribution Service). * **Nodes:** Independent processes. * **Topics:** Pub/Sub channels. * **Services:** RPC-style calls.

69.0.2 2. Zero-Copy Transport (Iceoryx)

Standard ROS2 serialization is slow for large data (LiDAR point clouds, 4K video). * **Solution:** Shared Memory. * **Mechanism:** 1. Publisher requests a memory chunk from shared segment. 2. Publisher writes data directly. 3. Publisher sends the *offset* (pointer) to Subscriber. 4. Subscriber reads directly. **Zero copies.**

69.0.3 3. Real-Time Executors

Standard ROS2 executors can suffer from priority inversion. * **Callback-group-level Executor:** Prioritize “Safety Stop” topic callbacks over “Camera Logging” callbacks.

69.0.4 4. Custom Allocators (`std::pmr`)

In the real-time control loop (1kHz+), heap fragmentation is fatal. * **Pattern:** Use `std::pmr::monotonic_buffer_resource` on the stack for message generation.

Chapter 70

MACHINE LEARNING INFRASTRUCTURE

Deep Learning frameworks (PyTorch, TensorFlow) are C++ engines wrapped in Python.

70.0.1 1. Tensor Memory Layout

A Tensor is a block of memory + a “View”. * **Strides:** The number of elements to skip to reach the next index in a dimension. * $\text{Element}(i, j) = \text{data}[i * \text{stride_i} + j * \text{stride_j}]$ * **Contiguity:** Transposing a tensor (`A.T`) usually just modifies strides, touching **zero** data.

70.0.2 2. Broadcasting Implementation

How to add `[32, 1]` vector to `[32, 100]` matrix? * **Virtual Expansion:** Set stride for the dimension of size 1 to 0. Accessing that dimension repeatedly reads the same value.

70.0.3 3. Automatic Differentiation (Autograd)

- **Computational Graph:** Directed Acyclic Graph (DAG) of operations.
- **Reverse Mode (Backprop):**
 1. Forward pass: Compute $y = f(x)$, store intermediate values (tape).
 2. Backward pass: Compute dL/dx using stored values and chain rule.
- **Implementation:** Node class with `virtual Tensor backward(Tensor grad)`.

70.0.4 4. Operator Fusion

Optimizing `ReLU(Add(MatMul(A, B), C))` into a single kernel launch to minimize VRAM bandwidth.

Chapter 71

DATABASE INTERNALS (LSM TREES)

How RocksDB and LevelDB achieve millions of writes per second.

71.0.1 1. The Write Problem

Random writes to disk (B-Tree update) are slow (IOPS bottleneck). Sequential writes are fast.

71.0.2 2. LSM Tree (Log-Structured Merge Tree)

- **MemTable:** In-memory sorted structure (SkipList or Red-Black Tree).
 - Writes go here first (fast RAM access).
 - WAL (Write Ahead Log) on disk for durability.
- **Immutable MemTable:** When MemTable is full, it becomes immutable and is flushed to disk.
- **SSTable (Sorted String Table):** The flushed file on disk. Key-Value pairs sorted by Key.
- **Compaction:** Background process merges multiple SSTables, discarding overwritten/deleted keys (Leveled Compaction).

71.0.3 3. Bloom Filters

To read a key, we might have to check *all* SSTables. Slow! * **Optimization:** Each SSTable has a Bloom Filter. * **Check:** If Bloom says “No”, key is definitely not in this file. Skip it.

71.0.4 4. Memory Mapped I/O (mmap)

Mapping the SSTable file directly into virtual address space. The OS manages paging. * **Benefits:** Zero-copy from disk cache to user space. * **Risks:** SIGBUS if file is truncated; lack of control over eviction.

Chapter 72

THE ULTIMATE ALGORITHM REFERENCE

Stop writing loops. Use the STL.

72.0.1 27.1 Non-Modifying Sequence Operations

- `std::all_of(begin, end, pred)`: True if all match.
- `std::any_of(begin, end, pred)`: True if any match.
- `std::none_of(begin, end, pred)`: True if none match.
- `std::for_each(begin, end, func)`: Apply function to all.
- `std::count(begin, end, val)`: Count occurrences.
- `std::mismatch(b1, e1, b2)`: Find first difference.
- `std::find(begin, end, val)`: Linear search.

72.0.2 27.2 Modifying Sequence Operations

- `std::copy(b, e, out)`: Copy range.
- `std::transform(b, e, out, op)`: Map function over range.
- `std::generate(b, e, gen)`: Fill with generator.
- `std::remove_if(b, e, pred)`: **Erase-Remove Idiom** step 1. Move valid elements to front.
- `std::replace(b, e, old, new)`: Replace values.
- `std::unique(b, e)`: Remove consecutive duplicates.

72.0.3 27.3 Partitioning

- `std::partition(b, e, pred)`: Reorder so true predicates come first. $O(N)$.
- `std::stable_partition`: Preserves relative order.

72.0.4 27.4 Sorting

- `std::sort(b, e)`: IntroSort (Quick + Heap + Insertion). $O(N \log N)$.
- `std::partial_sort(b, mid, e)`: Top K elements sorted.
- `std::nth_element(b, nth, e)`: Element at `nth` is what it would be if sorted. $O(N)$.

72.0.5 27.5 Binary Search (On Sorted Ranges)

- `std::lower_bound(b, e, val)`: First element $\geq$ val.
- `std::upper_bound(b, e, val)`: First element $>$ val.
- `std::binary_search(b, e, val)`: True/False existence.

72.0.6 27.6 Numeric Operations ()

- `std::iota(b, e, start)`: Fill with 0, 1, 2...
 - `std::accumulate(b, e, init)`: Sum (fold left).
 - `std::reduce(b, e)`: Parallelizable sum (C++17).
 - `std::inner_product`: Dot product.
-

Chapter 73

CAPSTONE PROJECT - HIGH-PERFORMANCE ORDER BOOK

This capstone project integrates C++20/23 features into a realistic high-frequency trading (HFT) component. It demonstrates Modules, Concepts, Ranges, Coroutines, and modern error handling.

73.0.1 Project Structure

```
1 order_book/  
2   src/  
3     types.cppm      (Module: Common types)  
4     order.cppm      (Module: Order definition)  
5     book.cppm       (Module: OrderBook logic)  
6     main.cpp        (Entry point)  
7   CMakeLists.txt  
8   README.md
```

73.0.2 1. Types Module (types.cppm)

```
1 export module types;  
2  
3 import <cstdint>;  
4 import <compare>;  
5  
6 export namespace hft {  
7     using Price = int64_t;  
8     using Quantity = uint32_t;  
9     using OrderId = uint64_t;  
10  
11     enum class Side : uint8_t { Buy, Sell };  
12 }
```

73.0.3 2. Order Module (order.cppm)

```
1 export module order;  
2  
3 import types;  
4 import <format>;  
5 import <string>;
```

```

6
7 export namespace hft {
8     struct Order {
9         OrderId id;
10        Side side;
11        Price price;
12        Quantity quantity;
13
14        // C++20 Spaceship for easy comparison
15        auto operator<=>(const Order&) const = default;
16
17        // C++23 Deducing This for generic accessors (example)
18        template<typename Self>
19        auto&& get_price(this Self&& self) {
20            return std::forward<Self>(self).price;
21        }
22    };
23 }
24
25 // C++20 Formatter specialization
26 template<>
27 struct std::formatter<hft::Order> {
28     constexpr auto parse(format_parse_context& ctx) { return ctx.begin(); }
29
30     auto format(const hft::Order& o, format_context& ctx) const {
31         return std::format_to(ctx.out(), "[ID:{}] {} @ {}",
32             o.id, (o.side == hft::Side::Buy ? "BUY" : "SELL"), o.price);
33     }
34 };

```

73.0.4 3. Order Book Module (book.cppm)

```

1 export module book;
2
3 import types;
4 import order;
5 import <vector>;
6 import <map>;
7 import <ranges>;
8 import <algorithm>;
9 import <expected>;
10 import <print>;
11 import <coroutine>;
12
13 export namespace hft {
14
15     // C++20 Concept for Order Container
16     template<typename T>
17     concept OrderContainer = requires(T c) {
18         c.push_back(std::declval<Order>());
19         c.size();
20     };
21
22     class OrderBook {
23     private:
24         // Use std::flat_map (C++23) for cache locality if available,
25         // else std::map. Simulated here as vector for simplicity + ranges
26         std::vector<Order> bids;
27         std::vector<Order> asks;
28
29     public:
30         // C++23 std::expected for error handling

```

```

31     std::expected<void, std::string> add_order(Order o) {
32         if (o.quantity == 0) return std::unexpected("Invalid quantity");
33
34         auto& side_vec = (o.side == Side::Buy) ? bids : asks;
35         side_vec.push_back(o);
36
37         // Keep sorted (simplified)
38         std::ranges::sort(side_vec, {}, &Order::price);
39         if (o.side == Side::Buy) std::ranges::reverse(side_vec);
40
41         return {};
42     }
43
44     // C++20 Coroutine Generator to stream top orders
45     // Note: Requires <generator> (C++23) or custom implementation
46     // Here we simulate a simple generator pattern or use ranges
47     auto top_levels(Side side, int depth) const {
48         const auto& vec = (side == Side::Buy) ? bids : asks;
49         return vec | std::views::take(depth);
50     }
51
52     void print_book() const {
53         std::println("--- Order Book ---");
54         std::println("ASKS:");
55         for (const auto& o : asks | std::views::reverse) std::println("  {}
56         ", o);
57         std::println("BIDS:");
58         for (const auto& o : bids) std::println("  {}", o);
59         std::println("-----");
60     }
61 }

```

73.0.5 4. Main Application (main.cpp)

```

1  import book;
2  import order;
3  import types;
4  import <print>;
5
6  int main() {
7      hft::OrderBook book;
8
9      book.add_order({1, hft::Side::Buy, 100, 10});
10     book.add_order({2, hft::Side::Buy, 99, 5});
11     book.add_order({3, hft::Side::Sell, 101, 20});
12     book.add_order({4, hft::Side::Sell, 102, 15});
13
14     book.print_book();
15
16     // Demonstrate Error Handling
17     if (auto res = book.add_order({5, hft::Side::Buy, 100, 0}); !res) {
18         std::println(stderr, "Error adding order: {}", res.error());
19     }
20
21     return 0;
22 }

```

Chapter 74

APPENDICES

Chapter 75

Appendix A: C++ Keywords & Operators Reference

75.0.1 Essential Keywords (Non-Exhaustive)

- **alignas** / **alignof**: Memory alignment queries and specifications.
- **asm**: Inline assembly block.
- **auto**: Type deduction (C++11).
- **const** / **volatile**: cv-qualifiers for type safety and hardware access.
- **constexpr** / **consteval** / **constinit**: Compile-time constant specifications.
- **decltype**: Inspect declared type of an entity.
- **explicit**: Prevent implicit conversions in constructors.
- **export**: Module interface export (C++20).
- **friend**: Allow access to private members.
- **inline**: Suggest inlining to compiler; allow definition in header.
- **mutable**: Allow modification of member in const object.
- **noexcept**: Specifier for functions that don't throw.
- **nullptr**: Null pointer literal (C++11).
- **operator**: Overload operators.
- **requires**: Constraint clause for Concepts (C++20).
- **static_assert**: Compile-time assertion.
- **template**: Define generic classes/functions.
- **thread_local**: Storage duration specifier.
- **typeid**: Runtime type identification (RTTI).
- **typename**: Declare a type parameter in templates.
- **virtual**: Declare virtual function for polymorphism.

75.0.2 Special Operators

- `::` Scope resolution
 - `->*` Pointer to member selection
 - `<=>` Three-way comparison (Spaceship) (C++20)
 - `co\_await`, `co\_yield`, `co\_return` Coroutine operators (C++20)
-

Chapter 76

Appendix B: Common Acronyms

- **ABI**: Application Binary Interface.
 - **API**: Application Programming Interface.
 - **COW**: Copy On Write.
 - **CRTP**: Curiously Recurring Template Pattern.
 - **CTAD**: Class Template Argument Deduction.
 - **UB**: Undefined Behavior (Avoid at all costs!).
 - **IB**: Implementation-defined Behavior.
 - **IIFE**: Immediately Invoked Function Expression (often with lambdas).
 - **NrvO** / **RVO**: (Named) Return Value Optimization.
 - **ODR**: One Definition Rule.
 - **PIMPL**: Pointer to Implementation (Opaque Pointer).
 - **RAII**: Resource Acquisition Is Initialization.
 - **RTTI**: Run-Time Type Information.
 - **SFINAE**: Substitution Failure Is Not An Error.
 - **SOO** / **SSO**: Small Object/String Optimization.
 - **STL**: Standard Template Library.
 - **TMP**: Template Metaprogramming.
 - **TU**: Translation Unit.
-

Chapter 77

Appendix C: Recommended Tooling

77.0.1 Compilers

- **GCC (GNU Compiler Collection)**: Standard on Linux.
- **Clang/LLVM**: Excellent error messages, widely used on macOS/Linux.
- **MSVC (Microsoft Visual C++)**: Standard on Windows.

77.0.2 Build Systems

- **CMake**: The industry standard meta-build system.
- **Meson**: Modern, fast, Python-based.
- **Bazel**: Google's build system, good for monorepos.

77.0.3 Package Managers

- **Conan**: Decentralized package manager for C/C++.
- **vcpkg**: Microsoft's C++ library manager.

77.0.4 Static Analysis & Sanitizers

- **AddressSanitizer (ASan)**: Detects memory errors (buffer overflows, use-after-free).
 - **UndefinedBehaviorSanitizer (UBSan)**: Detects undefined behavior.
 - **ThreadSanitizer (TSan)**: Detects data races.
 - **Clang-Tidy**: Linter and static analysis tool.
 - **Cppcheck**: Static analysis tool.
-

Chapter 78

Appendix D: Common C++ Traps & Pitfalls

78.0.1 I. General & Syntax Traps

1. Most Vexing Parse

- *Issue:* `MyClass obj();` declares a function returning `MyClass`, not a default-constructed object.
- *Fix:* Use brace initialization: `MyClass obj{\{\}};`.

2. The “dangling else” Problem

- *Issue:* Nested `if-else` without braces can associate `else` with the wrong `if`.
- *Fix:* Always use braces `\{\}` for control structures.

3. Integer Division

- *Issue:* `1/2` results in 0 (integer), not 0.5.
- *Fix:* Cast one operand to float/double: `1.0/2` or `static\_cast<double>(1)/2`.

4. Loop Variable Type Mismatch

- *Issue:* `for (unsigned i = v.size() - 1; i >= 0; --i)` causes an infinite loop because `unsigned` is never negative.
- *Fix:* Use `int` (and cast size) or standard iterators/ranges.

5. Shadowing Variables

- *Issue:* Declaring a local variable with the same name as a member or outer variable hides the outer one.
- *Fix:* Enable compiler warnings (`-Wshadow`) and use `this->member` if necessary.

78.0.2 II. Pointers, References & Memory

6. Object Slicing

- *Issue:* Assigning a `Derived` object to a `Base` value slices off the derived part.
- *Fix:* Use pointers `Base*` or references `Base\&` for polymorphism.

7. Dangling References

- *Issue*: Returning a reference to a local stack variable.
- *Fix*: Return by value or use smart pointers/dynamic allocation.

8. Iterator Invalidation

- *Issue*: Adding elements to a `std::vector` may reallocate memory, invalidating all pointers/iterators to elements.
- *Fix*: Don't cache iterators across mutating operations; use `reserve()` if possible.

9. `delete` VS `delete[]`

- *Issue*: Mismatching `new` with `delete[]` or `new[]` with `delete` causes undefined behavior.
- *Fix*: Use `std::vector` or `std::unique_ptr` instead of manual management.

10. Use-After-Move

- *Issue*: Accessing an object after `std::move()` (except for reassignment/destruction).
- *Fix*: Treat moved-from objects as empty; do not read their state.

78.0.3 III. Classes & OOP

11. Virtual Destructor Missing

- *Issue*: Deleting a derived class via a base pointer when the base destructor is not `virtual` leaks derived resources.
- *Fix*: Always mark base class destructors `virtual` (or `protected` if non-polymorphic).

12. Calling Virtual Functions in Constructor/Destructor

- *Issue*: Calls the *base* class version, not the derived one, because the derived part isn't initialized/is already destroyed.
- *Fix*: Use two-phase initialization or factory methods.

13. Copy Constructor/Assignment Missing

- *Issue*: Classes managing raw pointers will default to shallow copy (double free error).
- *Fix*: Follow the **Rule of Three/Five/Zero**.

14. Initialization Order

- *Issue*: Members are initialized in *declaration order*, not initializer list order.
- *Fix*: Keep initializer list order identical to member declaration order to avoid warnings.

78.0.4 IV. Concurrency

15. Data Races

- *Issue:* Multiple threads accessing shared memory without synchronization (at least one writer).
- *Fix:* Use `std::mutex`, `std::atomic`, or `std::shared_mutex`.

16. Deadlocks

- *Issue:* Two threads waiting on each other's locks.
- *Fix:* Acquire locks in a consistent global order; use `std::scoped_lock` (C++17) to lock multiple mutexes safely.

17. False Sharing

- *Issue:* Independent atomic variables on the same cache line degrade performance due to cache coherency protocols.
- *Fix:* Use `alignas(hardware_destructive_interference_size)` to pad variables.

78.0.5 V. Modern C++ & Macros

18. `std::vector<bool>` Weirdness

- *Issue:* It's a template specialization (bitfield), not a vector of bools. Returns a proxy object, not `bool&`.
- *Fix:* Use `std::deque<bool>` or `std::vector<char>` if you need real references.

19. Auto Type Deduction

- *Issue:* `auto` drops references and `const`.
- *Fix:* Use `auto&` or `const auto&` explicitly when needed.

20. Macro Side Effects

- *Issue:* `#define MAX(a,b) ((a) > (b) ? (a) : (b))` evaluates arguments twice. `MAX(x++, y)` increments `x` twice.
- *Fix:* Use `inline` functions or templates instead of macros.

21. Static Initialization Order Fiasco

- *Issue:* Global objects in different files have undefined initialization order.
 - *Fix:* Use the "Construct On First Use" idiom (Meyers Singleton).
-

Chapter 79

Appendix H: Professional C++ Idioms

79.0.1 1. RAII (Resource Acquisition Is Initialization)

- **Concept:** Bind resource lifecycle to object lifecycle. Constructor acquires, destructor releases.
- **Use Case:** Memory, file handles, mutex locks, sockets.
- **Example:** `std::lock_guard`, `std::unique_ptr`.

79.0.2 2. Pimpl (Pointer to Implementation)

- **Concept:** Hide private members in a separate class/struct, accessed via a pointer.
- **Benefit:** ABI stability, reduced compilation times (header dependency changes don't trigger rebuilds of clients).
- **Pattern:**

```
cpp      class Widget \{          struct Impl;          std::unique_ptr<Impl> pImpl
;      public:          Widget();          ~Widget(); // Defined in .cpp where Impl is visible
          \};
```

79.0.3 3. Copy-and-Swap

- **Concept:** Implement assignment operator in terms of copy constructor and swap.
- **Benefit:** Strong Exception Safety guarantee; removes code duplication.
- **Pattern:**

```
cpp      T& operator=(T other) \{ // Pass by value (copy)          swap(*this, other
);          return *this;          \}
```

79.0.4 4. NVI (Non-Virtual Interface)

- **Concept:** Public interface is non-virtual; virtual functions are private/protected.
- **Benefit:** Separation of interface (pre/post-conditions) from implementation.
- **Pattern:**

```
cpp      class Base \{      public:          void doWork() \{          // Pre-condition
          logic          doWorkImpl();          // Post-condition logic          \}          private
:          virtual void doWorkImpl() = 0;          \};
```

79.0.5 5. Erase-Remove Idiom

- **Concept:** Standard way to remove elements from a `std::vector` (before C++20 `std::erase`).
- **Pattern:** `v.erase(std::remove(v.begin(), v.end(), value), v.end());`

79.0.6 6. SFINAE (Substitution Failure Is Not An Error)

- **Concept:** Remove functions from overload resolution set if types don't match constraints.
- **Modern Replacement:** C++20 Concepts (`requires`).

79.0.7 7. CRTP (Curiously Recurring Template Pattern)

- **Concept:** Class `Derived` inherits from `Base<Derived>`.
 - **Use Case:** Static polymorphism (compile-time), adding functionality (mixins) without vtable overhead.
 - **Example:** `std::enable_shared_from_this`.
-

Chapter 80

Appendix E: C++ Interview Cheat Sheet

80.0.1 Core Concepts

1. **Virtual Functions:** Enable runtime polymorphism via `vtable/vptr`. Destructors must be `virtual` in base classes.
2. **Smart Pointers:**
 - `unique_ptr`: Exclusive ownership, no overhead.
 - `shared_ptr`: Shared ownership, ref-counted (atomic), control block overhead.
 - `weak_ptr`: Non-owning reference to `shared_ptr` (breaks cycles).
3. **Move Semantics:** Transfers resources (pointers) instead of deep copying. Enabled by rvalue references (`&&`) and `std::move`.
4. **RAII:** Resource Acquisition Is Initialization. Constructor acquires, destructor releases. Core to C++ safety.
5. **Cast Types:**
 - `static_cast`: Compile-time safe conversions.
 - `dynamic_cast`: Runtime checked downcasting (requires RTTI).
 - `reinterpret_cast`: Bitwise reinterpretation (unsafe).
 - `const_cast`: Remove/add constness.

80.0.2 Modern C++ (C++11/14/17/20)

1. **Lambdas:** Anonymous function objects. Capture `[=]`, `[&]`, or move-only `[x = std::move(y)]`.
2. **Auto:** Type deduction. Always initialize.
3. **Structured Bindings (C++17):** `auto [x, y] = pair;`
4. **Concepts (C++20):** Constrain templates for better errors/readability.
5. **Coroutines (C++20):** Functions that can suspend/resume.

80.0.3 System Design Questions

1. **Vector vs List:** Vector (contiguous, cache-friendly) is almost always better than List (node-based, cache misses) unless aggressive splicing is needed.
2. **Map vs Unordered Map:** Map (BST, $O(\log n)$, sorted) vs Unordered Map (Hash Table, $O(1)$ avg, unsorted).
3. **Handling 1M connections:** Use non-blocking I/O (epoll/kqueue) or `io_uring`, not one thread per connection.
4. **Memory Layout:** Stack (local vars) vs Heap (dynamic) vs Data (globals) vs Text (code).

80.0.4 Quick Coding

- **Implement Singleton:** Use static local variable (Thread-safe in C++11+).
 - **Implement String Class:** Handle deep copy, move semantics, and destructor.
 - **Reverse Linked List:** Classic pointer manipulation.
-

Chapter 81

Appendix F: The C++ Standard Evolution Matrix

81.0.1 1. Versioned Changelog

C++98 (ISO/IEC 14882:1998) - *The Foundation*

Released: 1998 * **Core:** Templates, Exceptions, Namespaces, `bool` type, cast operators (`static_cast`, etc.), `mutable`, `explicit`. * **STL:** Containers (`vector`, `list`, `map`, `set`, `deque`), Algorithms (`sort`, `find`, `transform`), Iterators, Strings (`std::string`), I/O Streams (`iostream`). * **Memory:** `std::auto_ptr` (Deprecated in C++11).

C++03 (ISO/IEC 14882:2003) - *The Bug Fix*

Released: 2003 * **Focus:** Defect Report (DR) fixes for C++98 to ensure consistency across compilers. * **Features:** Value initialization `T()`, fixes to `std::vector` contiguous memory guarantee.

C++11 (ISO/IEC 14882:2011) - *The Modern Revolution*

Released: September 2011 * **Language:** `auto`, `nullptr`, Range-based `for`, Lambda expressions, Rvalue references (`&&`) & Move semantics, Variadic templates, `constexpr` (limited), `decltype`, Uniform initialization `{}`, `static_assert`, `override`, `final`, `enum class`. * **Concurrency:** `std::thread`, `std::mutex`, `std::atomic`, `std::future`, `std::async`. * **Library:** Smart pointers (`unique_ptr`, `shared_ptr`, `weak_ptr`), `std::array`, `std::tuple`, `std::unordered_map/set`, `std::regex`, `std::chrono`.

C++14 (ISO/IEC 14882:2014) - *The Refinement*

Released: December 2014 * **Language:** Generic lambdas (auto params), Relaxed `constexpr` (loops/variables allowed), Binary literals (`0b1010`), Digit separators (`1'000`), Variable templates, Return type deduction. * **Library:** `std::make_unique`, `std::shared_timed_mutex`, `std::integer_sequence`, `std::exchange`, `std::quoted`.

C++17 (ISO/IEC 14882:2017) - *The Major Update*

Released: December 2017 * **Language:** Structured bindings `auto [x,y] = p;`, `if constexpr`, Fold expressions (`... + args`), Class Template Argument Deduction (CTAD), Inline variables, `__has_include`. * **Library:** `std::filesystem`, `std::optional`, `std::variant`, `std::any`, `std::string_view`, Parallel Algorithms (`std::execution::par`), `std::invoke`, `std::byte`, `std::pmr` (Polymorphic Memory Resources).

C++20 (ISO/IEC 14882:2020) - *The Gigantic Leap*

Released: December 2020 * **Language:** Concepts (Constraints), Modules (import/export), Coroutines (co_await), Three-way comparison (<=>), Designated initializers {x=1}, consteval (Immediate functions), constexpr, Range-based for with init. * **Library:** Ranges (std::ranges), std::span, std::format, std::jthread, std::stop_token, std::barrier, std::latch, std::semaphore, std::bit_cast, std::source_location, Calendars & Timezones.

C++23 (ISO/IEC 14882:2023) - *The Completion*

Released: October 2023 * **Language:** Deducing this (Explicit object parameter), if constexpr, Multidimensional subscript m[1,2], Static operator(), auto(x) decay copy. * **Library:** std::print, std::println, std::expected (Error handling), std::mdspan, std::flat_map, std::flat_set, std::generator (Synchronous coroutines), std::stacktrace, std::stdatomic.h.

81.0.2 2. Feature Matrix

Feature	C++98	C++11	C++14	C++17	C++20	C++23
Memory Variables	auto_ptr Type req.	unique_ptr auto	make_unique Var templates	pmr Structured Bindings	shared_ptr atomic constexpr	weak_ptr atomic constexpr
Loops	for(;;)	Range-for	-	-	Range-for init	for
Templates	Basic	Variadic	Variable	Fold Expressions	Concepts	Deduction Guides
Lambdas	-	Basic	Generic	constexpr	Template	Range-compare
Concurrency	-	thread	shared_lock	Parallel Algos	jthread, Latches	std::jthread
String	string	to_string	quoted	string_view	format	string_view
Metaprog.	Traits	static_assert	integer_seq	if constexpr	constexpr	if constexpr
Modules	-	-	-	-	Modules	std::literals
Coroutines	-	-	-	-	Async	std::generator

81.0.3 3. Timeline & Release Accuracy

Standard	ISO Publication	Codename	Compiler Flag (GCC/Clang)
C++98	1998-09	C++98	-std=c++98
C++03	2003-10	C++03	-std=c++03
C++11	2011-09	C++0x	-std=c++11
C++14	2014-12	C++1y	-std=c++14
C++17	2017-12	C++1z	-std=c++17
C++20	2020-12	C++2a	-std=c++20
C++23	2023-10	C++2b	-std=c++23
C++26	<i>Expected 2026</i>	C++2c	-std=c++26 / -std=c++2c

Chapter 82

Appendix G: C++ Standard Library Headers Reference

82.0.1 Concepts & Utilities

- `<concepts>` (C++20): Fundamental concepts library.
- `<coroutine>` (C++20): Coroutine support library.
- `<functional>`: Function objects, binder, and reference wrappers.
- `<memory>`: Smart pointers and allocators.
- `<tuple>`: Tuple library.
- `<type_traits>`: Compile-time type information.
- `<utility>`: Utility components (`std::pair`, `std::move`).

82.0.2 Containers

- `<array>` (C++11): Fixed-size array class.
- `<deque>`: Double-ended queue.
- `<list>`: Doubly-linked list.
- `<map>`: Associative containers (Red-Black Tree).
- `<queue>`: Queue adapter.
- `<set>`: Associative containers (Red-Black Tree).
- `<stack>`: Stack adapter.
- `<unordered_map>` (C++11): Hash map.
- `<vector>`: Dynamic array.
- `<span>` (C++20): Non-owning view of contiguous memory.

82.0.3 Algorithms & Iterators

- `<algorithm>`: Algorithms that operate on ranges.
- `<execution>` (C++17): Parallel algorithms.
- `<iterator>`: Iterator primitives.
- `<numeric>`: Numeric operations (`accumulate`, `reduce`).
- `<ranges>` (C++20): Range primitives and views.

82.0.4 Concurrency

- `<atomic>` (C++11): Atomic operations.
- `<barrier>` (C++20): Barriers.
- `<condition_variable>` (C++11): Condition variables.
- `<future>` (C++11): Futures and promises.
- `<latch>` (C++20): Latches.
- `<mutex>` (C++11): Mutual exclusion primitives.
- `<semaphore>` (C++20): Semaphores.
- `<shared_mutex>` (C++14): Shared mutexes.
- `<thread>` (C++11): Thread class.

82.0.5 Input/Output

- `<filesystem>` (C++17): File system operations.
- `<format>` (C++20): Formatting library.
- `<fstream>`: File stream classes.
- `<iostream>`: Standard I/O stream objects.
- `<print>` (C++23): Print functions.
- `<sstream>`: String stream classes.

82.0.6 Numerics & Math

- `<bit>` (C++20): Bit manipulation.
- `<complex>`: Complex number arithmetic.
- `<random>` (C++11): Random number generation.
- `<ratio>` (C++11): Compile-time rational arithmetic.
- `<valarray>`: Class for representing and manipulating arrays of values.
- `<numbers>` (C++20): Mathematical constants.